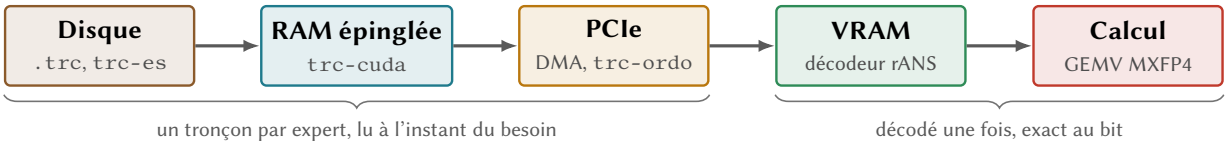
troncons

Tome II : le système

Architecture, format `.trc`, noyaux, cache et moteur ; la référence exacte et le juge ; le jalon J1 et son épreuve indépendante

Mémoire de recherche — le programme, construit et mesuré



63,4 Go
modèle servi (gpt-oss-120b), quatre fois la RAM

384 Mo
mémoire de pointe du processus ; llama.cpp : 10,9 Go

× 2,22
1,965 jeton/s, garde d'intégrité comprise, contre la meilleure base llama.cpp

0 / 5 120
pas dont les logits diffèrent entre fichier codé et brut

Stone

Toronto, le 16 octobre 2026

Fait suite au Tome I, *les fondations mesurées*. Chaque figure se régénère depuis les données de mesure commitées dans le dépôt `troncons` ; chaque nombre cite le fichier dont il vient.

Résumé

Le Tome I de *troncons* avait mesuré un portable ordinaire (16 Go de RAM, une RTX 4060 de 8 Go, un disque NVMe chiffré), démontré le toit du problème et décidé, par le calcul, de construire d'abord une architecture exacte : servir par la carte graphique un modèle bien plus gros que toute la mémoire de la machine, ses experts compressés sans perte sur le disque et décodés à l'instant où un calcul en a besoin, dans une fenêtre de mémoire qui ne dépend pas de la taille du modèle. Ce second tome est celui du système : le programme, construit en Rust avec des noyaux CUDA par une équipe de cinq lots aux interfaces gelées, et son premier jalon, jugé sur gpt-oss-120b (63,4 Go, quatre fois la RAM, huit fois la mémoire de la carte) contre douze critères déclarés avant la première ligne de code.

Le tome donne l'architecture et la structure du programme ; le format `.trc` au bit près, un tronçon par expert et des flux rangés par niveau de raffinement ; le chemin des octets, du disque à la mémoire de la carte, par la lecture sans tampon et la mémoire épinglée ; les noyaux, dont le décodeur rANS à table par voie, avec la preuve de leur exactitude et de leur déterminisme ; une référence en double précision et la théorie du juge, fondée sur la divergence de Kullback et Leibler ; le cache d'experts, l'ordonnanceur et le moteur, avec leurs lemmes.

Les résultats tiennent en quatre nombres, chacun recalculé par un second chemin depuis les passes commitées. *Mémoire* : 384 Mo de pointe pour le modèle de 63,4 Go, la même à 3,7 Mo près que pour un modèle cinq fois plus petit, quand llama.cpp en tient 10,9 Go (11,4 Go dans sa configuration la plus rapide) ; la mémoire ne suit plus la taille du modèle. *Exactitude* : des logits identiques au bit entre le fichier codé et le fichier brut sur 5 120 pas, d'une construction et d'un ordre de calcul à l'autre. *Vitesse* : 1,965 jeton par seconde avec la garde d'intégrité, 2,22 fois la meilleure configuration de llama.cpp, et un préremplissage sept fois plus rapide ; deux rapports antérieurs, ×3,85 puis ×2,79, sont barrés parce que la base était mal réglée (la balayer l'a portée de 0,57 à 0,88 jeton/s). *Justesse* : plus près des mathématiques exactes du modèle que llama.cpp (KL 0,0031 contre 0,0607 sur le 120b), et, avec un cache KV en f32, égal à elles au format près : tout l'écart venait de l'arrondi f16 du cache.

Un juge qui n'avait écrit ni la spécification ni le code a refait les chiffres : son décodeur, écrit depuis la seule spécification, redonne les tronçons au bit près ; sa propre passe avant en double précision confirme la référence à l'arrondi du format près. Il a trouvé deux défauts graves (une passe qui se fige sans message, un processus enfant orphelin qui bloque son pilote) et cinq moyens, dont une corruption d'octets bruts invisible jusque dans le texte. La ronde de corrections les a corrigés ou mesurés en une matinée, chacun avec un témoin qui tombe sur l'ancien binaire ; une garde de production rend désormais toute corruption d'un tronçon lu en erreur fatale nommée, pour 1,6 % de vitesse à disque égal. Dix critères tiennent sur douze ; les deux autres attendent une décision de l'arbitre.

Restent la contribution de recherche, la découpe base + complément du jalon J2 ; un disque non chiffré, qui dira ce que BitLocker coûte sur un vrai modèle ; l'étude biomimétique de Stone, le génome des poids ; et la porte qui rendra ce moteur à relais. Le mémoire existe en deux versions complètes, anglaise et française ; chaque figure se régénère depuis les données de mesure commitées.

Abstract

Volume I of *troncons* measured an ordinary laptop (16 GB of RAM, an 8 GB RTX 4060, an encrypted NVMe disk), proved the roof of the problem and decided, by calculation, to build an exact architecture first: serve through the graphics card a model much larger than all the memory of the machine, its experts compressed losslessly on disk and decoded at the moment a computation needs them, within a memory window that does not depend on the size of the model. This second volume is the volume of the system: the program, built in Rust with CUDA kernels by a team of five lots with frozen interfaces, and its first milestone, judged on gpt-oss-120b (63.4 GB, four times the RAM, eight times the card's memory) against twelve criteria declared before the first line of code.

The volume gives the architecture and structure of the program; the `.trc` format to the bit, one chunk per expert and streams stored by refinement level; the byte path, from the disk to the card's memory, through unbuffered reads and pinned memory; the kernels, among them the rANS decoder with per-lane tables, with the proof of their exactness and determinism; a double-precision reference and the theory of the judge, founded on the Kullback–Leibler divergence; the expert cache, the scheduler and the engine, with their lemmas.

The results fit in four numbers, each recomputed by a second path from the committed passes. *Memory*: a 384 MB peak for the 63.4 GB model, the same to within 3.7 MB as for a model five times smaller, where llama.cpp holds 10.9 GB (11.4 GB in its fastest configuration); memory no longer follows the size of the model. *Exactness*: logits identical to the bit between the coded and the raw file over 5 120 steps, from one build and one order of computation to another. *Speed*: 1.965 tokens per second with the integrity guard, 2.22 times the best configuration of llama.cpp, and a prefill seven times faster; two earlier ratios, ×3.85 then ×2.79, are struck out because the baseline was badly tuned (sweeping it raised it from 0.57 to 0.88 token/s). *Correctness*: closer to the exact mathematics of the model than llama.cpp (KL 0.0031 against 0.0607 on the 120b), and, with an f32 KV cache, equal to them up to the format: the whole gap came from the f16 rounding of the cache.

A judge who wrote neither the specification nor the code redid the numbers: his decoder, written from the specification alone, reproduces the chunks to the bit; his own double-precision forward pass confirms the reference up to the rounding of the format. He found two serious defects (a pass that hangs without a message, an orphan child process that blocks its driver) and five medium ones, among them a corruption of raw bytes invisible down to the text. The round of corrections corrected or measured them in one morning, each with a control that fails on the old binary; a production guard now turns any corruption of a chunk read into a named fatal error, for 1.6 % of speed at equal disk. Ten criteria out of twelve hold; the other two await a decision of the arbiter.

What remains is the research contribution, the base + complement split of milestone J2; an unencrypted disk, which will tell what BitLocker costs on a real model; Stone's biomimetic study, the weight genome; and the gate that will give this engine back to relais. The thesis exists in two complete versions, English and French; every figure is regenerated from the committed measurement data.

Table des matières

Résumé iii

Abstract v

Table des figures xiii

Liste des tableaux xvii

Liste des listings xix

Notations et conventions xxi

1 Introduction : du calcul au programme 1
1.1 La mission, et ce que Stone a demandé pour ce tome . . . 1
1.2 D'où l'on part : le Tome I . . . 1
1.3 La question de ce tome . . . 2
1.4 Ce que ce tome apporte . . . 2
1.5 La méthode . . . 3
1.6 Plan du tome . . . 3

Première partie Le programme 5

2 Architecture et structure du programme 7
2.1 Les exigences, et la décision qui en sort . . . 7
2.1.1 Pourquoi Rust et CUDA, par les mesures . . . 8
2.1.2 Les dépendances, et pourquoi si peu . . . 9
2.2 L'espace de travail : quatorze membres, un graphe sans cycle . . . 9
2.3 Le schéma d'ensemble . . . 10
2.4 Les coutures : ce qui est gelé entre les lots . . . 11
2.4.1 La règle de déterminisme, ou pourquoi l'égalité au bit a un sens . . . 11
2.5 Les interfaces, telles qu'elles sont . . . 12
2.6 La chaîne d'exactitude . . . 15
2.7 La fenêtre bornée, propriété de l'architecture . . . 16
2.8 Ce que l'épreuve indépendante a changé à l'architecture . . . 17

3 Le format `.trc` au bit près 23
3.1 Les principes . . . 23
3.2 La disposition générale . . . 24
3.3 L'en-tête fixe . . . 24
3.4 MÉTA, RÉSIDENTS, TABLES, RÉPERTOIRE . . . 25
3.4.1 MÉTA : les paires du GGUF, recopiées . . . 25

3.4.2 RÉSIDENTS : les tenseurs toujours lus . . . 25
3.4.3 TABLES : les fréquences du rANS . . . 27
3.4.4 RÉPERTOIRE : la table des matières . . . 28
3.5 Une partie de tronçon . . . 28
3.6 Le flux rANS . . . 29
3.6.1 La sérialisation . . . 29
3.6.2 Le décodage normatif . . . 30
3.6.3 Pourquoi c'est exact . . . 31
3.6.4 Trente-deux voies en même temps . . . 32
3.7 Les contextes : la table choisie par le décodeur . . . 33
3.8 Les sortes de flux . . . 33
3.9 La reconstruction exacte d'un expert . . . 33
3.9.1 Le procédé normatif . . . 33
3.9.2 L'expert d'origine, et pourquoi il ne faut rien réordonner . . . 34
3.9.3 L'emplacement d'expert sur la carte . . . 35
3.10 Les empreintes . . . 35
3.11 Le codeur à mémoire bornée . . . 36
3.11.1 Pourquoi deux passes . . . 36
3.11.2 Mêmes octets, mémoire bornée . . . 37
3.12 C1 sur 768 et 4 608 experts . . . 38
3.12.1 Le premier chemin : cinq contrôles par expert . . . 38
3.12.2 Le second chemin : aucune ligne commune . . . 39
3.13 Le décodeur indépendant de l'épreuve . . . 41
3.13.1 Ce que l'écriture du décodeur a appris sur le texte . . . 42
3.14 Les versions, et ce qui est réservé pour J2 . . . 43

4 Le chemin des octets 47
4.1 Ce que le chemin doit faire . . . 47
4.2 `trc-cuda` : l'API du pilote, sans bibliothèque liée . . . 48
4.3 `trc-es` : lire sans tampon, par port de complétion . . . 49
4.3.1 Les structures, au bit près . . . 50
4.3.2 L'exclusion, démontrée . . . 50
4.4 Épingler : l'essai E0 (a) . . . 53
4.5 Le lien : l'essai E0 (b) et le moteur de copie unique . . . 54
4.6 Lire le routeur, une fois par couche : l'essai E0 (c) . . . 55
4.6.1 La boîte aux lettres, au bit près . . . 55
4.6.2 Le seuil, et le premier modèle . . . 56
4.6.3 Ce que la mesure a changé au modèle . . . 57
4.7 Le disque seul : l'étape E1 . . . 59
4.7.1 Le seuil, et ce qu'est une passe propre . . . 59
4.7.2 Une séance sous charge, une séance au calme . . . 60
4.7.3 La loi de Little, second chemin du débit . . . 60
4.8 De bout en bout : l'étape E2 . . . 61
4.8.1 Le seuil, posé avant la mesure . . . 62
4.8.2 Le transport exact, et ce que chaque contrôle voit . . . 63
4.8.3 Le débit : le verdict tombe, et le témoin dit pourquoi . . . 63
4.9 La découverte : deux régimes selon la seule longueur . . . 64

4.10 Le chemin dans le moteur . . . 67
4.11 Ce que le chapitre a barré . . . 68

5 Les noyaux sur la carte 73
5.1 Ce que la carte doit faire . . . 73
5.2 Le décodeur à table par voie (K1) . . . 74
5.2.1 Le flux, la tuile, la table . . . 74
5.2.2 Deux noyaux, un emplacement . . . 75
5.2.3 L'exactitude, démontrée . . . 75
5.2.4 La fenêtre de mots . . . 77
5.2.5 La table par voie, et le déterminisme . . . 78
5.2.6 Témoins et cassures . . . 79
5.2.7 Pourquoi un expert seul est lent : le modèle de latence . . . 79
5.2.8 Les débits, et un verdict qui tombe de 0,9 et 1,1 % . . . 80
5.3 Le produit matrice-vecteur MXFP4 (K2) . . . 82
5.3.1 L'algorithme . . . 82
5.3.2 Exacte au bit sur entiers . . . 83
5.3.3 Sur poids réels : la borne d'erreur . . . 84
5.3.4 Le toit . . . 84
5.4 La passe avant : douze noyaux (K3) . . . 84
5.4.1 Les noyaux et leurs structures . . . 84
5.4.2 Le protocole du témoin . . . 86
5.4.3 Un seul arrondi : les noyaux en f64 . . . 87
5.4.4 Les noyaux exacts . . . 88
5.4.5 L'attention, la plus petite marge . . . 89
5.4.6 Les sélections, et le déterminisme d'ensemble . . . 90
5.4.7 Résultats, noyau par noyau . . . 92
5.4.8 La chaîne de bout en bout . . . 92
5.5 La ronde : le cache KV en f32 . . . 96
5.6 Ce que le chapitre a barré . . . 97

Deuxième partie La vérité et la mémoire 99

6 La référence exacte et le juge 101
6.1 Pourquoi juger contre une vérité mathématique . . . 102
6.2 La passe avant en double précision . . . 102
6.2.1 Des poids exacts . . . 102
6.2.2 Deux chemins, une fonction . . . 103
6.2.3 Le pavage du produit, et pourquoi il ne change aucun bit . . . 103
6.2.4 La rampe de YaRN : la définition d'OpenAI et la forme de llama.cpp . . . 104
6.3 La divergence de Kullback-Leibler, et ce qu'elle mesure . . . 107
6.3.1 Définition et premières propriétés . . . 107
6.3.2 La KL d'une petite perturbation des logits . . . 108
6.3.3 Le premier choix, et ses intervalles . . . 108
6.3.4 Juger deux moteurs sur les mêmes positions . . . 108
6.4 L'instrument et ses témoins . . . 109
6.4.1 Le fichier de logits et son plancher . . . 109

6.4.2 Le défaut que le témoin d'identité a trouvé : le jugé codé ne se renormalise pas . . . 109
6.4.3 Les témoins : ce que l'instrument voit, et ce qu'il ne voit pas . . . 110
6.5 La découverte : un mélange d'experts amplifie les plus petites erreurs . . 112
6.5.1 Les nombres . . . 112
6.5.2 Le mécanisme : la bascule de routage . . . 112
6.5.3 L'expérience du routage imposé . . . 115
6.5.4 Les bascules, comptées . . . 116
6.5.5 Ce que cela fait aux critères . . . 116
6.6 Le cache KV en f16 porte les bascules, le f32 rejoint la vérité . . . 117
6.6.1 Le premier jeton de C12 . . . 117
6.6.2 La part des bascules que porte l'arrondi du cache . . . 117
6.6.3 Pourquoi le f32 doit rejoindre la vérité . . . 117
6.6.4 Mesuré . . . 118
6.7 Ce qui reste . . . 119

7 Le cache, l'ordonnanceur et le moteur 123
7.1 Le problème du cache d'experts . . . 124
7.2 L'optimum hors ligne : Belady . . . 124
7.3 LRU : pile, borne compétitive, pire cas . . . 125
7.4 Le cache de troncons, et pourquoi il égale ses simulateurs . . . 127
7.4.1 L'interface . . . 127
7.4.2 Deux règles d'éviction, un seul résultat . . . 127
7.4.3 L'étage RAM inclusif . . . 128
7.4.4 Le témoin O1 . . . 128
7.5 La falaise du LRU partagé . . . 129
7.6 La localité de gpt-oss . . . 131
7.6.1 Réutilisation : popularité et récence . . . 131
7.6.2 Les politiques . . . 132
7.7 L'ordonnanceur à la demande . . . 134
7.7.1 Le chemin d'un jeton . . . 134
7.7.2 L'ordre est sûr . . . 135
7.7.3 Le moteur calcule ce que les noyaux ont validé, au bit . . . 135
7.7.4 Le plancher en série, et ce que le moteur en fait . . . 136
7.8 Le préremplissage par flux . . . 137
7.9 Le préchargement, essayé et abandonné . . . 138
7.10 La garde d'intégrité, et ce qu'elle coûte . . . 139
7.11 Le moteur contre la base : le critère C6 . . . 140
7.12 Ce qui reste . . . 141

Troisième partie Le jugement 143

8 Résultats du jalon J1 145
8.1 Ce qui se mesure, et comment . . . 145
8.2 Les critères, d'un coup d'œil . . . 146
8.3 Exactitude : codé égale brut, au bit, sur 5 120 pas . . . 148
8.4 Mémoire : 384 Mo pour 63 Go . . . 149

8.5 Vitesse . . . 151
8.5.1 Le plancher en série (C5) . . . 151
8.5.2 La base, et l'histoire de ses chiffres . . . 152
8.5.3 C6 refait : la garde comprise . . . 153
8.5.4 Le préremplissage par flux . . . 153
8.5.5 Le coût de la garde d'intégrité . . . 154
8.5.6 Ce que J1+ a gardé, tué, laissé ouvert . . . 155
8.6 Fidélité : à quelle distance des mathématiques exactes du modèle . . . 155
8.6.1 L'instrument . . . 155
8.6.2 C4 sur les deux modèles . . . 156
8.6.3 Le cache KV en f16 portait tout l'écart . . . 156
8.7 Ce que les résultats disent ensemble . . . 158

9 L'épreuve indépendante et la ronde de corrections 161
9.1 Le protocole . . . 161
9.2 Geste 1 : la spécification est-elle vraie ? . . . 162
9.3 Les témoins tombent-ils ? . . . 163
9.4 Une troisième passe avant en double précision . . . 163
9.5 La séance de vitesse et de mémoire . . . 165
9.6 La base est-elle honnête ? . . . 167
9.7 Le texte est-il juste ? . . . 167
9.8 Les constats . . . 168
9.9 La ronde de corrections . . . 168
9.9.1 G1 : la passe figée . . . 168
9.9.2 G2 : l'orphelin qui tient les tuyaux . . . 171
9.9.3 M1 : la garde de production . . . 172
9.9.4 M2 à M5, et les mineurs . . . 173
9.9.5 Ce qui reste à l'arbitre . . . 174
9.10 Ce que l'épreuve n'a pas pu vérifier . . . 174

10 Discussion, limites et travaux futurs 177
10.1 Ce que le Tome II établit . . . 177
10.2 Les prédictions du Tome I, face aux mesures . . . 177
10.3 Les chiffres barrés . . . 178
10.4 Les idées tuées par les nombres . . . 178
10.5 Les limites . . . 178
10.6 Travaux futurs . . . 179
10.6.1 Avant tout : une série unique, et les décisions de l'arbitre . . . 180
10.6.2 Le disque D : sans BitLocker . . . 180
10.6.3 J2 : la découpe base + complément . . . 181
10.6.4 La porte compatible pour relais . . . 181
10.6.5 Le génome des poids : l'étude biomimétique de Stone . . . 182

Conclusion 185

A Protocoles de mesure du moteur 187
A.1 Ce qui est hérité du Tome I . . . 187
A.2 Les critères de fin de chaque étape . . . 187

A.3 Le seuil précède sa passe : l'heure du commit fait foi 189
A.4 Séries ABBA et palindromes 189
A.5 Le verrou de la carte et la machine partagée 190
A.6 Le témoin des témoins 191
A.7 L'épreuve indépendante 191
A.8 La ronde de corrections, bloc par bloc 191
A.9 La reproduction des constructions 192

B Outils du moteur 195
B.1 La chaîne du Tome II 195
B.2 Le programme `troncons` 196
B.3 Les outils commités 197
B.4 Les figures et leurs données 202
B.5 Les journaux 204

Bibliographie 207

Table des figures

2.1 La chaîne de construction des noyaux . . . 9
2.2 Le graphe des dépendances internes . . . 19
2.3 Lignes de code, de témoins et d'outils par membre . . . 19
2.4 Le schéma d'ensemble . . . 20
2.5 Les cinq lots et leurs coutures . . . 20
2.6 La chaîne d'exactitude . . . 21

3.1 Le fichier .trc, ses sections . . . 24
3.2 L'en-tête fixe, octet par octet . . . 26
3.3 La section MÉTA . . . 27
3.4 Une entrée de la table des résidents . . . 27
3.5 Un enregistrement de la section TABLES . . . 28
3.6 Une entrée du répertoire . . . 28
3.7 L'en-tête d'une partie et un descripteur de flux . . . 28
3.8 Un tronçon moyen et un flux CODES_CTX . . . 29
3.9 L'en-tête d'un flux rANS . . . 29
3.10 L'entrée de LUT de 32 bits . . . 30
3.11 La tuile rANS et la table par voie . . . 32
3.12 Le codeur à deux passes et sa fenêtre . . . 36
3.13 Pointe de mémoire du codeur et de la vérification . . . 38
3.14 Bits par poids par tenseur et par couche . . . 40
3.15 Taille des tronçons par couche . . . 41
3.16 Les deux chemins de C1 . . . 42

4.1 La pile du chemin des octets . . . 48
4.2 L'automate d'une requête de `trc-es` . . . 51
4.3 E0 (b) : la DMA depuis la mémoire épinglée . . . 54
4.4 Le moteur de copie unique . . . 55
4.5 La case de la boîte aux lettres d'une couche . . . 56
4.6 Le modèle D2, à l'échelle . . . 57
4.7 E0 (c) : coût d'une synchronisation, à vide et sous charge . . . 58
4.8 E1 : le lecteur sur le disque seul, deux séances . . . 60
4.9 La loi de Little contre le mur . . . 61
4.10 La fenêtre d'E2 et la vie d'un emplacement . . . 62
4.11 E2 : experts servis par passe . . . 65
4.12 E2 : les étages . . . 66
4.13 E2 : chronogramme des 40 premiers tronçons . . . 67
4.14 La découverte : le débit selon la longueur de la lecture . . . 69
4.15 Le moteur : deux couches du 120b, chaque défaut daté . . . 70
4.16 Le moteur : lectures de 12 Mio (L12) . . . 71

5.1 La tuile rANS et la table par voie . . . 74
5.2 Les bits : entrée de LUT, bloc MXFP4, code, échelle E8M0 . . . 75
5.3 L'emplacement SoA d'un expert de gpt-oss, à l'échelle . . . 76
5.4 La fenêtre de mots à trois registres . . . 77
5.5 K1 : décomposition du temps d'un expert . . . 81
5.6 K1 : débit du décodeur par passe . . . 81
5.7 K2 : écart de la GEMV sur poids réels . . . 85
5.8 K2 : débit de la GEMV MXFP4 . . . 85
5.9 La passe avant d'une couche, noyau par noyau . . . 86
5.10 Les structures passées aux noyaux . . . 86
5.11 K3 : la marge de chaque noyau à son seuil . . . 93
5.12 K3 : KL par position, routage libre contre imposé . . . 94
5.13 K3 : les décisions de routage et leurs bascules . . . 95
5.14 K3 : la cassure « fenêtre » et sa marche à 128 . . . 95
5.15 Le cache KV : f16 contre f32, 120b . . . 97

6.1 La rampe de YaRN, OpenAI contre llama.cpp . . . 106
6.2 Une entrée du fichier .kld au bit près . . . 109
6.3 Les chemins de la vérité . . . 112
6.4 La KL par position est une queue . . . 114
6.5 La bascule de routage . . . 115
6.6 L'arbre des expériences de R1 . . . 115
6.7 KL par position contre la marge du routeur . . . 116
6.8 Accord de routage par couche, moteur contre vérité . . . 118
6.9 KL contre la vérité, cache KV f16 contre f32 . . . 120

7.1 L'automate d'un emplacement d'expert . . . 124
7.2 Les deux étages du cache . . . 128
7.3 La falaise du LRU partagé . . . 130
7.4 La falaise mesurée . . . 131
7.5 La proposition vérifiée sur 17 traces . . . 132
7.6 Succès selon le budget, cinq politiques . . . 133
7.7 Les trois flux, l'hôte et le disque . . . 134
7.8 Le chronogramme mesuré d'une couche . . . 137
7.9 troncons contre llama.cpp balayé . . . 140

8.1 Mémoire de pointe contre taille du modèle . . . 149
8.2 Le plancher en série, étage par étage . . . 152
8.3 Génération : troncons contre llama.cpp . . . 154
8.4 Accord de routage avec la vérité, par couche . . . 157
8.5 KL par position contre la vérité f64 . . . 159

9.1 Les chemins de la vérité . . . 165
9.2 La séance de l'épreuve, passe par passe . . . 166
9.3 La sortie sur erreur, avant et après la ronde . . . 170
9.4 Le tourniquet des tampons d'arrivée . . . 170
9.5 La garde de production . . . 172

10.1 Les jalons et les chantiers qui viennent 179

Liste des tableaux

2.1 CUDA contre Vulkan : ce qui a décidé . . . 8
2.2 Les dépendances (couture D) . . . 9
2.3 Les membres de l'espace de travail . . . 10
2.4 Les coutures et leur histoire . . . 12
2.5 Ce que la ronde de corrections a changé . . . 17

3.1 Les sections des deux fichiers . . . 25
3.2 Les sortes de flux . . . 34
3.3 C1 : le verdict . . . 39
3.4 Les six cassures du second chemin . . . 43
3.5 Les cassures du décodeur de l'épreuve . . . 43
3.6 Les précisions du texte en attente de l'arbitre . . . 44
3.7 Ce que coûte la découpe de la v1.1 . . . 45

4.1 Une lecture de `trc-es` en mémoire . . . 50
4.2 E0 (a) : épingler de 0,25 à 4 Go . . . 53
4.3 E2 : la table des détections de la cassure . . . 64
4.4 Les hypothèses, tuées ou gardées par les nombres . . . 68

5.1 Ressources des noyaux sur Ada . . . 74
5.2 K1 : témoins et cassures . . . 80
5.3 K1 et K2 : débits contre seuils . . . 82
5.4 Les tampons de la passe avant . . . 87
5.5 K3 : les douze noyaux contre la référence f64 . . . 92
5.6 K3 : la chaîne de bout en bout . . . 93
5.7 Le cache KV en f16 contre f32 . . . 96

6.1 Les deux blocs que lit la référence . . . 103
6.2 R0 et R1 sur gpt-oss-20b . . . 111
6.3 gpt-oss amplifie les petites erreurs . . . 113
6.4 Le cache KV en f16 contre f32, contre la vérité f64 . . . 119

7.1 L'interface du cache . . . 127
7.2 Le témoin O1 . . . 129
7.3 La réutilisation d'un jeton au suivant . . . 132
7.4 Le plancher en série du 120b . . . 137

8.1 Les séances de mesure du jalon J1 . . . 146
8.2 Le jalon J1, critère par critère . . . 147
8.3 C3 et C11 : codé contre brut . . . 148
8.4 Les termes de la fenêtre qui croissent avec le modèle . . . 150
8.5 La base llama.cpp balayée . . . 153

8.6 C4 contre la vérité f64 . . . 156
8.7 Cache KV en f16 contre f32 . . . 158

9.1 Geste 1 : un troisième décodeur depuis le seul § 4 . . . 162
9.2 Les cassures du décodeur du juge . . . 163
9.3 Les cassures rejouées par le juge . . . 164
9.4 C8 par deux instruments . . . 167
9.5 Le premier jeton de C12 . . . 167
9.6 Les constats de l'épreuve, et ce que la ronde en a fait . . . 169

10.1 Les prédictions du Tome I et les mesures du Tome II . . . 178
10.2 Les chiffres barrés du Tome II . . . 179
10.3 Les idées tuées par les nombres . . . 180
10.4 Les mécanismes du vivant et leurs hypothèses mesurables . . . 182

A.1 Les critères de fin des étapes . . . 188
A.2 Les fichiers de seuils et leur date . . . 189
A.4 Les outils qui cassent les témoins . . . 191
A.5 Les blocs de la ronde de corrections . . . 192

B.1 Les options du programme troncons . . . 196
B.2 Les outils du moteur et de ses mesures . . . 197
B.4 Les figures et les fichiers qu'elles lisent . . . 202
B.6 Les journaux commités du moteur . . . 204

Liste des listings

2.1 L'interface de `trc-format` . . . 12
2.2 L'interface de `trc-codec` . . . 13
2.3 Les interfaces de `trc-cuda` et `trc-es` . . . 13
2.4 L'interface de `trc-cache` . . . 14
2.5 La configuration du moteur . . . 14

3.1 Le cœur du décodeur rANS . . . 30
3.2 La reconstruction d'une matrice v1.0 . . . 33
3.3 Le rANS vectorisé de l'épreuve . . . 41

4.1 Chargement de la table du pilote . . . 49
4.2 Soumettre, lancer, récolter . . . 50
4.3 Le routeur simulé de l'essai E0 (c) . . . 57
4.4 Le cumul brut du compteur de disque . . . 59

5.1 Le pas d'un warp . . . 76
5.2 La boucle de `corps_codes` . . . 78
5.3 L'échelle E8M0 exacte et le produit d'un bloc . . . 83
5.4 La fréquence de YaRN en f64 . . . 87
5.5 Le combiner . . . 88
5.6 L'attention de décodage . . . 89
5.7 Le routeur . . . 91

6.1 Le noyau 2 × 2 de la référence . . . 105
6.2 La rampe de YaRN dans la référence . . . 107
6.3 KL d'une entrée décodée . . . 110

7.1 Le prochain usage dans Belady . . . 126
7.2 La décision d'une couche . . . 134

9.1 Le choix d'un tampon d'arrivée au préremplissage par flux, après la ronde (`moteur/crates/trc-ordo/src/lib.rs`, extrait). . . . 171
9.2 La garde rend un tronçon faux en erreur fatale nommée (`moteur/crates/trc-ordo/src/garde.rs`, extrait). . . . 172

Notations et conventions

Les conventions sont celles du Tome I. Les unités sont décimales : 1 Go = 10^9 octets, 1 Go/s = 10^9 octets par seconde ; les multiples binaires s'écrivent Kio, Mio, Gio (une lecture de tronçon de 12 Mio fait 12×2^{20} octets). Chaque nombre porte sa provenance : [mes.] mesuré ici, sur la machine de Stone, par un outil et un journal commités ; [calc.] calculé à partir de nombres nommés ; [litt.] littérature (chiffre d'éditeur, non reproduit). Une phrase qu'une mesure plus récente a démentie est barrée à sa place (~~ainsi~~), avec ce qui l'a remplacée. Un fichier du dépôt cité comme source d'un nombre s'écrit `mesures/moteur/0/csv/` ; son chemin part de la racine du dépôt `troncons`.

Les noms du travail

Nom	Signification
troncons	le programme (un espace de travail Rust avec des noyaux CUDA, sous `moteur/`), et le projet
tronçon	les octets d'un expert (ses trois matrices et leurs biais) dans un fichier `.trc`, lus en une seule requête
emplacement	la place d'un expert décodé en mémoire vidéo (13,25 Mo) ; le cache décide quel expert y est
lots E, F, K, O, R	les cinq lots de l'équipe, chacun seul propriétaire de ses fichiers : E le chemin des octets (`trc-cuda`, `trc-es`), F le format et le codeur, K les noyaux, O le cache, l'ordonnanceur et le moteur, R la référence et le juge
coutures F, C, N, A, J, D	les interfaces gelées entre les lots avant la construction (tableau 2.4)
E0, E1, E2, F1–F3, K1–K3, R0–R2, O1–O4	les étapes de chaque lot, chacune avec son témoin, sa cassure et son seuil déclaré (`decisions/ARCHITECTURE.md` § 6.5 ; chapitre A)
J1, J1+, J2	les jalons : J1 le service exact de gpt-oss-120b dans une fenêtre bornée ; J1+ la vitesse contre la base ; J2 la découpe base + complément dans le moteur (à venir)
C1–C12	les douze critères de J1 et J1+, déclarés le 15 octobre 2026 avant la première ligne de code (chapitre 8)
G1, G2 ; M1–M5 ; m1–m6	les constats de l'épreuve indépendante : graves, moyens, mineurs (chapitre 9)
mode A0	aucun cache d'experts en RAM : tout défaut du cache de la carte est lu du disque
fichier codé / brut	le même `.trc`, flux codés par rANS, ou rangés tels quels (drapeau bit 2) ; le fichier brut est le chemin de référence de C3

Nom	Signification
la référence	la passe avant de gpt-oss en f64 sur le processeur (`trc-reference`), la « vérité » du critère C4
la base	llama.cpp b11515 (CUDA 12.4), meilleure configuration mesurée `fit -t 16` sur le 120b

Symboles

Chaque chapitre définit ses symboles là où il s'en sert. Ceux qui suivent sont partagés par plusieurs chapitres ; une lettre qui a deux sens dans le tome est donnée avec la portée de chacun.

Symbole	Signification
Le modèle (gpt-oss)	
L	nombre de couches (36 pour le 120b, 24 pour le 20b), sauf dans les passages sur le rANS des chapitres 3 à 5 et 9, où $L = 2^{16}$ est la borne basse de l'état (plus bas)
E	experts routés par couche (128 et 32)
k	experts choisis par jeton et par couche (4) ; au chapitre 3, k compte aussi les sous-pas d'un macro-pas rANS
ℓ, t	indice d'une couche, indice d'un jeton (un pas)
d	couches d'avance des tampons d'arrivée ($d = 3$, proposition 2.3) ; ailleurs une dimension, définie sur place
z, p, q	logits ; la loi du jeton suivant de la vérité (p) et d'un moteur (q)
Le format et le rANS	
M	précision des probabilités du rANS, $M = 2^{12} = 4\,096$
x, x_l	état rANS sur 32 bits, de la voie $l \in \{0, \dots, 31\}$
L (rANS)	borne basse de l'état normalisé, 2^{16} ; chaque voie finit une tuile sur L
f_s, c_s	fréquence quantifiée et cumul du symbole s ($\sum_s f_s = M$)
P, N, T, W	dans l'en-tête d'un flux : octets produits, macro-pas par tuile (256), tuiles, mots de 16 bits ; N a un autre sens au chapitre 7 (plus bas)
e	échelle E8M0 d'un bloc MXFP4 : valeur 2^{e-127} dans la spécification de l'OCP [1] ; le code et les chapitres 5 et 6 écrivent un poids $K[c]\,2^{e-128}$, où $K = \{0, \pm1, \pm2, \pm3, \pm4, \pm6, \pm8, \pm12\}$ porte les grandeurs E2M1 doublées (la même valeur) ; médiane de sa ligne et écart comme au Tome I
L'arithmétique et le juge	
u, u_{16}, u_{32}	unité d'arrondi ; du f16 (2^{-11}) et du f32 (2^{-24})
$\mathrm{KL}(p \,\|\, q)$	divergence de Kullback-Leibler $\sum_i p_i \ln(p_i/q_i)$, en nats, d'un moteur q à la vérité p, par position [2]
G_{moteurs}, G_{cfg}	KL mesurée entre deux moteurs justes, entre deux configurations d'un même moteur (étape R0)
Δ, δ	un écart (de logits, de marges) ; défini sur place
Cache, ordonnanceur, disque	

Symbole	Signification
N (cache)	capacité d'un cache, en emplacements (326 en J1 sur le 120b)
h, h_v	taux de succès du cache d'experts ; celui que le Tome I prédisait pour le cache de la carte
X, N, R	dans la loi de Little $X = N/R$ (chapitres 4 et 8) : débit, nombre moyen de requêtes en vol, temps de séjour [3]
W_i, O_i	débit de l'étage i, octets qu'un jeton lui fait traverser (Tome I) ; le toit $t \geq \max_i O_i/W_i$
B	octets de tronçons lus par pas ($\approx$ 0,71 Go sur le 120b, chapitre 2) ; au chapitre 3, blocs par ligne d'une matrice

Les renvois au Tome I

Ce tome cite le Tome I de deux façons. « Tome I, chapitre n » est un chapitre du premier mémoire. « L'étude 0n » (ou « 0n § m ») est l'une des sept études commitées d'où le Tome I a été écrit, sous `fondations/` ; ses paragraphes sont ceux du fichier Markdown. La correspondance :

Étude	Fichier	Tome I, chapitre
01	`fondations/01-machine.md`	3 La machine et sa physique
02	`fondations/02-quantification.md`	5 Quantification extrême
03	`fondations/03-entropie.md`	6 Compression sans perte
04	`fondations/04-chargement.md`	7 Chargement à la demande
05	`fondations/05-sparsite.md`	8 Sparsité et décodage spéculatif
06	`fondations/06-modeles.md`	9 Les modèles visés
07	`fondations/07-ce-qui-manque.md`	4 Formulation, 10 Architecture
décision	`decisions/ARCHITECTURE.md`	10 ; dans ce tome, chapitres 2 et 3

La règle de décision employée partout, « 06 § 7.5 », est celle de l'étude 06 : une différence n'est réelle que si elle dépasse deux fois le plus grand bruit des deux programmes et 3 % (chapitre A).

Chapitre 1

Introduction : du calcul au programme

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, le Tome I : une machine mesurée étage par étage, un toit démontré, quatre leviers mesurés, une équation du jeton, trois architectures calculées et une décision, construire d'abord l'architecture exacte dans un format qui prépare la contribution. Aucun programme ne tournait. **Ce tome établit** le programme, troncons, construit par une équipe de cinq lots ; son format, son chemin des octets, ses noyaux, sa référence, son cache et son moteur, chacun avec ses démonstrations et ses bancs ; les résultats de son premier jalon sur gpt-oss-120b ; l'épreuve qu'un juge indépendant lui a fait passer, et la ronde de corrections qui a suivi. **Il reste** la contribution de recherche (la découpe base + complément, jalon J2), le disque non chiffré, l'étude du génome des poids et la porte pour relais (chapitre 10).

1.1 La mission, et ce que Stone a demandé pour ce tome

La mission a été écrite par Stone le 15 octobre 2026, et le Tome I la cite en entier. Elle demande « une vrai solution pour prendre un modele ultra lourd quelcoque d'une ai le faire tourner proprement sans ram ni rien », puis : « je ne veux pas que des recherches déja existante je veux que tu fasse une vrai recherche de spécialiste puis tu implémente la solution fonctionnelle avec une team d'ingénieur ». Le Tome I était la recherche ; ce tome est l'implémentation. Stone a demandé qu'elle soit écrite avec la même exigence :

> *pareil tout ce que l'équipe fera calcul et tout ça on fera un mémoire comme l'autre architectuew areucture genre vraiment comme un projet de recherche c'est dailleur le cas mais vraiment calcul theorie et tout*

et il avait fixé la barre le même jour : « des shemas des explications des commentaires he veux que ce soit légendaire ce projet ». Ce tome suit donc la forme du premier : chaque chapitre dit d'où il part, ce qu'il établit et ce qui reste ; chaque structure est donnée au bit près, chaque calcul posé pas à pas, chaque mesure tracée depuis ses données commitées ; chaque chapitre se ferme sur ce qui pourrait être faux.

1.2 D'où l'on part : le Tome I

La machine. Un portable ordinaire : processeur i7-13620H, 16 Go de RAM sur un seul module, carte RTX 4060 Laptop de 8 Go, disque NVMe de 1 To chiffré par BitLocker. Le Tome I en a mesuré chaque étage avec des outils écrits pour cela, chacun avec un témoin qu'on a forcé à échouer : le disque livre 2,82 Go/s, 47 % de son annonce, parce que le volume est chiffré et passe par Intel RST VMD ; le lien hôte-carte 12,2 Go/s ; la mémoire de la carte 251,3 Go/s.

Le toit. De ces débits, le Tome I a démontré la borne d'une chaîne d'étages : le temps d'un jeton est au moins $\max_i O_i/W_i$, où W_i est le débit de l'étage i et O_i les octets qu'un jeton doit lui faire traverser. Si la carte décompresse, la compression ne devient jamais inutile ; si le processeur décompresse, elle l'est au-delà d'un facteur $r^\star = 4{,}33$.

Les leviers. La quantification extrême (un treillis à 2 bits à 90 % du chemin entre le scalaire optimal et la borne de Shannon) ; le codage sans perte (les échelles du MXFP4 codées par la médiane de leur ligne, un taux de 1,102 6 sur les experts de gpt-oss-120b, un décodeur rANS démontré exact à 82 Go/s sous CUDA) ; le chargement à la demande, où le succès du cache décide de tout ; la spéculation, qui ne paie que sous une réutilisation des experts que le routage indépendant ne donne pas.

L'équation, et la décision. Une seule équation, le temps d'un jeton d'un mélange d'experts servi par la carte, contient les quatre leviers ; ses élasticités démontrées disent que sur cette machine le disque tient le maximum, et que des noyaux plus rapides rapporteraient moins de 2 %. Trois architectures y ont été calculées : A, exacte ; B, qui sert une base sans son complément là où l'octet est cher, mécanisme neuf ; C, qui lit moins souvent. Après le débat de deux partisans, l'arbitre a décidé de construire A d'abord, en Rust avec des noyaux CUDA, dans un format qui prépare B (`decisions/ARCHITECTURE.md`). C'est là que ce tome commence.

1.3 La question de ce tome

> *Le programme fait-il ce que le calcul promettait : servir un modèle plusieurs fois plus gros que toute la mémoire de la machine, exactement, dans une fenêtre de mémoire qui ne dépend pas de sa taille, plus vite que la meilleure base mesurée ; et un juge qui ne l'a pas écrit le confirme-t-il ?*

La question se juge sur douze critères, déclarés le 15 octobre 2026 avant la première ligne de code, chacun avec un seuil dérivé d'une grandeur nommée (`decisions/ARCHITECTURE.md`, § 7). La réponse est au chapitre 8 ; l'épreuve, au chapitre 9.

1.4 Ce que ce tome apporte

1. **Un programme qui marche** : troncons fait tourner gpt-oss-120b (63,4 Go, quatre fois la RAM et huit fois la mémoire de la carte) sur la machine de Stone, avec 384 Mo de mémoire de pointe, la même à 3,7 Mo près que pour un modèle cinq fois plus petit.
2. **Un format autonome au bit près**, `.trc`, un tronçon par expert, des flux rangés par niveau de raffinement pour accueillir B sans rien coûter à A, et un troisième décodeur, écrit par le juge depuis la seule spécification, qui le redonne au bit près.
3. **L'exactitude, démontrée et mesurée** : logits identiques au bit entre le fichier codé et le fichier brut sur 5 120 pas, d'une construction et d'un ordre de calcul à l'autre.
4. **La vitesse** : 1,965 jeton par seconde garde comprise, 2,22 fois la meilleure base llama.cpp mesurée, et un préremplissage par flux sept fois plus rapide ; deux rapports antérieurs, ×3,85 puis ×2,79, barrés à leur place parce que la base était mal réglée.
5. **La justesse, contre les mathématiques exactes du modèle**, par une référence en double précision confirmée par un troisième calcul indépendant à l'arrondi près ; et la mesure que l'arrondi f16 du cache KV portait tout l'écart du moteur à la vérité.

6. **Une garde d'intégrité de production** qui rend toute corruption d'un tronçon lu en erreur fatale nommée, démontrée, pour 1,6 % de vitesse à disque égal.
7. **Une épreuve indépendante et une ronde de corrections** : deux défauts graves et cinq moyens trouvés par un juge, corrigés ou mesurés en une matinée, chacun avec un témoin qui tombe sur l'ancien binaire.

1.5 La méthode

Les règles sont celles du Tome I : chaque mesure a son témoin, et tout témoin neuf doit pouvoir échouer ; un seuil dérive d'une grandeur nommée et se déclare avant la mesure, par un commit daté ; un coût se juge en séries ABBA, avec l'écart d'un programme à lui-même comme bruit ; tout chiffre dont on dépend se refait par un second chemin ; une phrase démentie par une mesure se barre, et l'on dit ce qui a changé. Ce tome en ajoute trois. *Les coutures gelées* : les interfaces entre les lots sont fixées par écrit avant la construction, et ne changent que par une décision de l'arbitre. *L'épreuve indépendante* : un agent qui n'a écrit ni la spécification ni le code refait les chiffres avec ses propres outils. *Le second chemin des chiffres du mémoire* : les nombres des chapitres de résultats sont recalculés depuis les passes elles-mêmes par un outil commité (`memoire/outils/tome2_resultats.py`), qui s'arrête en erreur au premier écart avec un chiffre publié.

Chaque nombre porte sa provenance : [mes.] mesuré ici, par un outil et un journal commités ; [calc.] calculé à partir de nombres nommés ; [litt.] littérature, non reproduite.

1.6 Plan du tome

Le tome suit le chemin d'un octet, du programme au résultat. Chapitre 2 donne l'architecture et la structure du programme : les exigences, la langue et les noyaux, l'espace de travail, les lots et leurs coutures. Chapitre 3 donne le format `.trc` au bit près, section par section, et la reconstruction exacte d'un expert. Chapitre 4 suit le chemin des octets, du disque à la mémoire de la carte, par la lecture sans tampon et la mémoire épinglée. Chapitre 5 décrit les noyaux sur la carte, le décodeur rANS à table par voie et la passe avant de gpt-oss, avec la preuve de leur exactitude et de leur déterminisme. Chapitre 6 pose la référence exacte en double précision et la théorie du juge. Chapitre 7 décrit le cache d'experts, l'ordonnanceur et le moteur, avec la théorie du cache. Les trois derniers chapitres jugent : chapitre 8 donne les résultats du jalon J1, chapitre 9 l'épreuve indépendante et la ronde de corrections, chapitre 10 la discussion, les limites et les travaux futurs, avant la conclusion.

Première partie

Le programme

Chapitre 2

Architecture et structure du programme

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, le Tome I s'achevait sur une décision et une spécification : construire **troncons**, un moteur à nous, en Rust avec des noyaux CUDA, dont le premier jalon (J1) sert gpt-oss-120b (63,4 Go) *exactement*, experts codés sans perte, lus du disque à la taille d'un expert, dans une fenêtre de mémoire qui ne dépend pas de la taille du modèle. La spécification (`decisions/ARCHITECTURE.md`, commit `f0b0fea`) fixait les modules, leurs interfaces et cinq lots d'ingénieurs aux fichiers disjoints ; aucune ligne du programme n'existait. **Ce chapitre établit** ce qui a été construit : quatorze membres d'un espace de travail Rust (onze bibliothèques, trois exécutables, 15 655 lignes de code, 2 194 de témoins, 2 151 d'outils), vingt noyaux CUDA chargés par le seul pilote, six coutures gelées entre les lots et ce qu'elles sont devenues ; il démontre que l'égalité au bit des logits entre fichier codé et fichier brut (le critère C3) n'est pas une chance mais la conséquence de quatre contrats de couture (théorème 2.2), et il dit ce que l'épreuve indépendante a changé à l'architecture (une garde d'intégrité, une sortie sur erreur, un cache KV en f32 en option). **Il reste** les décisions de l'arbitre que la ronde de corrections lui a portées (couture N pour le cache KV en f32 ; précisions du format ; critères C7 et C8), le terme de la fenêtre qui croît avec le modèle (0,9 Mo non expliqués), et la cause première du blocage G1. Sources : `decisions/ARCHITECTURE.md` § 0 à 6 ; `moteur/Cargo.toml` et les `Cargo.toml` des membres ; `moteur/O-pour-le-memoire.md` (parties II et III) ; `moteur/E-pour-le-memoire.md` ; `moteur/K-pour-le-memoire.md` ; `mesures/epreuve-J1.md`.

En bref

Un programme qui fait tourner un modèle cinq fois plus gros que la mémoire de la machine se découpe en *qui sait quoi*. Le format sait où est chaque octet ; le chemin des octets sait le transporter du disque à la carte sans l'abîmer ; les noyaux savent calculer avec, toujours dans le même ordre ; le cache sait *où* est un expert, jamais *ce qu'il contient* ; le juge sait mesurer sans croire personne. Chaque savoir est un lot, chaque frontière une couture écrite à l'avance. C'est cette découpe qui rend l'exactitude démontrable : on ne prouve pas un programme de vingt mille lignes, on prouve que ses frontières transmettent l'égalité.

2.1 Les exigences, et la décision qui en sort

Les mots de Stone (SOCLE, 2026-10-15) demandent « une vrai solution pour prendre un modele ultra lourd quelcoque d'une ai le faire tourner proprement », et « je ne veux pas que des recherches déja existante ». L'arbitre en a tiré trois exigences qui doivent tenir ensemble (`ARCHITECTURE.md` § 1.1) :

E1, une solution qui marche : un programme qui sert un modèle bien plus gros que la mémoire, proprement : sortie juste, mémoire bornée, vitesse au-dessus de la base connue ;
E2, une contribution : un mécanisme ou un assemblage qui n'existe pas encore, prouvé par la mesure ;
E3, vite un premier programme qui marche, sans fermer la porte à E2.

L'ordre choisi (réponse par délégation du 2026-10-15, réversible) est **A exact d'abord** (J1), la qualité de la contribution B mesurée en parallèle sans moteur, puis B dans le moteur au jalon J2. Le Tome I en donne les chiffres ; ce chapitre décrit ce qu'est devenu A.

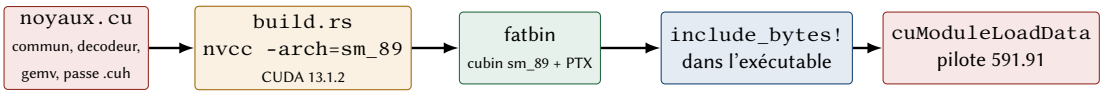
2.1.1 Pourquoi Rust et CUDA, par les mesures

Le choix de la langue n'est pas un goût. Les briques déjà mesurées du projet (codeur rANS, banc disque IOCP, lecteur GGUF) étaient en Rust ; un programme qui jongle avec des tampons alignés, des files d'entrées-sorties asynchrones et des pointeurs de carte gagne à ce que le compilateur refuse les durées de vie fausses. Le choix de CUDA contre Vulkan, lui, est tranché par des mesures (tableau 2.1) : le recouvrement du disque, du lien et du calcul, que suppose le maximum de l'équation du jeton, n'avait été mesuré sur cette machine que par des flux CUDA.

Question	Vulkan (wgpu 30)	CUDA 13.1
Décodeur rANS v4, même flux, ABBA	70,2 et 71,6 Go/s	**80,8 et 82,1 Go/s** (+14,9 %) [mes.]
GEMV 4 bits en VRAM	249 Go/s	231 à 237 Go/s (carte partagée) : à égalité [mes.]
Recouvrement copie-calcul	non mesuré	part cachée 0,98 à 1,01 [mes.]
Mémoire épinglée	tampon du pilote, 12,2 Go/s	`cuMemHostAlloc`, +21 % sur la paginable [mes.]
Lecture du disque directement dans la mémoire que lit la DMA	impossible sans import de mémoire hôte (absent de wgpu)	mémoire épinglée alignée, donnée telle quelle à `ReadFile`
Copie et calcul sur deux files	une seule file par périphérique	flux et moteur de copie

Table 2.1 : CUDA contre Vulkan, les mesures qui ont décidé de la langue des noyaux (`ARCHITECTURE.md` § 2.3, d'après les études 01 à 04 du Tome I). Le prix, dit dès la décision : le programme exige une carte NVIDIA ; les noyaux WGSL mesurés restent le second chemin, et un portage Vulkan s'ouvre après J2.

Les noyaux sont compilés *à la construction* par `nvcc` et embarqués dans l'exécutable ; à l'exécution, le pilote seul les charge (figure 2.1). La page du manuel de l'API du pilote (v13.4, mise à jour le 2026-10-12, consultée le 2026-10-16) dit que `cuModuleLoadData` accepte une image cubin ou fatbin produite par `nvcc`, ou du PTX terminé par un octet nul [4] : c'est ce qui permet de ne livrer *aucune* DLL d'exécution de CUDA. L'image contient le cubin `sm_89` (Ada, la RTX 4060 de la machine) et le PTX `compute_89`, que le pilote peut recompiler pour une carte plus récente. Pas de cuBLAS (aucune multiplication de matrices pleines en J1), pas de NVRTC.



À la construction seulement : `nvcc` est appelé directement quand `cl.exe` est dans le PATH (l'ancienne ligne `cmd /C` cassait la recette, constat mineur m2 de l'épreuve). À l'exécution : aucune DLL d'exécution de CUDA, le pilote seul charge l'image. À gauche de la case de l'exécutable, la machine de construction ; à droite, toute machine dont le pilote NVIDIA porte CUDA 13.1.

Figure 2.1 : La chaîne de construction des noyaux. Le `build.rs` de `trc-noyaux` (options `-O3 -std=c++17`, *sans* `--use_fast_math`, pour que les échelles E8M0 sous-normales restent exactes et que le binaire calcule ce que le banc a mesuré) produit un fatbin ; `include_bytes!` le place dans l'exécutable ; `trc_cuda::Module::charger` le donne au pilote. Source : `moteur/crates/trc-noyaux/build.rs`.

2.1.2 Les dépendances, et pourquoi si peu

La couture D (section 2.4) gèle les dépendances et leurs versions exactes. Le tableau 2.2 les donne avec leur licence ; toutes sont dans le cache cargo de la machine, et la construction se fait hors ligne (`--offline`). Un espace de travail Cargo partage un `Cargo.lock` et un dossier de sortie, et ses membres héritent les versions de la table `[workspace.dependencies]` par `workspace = true` [5] : une version ne s'écrit qu'à un endroit.

Crate	Version	Licence	Usage
`rayon`	1.12.0	MIT ou Apache-2.0	parallélisme du codeur, de la vérification, de la référence
`sha2`	0.10.9	MIT ou Apache-2.0	SHA-256 et empreinte64
`windows-sys`	0.61.2	MIT ou Apache-2.0	IOCP, `VirtualAlloc`, PDH, mémoire du processus
`half`	2.7.1	MIT ou Apache-2.0	f16 de la référence
`libloading`	0.8.9	ISC	chargement de `llama.dll` (découpage en jetons)
`serde`, `serde_json`	1.0.229, 1.0.151	MIT ou Apache-2.0	journaux
`anyhow`, `thiserror`	1.0.104, 2.0.21	MIT ou Apache-2.0	erreurs

Table 2.2 : Les dépendances (couture D), versions lues dans `moteur/Cargo.toml`. Aucune autre sans décision écrite ; pas de xxHash (absent du cache) : le préfixe de SHA-256, accéléré par les instructions SHA du processeur, suffit à un témoin. `llama.dll` b11515 (MIT) ne sert qu'au découpage en jetons, par son API publique, et comme base de comparaison.

2.2 L'espace de travail : quatorze membres, un graphe sans cycle

Le moteur vit sous `moteur/`, un espace de travail Cargo dont les membres sont trouvés par deux motifs, `crates/*` et `bins/*`. Une règle de construction en est sortie, écrite par le lot O en tête de `moteur/Cargo.toml` après l'avoir mesurée avec cargo 1.98.1 : Cargo refuse l'espace de travail *entier* tant qu'un dossier qui répond au motif n'a pas de `Cargo.toml` ; chaque lot écrit donc le `Cargo.toml` de sa crate avant tout autre fichier. Le lot F l'a appris à ses dépens : tant que `crates/trc-noyaux/` n'avait pas son manifeste, il a dû construire par un espace de brouillon hors du dépôt (`moteur/REPRISE-F.md`).

Le tableau 2.3 dit ce que fait chaque membre et qui le possède ; la figure 2.3 en donne la taille. Le décompte vient d'un outil commité (`memoire/outils/tome2_format.py`) qui compte les lignes des fichiers `.rs`, `.cu` et `.cuh` et les range en code, témoins (dossier `tests/`, fichier `tests.rs`) et outils (`banc/`, `examples/`, `src/bin/`) : 15 655 lignes de code, 2 194 de témoins, 2 151 d'outils, 20 000 en tout [calc.].

Membre	Lot	Rôle	Code
`trc-format`	F	structures du fichier `.trc`, lecture vérifiée, écriture, empreintes	1 303
`trc-codec`	F	rANS processeur (codeur, décodeur à contexte), reconstruction d'un expert : l'oracle	907
`trc-codeur`	F	exécutable : GGUF → `.trc`, compagnon `.vocab.gguf`, vérification C1	1 141
`trc-cuda`	E	l'API du pilote CUDA, enveloppée ; seule crate qui appelle `nvcuda.dll`	897
`trc-es`	E	lectures sans tampon par port de complétion (IOCP)	314
`banc-chemin`	E	exécutable : essais E0, banc de bout en bout E2 (disque → VRAM décodée)	2 341
`trc-noyaux`	K	vingt noyaux CUDA, leurs enveloppes, leur banc	2 219
`trc-reference`	R	passe avant de gpt-oss en f64 sur le processeur, lue dans le GGUF	1 592
`trc-jetons`	R	découpage en jetons par `llama.dll`	145
`trc-juge`	R	logits, KL, compteurs du système, séries ABBA	874
`trc-cache`	O	cache d'experts, politiques, simulateur `rejouer` (même code)	752
`trc-modele`	O	gpt-oss : hyperparamètres, tampons de carte, couches	737
`trc-ordo`	O	le moteur : ordonnanceur de couche, préremplissage, garde	1 750
`troncons`	O	exécutable : `generer`, `evaluer`, `mesurer`, `disque`	683

Table 2.3 : Les quatorze membres, leur lot (`ARCHITECTURE.md` § 6.1 : chaque lot est seul propriétaire de ses fichiers) et leurs lignes de code [calc.]. Les témoins et outils sont à la figure 2.3.

2.3 Le schéma d'ensemble

La figure 2.4 met les membres à leur place dans le temps et sur la machine. **Hors ligne**, une fois par modèle, `trc-codeur` réécrit le GGUF d'origine en un fichier `.trc` autonome (le chapitre suivant le décrit au bit près) et vérifie, expert par expert, que le décodage redonne l'original (C1). **En ligne**, à chaque jeton, `troncons` appelle le moteur de `trc-ordo`, qui demande au cache où sont les quatre experts choisis par le routeur de chaque couche, lit les absents par `trc-es`, les fait copier et décoder par `trc-cuda` et `trc-noyaux`, puis lance la couche par `trc-modele`.

Trois flux de carte et un fil d'hôte. Le moteur ordonne la carte par trois flux CUDA : *calcul* (priorité haute), *copie*, *décodage*. Ils ne se rencontrent que par des événements. L'hôte, lui, attend une fois par couche l'événement « routeur ℓ » enregistré juste après le noyau `routeur_topk`, lit les quatre indices dans une boîte aux lettres en mémoire projetée (protocole de l'essai E0 c du lot E : numéro de séquence écrit en dernier, après une barrière système), demande au cache, soumet les lectures des défauts et met en file copies et décodages. Le coût de cette synchronisation est mesuré : 18,5 µs par couche sous charge, 0,67 ms par jeton du 120b, 23 fois sous le seuil déclaré de 15,7 ms [mes.] (`moteur/E-pour-le-memoire.md`, D2 bis).

Lemme 2.1 (L'ordre des trois flux est sûr)**.** *Sous les hypothèses (H1) un flux exécute ses opérations dans l'ordre et sans recouvrement ; (H2) une attente d'événement sur un flux retient tout ce qui y est mis ensuite jusqu'à la fin de ce que l'événement a capturé ; (H3) l'hôte ne décide la couche ℓ qu'après avoir vu fini l'événement « routeur ℓ » ; (H4) un expert demandé par la couche ℓ reste épinglé jusqu'à* `liberer`*(ℓ), appelé après la mise en file de la fin de la couche, et une victime d'éviction n'est jamais épinglée ni en vol ; aucun emplacement d'expert n'est lu par une GEMV pendant qu'un décodage l'écrit, et aucun tampon d'arrivée n'est récrit avant la fin de la copie ou du décodage qui le lisait.*

Démonstration. Soit s un emplacement décodé pour la couche ℓ' et évincé par une demande de la couche ℓ, plus tardive dans le flux des couches. Par (H4), l'éviction suit `liberer`(ℓ'), donc la mise en file de la fin de la couche ℓ' sur le flux de calcul ; le nouveau décodage dans s est mis en file après la décision de la couche ℓ, donc, par (H3), après que l'hôte a vu fini « routeur ℓ », lui-même mis en file après la fin de ℓ' sur le même flux : par (H1), toutes les GEMV qui lisaient s sont finies quand le décodage commence. Réciproquement, les GEMV de la couche ℓ qui lisent s sont mises en file après l'attente de l'événement « décodé » de s (H2). Pour les tampons d'arrivée, chaque réutilisation suit la décision d'une couche, donc la fin des couches précédentes, qui ont attendu leurs décodages, qui ont attendu leurs copies ; des attentes explicites (copie après décodage précédent du même tampon, lecture après copie du même tampon épinglé) le rendent vrai même sans (H3). Démonstration complète du lot O : `moteur/O-pour-le-memoire.md` § 6.1. ■

La ronde de corrections a trouvé que ce raisonnement ne couvrait pas le *préremplissage par flux* (J1+), où plusieurs lectures d'un paquet sont en vol en même temps : le tampon de la s-ième lecture était s mod 12, et une lecture lente pouvait encore écrire son tampon quand la douzième suivante le reprenait. La règle corrigée prend le tampon libre suivant ; elle est démontrée (lemme 4 bis du lot O), et le cas ne s'est pas produit dans la passe rejouée sous la charge de l'épreuve (0 conflit) [mes.]. C'est l'une des deux causes possibles du blocage G1 ; la cause première n'est pas établie (section 2.8).

2.4 Les coutures : ce qui est gelé entre les lots

Cinq lots ont construit troncons en parallèle, chacun propriétaire de ses fichiers (figure 2.5). Ce qui les relie est écrit à l'avance, dans la spécification, et ne change que par une décision datée de l'arbitre : ce sont les *coutures* (tableau 2.4). Une couture peut s'*étendre* sans décision (une fonction ajoutée, un champ ajouté) si rien d'existant ne change ; elle ne change jamais dans un commit de code.

2.4.1 La règle de déterminisme, ou pourquoi l'égalité au bit a un sens

La couture N contient une règle qui n'est pas une interface mais une discipline : *aucun noyau n'utilise d'addition atomique en virgule flottante ; toute réduction suit un arbre fixe qui ne dépend que des tailles.* L'addition flottante n'est pas associative : $(a + b) + c$ et $a + (b + c)$ diffèrent en général d'un arrondi. Une réduction par atomiques additionne dans l'ordre où les fils arrivent, qui change d'une exécution à l'autre ; deux passes identiques rendraient des logits différents, et le critère C3 (« zéro bit différent ») n'aurait plus de sens. Avec un arbre fixe, chaque noyau est une *fonction* des octets qu'il lit. Le lot K l'a éprouvé jusqu'au détail : le noyau `combiner` emploie `__fmul_rn` et `__fadd_rn`, jamais contractés en fma ; une fma remise à la place fait tomber l'égalité avec la même somme sur l'hôte dès l'élément 649 (`moteur/K-pour-le-memoire.md`, § 7.3.6) [mes.]. Un arrondi de moins change des bits.

	Entre	Ce qui est gelé	Ce qu'elle est devenue (au 2026-10-16)
F	F → E, K, O, R	le format `.trc` au bit près	aucun octet changé ; neuf précisions du lot F et trois imprécisions relevées par l'épreuve (m4), toutes compatibles avec les fichiers écrits, en attente de l'arbitre (chapitre suivant)
C	E → K, O	l'API de `trc-cuda`	étendue sans rien changer (contexte avec mode d'attente, attributs, `Flux::fini`, `Evenement::nouveau_bloquant`, `MemoireCarte::remplir`, `HoteEpingle::lire_volatil_u32`, `copier_carte_carte`…)
N	K → O	interface binaire des noyaux, disposition des tampons, règle de déterminisme	tenue ; le cache KV en f32 (constat M5) est ajouté *hors couture*, en option (`--kv-f32`, deux noyaux neufs), sans toucher aux noyaux f16 ; la décision est à l'arbitre
A	O ↔ R	API de `trc-cache`, format des traces	inchangée ; le simulateur est le code même du cache
J	O, E → R	format des journaux de mesure	étendue (compteurs d'entrées-sorties du processus, journal d'erreur `<journal>.erreur.json`)
D	tous	dépendances et versions	inchangée (tableau 2.2)

Table 2.4 : Les six coutures et ce qu'elles sont devenues. Sources : `ARCHITECTURE.md` § 6.2 ; `moteur/REPRISE-F.md` ; `moteur/E-pour-le-memoire.md` § 2.3 ; `moteur/O-pour-le-memoire.md` § 14 ; `mesures/epreuve-J1.md`, constat m4.

2.5 Les interfaces, telles qu'elles sont

Les listings qui suivent sont des extraits du code commité, raccourcis (commentaires résumés, corps omis) ; ils montrent ce qu'un lot *promet* aux autres.

Le format (`trc-format`, lot F). Un `Fichier` s'ouvre en vérifiant tout ce qui peut l'être sans lire les tronçons : en-tête, empreintes des sections, sommes des tables, rangs du répertoire, taille du fichier. Le reste du programme ne lit jamais un octet du `.trc` sans passer par ses structures.

Listing 2.1 : L'interface de `trc-format` (extrait de `moteur/crates/trc-format/src/lib.rs`).

```
pub const MAGIE: [u8; 8] = *b"TRONCONS";
pub const ALIGNEMENT: u64 = 4096;
pub const VERSION_MAJEURE: u16 = 1;   // un lecteur 1.x lit toute 1.y, y <= 1
pub const VERSION_MINEURE: u16 = 1;
impl Fichier {
    pub fn ouvrir(chemin: &Path) -> Result<Fichier, ErreurFormat>; // tout se vérifie
    pub fn entete(&self) -> &Entete;
    pub fn residents(&self) -> &[TenseurResident];
    pub fn table(&self, couche: u16, matrice: u8, sorte: Sorte, contexte: u8) -> Option<&Table>;
    pub fn entree(&self, couche: u16, expert: u16) -> &EntreeRepertoire;
```

```
}
pub fn lire_partie(octets: &[u8]) -> Result<Partie<'_>, ErreurFormat>;
pub fn empreinte64(octets: &[u8]) -> u64;   // 8 premiers octets de SHA-256, petit-boutiste
```

Le codec (`trc-codec`, lot F). Il contient le codeur et le décodeur processeur du rANS, et surtout *la reconstruction de référence d'un expert* : l'oracle que le décodeur de la carte doit égaler octet pour octet. Le contexte d'un symbole est une fonction que le décodeur sait calculer avant de le décoder (`None` : symbole sauté).

Listing 2.2 : L'interface de `trc-codec` (extrait de `moteur/crates/trc-codec/src/lib.rs` et `rans.rs`).

```
pub fn coder(source: &[u8], geometrie: u8, contexte: &dyn Fn(usize) -> Option<u8>,
             tables: &[TableRans]) -> Vec<u8>;            // flux serialise
pub fn decoder(flux: &[u8], contexte: &mut dyn FnMut(usize, &[u8]) -> Option<u8>,
               tables: &[TableRans]) -> Result<Vec<u8>, ErreurCodec>; // voies hors de L : erreur
pub fn reconstruire(fichier: &Fichier, couche: u16, expert: u16, parties: &[&[u8]],
                    exact: bool) -> Result<ExpertDecode, ErreurCodec>;
impl ExpertDecode { pub fn octets_origine(&self) -> Vec<u8>; pub fn empreinte(&self) -> u64; }
```

Le chemin des octets (`trc-cuda`, `trc-es`, lot E). `trc-cuda` charge `nvcuda.dll` depuis System32 et en enveloppe chaque point d'entrée par son nom exporté exact (une macro `table!` nomme au chargement tout symbole absent) ; aucune autre crate n'appelle le pilote. `trc-es` lit sans tampon (`FILE_FLAG_NO_BUFFERING | FILE_FLAG_OVERLAPPED`) : décalage, longueur et adresse de destination doivent être multiples de 4 096, ce qui fixe l'alignement du format [6].

Listing 2.3 : Les interfaces de `trc-cuda` et `trc-es` (couture C ; extraits).

```
impl HoteEpingle { pub fn allouer(c: &Contexte, octets: usize, projetee: bool) -> Result<Self>; }
pub fn copier_vers_carte(dst: VueCarte, src: &HoteEpingle, decalage: usize, octets: usize,
                         f: &Flux) -> Result<()>;
impl Evenement { pub fn enregistrer(&self, f: &Flux) -> Result<()>; pub fn fini(&self) -> Result<bool>; }
impl Flux { pub fn attendre(&self, e: &Evenement) -> Result<()>; }
pub struct Requete { pub jeton: u64, pub decalage: u64, pub longueur: u32, pub destination: *mut u8 }
impl Lecteur {
    pub fn ouvrir(chemin: &Path, profondeur: usize) -> Result<Self>;
    pub fn soumettre(&self, r: Requete) -> Result<()>;   // multiples de 4 096
    pub fn recolter(&self, attente_ms: u32, sortie: &mut Vec<Achevement>) -> Result<()>;
}
```

Le cache (`trc-cache`, lot O, couture A). Le cache rend une *décision* par expert demandé : succès (l'emplacement où il est), défaut servi par la RAM tiède, ou défaut servi par le disque. Un expert demandé par la couche en cours est épinglé jusqu'à `liberer`. Le simulateur `rejouer` est le même

code, appelé sur une trace : le témoin « compte du moteur = compte du simulateur, à l'unité » est alors possible, et il a tenu sur 5 120 pas du 20b (454 210 succès, 37 310 défauts, 36 948 évictions des deux côtés ; cassure : 427 984 / 63 536 / 63 174) [mes.] (`moteur/O-pour-le-memoire.md` § 6.4).

Listing 2.4 : L'interface de `trc-cache` (couture A, ARCHITECTURE.md § 5.7).

```
pub enum Decision {
    Succes { emplacement: u32 },
    DefautRam { emplacement: u32, tiede: u32 },        // troncon code deja en
    RAM epinglee
    DefautDisque { emplacement: u32, tiede: Option<u32> } }
impl<P: Politique> Cache<P> {
    pub fn demander(&mut self, couche: u16, experts: &[u16], t: u64) -> Vec<
    Decision>;
    pub fn pret(&mut self, emplacement: u32);           // decodage termine
    pub fn liberer(&mut self, couche: u16);             // fin des GEMV :
    desepingle
}
pub fn rejouer<P: Politique>(trace: &Trace, emplacements: u32, tiedes: u32, vram:
     P, ram: P) -> Compteurs;
```

Le moteur (`trc-ordo`, lot O). Sa configuration montre l'architecture mieux qu'un long texte : chaque champ après `brut` est une extension, ajoutée sans rien changer, et presque chacune porte le nom d'une mesure ou d'un constat (listing 2.5).

Listing 2.5 : La configuration du moteur (`moteur/crates/trc-ordo/src/lib.rs`, commentaires résumés).

```
pub struct Config {
    pub fichier: PathBuf,
    pub ram_tiede_go: f64,          // 0 : mode A0, aucun cache en RAM
    pub enchassement_en_ram: bool,  // sinon une ligne de token_embd lue par
    jeton
    pub profondeur_es: usize,       // lectures confiees au systeme a la fois (4)
    pub couches_d_avance: usize,    // d : fenetre de d * k tampons d'arrivee (3)
    pub prechargement: bool,        // J1+ : mesure, tue (-11 %)
    pub contexte: u32,              // 4 096 positions en J1
    pub temoin: bool,               // chaque expert decode redescend et se
    compare
    pub brut: bool,                 // fichier brut exige (drapeau bit 2)
    // ---- extensions
    pub emplacements: Option<u32>,  // sinon : autant que le plafond de VRAM le
    permet
    pub vram_plafond_mio: u64,      // 8 188 - 909 - 100 (critere C10)
    pub longueur_lecture: Option<u32>, // 12 Mio : regime rapide du disque (lot E
    )
    pub chronogramme: bool,
    pub battement: bool,            // constat G1 : dater un blocage
    pub garde: bool,                // constat M1 : vrai par defaut
    pub casser: Option<(String, u64)>, // temoin de la garde, rien en production
    pub kv_f32: bool,               // constat M5 : hors couture N, faux par
    defaut
}
impl Moteur {
    pub fn nouveau(c: &Config) -> Result<Moteur>;
    pub fn pas(&mut self, jeton: u32, position: u32, logits: Option<&mut [f32]>)
    -> Result<u32>;
```

```
    pub fn preremplir(&mut self, jetons: &[u32], logits: Option<&mut dyn FnMut(
    usize, &[f32]) -> Result<()>>) -> Result<u32>;
}
```

Les noyaux (`trc-noyaux`, lot K, couture N). Vingt noyaux `extern "C"` : le décodeur en deux temps (`decoder_echelles` puis `decoder_codes`, et leurs versions par lot), `copier_brut`, trois GEMV (MXFP4, Q8_0, f32, plus une GEMV MXFP4 par lot), `rmsnorm`, `rope_yarn`, `ecrire_kv` et `attention` (et leurs variantes f32 de la ronde), `routeur_topk`, `swiglu_oai`, `combiner`, `ligne_q8_0`, `argmax` [calc.] (relevé dans `moteur/crates/trc-noyaux/noyaux/*.cuh`). La disposition qu'ils se transmettent, l'emplacement d'expert, est gelée au chapitre suivant (section 3.9.3).

2.6 La chaîne d'exactitude

Le critère C3 du jalon demandait que les logits du fichier codé et ceux du fichier brut, dans le même moteur, ne diffèrent d'*aucun bit* sur 5 120 pas. C'est une exigence étrange au premier regard : deux calculs en virgule flottante sur des chemins différents (un décodeur entropique d'un côté, une copie de l'autre) rendent-ils vraiment les mêmes bits ? La réponse est oui, et elle ne repose sur aucune chance : elle se démontre à partir des coutures (figure 2.6).

Théorème 2.2 (La chaîne d'exactitude)**.** *Supposons :*

(C1) pour tout expert x, la reconstruction processeur du tronçon de x égale l'expert d'origine du GGUF, octet pour octet ;
(K1) le décodeur de la carte écrit dans l'emplacement exactement les octets (codes, échelles, biais) de la reconstruction processeur, et `copier_brut` y écrit les flux bruts dans la même disposition ;
(C2) les octets d'un tronçon arrivés en VRAM sont ceux du fichier ;
(N) chaque noyau est une fonction déterministe des octets qu'il lit et de ses paramètres ;
(L) aucun emplacement n'est lu pendant qu'on l'écrit (lemme 2.1) ;
(A) le cache ne choisit que où *est un expert, jamais* quels octets *il porte.*

Alors, pour toute suite de jetons, les logits rendus par le moteur sur le fichier codé et sur le fichier brut sont égaux au bit, quels que soient le nombre d'emplacements, la politique du cache et l'ordre d'achèvement des lectures.

Démonstration. Les emplacements. Soit x un expert et $E_c(x)$, $E_b(x)$ les octets de son emplacement après décodage (passe codée) et après copie (passe brute). Par (C2), le décodeur et la copie lisent les octets du fichier. Par (K1) et (C1), $E_c(x)$ est la disposition SoA des codes, échelles et biais de l'expert d'origine ; les flux bruts du fichier brut sont, par construction du codeur (`flux_bruts`), les mêmes échelles, codes et biais, que `copier_brut` range dans la même disposition : $E_b(x) = E_c(x)$. *Les autres entrées.* Les tenseurs résidents sont recopiés tels quels dans les deux fichiers ; le cache KV ne dépend que des calculs précédents ; les paramètres des noyaux (positions, tailles) ne dépendent pas du fichier. *Récurrence.* Ordonnons les calculs par (jeton t, couche ℓ). Supposons les états (résidu, cache KV) égaux au bit dans les deux passes avant (t, ℓ). Les noyaux de l'attention et du routeur lisent des octets égaux : par (N), ils rendent les mêmes bits, donc les mêmes quatre indices et les mêmes poids. Par (A), les emplacements choisis peuvent différer d'une passe à l'autre, mais chacun porte $E(x)$ pour l'expert x demandé, et, par (L), ces octets sont ceux du décodage fini au moment de la lecture. Les GEMV des experts lisent donc des octets égaux et rendent les mêmes bits ; `combiner` les additionne dans l'ordre $i = 0, 1, 2, 3$ du routeur, qui est le même, pas dans l'ordre des emplacements. L'état après (t, ℓ) est égal au bit. La sortie (norme, GEMV de l'enchâssement de sortie, argmax) ne lit que le résidu final. ■

Le théorème dit pourquoi C3 est *attendu*, pas toléré : une seule différence de bit est un défaut, à chercher. Il dit aussi ce qu'il ne dit pas : il ne juge pas la justesse du modèle (c'est le critère C4, contre une référence f64), seulement que le codage entropique ne change rien au calcul.

Ce qui a été mesuré. Sur gpt-oss-20b et gpt-oss-120b, 10 fenêtres de 512 jetons (5 120 pas) : 0 empreinte de logits différente entre codé et brut, fichiers `.kld` égaux octet pour octet, perplexité 461,978277060145 (20b) et 422,20558522791305 (120b) des deux côtés [mes.]. Les passes n'avaient pas le même nombre d'emplacements (362 contre 361 sur le 20b, 325 contre 326 sur le 120b), donc pas la même suite de défauts : la partie « quel que soit le cache » est observée, pas seulement déduite (`moteur/O-pour-le-memoire.md` § 6.3). L'épreuve indépendante l'a refait depuis un clone propre, par le préremplissage par flux au lieu de la boucle de pas : ses `.kld` égalent ceux du lot O octet pour octet (SHA-256 `5b212fe9`…) [mes.] (`mesures/epreuve-J1.md` § 0).

2.7 La fenêtre bornée, propriété de l'architecture

« sans ram ni rien », a demandé Stone. La lecture retenue par le projet (SOCLE) : *la mémoire occupée ne dépend plus de la taille du modèle*, seulement d'une fenêtre de travail bornée. Cette propriété n'est pas une mesure heureuse : elle se lit dans la structure.

Proposition 2.3 (La mémoire de l'hôte, terme par terme)**.** *En mode A0 (aucun cache tiède), la mémoire de l'hôte du moteur vaut*

$$M = M_{\text{fixe}} + d \cdot k \cdot \ell_{\text{al}} + M_{\text{rép}},$$

où M_{fixe} ne dépend pas du modèle (contexte CUDA, image des noyaux, bibliothèques), $d \cdot k$ est le nombre de tampons d'arrivée épinglés ($d = 3$ couches d'avance, $k = 4$ experts actifs) de longueur ℓ_{al} (le plus grand tronçon arrondi à 4 096, ou une longueur de lecture fixe), et $M_{\text{rép}}$ regroupe ce qui croît comme le nombre d'experts $L \cdot E$ (répertoire, longueurs du cache, tables, trace).

Démonstration. Les poids d'experts ne séjournent dans la mémoire de l'hôte que le temps de leur copie vers la carte, dans les $d \cdot k$ tampons d'arrivée (lemme 2.1 : un tampon n'est réutilisé qu'après la copie qui le lisait) ; les emplacements décodés sont en VRAM. Les tenseurs résidents sont montés en VRAM à l'ouverture, sauf l'enchâssement d'entrée (`token_embd`, 0,615 Go), dont la ligne d'un jeton est lue du disque dans un tampon de 8 Kio. Le fichier ouvert ne garde en mémoire que ses structures : en-tête, tables, répertoire, table des résidents. Seuls le répertoire et les objets par expert croissent avec $L \cdot E$. ■

Ce qui a été mesuré, et ce que la proposition a mal compté. Pointe de l'ensemble de travail : **384,2 Mo** pour gpt-oss-120b (63,4 Go) et 380,5 Mo pour gpt-oss-20b (12,1 Go), contre 10,9 et 9,42 Go pour llama.cpp sur les mêmes modèles [mes.] (`moteur/O-pour-le-memoire.md` § 5 et § 6.6) ; la configuration la plus rapide de llama.cpp sur le 120b, `fit -t 16`, balayée par la ronde, en tient 11,4 (section 8.4). Un modèle 5,2 fois plus gros coûte 1 % de mémoire de plus. Le plafond déclaré de C7 (1,9 Go) tient, avec une marge de cinq (386,8 Mo au plus, garde comprise). Mais l'*écart* 120b − 20b, déclaré à 0,5 Mo plus deux fois le bruit, est mesuré à 3,70 Mo (lot O) et 2,94 Mo (épreuve) : **il tombe**. La proposition avait oublié des termes : la fenêtre elle-même croît avec le plus grand tronçon (12 × (12 251 136 − 12 144 640) = 1 277 952 o), les événements horodatés par défaut possible (657 à 660 o chacun), le répertoire tenu deux fois. La somme des termes nommés fait ≈ 2,05 Mo ; il reste ≈ 0,9 Mo que ces termes n'expliquent pas (granularité des petites allocations) : **le modèle à termes nommés n'est pas fermé** (`moteur/O-pour-le-memoire.md` § 13.6). La physique reste : la mémoire ne suit pas la taille du modèle ; ce qui était mal posé, c'est la tolérance.

2.8 Ce que l'épreuve indépendante a changé à l'architecture

L'épreuve (`mesures/epreuve-J1.md`, 2026-10-16) a été tenue par un juge qui n'avait écrit ni la spécification ni une ligne du code. Elle a confirmé la spécification et les critères d'exactitude ; elle a aussi trouvé deux constats graves et un moyen qui touchaient l'architecture, non le calcul. La ronde de corrections les a traités (tableau 2.5).

	Constat	Correction	Témoin et coût
G1	une passe peut se figer sans fin et sans message : sur une erreur, la destruction du moteur tournait dans `nvcuda64.dll`	l'erreur est écrite avant toute destruction (`<journal>.erreur.json`), puis le processus se termine sans détruire le contexte ; battement par couche du préremplissage ; tampons d'arrivée du flux corrigés	l'ancien binaire attend 90 s les tuyaux, le nouveau rend en 1,5 à 4,9 s ; cause première non reproduite
G2	après une erreur, un `nvidia-smi` orphelin garde les tuyaux du parent	l'enfant est tué par `Drop` et par une tâche « tuer à la fermeture » ; entrée nulle ; poignées standard non héritables	trois cas, 0 orphelin (l'ancien : un par cas)
M1	la production ne garde pas les flux bruts d'un tronçon (MEDIANES, BIAIS : 0,36 % des octets) ni la ligne d'enchâssement : une corruption passe sans erreur	une **garde** : l'empreinte64 des octets utiles de chaque tronçon lu est comparée au répertoire par quatre fils, avant de rendre un jeton ; chaque ligne lue est comparée à une table bâtie à l'ouverture	erreur nommée sur la copie corrompue (couche 0, expert 30) ; −1,6 % de jetons/s à débit de disque égal, 5,2 à 5,4 ms d'attente par jeton, +0,95 s à l'ouverture, +1,9 Mo
M5	le cache KV en f16 décide du premier jeton du texte de C12	cache KV en f32 en option (`--kv-f32`), hors couture N	troncons égal à la vérité f64 au format près sur le 120b (KL au plancher du format, 100 % de premiers choix) ; 326 → 314 emplacements

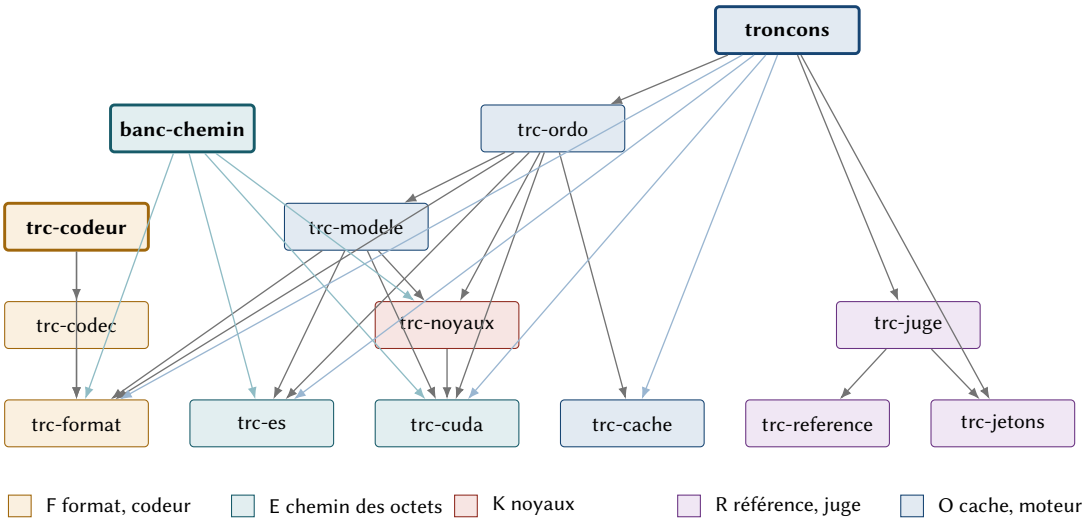
Table 2.5 : Les constats de l'épreuve qui touchaient l'architecture, et leur correction (`moteur/O-pour-le-memoire.md` partie III ; journaux sous `mesures/moteur/O/corrections/`).

La garde, une conséquence du format. La garde n'a demandé aucun octet de plus : le répertoire portait déjà l'empreinte64 des octets utiles de chaque partie (champ 32, chapitre suivant). Son coût se pose avant de se mesurer : un pas du 120b lit $B \approx 0{,}71$ Go de tronçons ; l'empreinte va à 1,07 à 1,10 Go/s par fil [mes.] (`examples/debit_empreinte.rs`), soit $\approx 0{,}65$ s de cœur par pas, étalées sur quatre fils ($\approx 0{,}17$ s de mur, sous les $\approx 0{,}5$ s d'un pas) ; au chemin critique ne reste que le hachage de la dernière lecture, au plus 12,25 Mo/1,07 Go/s $\approx$ 11,4 ms [calc.]. Mesuré : 5,2 à 5,4 ms d'attente par jeton sur les quatre passes gardées de la ronde, et −1,6 % de jetons/s à débit de disque égal [mes.] (`mesures/moteur/O/corrections/csv/vitesse-2026-10-16-passes.csv`). La note du lot O écrivait ~~5,2 à 5,3 ms~~ et ~~−1,7 %~~ : la passe 07 (5,42 ms) n'y était pas comptée, et −1,7 % venait des débits arrondis (section 8.5.5).

La vitesse contre la base, refaite. Le jalon J1+ (C6) comparait troncons à la meilleure configuration mesurée de llama.cpp. Le chiffre a changé deux fois, et chaque fois il est barré, pas retiré. Le lot O avait publié ~~×3,85~~ contre `fit -t 6` (0,568 j/s), une base sous-réglée ; l'épreuve a trouvé `-t 10` meilleure et écrit ~~×2,79 à 2,89~~ ; la ronde a balayé la base (`mesures/moteur/O/corrections/csv/base-2026-10-16-passes.csv`, onze passes, palindrome déclaré) : la meilleure est `fit -t 16`, **0,884 j/s** (0,900 et 0,867), qui bat `-t 6` de 54 % [mes.]. Contre elle, troncons *garde comprise* fait **1,965 j/s** en moyenne sur quatre passes (1,703 ; 1,900 ; 2,126 ; 2,131 ; bruit 10,97 %, `csv/vitesse-2026-10-16-passes.csv`) : × **2,22**, au-delà du seuil de la règle (2 × 10,97 % + 3 % = 24,9 %) [mes.]. C6 tient. La base peut encore être meilleure (plus de 16 fils, `--mlock`) : non balayée.

Ce qui pourrait être faux

- **Le théorème de la chaîne suppose (N) pour tous les noyaux.** Il est vérifié noyau par noyau (chaque appel fait deux fois, sorties égales au bit, lot K) et par C3 de bout en bout, pas démontré pour le compilateur : un `nvcc` qui contracterait une opération autrement d'une construction à l'autre changerait des bits. Les trois images fatbin construites avec et sans `vcvars` sont égales octet pour octet (constat m2), ce qui borne ce risque pour une version de `nvcc`, pas pour une autre.
- **La cause première de G1 n'est pas établie** : le défaut des tampons d'arrivée est démontré possible, pas observé ; l'ancien binaire, rejoué sous la même charge, a fini aussi (338 s).
- **Le modèle de la fenêtre laisse 0,9 Mo inexpliqués**, et l'écart 120b − 20b de C7 tombe tant que l'arbitre n'a pas tranché (fenêtre de taille fixe proposée : 0,99 Mo d'écart mesurés).
- **Le décompte des lignes** classe un fichier par sa place (`tests/`, `banc/`…), pas par son contenu : un module de test logé dans `src/` compte comme code.
- **Le coût de la garde** repose sur une paire de passes à disque égal ; la série déclarée dit −12,3 % sous un seuil de 26,8 % (bruit 11,9 %), non réel (section 8.5.5).
- **Le choix de CUDA** repose sur le décodeur (+14,9 %) et sur deux fonctions absentes de wgpu, pas sur une mesure du chemin complet sous Vulkan ; un Vulkan sans wgpu (avec import de mémoire hôte et une file de transfert) aurait peut-être fait aussi bien.



Bord épais : exécutable. Une flèche A → B se lit « A dépend de B » (son Cargo.toml).

Figure 2.2 : Le graphe des dépendances internes de l'espace de travail, relevé dans les `Cargo.toml` (`memoire/figures/donnees/tome2/caisses.csv`). Six feuilles sans dépendance interne, cinq niveaux, aucun cycle. Deux faits structurels s'y lisent. **Le moteur ne dépend pas du codec processeur** : `trc-ordo` et `trc-modele` n'importent pas `trc-codec` ; le décodeur de la carte (`trc-noyaux`) est un second décodeur, écrit depuis la spécification, et son égalité avec `trc-codec` est un témoin, pas une évidence. **La référence ne dépend pas du format** : `trc-reference` lit le GGUF d'origine par son propre lecteur, ce qui fait d'elle un chemin indépendant pour juger le moteur.

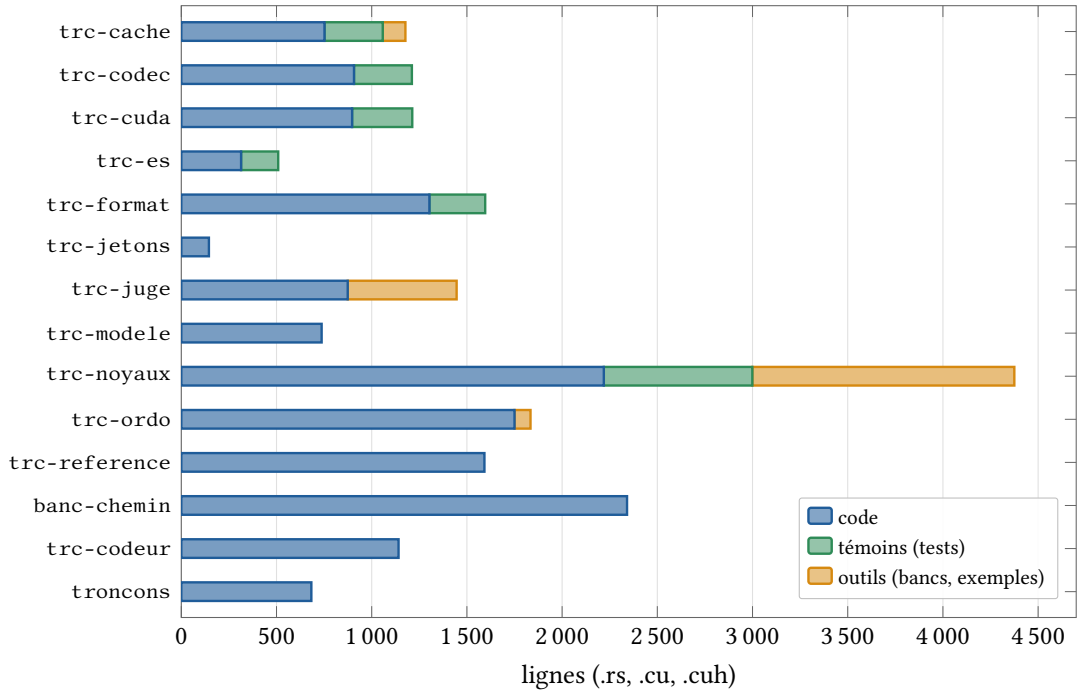


Figure 2.3 : Lignes de code, de témoins et d'outils par membre de l'espace de travail [calc.]. `trc-noyaux` est le plus gros : ses noyaux, leurs enveloppes, 780 lignes de témoins sur la carte et un banc de 1 376 lignes. Les crates sans témoin dans leur dossier (`trc-modele`, `trc-ordo`, `trc-reference`) sont jugées par des témoins extérieurs (chaîne K3, critères C3 et C4, témoin O2), commités sous `mesures/moteur/`. Données : `memoire/figures/donnees/tome2/caisses.csv`.

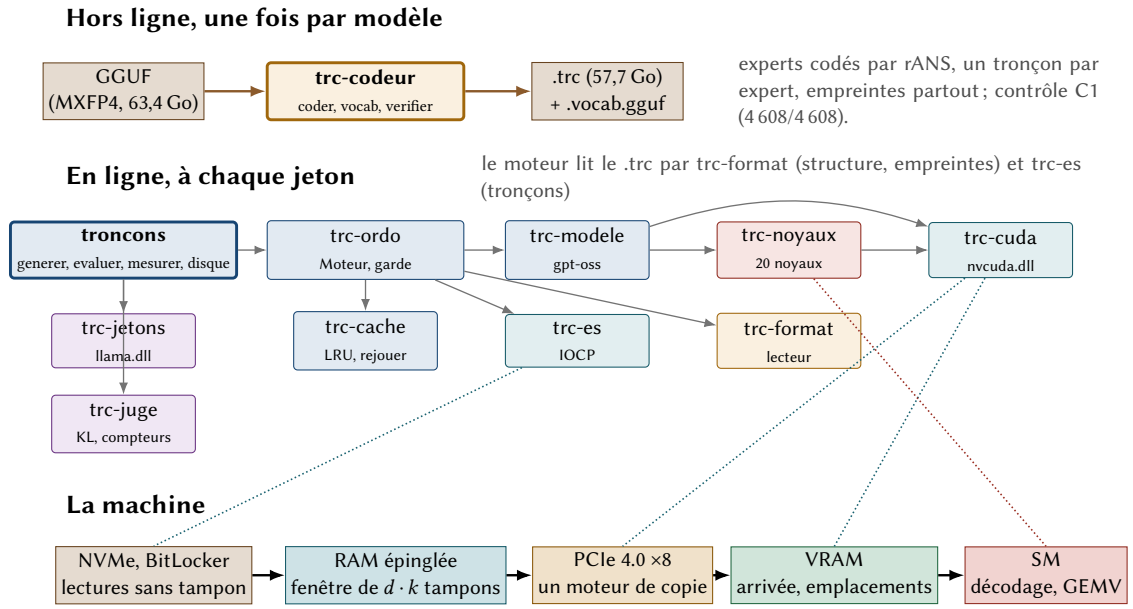
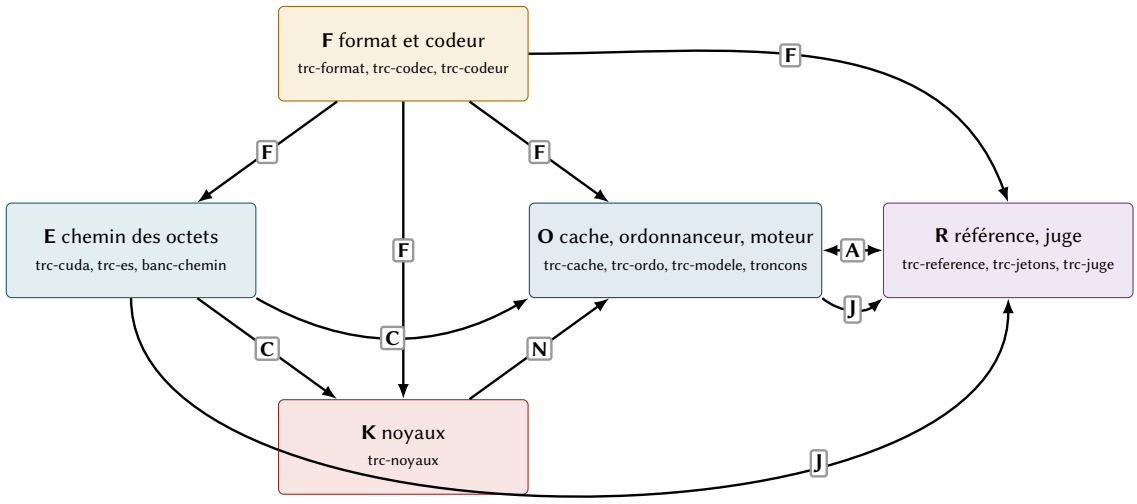


Figure 2.4 : Le schéma d'ensemble : le codeur hors ligne, le moteur en ligne, et la machine dessous (pointillés : quelle crate parle à quel organe). Un tronçon lu du disque arrive dans la RAM épinglée, que le disque écrit et que le moteur de copie de la carte lit : c'est la même page physique, sans copie par le processeur. La carte n'a qu'un moteur de copie sous WDDM (attribut ASYNC_ENGINE_COUNT = 1, relevé par le lot E [mes.]) : copies aller et retour forment une seule file.



F : le format .trc au bit près (§ 4 de la spécification). C : l'API de trc-cuda. N : l'interface binaire des noyaux, la disposition des tampons et la règle de déterminisme. A : l'API du cache et le format des traces de routage. J : le format des journaux de mesure. D, non dessinée, lie tout le monde : les dépendances et leurs versions exactes. Une flèche va du lot qui définit la couture au lot qui s'en sert.

Figure 2.5 : Les cinq lots et leurs coutures gelées (version 1, 2026-10-15 ; ARCHITECTURE.md § 6.2). Repli prévu par la spécification : à quatre ingénieurs, F et R fusionnent ; à trois, {F, R}, {E, O}, {K}, le lot K restant seul parce que la passe avant est le plus long pôle.

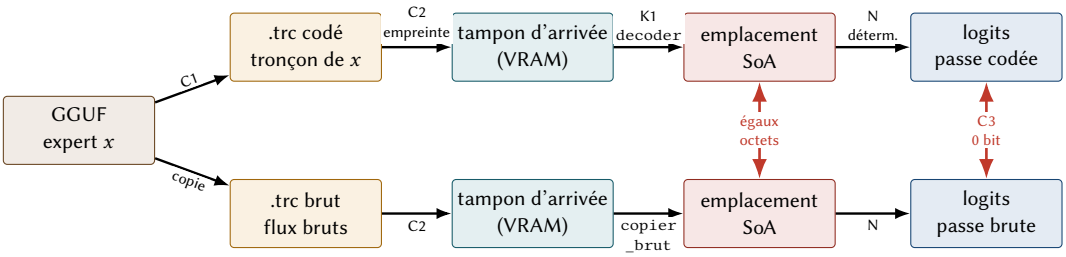


Figure 2.6 : La chaîne d'exactitude. En haut, la passe codée ; en bas, la passe brute (le même modèle écrit sans entropie, drapeau bit 2). Chaque flèche porte l'égalité qui la garde et le lot qui l'a prouvée. Les deux chemins se rejoignent à l'emplacement d'expert, qui porte les mêmes octets ; à partir de là, la règle de déterminisme rend l'égalité des logits inévitable.

Chapitre 3

Le format `.trc` au bit près

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, la spécification de l'arbitre (`decisions/ARCHITECTURE.md` § 4) décrivait au bit près un fichier qui n'existait pas : un fichier autonome par modèle, aligné sur 4 096 octets, un tronçon par expert, des flux rANS à table choisie par voie. La mesure F1 de l'arbitre en avait fixé le codage (même taux que les trois flux de classe de l'étude 03, sans réassemblage) ; personne n'avait écrit le codeur, ni un décodeur de ce format. **Ce chapitre établit** le format tel que les fichiers réels le portent : chaque champ de chaque structure, dessiné et rempli avec les valeurs lues dans `gpt-oss-120b.trc` (57 730 662 400 octets) et `gpt-oss-20b.trc` ; les lemmes qui font du rANS entrelacé un code exact, et pourquoi ses deux contrôles gratuits ne gardent pas la table ; la reconstruction exacte d'un expert, démontrée ; le codeur à mémoire bornée et ses deux propositions (mêmes octets, mémoire indépendante du modèle) ; le critère C1 tenu sur 768 et 4 608 experts par deux chemins qui ne partagent aucune ligne ; et le geste qui dit si une spécification est vraie : un juge qui n'en a écrit aucune ligne l'a décodée, depuis le seul texte, au bit près. **Il reste** la version 1.1 à deux niveaux (J2), réservée au bit près mais non écrite ; treize précisions du texte (neuf du lot F, trois du juge de l'épreuve, une du juge du Tome II), compatibles avec les fichiers écrits, que l'arbitre doit porter dans la spécification. Sources : `decisions/ARCHITECTURE.md` § 3 et 4 ; `moteur/F-pour-le-memoire.md`, `moteur/REPRISE-F.md` ; `mesures/moteur/F/derivations.md`, `verdicts.md`, `00-seuils-declares.md`, `journaux/`, `csv/` ; `mesures/epreuve-J1.md` § 2 ; `mesures/moteur/epreuve-J1/decodeur_s4.py`.

En bref

Un fichier `.trc` est un livre dont chaque page est un expert. La table des matières (le répertoire) dit où commence chaque page, combien elle pèse et quelle empreinte elle doit avoir ; chaque page se lit d'un seul geste, parce qu'elle tient d'un seul tenant sur le disque ; et chaque page est écrite dans une sténographie (le rANS) que trente-deux lecteurs déchiffrent en même temps, chacun sachant d'avance, par la ligne où il se trouve, quel alphabet utiliser. Rien ne se devine : toute structure porte sa taille, son nombre d'entrées et son empreinte, et un lecteur refuse ce qu'il ne connaît pas.

3.1 Les principes

Six principes, chacun avec sa raison (`ARCHITECTURE.md` § 4.1) :

1. **Un fichier autonome par modèle**, écrit une fois depuis le GGUF d'origine par `trc-codeur`. Il contient tout ce que le moteur lit ; un compagnon `<nom>.vocab.gguf` porte le vocabulaire pour `llama.dll`. Raison (réponse par délégation du 2026-10-15, réversible) : un instrument général a un seul artefact par modèle ; le coût est de recopier les tenseurs toujours lus (2,3 Go sur le 120b).

2. **Petit-boutiste** partout ; chaînes en UTF-8 ; tailles en octets.
3. **Alignement de 4 096 octets** pour toute section et tout ce que lit une lecture sans tampon : Windows exige alors que décalage, longueur et adresse soient multiples de la taille du secteur [6], et 4 096 couvre le NVMe de la machine.
4. **Un tronçon = un expert** (ses trois matrices et leurs biais), d'un seul tenant : un défaut coûte une requête, une complétion, une empreinte, un emplacement. Le gain de débit d'une lecture de 12 Mio contre trois de 4,2 Mio (+9 %) était sous le seuil de décision (20 %) : le choix est fait pour la simplicité, pas pour la vitesse.
5. **Niveaux de raffinement** : un tronçon est la suite de ses parties de niveau 0, 1, … ; chaque partie est alignée sur 4 096, et tout préfixe qui s'arrête à la fin d'une partie se décode seul. Le lecteur, le cache et l'ordonnanceur ne connaissent que « lire jusqu'à la fin du niveau j ».
6. **Rien ne se devine** : un lecteur refuse une version majeure, une sorte de flux ou un drapeau inconnus.

3.2 La disposition générale

Un fichier `.trc` est un en-tête fixe de 4 096 octets suivi de cinq sections, chacune alignée sur 4 096 (figure 3.1). Les décalages et longueurs du tableau 3.1 sont ceux que portent les en-têtes des deux fichiers réels, relus par le second chemin de C1 (section 3.12).

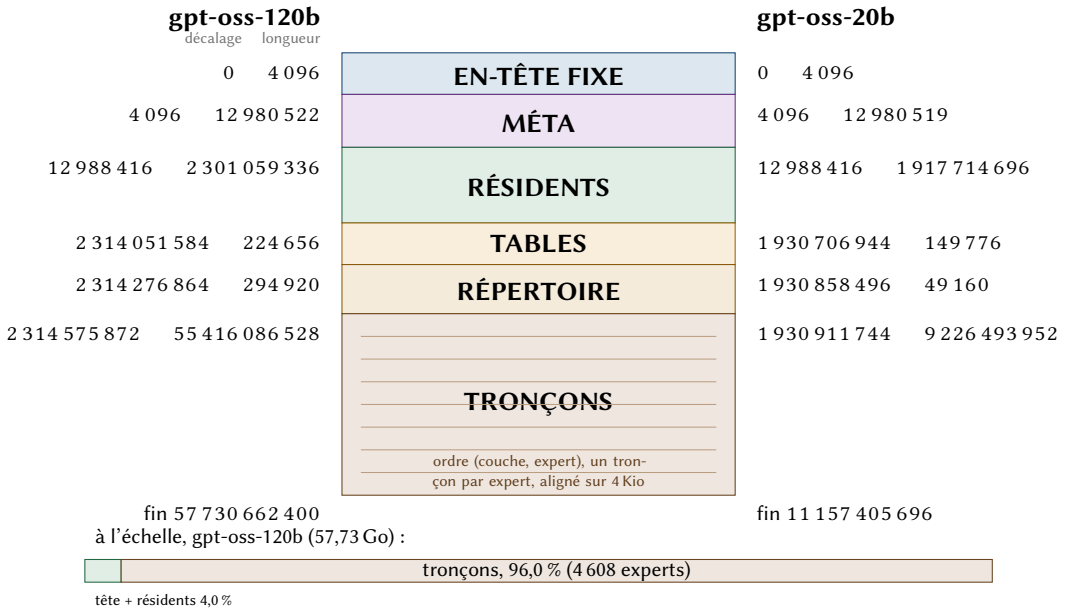


Figure 3.1 : Le fichier `.trc` et ses sections, aux décalages réels des deux fichiers (en octets ; hauteurs des cases non proportionnelles). La barre du bas est à l'échelle : sur gpt-oss-120b, 96,0 % du fichier sont des tronçons. Données : `mesures/moteur/F/journaux/second-chemin-{20b,120b}.json`, champ `entete.sections`.

3.3 L'en-tête fixe

L'en-tête tient en 4 096 octets (figure 3.2). Il dit ce qu'est le fichier (magie, version, drapeaux), d'où il vient (SHA-256 et taille du GGUF source), la géométrie du modèle (couches, experts, experts actifs, dimensions des trois matrices, format MXFP4), et où sont les sections. Ses 3 824 octets réservés doivent

	gpt-oss-120b		gpt-oss-20b	
Section	décalage	longueur	décalage	longueur
0 MÉTA	4 096	12 980 522	4 096	12 980 519
1 RÉSIDENTS	12 988 416	2 301 059 336	12 988 416	1 917 714 696
2 TABLES	2 314 051 584	224 656	1 930 706 944	149 776
3 RÉPERTOIRE	2 314 276 864	294 920	1 930 858 496	49 160
4 TRONÇONS	2 314 575 872	55 416 086 528	1 930 911 744	9 226 493 952
fin du fichier		57 730 662 400		11 157 405 696

Table 3.1 : Les sections des deux fichiers, en octets (`memoire/figures/donnees/tome2/sections.csv`). Les longueurs sont *utiles* : elles comptent l'empreinte finale de la section, pas le bourrage qui mène à la section suivante (précision du lot F, constat m4 de l'épreuve). Exemple : MÉTA du 120b finit à 4 096 + 12 980 522 = 12 984 618, et RÉSIDENTS commence au multiple de 4 096 suivant, 3 171 × 4 096 = 12 988 416. La dernière section finit exactement à la fin du fichier.

être nuls, ce que le lecteur vérifie ; ses huit derniers octets sont l'empreinte64 (section 3.10) des 4 088 premiers. Un fichier tronqué est refusé dès l'ouverture : sa taille réelle diffère du champ 24.

Les drapeaux. Bit 0 : tronçons alignés sur 4 096 (toujours 1). Bit 1 : au moins deux niveaux (v1.1). Bit 2 : fichier *brut*, dont les flux ne sont pas codés, et qui sert de chemin de référence au critère C3. Un drapeau inconnu fait refuser le fichier, même ré-empreint : c'est l'une des cassures du témoin de `trc-format` (`moteur/REPRISE-F.md`, F1).

3.4 MÉTA, RÉSIDENTS, TABLES, RÉPERTOIRE

3.4.1 MÉTA : les paires du GGUF, recopiées

La section MÉTA commence par un `u64` (le nombre de paires), puis recopie octet pour octet les paires clé-valeur du GGUF, dans leur ordre et au format GGUF v3 (longueur de la clé sur 64 bits, octets de la clé, type sur 32 bits, valeur) [7]. Le codeur ajoute trois paires : `troncons.codeur.version` (« trc-codeur 0.1.0 »), `troncons.codeur.date` (« 2026-10-15 ») et `troncons.source.nom`. Les deux fichiers ont 36 + 3 = 39 paires [mes.] (figure 3.3). La section finit par l'empreinte64 de tout ce qui précède. Ses 12,98 Mo sont presque tous le vocabulaire (201 088 jetons) ; les trois octets d'écart entre 120b et 20b viennent des chaînes qui nomment le modèle. Le moteur y lit les hyperparamètres : 64 têtes de requête, 8 têtes de clés et valeurs, dimension de tête 64, base de rotation 150 000, YaRN de facteur 32 sur un contexte d'origine de 4 096 (β rapide 32, β lent 1), fenêtre glissante de 128.

3.4.2 RÉSIDENTS : les tenseurs toujours lus

Tous les tenseurs du GGUF sauf les experts (`blk.*.ffn_{gate,up,down}_exps.{weight,bias}`) sont recopiés tels quels : 687 − 36 × 6 = 471 sur le 120b, 459 − 24 × 6 = 315 sur le 20b [calc.]. La section commence par une table (un `u32` de compte, un `u32` réservé, puis une entrée de 128 octets par tenseur, figure 3.4), des zéros jusqu'au multiple de 4 096 depuis le début de la section, puis la zone de données, où chaque tenseur est aligné sur 256 dans sa disposition ggml. L'empreinte64 de la table est rangée aux huit derniers octets de la section : sur le 120b, à 2 301 059 328 octets de son début, pour une longueur de 2 301 059 336 [mes.] (précision 2 du lot F : la spécification dit « après la zone de données » ; la fin du dernier tenseur étant alignée sur 256, les trois lectures possibles coïncident ici).

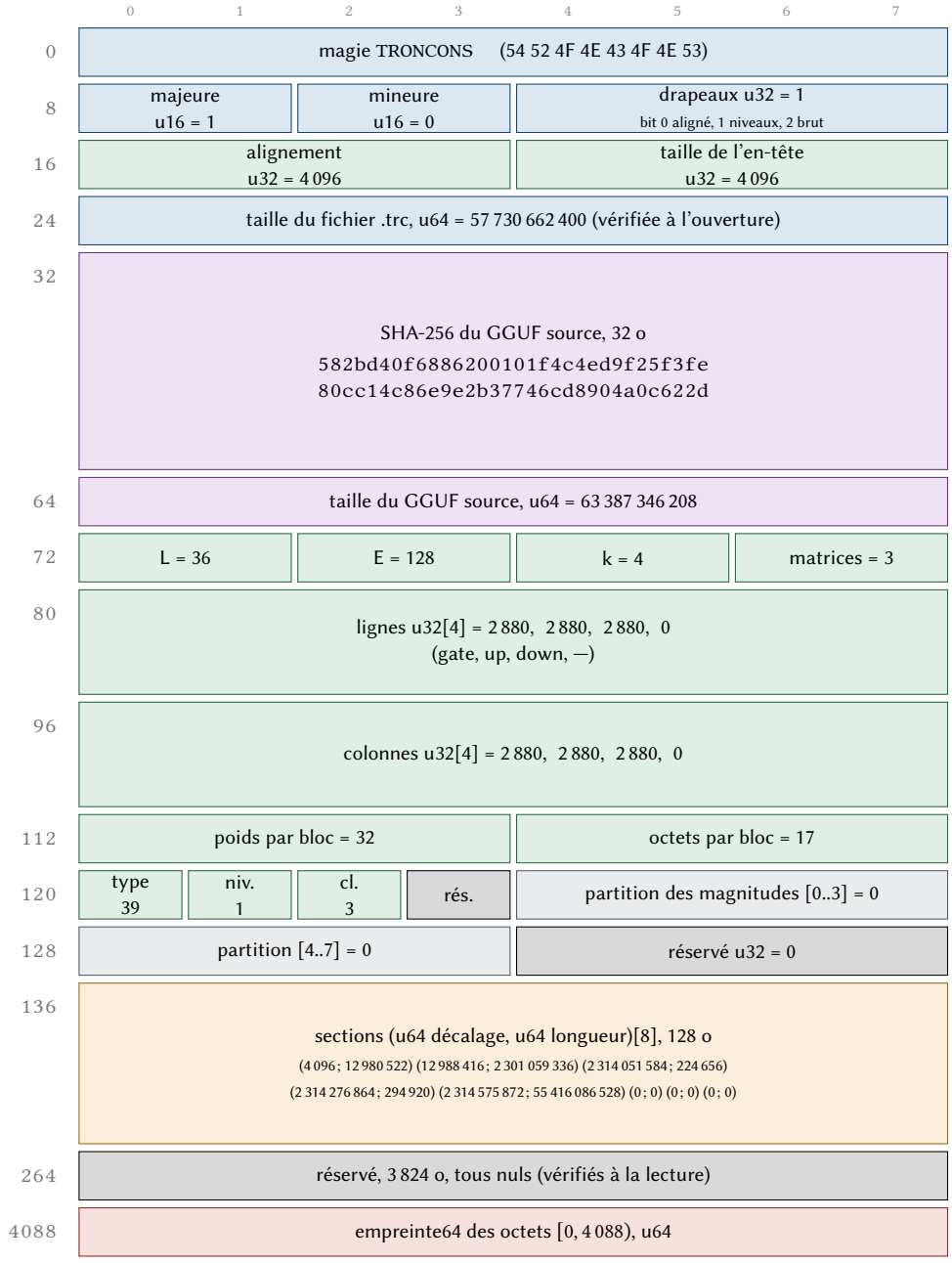


Figure 3.2 : L'en-tête fixe, huit octets par rangée (décalages à gauche), sauf deux rangées comprimées : les sections (128 octets, décalages 136 à 263, sur trois rangées) et le réservé (3 824 octets, de 264 à 4 087, sur une seule), avec les valeurs lues dans `gpt-oss-120b.trc`. Pour le 20b : taille 11 157 405 696, SHA-256 `27cd6c43…5901`, taille source 12 109 566 624, L = 24, E = 32, les mêmes dimensions. Le type 39 est MXFP4 dans la numérotation de ggml ; la partition des magnitudes ne sert qu'à la v1.1. Couleurs : identité (bleu), provenance (violet), géométrie (vert), sections (orange), réservé (gris), empreinte (rouge).

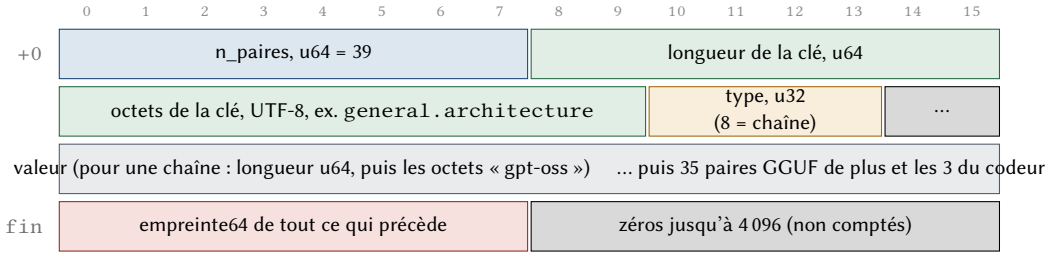


Figure 3.3 : La section MÉTA, en rangées symboliques (les clés et les valeurs ont des longueurs variables) : le compte, les paires au format GGUF v3 recopiées octet pour octet, l'empreinte64 finale, puis des zéros que la longueur de la section ne compte pas.

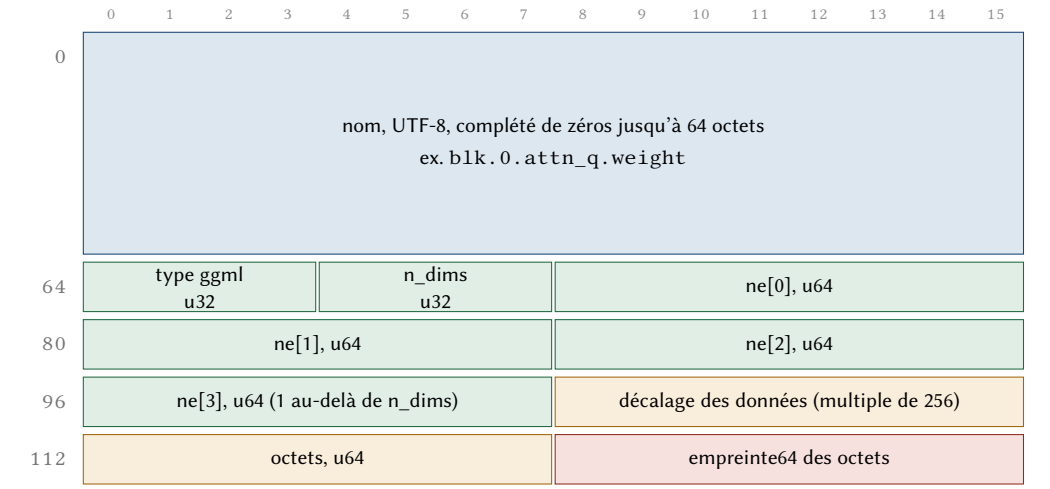


Figure 3.4 : Une entrée de la table des résidents (128 octets, seize par rangée). `ne` suit la convention de ggml : `ne[0]` est la dimension la plus rapide. Le décalage est compté depuis le début de la zone de données.

L'enchâssement d'entrée `token_embd` (Q8_0, 0,615 Go) n'est *jamais* monté en VRAM : le moteur en lit une ligne par jeton. Une ligne de 2 880 poids en Q8_0 fait 90 blocs de 34 octets, soit 3 060 octets, lus dans une lecture alignée de 8 Kio [calc.].

3.4.3 TABLES : les fréquences du rANS

Un enregistrement (figure 3.5) porte la couche, la matrice, la sorte du flux qu'il sert, le contexte, et une charge : 16 ou 256 fréquences sur 16 bits, ou 4 centres en f32 (v1.1). Une fréquence est un entier de 0 à 4 096 ; la somme d'une table vaut exactement 4 096 ; 0 marque un symbole absent. Les tables sont communes à tous les experts d'une couche, une par (couche, matrice, sorte, contexte). En v1.0, chaque matrice d'une couche a quatre tables : DELTA (contexte 0) et CODES_CTX (contextes 0, 1, 2), soit $36 \times 3 \times 4 = 432$ enregistrements de $8 + 512 = 520$ octets sur le 120b ; la section fait $8 + 432 \times 520 + 8 = 224\,656$ octets, compte et empreinte compris, exactement la longueur relevée [calc.]. Le 20b en a 288 (149 776 octets). Le moteur les monte en VRAM sous forme de LUT de 4 096 mots de 32 bits (16 Kio chacune) : $432 \times 16\,384 = 7\,077\,888$ octets (7,08 Mo, 6,75 Mio) pour le 120b [calc.]. La spécification (`decisions/ARCHITECTURE.md` § 4.6) écrit « $36 \times 3 \times 4 \times 16$ Kio = ~~6,9 Mo~~ » : le produit est juste, son écriture ne l'est ni en Mo ni en Mio ; c'est la précision T2 du tableau 3.6.

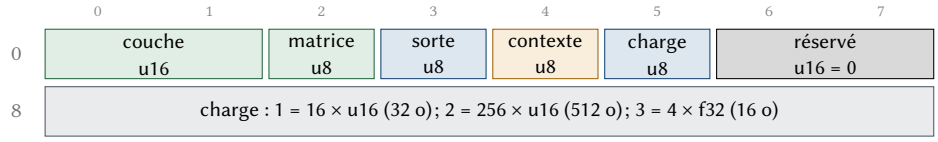


Figure 3.5 : Un enregistrement de la section TABLES : huit octets d'en-tête, puis la charge.

3.4.4 RÉPERTOIRE : la table des matières

Le répertoire a une entrée de 64 octets par (couche, expert), dans cet ordre, puis l'empreinte64 de la section : $4\,608 \times 64 + 8 = 294\,920$ octets sur le 120b, $768 \times 64 + 8 = 49\,160$ sur le 20b [calc.]. Chaque entrée (figure 3.6) dit où commence le tronçon, la longueur et les octets utiles de chacun de ses deux niveaux, et trois empreintes : celle de l'expert *d'origine*, celle des octets utiles de chaque niveau. La partie de niveau 1 suit immédiatement celle de niveau 0 : le mode exact lit les deux en une seule requête. C'est la seule structure du fichier qui croisse avec le nombre d'experts, et elle reste sous 0,3 Mo sur le 120b.

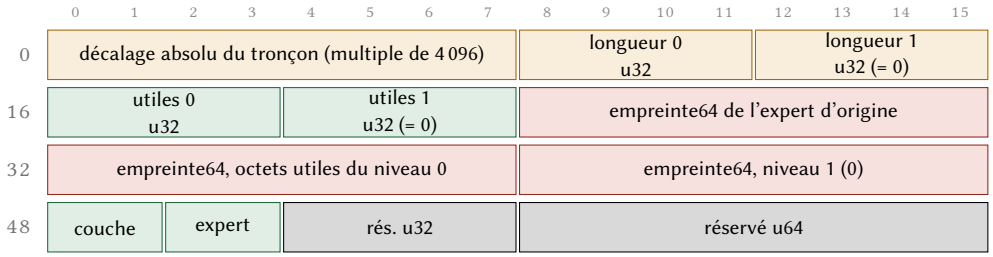


Figure 3.6 : Une entrée du répertoire (64 octets). En v1.0, les champs du niveau 1 valent zéro.

3.5 Une partie de tronçon

Une partie commence par un en-tête de 64 octets et une table de descripteurs de flux de 16 octets chacun (figure 3.7), puis les flux, chacun aligné sur 16, puis des zéros jusqu'au multiple de 4 096. En v1.0, une partie de niveau 0 porte douze flux, quatre par matrice : MEDIANES_j, BIAIS_j, DELTA_j, CODES_CTX_j pour $j = 0$ (gate), 1 (up), 2 (down). La figure 3.8 en donne les tailles moyennes, mesurées sur les 13 824 matrices du 120b.

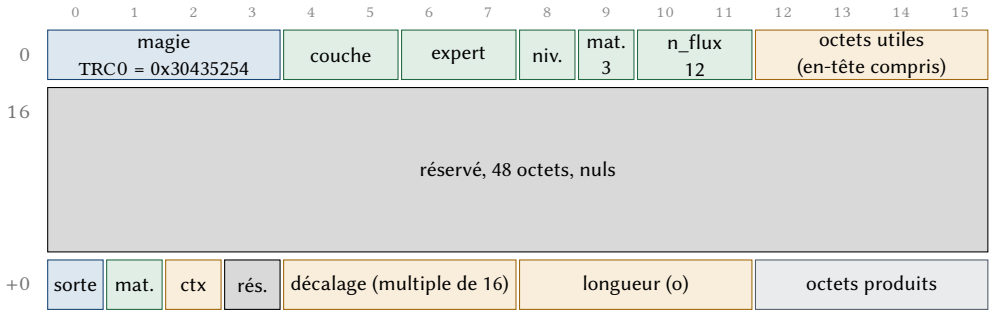
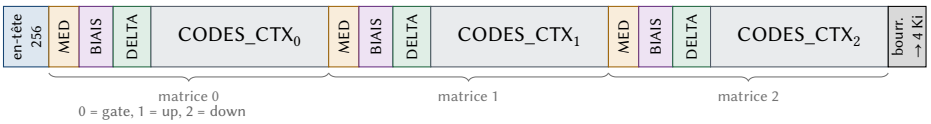


Figure 3.7 : En haut, l'en-tête d'une partie (64 octets : magie TRC0 au niveau 0, TRC1 au niveau 1). En bas, un descripteur de flux (16 octets). Le contexte fixe vaut 0 pour DELTA et 0xFF (aucun) pour les flux bruts et les flux à contexte variable (précision 4 du lot F) ; les octets produits d'un flux rANS valent son champ P, bourrage compris (précision 5).



Tailles moyennes sur les 13 824 matrices de gpt-oss-120b : MEDIANES 2 880 o (brut), BIAIS 11 520 o (brut), DELTA 32 772 o, CODES_CTX 3 960 750 o ; en-tête de partie et table des flux 64 + 12 × 16 = 256 o ; bourrage de zéros jusqu'au multiple suivant de 4 096 : 2 036 o en moyenne. Tronçon moyen : 12 026 060 o, contre 13 253 760 o d'expert MXFP4 d'origine. Largeurs des cases hors d'échelle (CODES_CTX fait 98,8 % du tronçon).

Un flux CODES_CTX, sérialisé



Pour une matrice 2 880 × 2 880 : P = 16 · 259 200 = 4 147 200 octets produits, 32 400 macro-pas, T = ⌈32 400/256⌉ = 127 tuiles ; partie fixe 16 + 4T + 128T = 16 780 o, soit 0,42 % du flux. La dernière case est le bourrage jusqu'au multiple de 16 (au plus 15 o).

Figure 3.8 : Un tronçon moyen de gpt-oss-120b, flux par flux, puis le flux CODES_CTX sérialisé. Données : `memoire/figures/donnees/tome2/flux-moyens.csv` et `calculs-format.csv`, tirés de `mesures/moteur/F/journaux/verifier-120b.json`.

Le bourrage, un contrôle de vraisemblance. Si la longueur utile d'une partie était uniformément répartie modulo 4 096, le bourrage moyen vaudrait $\frac{1}{4\,096}\sum_{i=0}^{4\,095} i = 2\,047{,}5$ octets. Mesuré : 2 036 octets sur le 120b, 2 029 sur le 20b (tronçon moyen moins octets utiles moyens) [calc.]. Le bourrage coûte donc 0,017 % du fichier, et le principe 3 se paie à ce prix.

3.6 Le flux rANS

3.6.1 La sérialisation

Un flux rANS (figure 3.9) porte en 16 octets ce qu'il faut pour le décoder : P, le nombre d'octets qu'il produit (multiple de 128) ; N, le nombre de macro-pas par tuile (256) ; la géométrie (1 : un octet par symbole, 4 sous-pas par mot de sortie ; 2 : un demi-octet, 8 sous-pas) ; T, le nombre de tuiles, $T = \lceil (P/128)/N \rceil$; W, le nombre de mots de 16 bits du flux partagé. Suivent `début[T]` (l'indice du premier mot de chaque tuile), `états[32 T]` (l'état initial de chaque voie de chaque tuile), `mots[W]`, puis des zéros jusqu'au multiple de 16.

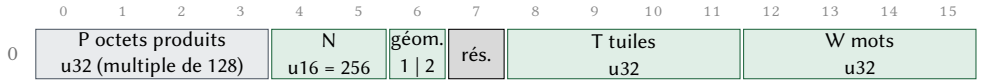


Figure 3.9 : L'en-tête d'un flux rANS (16 octets).

Le calcul, pour une matrice de gpt-oss. Une matrice de $R = 2\,880$ lignes et $C = 2\,880$ colonnes a $B = C/32 = 90$ blocs par ligne, $R \cdot B = 259\,200$ blocs. Ses codes font $16 \times 259\,200 = 4\,147\,200$ octets, soit 32 400 macro-pas de 128 octets, donc $T = \lceil 32\,400/256 \rceil = 127$ tuiles (126 pleines et une de 144 macro-pas). La partie fixe du flux vaut $16 + 4T + 128T = 16 + 508 + 16\,256 = 16\,780$ octets. Le flux CODES_CTX moyen fait 3 960 750 octets : il reste $(3\,960\,750 - 16\,780)/2 \approx 1\,971\,986$ mots (bourrage d'au plus 15 octets compris). Rapporté aux 8 294 400 poids de la matrice, le flux coûte $3\,960\,750 \times 8/8\,294\,400 = 3{,}8202$ bits par poids, dont $1\,971\,986 \times 16/8\,294\,400 = 3{,}8040$ de mots

entropiques et 0,0162 de partie fixe [calc.]. Le DELTA d'une matrice a 259 200 octets, 8 tuiles, une partie fixe de 1 072 octets, et coûte $32\,772 \times 8/8\,294\,400 = 0{,}0316$ bit par poids [calc.].

3.6.2 Le décodage normatif

Constantes : probabilités sur 12 bits ($M = 4\,096$), état x sur 32 bits, borne basse $L = 2^{16}$, mots de 16 bits. Une tuile t couvre les macro-pas tN à $\min((t+1)N, P/128) - 1$. La voie $l \in \{0, \dots, 31\}$ du macro-pas m produit le mot de sortie de 32 bits d'indice $w = 32m + l$; ses sous-pas $k = 0, 1, \dots$ en produisent, du poids faible au poids fort, l'octet k (géométrie 1) ou le demi-octet k (géométrie 2). La table d'un symbole est rangée en LUT de 4 096 entrées de 32 bits (figure 3.10) : pour chaque fente $i \in [c[s], c[s] + f[s])$, l'entrée porte le symbole s, sa fréquence et l'écart $i - c[s]$, où $c[s] = \sum_{u<s} f[u]$ est le cumul.

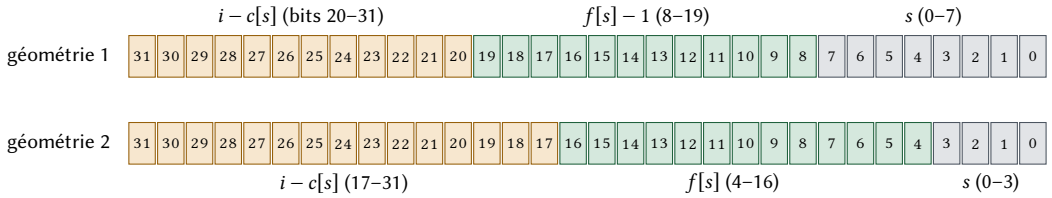


Figure 3.10 : L'entrée de LUT de 32 bits, poids fort à gauche. En géométrie 1, la fréquence est rangée moins un : $f = 4\,096$ (un symbole seul) tient alors en 12 bits (test `symbole_unique_a_frequence_pleine` du lot K).

Le décodage d'une tuile part de $p \leftarrow \texttt{début}[t]$ et $x_l \leftarrow \texttt{états}[32t+l]$, puis, pour chaque macro-pas, pour chaque sous-pas, **pour $l = 0$ à 31 dans l'ordre** : lire l'entrée e de la LUT du contexte du symbole à la fente $x_l \bmod 4\,096$, en tirer s, f et $b = i - c[s]$, poser $y = f \cdot \lfloor x_l/4\,096 \rfloor + b$; si $y < L$, lire un mot ($x_l \leftarrow 2^{16}y + \texttt{mots}[p]$, $p \leftarrow p + 1$), sinon $x_l \leftarrow y$; écrire s. Le listing 3.1 est ce cœur, tel que l'écrit `trc-codec`.

Listing 3.1 : Le cœur du décodeur normatif (`moteur/crates/trc-codec/src/rans.rs`, `decoder_avec`, raccourci). Un mot manquant se lit 0 et se compte ; aucun flux ne fait paniquer le décodeur.

```
for m in 0..nb {                          // macro-pas de la tuile
  for k in 0..spm {                       // sous-pas : 4 (geometrie 1) ou 8 (2)
    for l in 0..VOIES {                   // les 32 voies, DANS L'ORDRE
      let w = (m0 + m) * VOIES + l;       // mot de sortie de 32 bits
      let n = w * spm + k;                // indice du symbole
      let Some(c) = contexte(n, &sortie) else { continue };   // saute
      let e = luts[c as usize][(x[l] & (M - 1)) as usize];
      let (s, f, b) = (e & 0xFF, ((e >> 8) & 0xFFF) + 1, e >> 20); // geometrie 1
      let y = f * (x[l] >> 12) + b;
      if y < L { x[l] = (y << 16) | mot(p); p += 1; } else { x[l] = y; }
      sortie[n] = s as u8;
    }
  }
}
// fin de tuile : chaque voie doit valoir L, et p doit egaler debut[t + 1]
let hors = x.iter().filter(|&&v| v != L).count();
let mots_faux = deborde || p != fin;
```

Le codeur fait le chemin exactement inverse : il parcourt les symboles d'une tuile par macro-pas, sous-pas et voies *décroissants*, part de $x = L$ sur chaque voie, et, pour chaque symbole s de

fréquence f et de cumul c, émet le mot $x \bmod 2^{16}$ puis pose $x \leftarrow \lfloor x/2^{16} \rfloor$ si $x \geq f \cdot 2^{20}$, avant de coder $x \leftarrow \lfloor x/f \rfloor \cdot 4\,096 + (x \bmod f) + c$. Il renverse enfin sa pile de mots.

3.6.3 Pourquoi c'est exact

Les quatre énoncés suivants sont ceux du lot F (`mesures/moteur/F/derivations.md` § 1 à 3), écrits en entier. Notations : un symbole s a une fréquence $f \in [1, M]$ et un cumul c ; codage $C(s, x') = \lfloor x'/f \rfloor M + (x' \bmod f) + c$; décodage depuis x : fente $i = x \bmod M$, symbole s tel que $c \leq i < c + f$, puis $D(x) = f\lfloor x/M \rfloor + i - c$. Le principe de l'ANS est de Duda [8] ; l'entrelacement de plusieurs états sur un seul flux de mots, de Giesen [9].

Lemme 3.1 (Inverse exact)**.** *Pour tout $x' \geq 0$, $D(C(s, x')) = (s, x')$.*

Démonstration. Posons $q = \lfloor x'/f \rfloor$ et $r = x' \bmod f$, donc $x' = qf + r$ avec $0 \leq r < f$. Alors $C = qM + (r + c)$ avec $0 \leq r + c < c + f \leq M$: la fente vaut $i = r + c \in [c, c + f)$, donc le symbole lu est s, et $\lfloor C/M \rfloor = q$. D'où $D = fq + (r + c) - c = qf + r = x'$. ∎

Lemme 3.2 (L'intervalle des états)**.** *Avec la règle d'émission du codeur, si $x \in I = [L, 2^{32})$ avant le codage d'un symbole, alors l'état réduit x' vérifie $x' \in [f \cdot 2^4, f \cdot 2^{20})$, et $C(s, x') \in I$.*

Démonstration. Sans émission, $L \leq x < f \cdot 2^{20}$; or $f \cdot 2^4 \leq M \cdot 2^4 = 2^{16} = L \leq x$, donc $x' = x \in [f \cdot 2^4, f \cdot 2^{20})$. Avec émission, $f \cdot 2^{20} \leq x < 2^{32}$, donc $x' = \lfloor x/2^{16} \rfloor \in [f \cdot 2^4, 2^{16})$, et $2^{16} \leq f \cdot 2^{20}$ puisque $f \geq 1$. Enfin $C(s, x') \geq \lfloor x'/f \rfloor M \geq 2^4 \cdot 2^{12} = L$ et $C(s, x') < (\lfloor x'/f \rfloor + 1)M \leq 2^{20} \cdot 2^{12} = 2^{32}$. ∎

Lemme 3.3 (Le décodeur lit exactement quand le codeur a émis)**.** *Au pas qui décode un symbole, le décodeur lit un mot si et seulement si le codeur en a émis un en codant ce symbole, et une seule lecture suffit.*

Démonstration. Par le lemme 3.1, $y = D(x)$ est le x' du codeur. Si le codeur a émis, $x' < 2^{16} = L$: le décodeur lit, et $2^{16}y + w$ reconstitue exactement l'état d'avant l'émission ($w = x \bmod 2^{16}$), qui est dans I. Sinon $x' = x \geq L$ et le décodeur ne lit pas. Une seconde lecture n'est jamais nécessaire : pour $y < L$, $2^{16}y + w \geq 2^{16} = L$ dès que $y \geq 1$, et $y \geq 16f \geq 16$ puisque $x \geq L$ donne $\lfloor x/M \rfloor \geq 16$. ∎

Proposition 3.4 (Les deux contrôles gratuits d'une tuile)**.** *Sur un flux intègre, le décodeur (a) finit chaque voie sur $x = L$; (b) consomme exactement les $\mathtt{début}[t+1] - \mathtt{début}[t]$ mots de la tuile t.*

Démonstration. (a) Par récurrence sur les pas inverses, avec le lemme 3.3 : l'état du décodeur après le k-ième symbole décodé égale l'état du codeur avant le codage de ce symbole ; après le dernier symbole décodé, qui est le premier codé, c'est l'état initial du codeur, L. (b) Le décodeur lit un mot à chaque pas où le codeur en a émis un, et seulement là ; le nombre de lectures est donc celui des émissions, la taille de la pile de la tuile. L'ordre des lectures entre voies est l'ordre des émissions renversé : les 32 voies partagent une seule pile parce que l'ordre inverse du codeur, renversé par la pile, est exactement l'ordre direct du décodeur. ∎

Ce que les contrôles ne gardent pas. Ni (a) ni (b) ne garde la *table* : un décodeur qui lit une table faussée reste dans I et peut finir sur L. Le lot K l'a vu sur la carte : une fente de LUT corrompue rend une sortie fausse avec **0 voie hors de** L [mes.] (`moteur/K-pour-le-memoire.md` § 3). C'est pourquoi le format porte les empreintes des tables, vérifiées à l'ouverture, et celles des parties. Le témoin des témoins du lot F l'a vérifié sur le code : retirer le compte des voies, ou celui des mots, fait tomber un test chacun ; sa première passe avait montré qu'un test qui ne regardait que leur conjonction ne voyait pas la perte du premier (`mesures/moteur/F/casser-temoins-f1-passe1.log`). Le second

contrôle (b) n'est pas dans la spécification : le lot F et le lot K l'ont ajouté chacun de leur côté, sans changer un octet du format, et le juge l'a vérifié vrai sur 100 % des tuiles décodées (constat m4).

3.6.4 Trente-deux voies en même temps

La règle « pour $l = 0$ à 31 dans l'ordre » semble interdire de décoder les voies en parallèle. Elle ne l'interdit pas : seul l'ordre des *lectures de mots* lie les voies (figure 3.11).

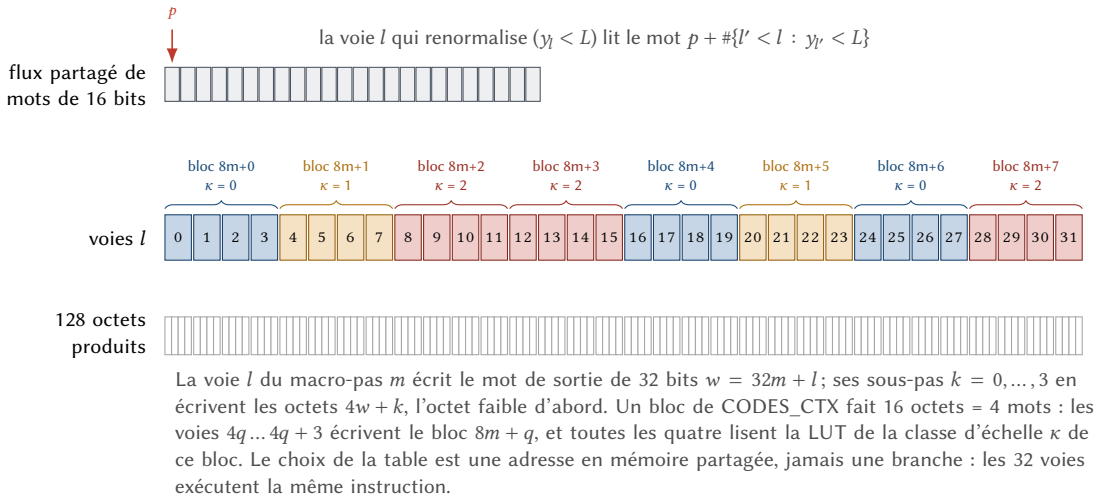


Figure 3.11 : La tuile rANS et la table par voie (figure M4 de la spécification). Exemple de classes κ pour les huit blocs d'un macro-pas.

Proposition 3.5 (Décodage parallèle des voies). *Au sous-pas (m, k), soit $v_l = [y_l < L]$ l'indicateur de renormalisation de la voie l. Un décodeur qui calcule les 32 voies en même temps et donne à la voie l le mot $p + \sum_{l'<l} v_{l'}$, puis avance p de $\sum_l v_l$, produit les mêmes symboles et les mêmes états que le décodeur normatif.*

Démonstration. Le calcul de y_l ne dépend que de x_l et de la LUT : il commute entre voies. Dans le décodeur normatif, au sous-pas (m, k), les voies qui renormalisent lisent les mots $p, p+1, \ldots$ dans l'ordre croissant des voies : la voie l lit le mot p plus le nombre de voies d'indice inférieur qui lisent aussi, soit $p + \sum_{l'<l} v_{l'}$; après les 32 voies, p a avancé de $\sum_l v_l$. Les deux décodeurs font donc les mêmes lectures ; par récurrence sur les sous-pas, états et pointeur sont égaux à chaque étape. Les tuiles, elles, sont indépendantes : chacune a son début et ses états. ■

Deux décodeurs l'emploient sans s'être lus. Sur la carte, un warp calcule v par `ballot` et le rang par `popc` du vote masqué (`moteur/K-pour-le-memoire.md` § 2.1) ; dans l'épreuve, numpy le calcule par une somme cumulée sur l'axe des voies, $p + \mathrm{cumsum}(v)_l - 1$ pour une voie qui renormalise, sur les 127 tuiles d'un flux à la fois (listing 3.3). Le format est conçu pour cela : 32 voies par tuile, c'est la largeur d'un warp.

3.7 Les contextes : la table choisie par le décodeur

Pour une matrice de R lignes et $B = C/32$ blocs par ligne, le bloc $\beta = rB + b$ a un écart d'échelle $\delta[\beta]$ et une *classe d'échelle*

$$\kappa(\beta) = \begin{cases} 0 & \text{si } \delta[\beta] < 128 \text{ (échelle sous la médiane de sa ligne),} \\ 1 & \text{si } \delta[\beta] = 128 \text{ (à la médiane),} \\ 2 & \text{si } \delta[\beta] > 128 \text{ (au-dessus).} \end{cases}$$

Pour CODES_CTX, le contexte d'un symbole est choisi **par voie** : le mot w appartient au bloc $\lfloor w/4 \rfloor$ (un bloc fait 16 octets, donc 4 mots), et sa table est celle de $\kappa(\lfloor w/4 \rfloor)$. Le décodeur décode toujours DELTA avant CODES_CTX : il connaît κ avant d'en avoir besoin.

Proposition 3.6 (Contexte connu du décodeur : exactitude et taux)**.** *Si le contexte $k(n)$ du symbole n est une fonction de données que le décodeur possède avant de décoder n, le décodage reste exact, et le nombre de bits émis vaut $\sum_n -\log_2(f_{k(n)}[s_n]/M)$ à l'en-tête et aux états finaux près.*

Démonstration. Exactitude : le lemme 3.1 vaut pour toute table ; codeur et décodeur appliquent au symbole n la même table puisqu'ils calculent le même $k(n)$, le codeur depuis la source, le décodeur depuis ce qu'il a déjà rendu, identique par récurrence. Taux : un pas de codage multiplie l'état par M/f à l'arrondi près ($x' \mapsto\approx x'M/f$), et chaque mot émis le divise par 2^{16} ; le nombre de mots vaut donc, à une constante par voie près, $\sum_n \log_2(M/f_{k(n)}[s_n])/16$. ■

Le choix de la table est une *adresse*, pas une branche : sur la carte, les 32 voies exécutent la même instruction et ne diffèrent que par la LUT qu'elles lisent en mémoire partagée. Contre les trois flux de classe de l'étude 03, la table par voie évite le réassemblage et la somme préfixe, pour le même taux : 3,8431 contre 3,8434 bits par poids sur les 12 tenseurs de F1 [mes.] (ARCHITECTURE.md § 3.3).

Les blocs de bourrage. La spécification ne disait pas la classe d'un bloc de bourrage, quand $16RB$ n'est pas multiple de 128. Le DELTA d'une matrice produit $\lceil RB/128 \rceil \cdot 128$ octets et ses codes $\lceil 16RB/128 \rceil \cdot 128$, soit $\lceil RB/8 \rceil \cdot 8$ blocs. Comme $\lceil RB/8 \rceil \cdot 8 \leq \lceil RB/128 \rceil \cdot 128$, tout bloc de bourrage des codes a un δ décodé, qui vaut 0 (bourrage), donc $\kappa = 0$: le décodeur n'a aucune règle de plus à connaître (précision 6 du lot F). Pour gpt-oss, $RB = 259\,200 = 2\,025 \times 128$: aucun bourrage.

3.8 Les sortes de flux

3.9 La reconstruction exacte d'un expert

3.9.1 Le procédé normatif

Pour chaque matrice j (R lignes, C colonnes, $B = C/32$) :

1. lire $\text{MEDIANES}_j \to \text{méd}[r]$ et $\text{BIAIS}_j \to \text{biais}[r]$;
2. décoder $\text{DELTA}_j \to \delta[\beta]$, puis $e[\beta] = (\text{méd}[r] + \delta[\beta] - 128) \bmod 256$ et $\kappa(\beta)$;
3. décoder CODES_CTX_j avec la table par voie $\kappa \to \texttt{qs}[16\beta \ldots 16\beta + 15]$;
4. écrire l'emplacement de la carte : codes, échelles, biais (section 3.9.3).

Listing 3.2 : La reconstruction d'une matrice v1.0 (`moteur/crates/trc-codec/src/lib.rs`, `decoder_matrice_v10`, raccourci).

Code	Sorte	Contenu, octets produits pour une matrice $R \times C$ (gpt-oss : $R = C = 2\,880$, $B = 90$)	Niveau
0x01	MEDIANES	brut : R octets, la médiane E8M0 de chaque ligne (`tri[B/2]` des B échelles) ; 2 880	0
0x02	BIAIS	brut : R f32 ; 11 520	0
0x03	DELTA	rANS géom. 1 : RB octets, $\delta = (e - \text{méd} + 128) \bmod 256$; 259 200	0
0x04	CODES_CTX	rANS géom. 1, table par voie : $16RB$ octets, les `qs` des blocs MXFP4 dans l'ordre β ; 4 147 200	0 (v1.0)
0x05	(réservé)	trois flux de classe réassemblés : non produit, non lu	—
0x06	CODES_BRUTS	brut : $16RB$ octets (fichier brut seulement)	0
0x07	ECHELLES_BRUTES	brut : RB octets E8M0 (fichier brut seulement)	0
0x10	BASE_CTX	rANS géom. 1, table par voie : chaque demi-octet = signe et classe de magnitude	0 (v1.1)
0x11	COMPL_CTX	rANS géom. 2, table par symbole : rang de la magnitude dans sa classe	1 (v1.1)
0x12	CENTRES	sorte de table seulement : les 4 valeurs de reconstruction de la base	—

Table 3.2 : Les sortes de flux (ARCHITECTURE.md § 4.10). Les sortes 0x10 à 0x12 sont réservées au bit près pour la v1.1 (J2) et ne sont produites par aucun fichier écrit.

```rust
let (delta, bd) = rans::decoder_une_table(&flux_d, &t.delta.lut)?;   // 2. les deltas
let kappa: Vec<u8> = delta.iter().map(|&d| classe_echelle(d)).collect();
let mut echelles = vec![0u8; n_blocs];
for (beta, e) in echelles.iter_mut().enumerate() {                   // e = med + delta - 128
    *e = med[beta / b].wrapping_add(delta[beta]).wrapping_sub(128);   // (mod 256)
}
let luts: [&[u32]; 3] = [&t.codes[0].lut, &t.codes[1].lut, &t.codes[2].lut];
let (mut codes, bc) = rans::decoder_par_voie(&flux_c, &kappa, &luts)?; // 3. table par voie
codes.truncate(16 * n_blocs);                                        // sans le bourrage
```

3.9.2 L'expert d'origine, et pourquoi il ne faut rien réordonner

L'expert d'origine, au sens des empreintes, est la suite, pour $j = 0, 1, 2$, des RB blocs MXFP4 de 17 octets $[e[\beta], \mathrm{qs}[16\beta \dots 16\beta + 15]]$ dans l'ordre β, puis, pour $j = 0, 1, 2$, des R biais f32 : $3 \times (17 \times 259\,200 + 4 \times 2\,880) = 13\,253\,760$ octets pour gpt-oss [calc.]. Un bloc MXFP4 porte 32 poids E2M1 (4 bits chacun, deux par octet) et une échelle commune E8M0 (un octet, puissance de deux) [10].

Lemme 3.7 (L'expert d'origine dans le GGUF)**.** *Dans le GGUF, le tenseur `blk.ℓ.ffn_*_exps.weight` a pour dimensions ggml* $[C, R, E]$*, rangé* $\mathtt{ne}_0 = C$ *le plus rapide. Le bloc* b *de la ligne* r *de l'expert* x *est le bloc* $(xR + r)B + b$ *du tenseur, et l'expert* x *occupe les octets* $[17xRB, 17(x + 1)RB)$*, d'un seul tenant ; ses biais (tenseur F32* $[R, E]$*) occupent* $[4xR, 4(x + 1)R)$*.*

Démonstration. Une ligne de C poids MXFP4 fait $B = C/32$ blocs de 17 octets ; une matrice d'expert, R lignes ; les experts se suivent selon la dimension la plus lente. L'indice de bloc est donc $((x \cdot R) + r) \cdot B + b$, et les RB blocs de x sont consécutifs. Même raisonnement pour les biais, à 4 octets par valeur. ∎

Le lemme fait que l'empreinte d'origine se calcule sans rien réordonner :

$$r(x) = 8 \text{ premiers octets de SHA-256}\big(\text{gate}_x \,\|\, \text{up}_x \,\|\, \text{down}_x \,\|\, \text{biais}_{\text{gate},x} \,\|\, \text{biais}_{\text{up},x} \,\|\, \text{biais}_{\text{down},x}\big).$$

Le codeur la calcule sur les tranches du GGUF ; la vérification la recalcule sur l'expert décodé *et* sur les tranches, indépendamment.

Théorème 3.8 (Reconstruction exacte)**.** *Pour tout expert x et tout fichier v1.0 écrit par le codeur depuis un GGUF, le procédé normatif appliqué à la partie de niveau 0 de x rend exactement les octets de l'expert d'origine.*

Démonstration. *Biais* : le flux BIAIS_j est la tranche de biais recopiée, et se relit telle quelle. *Échelles* : le codeur pose $\delta = (e - \text{méd} + 128) \bmod 256$, le décodeur $e' = (\text{méd} + \delta - 128) \bmod 256$; dans $\mathbb{Z}/256\mathbb{Z}$, $e' = \text{méd} + (e - \text{méd} + 128) - 128 = e$. La médiane lue est celle que le codeur a écrite (flux brut), quelle que soit sa valeur : l'identité ne dépend pas de ce qu'elle soit vraiment la médiane. *δ décodés* : DELTA_j est un flux à une table ; par les lemmes 3.1 et 3.3 et la proposition 3.4, le décodeur rend la suite codée. *Codes* : CODES_CTX_j est codé avec la table de $\kappa(\lfloor w/4 \rfloor)$, fonction des δ déjà décodés, égaux à ceux du codeur ; par la proposition 3.6, le décodage rend les `qs` codés, bourrage compris, que la troncature à $16RB$ retire. *Assemblage* : l'expert d'origine est la concaténation des blocs $[e[\beta], \texttt{qs}[16\beta \ldots]]$ dans l'ordre β puis des biais, dans l'ordre des matrices ; c'est ce que rend `ExpertDecode::octets_origine`, et c'est la disposition du GGUF par le lemme 3.7. ■

3.9.3 L'emplacement d'expert sur la carte

Le décodeur de la carte n'écrit pas l'expert d'origine mais une disposition « structure de tableaux » (SoA) que lit la GEMV (couture N, ARCHITECTURE.md § 5.6). Pour chaque matrice : les codes ($16RB$ octets, ordre β) au décalage 0 ; les échelles (RB octets) au décalage $\text{al}_{256}(16RB)$; les biais ($4R$ octets) au décalage $\text{al}_{256}(\text{al}_{256}(16RB) + RB)$; pas de matrice $\text{al}_{256}(\text{décalage des biais} + 4R)$, où $\text{al}_{256}(x)$ arrondit au multiple de 256 supérieur. Pour gpt-oss : codes 4 147 200 octets à 0 ; échelles 259 200 à 4 147 200 ; $4\,147\,200 + 259\,200 = 4\,406\,400 = 17\,212{,}5 \times 256$, donc biais à $17\,213 \times 256 = 4\,406\,528$; $4\,406\,528 + 11\,520 = 4\,418\,048 = 17\,258 \times 256$, donc pas de 4 418 048 octets et **emplacement de 13 254 144 octets** [calc.] (test `disposition_de_gpt_oss_egale_la_specification` du lot K). La valeur d'un poids est $k[\text{code}] \cdot 2^{e-128}$ avec $k \in \{0, 1, 2, 3, 4, 6, 8, 12\}$ signé par le bit 3 : les grandeurs E2M1 sont doublées, d'où l'exposant $e - 128$ au lieu de $e - 127$.

3.10 Les empreintes

Définition 3.9 (Empreinte64)**.** Pour une suite d'octets o, empreinte64(o) est l'entier petit-boutiste formé des 8 premiers octets de SHA-256(o) [11].

Le format en porte partout : l'en-tête (sur ses 4 088 premiers octets), la fin de MÉTA, de la table des RÉSIDENTS, de TABLES, de RÉPERTOIRE ; chaque tenseur résident ; chaque partie de tronçon (octets utiles) ; chaque expert d'origine. Le SHA-256 complet du GGUF source est dans l'en-tête. À l'ouverture, le moteur vérifie l'en-tête, les tables, le répertoire et la taille du fichier (quelques Mo hachés). En production, depuis la ronde de corrections, la garde compare l'empreinte de chaque partie lue à celle du répertoire (chapitre précédent).

Proposition 3.10 (Ce qu'une égalité d'empreintes exclut)**.** *Si l'empreinte64 se comporte comme une fonction aléatoire sur les erreurs considérées (une erreur qui n'est pas fabriquée contre SHA-256 : un décalage, un ordre de matrices, un bit retourné), la probabilité qu'au moins un expert faux parmi n ait*

l'empreinte de son original est au plus $n \cdot 2^{-64}$ *; pour les 4 608 experts du 120b,* $4\,608/1{,}8447 \cdot 10^{19} \approx 2{,}50 \cdot 10^{-16}$ *[calc.].*

Démonstration. Pour un couple distinct donné, la probabilité d'une même image de 64 bits vaut 2^{-64} ; la borne de l'union sur les n experts donne $n \cdot 2^{-64}$. ■

Ce n'est pas une garantie contre un adversaire : le préfixe de 64 bits se casse en environ 2^{32} essais par le paradoxe des anniversaires, si l'on choisit les deux contenus. Personne ici ne fabrique les octets du GGUF.

Ce que chaque contrôle voit. Une empreinte n'a pas la même portée selon ce qu'elle hache (`moteur/E-pour-le-memoire.md`, D14). L'empreinte de la partie voit tout bit retourné dans ses octets utiles. L'empreinte de l'expert d'origine, refaite depuis le décodé, ne voit que ce qui change la sortie : un champ que le décodeur ne lit pas (la couche ou l'expert d'un en-tête de partie, un octet réservé) ne change rien. Les compteurs du décodeur de la carte voient les fautes de format et les voies hors de L, ni les flux bruts ni l'identité de la partie. La cassure du banc E2 l'a établi sur le vrai chemin : douze bits retournés, un par sorte de région, **12 détectés sur 12**, dont 12 par l'empreinte de la partie, 11 par celle de l'expert, 9 par les compteurs ; en production (avant la garde), 9 [mes.]. Le décodeur de l'épreuve, sur la même copie, nomme qui voit quoi : 6 tronçons par les voies hors de L, 2 par une erreur de format, 2 par l'en-tête de partie (leurs octets décodés égalent pourtant le GGUF), et **2 seulement par l'empreinte de l'expert** : un octet de MEDIANES (couche 23, expert 4) et un octet de BIAIS (couche 21, expert 24), toutes voies sur L [mes.] (`mesures/epreuve-J1.md` § 2.4). C'est le constat M1, que la garde a fermé.

3.11 Le codeur à mémoire bornée

3.11.1 Pourquoi deux passes

À 15 h 10 le 2026-10-15, la machine n'avait que 336 Mo de mémoire disponible : un traceur d'un autre lot lisait gpt-oss-120b et tenait 8,7 Go privés [mes.] (`mesures/moteur/F/00-seuils-declares.md`). La première version du codeur gardait les 128 experts préparés d'une couche, soit $3 \times 128 \times 4\,406\,400 \approx 1{,}69$ Go sur le 120b [calc.] : elle aurait fait paginer la mesure d'un autre. Le lot F l'a réécrite en deux passes par paquets de $p = 2 \times$ fils experts (figure 3.12) ; le format n'a pas changé d'un octet.

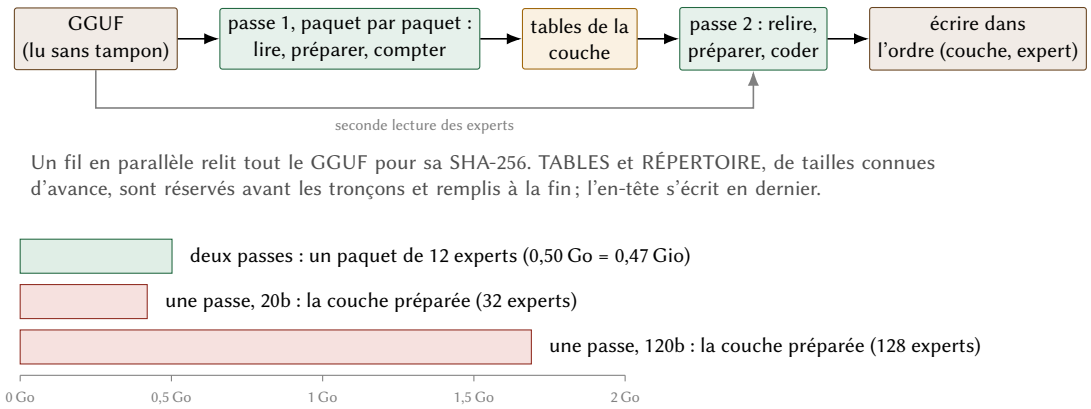


Figure 3.12 : Le codeur à deux passes d'une couche, et sa fenêtre de mémoire à l'échelle : un paquet de 12 experts (≈ 0,50 Go, soit 0,47 Gio : 502 252 544 octets ; le lot F écrivait ~~0,47 Go~~) contre la couche entière de la version à une passe (0,42 Go sur le 20b, 1,69 Go sur le 120b).

3.11.2 Mêmes octets, mémoire bornée

Notations : une couche a E experts ; pour la matrice j, l'expert x donne une préparation $P_j(x)$ (médianes, δ, `qs`, fonction de ses seuls blocs d'origine) et des comptes $N_j(x)$ (histogramme de δ, histogrammes des codes par classe κ), vecteurs d'entiers.

Proposition 3.11 (Identité des deux codeurs)**.** *Les octets écrits par le codeur à deux passes sont ceux du codeur à une passe.*

Démonstration. (1) *Comptes.* À une passe, $N_j = \sum_x N_j(x)$, sommé par un repli parallèle dans un ordre quelconque ; à deux passes, $N_j = \sum_{\text{paquets}} \sum_{x \in \text{paquet}} N_j(x)$. L'addition d'entiers (u64, sans débordement : au plus $E \cdot RB \cdot 16 \approx 5{,}3 \cdot 10^8$ par case pour le 120b) est associative et commutative : les deux N_j sont égaux. (2) *Tables* : une fonction déterministe de N_j (règle d'arrondi de l'étude 03) : égales. (3) *Tronçons* : un tronçon est une fonction de $(P_0(x), P_1(x), P_2(x)$, biais de x, tables de la couche$)$; $P_j(x)$ se recalcule à la seconde passe depuis les mêmes octets (lemme 3.7) : mêmes tronçons. (4) *Ordre* : (couche, expert) dans les deux cas, résultats d'un paquet recueillis dans l'ordre de x : mêmes décalages, même répertoire, mêmes empreintes, même en-tête. L'empreinte d'origine, calculée à une passe par un hacheur nourri matrice après matrice et à deux passes d'un seul appel sur la suite, porte sur la même suite d'octets ; SHA-256 ne dépend pas du découpage en appels. ■

Le témoin. Le 20b recodé par la version à deux passes doit avoir la SHA-256 du fichier de F2, calculée par `Get-FileHash` (un chemin indépendant de notre code) : `e8bd6ae1…169f` des deux côtés ; la copie au bit retourné donne `c40f1163…7123` : l'instrument tombe [mes.] (`journaux/identite-20b-2passes.txt`).

Proposition 3.12 (Borne de mémoire)**.** *Avec p experts par paquet, la pointe de la boucle des experts est bornée par*

$$\underbrace{p \sum_j 17 R_j B_j}_{\text{blocs lus}} + \underbrace{p \sum_j 17 R_j B_j}_{\text{préparations}} + \underbrace{p\, T_{\max}}_{\text{tronçons assemblés}} + \underbrace{3 \cdot 4RE}_{\text{biais de la couche}} + \underbrace{2 \times 16\,\text{Mio}}_{\text{rebond, morceau}},$$

où seul le terme des biais dépend de E.

Démonstration. Un paquet est lu, préparé (au plus une préparation en vol par expert du paquet, $(16+1)R_jB_j$ octets par matrice), codé et écrit avant que le suivant soit lu ; les tronçons d'un paquet sont tenus jusqu'à leur écriture ; les biais de la couche sont lus une fois ; la lecture sans tampon passe par un rebond aligné, et la recopie des résidents par morceaux de 16 Mio. Rien d'autre ne dépend de la taille d'un tenseur. ■

Le calcul, pour gpt-oss et $p = 12$. $\sum_j 17 R_j B_j = 3 \times 17 \times 259\,200 = 13\,219\,200$ octets. Blocs lus et préparations : $2 \times 12 \times 13\,219\,200 = 317\,260\,800$; tronçons : $12 \times 12\,251\,136 = 147\,013\,632$; biais : $34\,560\,E$, soit 1 105 920 (20b) et 4 423 680 (120b) ; rebond et morceau : 33 554 432. Total sur le 120b : 502 252 544 octets [calc.]. Le lot F écrivait ~~≈ 0,47 Go~~ : c'est 0,47 *Gio* ; en octets, la borne vaut 0,50 Go (correction du rédacteur, 2026-10-16 : unité). Le terme qui dépend du modèle ne pèse que 3,3 Mo d'écart entre les deux. La version à une passe gardait en plus $3E \times 4\,406\,400$ octets de préparations : 0,42 Go (20b), 1,69 Go (120b). D'où le seuil, déclaré avant le lancement de F3 : pointe 120b ≤ 1,25× pointe 20b, la marge de 25 % couvrant le bruit de l'ensemble de travail (allocateur, fils), déclarée et non mesurée.

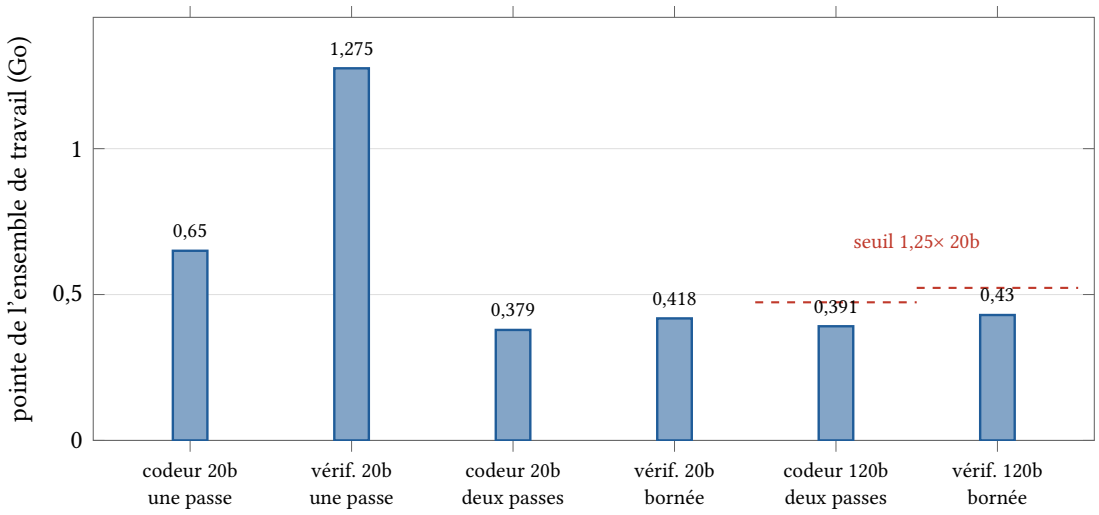


Figure 3.13 : Pointe de l'ensemble de travail du codeur et de la vérification, une passe contre deux passes (relevée toutes les 0,5 s par le lanceur). Le 120b, cinq fois plus gros, se code et se vérifie dans la même fenêtre que le 20b : rapports 1,033 (codeur) et 1,028 (vérification), sous le seuil de 1,25 [mes.]. Données : `mesures/moteur/F/csv/pointes-memoire.csv`.

Le prix, et une unité fausse deux fois. Les experts sont lus deux fois : 2×61,0 Go sur le 120b au lieu de 61,0 Go. Au débit du disque en lectures aléatoires de 4 Mio en profondeur 4 (2,58 Go/s, étude 01 ; ~~débit séquentiel~~ : le séquentiel en blocs de 4 Mio y vaut 1,04 à 1,63 Go/s), la seconde lecture coûte 61,0/2,58 = 23,6 secondes [calc.]. Le lot F avait écrit ~~≈ 24 minutes de plus~~ dans ses dérivations, et l'a barré lui-même (« l'unité était fausse ») ; le commentaire de tête de `moteur/bins/trc-codeur/src/coder.rs` dit encore « ~~de l'ordre de 25 minutes de disque de plus~~ » : c'est la même erreur, relevée par le rédacteur le 2026-10-16 et signalée au lot F (le fichier n'est pas à lui). Le codage du 120b a duré 1 153 s en tout : la seconde lecture y pèse environ 2 % (23,6/1 153 = 2,05 % [calc.]), le rANS du processeur fait le reste.

> **Ce que le témoin n'avait pas couvert : l'erreur 87**
>
> La première version à deux passes calculait la SHA-256 du GGUF par morceaux de 16 Mio lus sans tampon, et relisait après la lecture courte de fin de fichier, à un décalage qui n'était plus aligné : Windows refuse (erreur 87, paramètre incorrect). Le défaut a été vu sur le 20b, corrigé (une lecture courte n'arrive qu'en fin de fichier ; l'appelant compare le total à la taille), et une quatrième cassure du témoin de la lecture sans tampon le couvre désormais : quatre cassures sur quatre tombent, le test restauré tient [mes.] (`journaux/casser-lecture.log`).

3.12 C1 sur 768 et 4 608 experts

3.12.1 Le premier chemin : cinq contrôles par expert

La vérification (`trc-codeur verifier`) ne fait pas confiance au codeur : elle relit le `.trc` par `trc-format` (toutes empreintes), décode chaque expert par `trc-codec`, et compare aux tranches lues dans le GGUF. Cinq contrôles par expert : (1) l'empreinte des octets utiles de la partie égale celle du répertoire ; (2) le décodage réussit (voies sur L, mots consommés) ; (3) les octets d'origine reconstruits égalent, octet pour octet, les tranches du GGUF ; (4) l'empreinte du décodé égale l'empreinte d'origine du répertoire ; (5) l'empreinte des tranches du GGUF l'égale aussi.

Dans chaque cassure, les contrôles « partie », « octets » et « décodé » tombent ; « décodage » tient

Grandeur (seuil déclaré avant)	gpt-oss-20b	gpt-oss-120b
Experts exacts, cinq contrôles chacun (0 faux)	768 sur 768	**4 608 sur 4 608**
Contrôles faux : partie, décodage, octets, décodé, GGUF	0, 0, 0, 0, 0	0, 0, 0, 0, 0
Résidents égaux, octets et empreinte	315 sur 315	471 sur 471
SHA-256 du GGUF, recalculée par le codeur puis la vérification	`27cd6c43…5901`	`582bd40f…622d`
r_{03} (à 0,1 % de l'étude 03 : 1,1037 et 1,1026)	1,103732	1,102592
Bits par poids : codes, δ, médianes, total	3,8135 ; 0,0343 ; 0,0028 ; 3,8506	3,8202 ; 0,0316 ; 0,0028 ; 3,8546
Tronçons : min, moyenne, max (octets)	11 837 440 ; 12 013 664 ; 12 144 640	11 763 712 ; 12 026 060 ; 12 251 136
Taille du `.trc` (repère de l'arbitre à 1 %)	11 157 405 696 (+0,52 %)	57 730 662 400 (+0,14 %)
Cassure en mémoire (une médiane faussée)	(7, 13) seul faux	(29, 101) seul faux

Table 3.3 : Le critère C1 et les taux, contre les seuils déclarés avant les lancements (`mesures/moteur/F/verdicts.md` ; journaux `verifier-20b.json`, `verifier-120b.json`, `comparer-03-120b.json`). Sur une vraie copie du 20b dont un seul octet est changé (décalage 8 818 286 848, 120 → 121, vu par `cmp -l`), exactement l'expert (17, 29) tombe, code de sortie 1.

(une médiane faussée change $e = \text{méd} + \delta - 128$, pas les flux rANS) ; « GGUF » tient (la source n'est pas touchée). C'est le motif attendu, et il dit que les cinq contrôles ne sont pas redondants.

Le taux, mot à mot. Sur le 120b, la comptabilité de l'étude 03 (blocs MXFP4 d'origine contre flux MEDIANES, DELTA et CODES_CTX tels que rangés) donne codes 3,820169, δ 0,031609, médianes 0,002778, soit **3,854555 bits par poids** ; avec les tables (224 656 octets), 3,854571 [mes.]. Les médianes coûtent exactement un octet par ligne de 2 880 poids : $8/2\,880 = 0{,}0027778$ [calc.]. Le rapport r_{03} se lit directement : 17 octets d'origine pour 32 poids font $17 \times 8/32 = 4{,}25$ bits par poids, et $4{,}25/3{,}854555 = 1{,}102592$, le r_{03} publié [calc.]. Le rapport des tronçons complets (en-têtes, biais, bourrage compris) vaut $13\,253\,760/12\,026\,060{,}4 = 1{,}102087$ [calc.]. Les douze tenseurs de F1 du 20b sont égaux au millionième à la mesure de l'arbitre, ce qui fait du codeur de production un second chemin de F1 (`csv/f1-contre-trc-20b.csv`).

3.12.2 Le second chemin : aucune ligne commune

Le premier chemin lit le GGUF par la fonction même du codeur (`gguf::lire_dans`) : une erreur systématique de lecture (lire la tranche d'un autre expert, un décalage faux) serait commune au codeur et à la vérification, et C1 tiendrait à tort. Le lot F a donc écrit un second chemin (`mesures/moteur/F/second_chemin_c1.py`) : Python, bibliothèque standard seule (`hashlib` sur OpenSSL 3.0.18, pas la crate `sha2`), écrit depuis le seul § 4 et la spécification GGUF v3, sans relire `trc-format`, `trc-codec` ni le lecteur du codeur ; le GGUF lu sans tampon ; les décalages des tenseurs refaits une troisième fois par le lecteur GGUF de llama.cpp lui-même (`gguf_init_from_file` de `ggml-base.dll` b11515, MIT) [12]. La figure 3.16 montre comment les deux chemins se rejoignent.

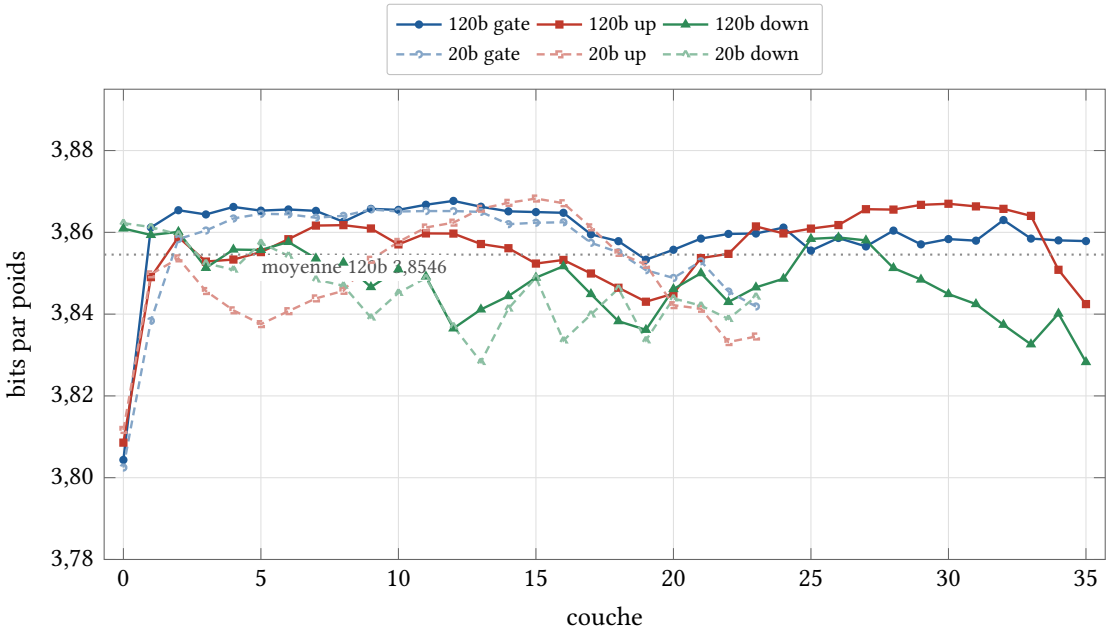


Figure 3.14 : Bits par poids de chaque tenseur d'experts (codes, δ et médianes), couche par couche, pour les deux modèles. Le taux est régulier entre couches et entre gate, up et down, à 0,05 bit près, sauf la couche 0 du 120b (gate et up près de 3,80). Données : `mesures/moteur/F/csv/taux-par-tenseur-{120b,20b}.csv`.

Proposition 3.13 (Le second chemin ferme la faille d'une lecture commune)**.** *Soit $G(x)$ les octets de l'expert d'origine, $D(x)$ l'expert décodé, $r(x)$ l'empreinte du répertoire, h l'empreinte64. Si $h(D(x)) = r(x)$ (premier chemin) et $h(G_2(x)) = r(x)$ (second chemin, G_2 lu par un autre programme), alors $D(x) \neq G_2(x)$ exigerait une collision de h ; et une erreur de lecture du codeur ne passe que si G_2 se trompe exactement de la même façon.*

Démonstration. Les deux égalités donnent $h(D(x)) = h(G_2(x))$; la proposition 3.10 borne la probabilité d'une collision par $4\,608 \cdot 2^{-64}$ sur le modèle. Une erreur systématique de `gguf::lire_dans` rendrait $G_1(x) \neq G(x)$; le codeur aurait rangé $r(x) = h(G_1(x))$, et la vérification aurait trouvé $D = G_1$, à tort. Le second chemin compare $r(x)$ à $h(G_2(x))$: il faudrait que deux programmes sans ligne commune, dont l'un (llama.cpp) fait référence pour le format GGUF, se trompent de la même façon. ■

Ce qu'il a mesuré. Sur les deux modèles, sur secteur : 0 écart entre le lecteur Python et llama.cpp (459 et 687 tenseurs) ; zone de données pavée sans trou ; SHA-256 par `hashlib` égale au registre des téléchargements (troisième calcul, le premier hors de `sha2`) ; en-tête, MÉTA (36 paires octet pour octet, plus 3), TABLES (288 et 432), RÉPERTOIRE sans écart ; **0 empreinte d'expert fausse sur 768, puis sur 4 608** ; BIAIS et MEDIANES égaux octet pour octet à leurs sources refaites [mes.] (`journaux/second-chemin-{20b,120b}.json`). Les médianes, refaites : pour $B = 90$, la médiane est `tri[45]`, le 46e plus petit des 90 octets d'échelle d'une ligne.

Ce que le témoin a attrapé chez son auteur

La première version du second chemin lisait le numéro de version du GGUF sur la magie (curseur à 0 au lieu de 4). Sa relecture croissante de la tête, faite pour les métadonnées longues, a alors grossi sans borne jusqu'au fichier entier : 9,6 Go de RAM sur une machine de 16, avant d'échouer. Corrigé (curseur à 4, relecture bornée à 256 Mio). Leçon : une boucle de repli sans borne

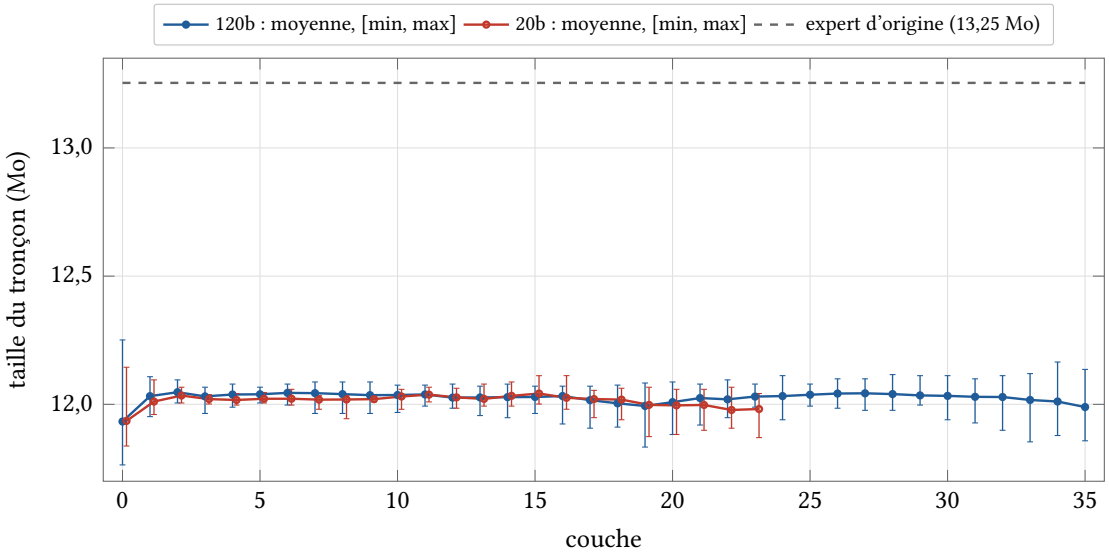


Figure 3.15 : Taille des tronçons par couche (moyenne, minimum et maximum sur les experts de la couche), contre l'expert MXFP4 d'origine (13 253 760 octets). Le plus grand tronçon du 120b (12 251 136 octets, couche 0) fixe la taille des tampons d'arrivée du moteur. Données : `mesures/moteur/F/csv/troncons-par-couche-{120b,20b}.csv`.

> transforme une erreur d'analyse en charge pour toute la machine (`moteur/REPRISE-F.md`).

Il manquait encore un maillon : « octets du tronçon → expert décodé » n'avait sur le 120b qu'un décodeur processeur. Le décodeur de la carte du lot K l'a refait : **4 608 sur 4 608** experts du vrai fichier égaux à `trc-codec` et au répertoire (`mesures/moteur/K/k1-trc-gptoss120b.json`) [mes.].

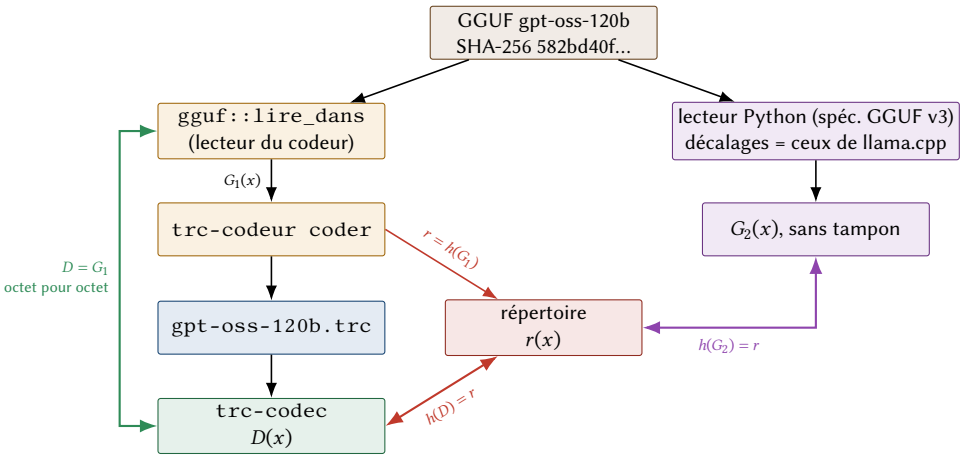
3.13 Le décodeur indépendant de l'épreuve

La spécification prévoyait un geste pour savoir si elle était vraie (ARCHITECTURE.md § 8.1) : un juge qui ne l'a pas écrite code un décodeur Python *à partir du seul § 4*, sans lire `trc-format` ni `trc-codec`, et doit redonner au bit près des tronçons tirés au hasard. Un échec aurait été un défaut de la spécification ou du codeur.

Ce que fait l'outil. `mesures/moteur/epreuve-J1/decodeur_s4.py` (Python 3.13, numpy) contient son propre lecteur GGUF v3, vérifie l'en-tête champ par champ, MÉTA, RÉSIDENTS, TABLES et RÉPERTOIRE, puis décode chaque tronçon tiré par le rANS du § 4.9.2, *vectorisé sur les tuiles* (127 tuiles × 32 voies en parallèle, 1 024 pas séquentiels par flux, 0,8 s par expert), la table par voie et la reconstruction. L'expert est comparé à l'empreinte d'origine du répertoire et aux octets du GGUF au décalage du lemme 3.7 ; s'y ajoutent deux contrôles que le décodeur n'exige pas (médiane égale à `tri[B/2]`, fin de chaque tuile égale au début de la suivante).

Listing 3.3 : Le cœur du rANS vectorisé du juge (`mesures/moteur/epreuve-J1/decodeur_s4.py`, `decoder_flux`, géométrie 1, raccourci) : la règle « pour $l = 0$ à 31 dans l'ordre » devient une somme cumulée sur l'axe des voies (proposition 3.5).

```
for ml in range(N):                          # macro-pas local, toutes les tuiles ensemble
    m = tuiles * N + ml
    act = np.nonzero(m < total_m)[0]         # tuiles encore actives
```



À gauche, le premier chemin (`trc-codeur verifier`, 4 608 experts, cinq contrôles chacun). À droite, le second (`second_chemin_c1.py`), qui ne partage aucune ligne avec le lot F et dont les décalages de tenseurs égalent ceux du lecteur GGUF de llama.cpp. Les deux chemins ne se rejoignent que dans le répertoire ; ensemble, ils donnent $h(D(x)) = h(G_2(x))$ pour chaque expert (proposition 3.13). Le seul maillon du décodage a été refait par deux autres décodeurs : celui de la carte du lot K (4 608/4 608) et le décodeur Python de l'épreuve (20 tronçons tirés).

Figure 3.16 : Les deux chemins de C1 sur le 120b.

```
    w = m[act, None] * 32 + voies[None, :] # mot de sortie de chaque voie
    ctx = kappa[w >> 2]                    # table par voie : bloc = w / 4
    xa, pa = x[act], p[act]
    for k in range(4):                     # sous-pas
        e = luts[ctx, (xa & 4095).astype(np.int64)].astype(np.uint64)
        s, f, b = e & 0xFF, ((e >> 8) & 0xFFF) + 1, e >> 20
        y = f * (xa >> 12) + b
        besoin = y < L_RANS                # voies qui renormalisent
        cs = np.cumsum(besoin, axis=1)     # rang de chaque lecture dans la tuile
        idx = np.clip(pa[:, None] + cs - 1, 0, max(W - 1, 0))
        xa = np.where(besoin, (y << 16) | mots[idx], y)
        pa = pa + cs[:, -1]
        sortie[4 * w + k] = s.astype(np.uint8)
    x[act], p[act] = xa, pa
```

Les résultats. **20 tronçons sur 20** tirés dans `gpt-oss-120b.trc` (graine 20261016) égaux au GGUF au bit près, et 20 sur 20 dans le 20b ; 0 voie hors de L, 0 tuile dont la fin diffère du début de la suivante ; 172 800 lignes sur 172 800 dont la médiane égale `tri[B/2]` ; les 471 résidents du 120b et les 315 du 20b égaux au GGUF octet pour octet ; en-tête, MÉTA (39 = 36 + 3 paires), TABLES (432 tables de somme 4 096), RÉPERTOIRE (4 608 entrées contiguës du début de TRONÇONS à la fin du fichier) et compagnons `.vocab.gguf` conformes [mes.] (`mesures/epreuve-J1.md` § 2.2 ; `journaux/s4-gptoss120b-20.json`).

3.13.1 Ce que l'écriture du décodeur a appris sur le texte

« Rien n'a manqué pour décoder au bit près », écrit le juge. Mais un second implémenteur a dû deviner trois choses, que le lot F avait tranchées de son côté parmi neuf précisions (tableau 3.6). Aucune ne change un octet des fichiers écrits ; toutes attendent que l'arbitre les porte dans la spécification.

Cassure	Attendu	20b	120b
bit de codes	cet expert, empreinte seule	(3, 7) : empreinte seule	(3, 7) : empreinte seule
médiane refaite	cet expert, MEDIANES seul	(5, 9) : MEDIANES seul	(29, 101) : MEDIANES seul
bit d'un biais	cet expert, empreinte et BIAIS	(11, 13)	(35, 127)
lecteur décalé	1 écart contre llama.cpp	1 (`output.weight`)	1
SHA-256	différente	différente	différente
tranche de $x + 1$	tous les experts lus faux	768 sur 768 (et MEDIANES 768)	256 sur 256 (couches 0 et 1, déclaré)

Table 3.4 : Les six cassures du second chemin : chacune tombe exactement où elle vise, et les autres experts restent justes (`verdicts.md`; `csv/second-chemin-c1.csv`). Le décalage fait aussi tomber les médianes (elles viennent des mêmes tranches) et laisse les biais justes (ils ne sont pas décalés) : les contrôles voient chacun leur propre octet.

Cassure (tronçon couche 12, expert 54)	Voies hors de L	Octets faux	Le second tient
un bit retourné dans les mots de CODES_CTX	32 (une tuile)	31 122	oui
deux fréquences d'une table échangées	4 064	3 624 195	oui
une médiane changée d'une unité	**0**	90 (une ligne d'échelles)	oui
une fente de LUT corrompue	4 032	3 262 607	oui

Table 3.5 : Les cassures du décodeur de l'épreuve (`mesures/epreuve-J1.md` § 2.3). La médiane faussée laisse toutes les voies sur L et change exactement les 90 échelles d'une ligne : seule l'empreinte de l'expert la voit. Le bit retourné ne fausse qu'une tuile : les tuiles sont indépendantes, et l'erreur ne passe pas la frontière.

3.14 Les versions, et ce qui est réservé pour J2

La v1.0 (J1) a un niveau et la sorte CODES_CTX ; un lecteur 1.x lit toute version 1.y inférieure ou égale à la sienne, et refuse 2.x. La v1.1 (J2) est réservée au bit près : deux niveaux, BASE_CTX (signe et classe de magnitude, en demi-octet, table par voie) au niveau 0, COMPL_CTX (rang de la magnitude dans sa classe, table par symbole, jusqu'à 192 contextes, sauts pour la classe à une seule magnitude) au niveau 1, et les tables CENTRES. La mesure F1 de l'arbitre, rapportée au Tome I, en a fixé le coût (tableau 3.7) ; le passage demandera de recoder le fichier une fois.

Ce qui pourrait être faux

- **Le décodeur de l'épreuve n'a décodé que 40 tronçons** (20 par modèle) et 13 nommés ; C1 sur les 4 608 experts repose sur les deux chemins du lot F et sur le décodeur de la carte du lot K.
- **Le second chemin est écrit par le lot F** : son auteur connaît les choix du codeur. Quatre de ses contrôles de structure en dépendent (précisions 1, 2, 4, 5) ; aucun ne touche les empreintes d'experts, qui ne dépendent que des § 4.7, 4.11, 4.12 et de la spécification GGUF.
- **Le lecteur Python et le codeur partagent une lecture du § 4.11** (blocs puis biais, ordre gate, up, down ; `ne[0]` = colonnes) : si elle était fausse au regard de llama.cpp, les deux seraient faux ensemble. Le témoin qui la valide est la référence f64 du lot R, lue dans le GGUF

	Précision (réponse par délégation du 2026-10-15, réversible)	Vue aussi par le juge
1	la longueur d'une section est utile : empreinte comprise, bourrage exclu	oui (m4.1)
2	l'empreinte de la table des résidents est aux 8 derniers octets de la section	oui (m4.2)
3	les octets utiles d'une partie finissent avec le dernier flux	—
4	contexte fixe : 0 pour DELTA, `0xFF` sinon	—
5	octets produits d'un flux rANS = son champ P (bourrage compris) ; d'un flux brut, sa longueur	—
6	un bloc de bourrage a $\kappa = 0$ (section 3.7)	—
7	structures du § 5.3 étendues (champs et fonctions ajoutés, rien de changé)	—
8	règle d'arrondi des tables : celle de l'étude 03 (non normative, les fréquences sont stockées)	—
9	ordre des flux du fichier brut : pour chaque j, ses trois flux à la suite ; TABLES vide	—
m4.3	« la fin de la tuile t est le début de la tuile $t + 1$ » : vrai sur 100 % des tuiles, non normatif	second contrôle gratuit (proposition 3.4)
T2	taille des LUT : § 4.6 écrit 6,9 Mo ; c'est 7 077 888 octets, 7,08 Mo ou 6,75 Mio	relevée par le juge du Tome II (m17)

Table 3.6 : Les précisions du texte du format en attente de l'arbitre : les neuf du lot F (`moteur/REPRISE-F.md`), les trois imprécisions du juge de l'épreuve (constat m4, `mesures/epreuve-J1.md` § 2.5) et celle du juge du Tome II (T2, `memoire/tome2/JUGEMENT.md`, m17).

et jugée contre llama.cpp.

- **Les pointes de mémoire du codeur** sont des ensembles de travail relevés toutes les 0,5 s : une pointe plus brève peut échapper ; la machine était partagée.
- **La borne de collision** suppose que SHA-256 se comporte comme une fonction aléatoire sur des erreurs non fabriquées ; elle ne vaut rien contre un adversaire qui choisit les octets.
- **Le taux de la v1.1** est mesuré sur 12 tenseurs du 20b par l'outil de l'arbitre, pas par le codeur de production, qui ne l'écrit pas encore ; si ce codeur dépasse 1,004× le niveau unique sur le 120b, l'arbitre redécide.
- **La cassure de copie** n'est faite que sur le 20b ; sur le 120b, la cassure est en mémoire (même code de vérification, un bit retourné après la lecture).

Variante (12 tenseurs de gpt-oss-20b)	Total	Codes ou base	Complément	Rapport à A_ctx
A_flux (trois flux de classe, étude 03)	3,8434	3,8059	—	1,00008
A_ctx (table par voie, v1.0)	**3,8431**	3,8056	—	1
B_cond (complément selon κ et la classe)	3,8591	2,6066	1,2149	1,00414
B_mag (complément selon la classe seule)	3,8733	2,6066	1,2291	1,00785
B_riche (v1.1)	**3,8550**	2,6066	1,2107	**1,00308**

Table 3.7 : Bits par poids de la mesure F1 de l'arbitre (ARCHITECTURE.md § 3.3 ; `decisions/journaux/arbitre/decoupe-gptoss20b.json`), tous les flux décodés et comparés octet à octet. Le format unique de J2 paiera 0,31 % d'octets au mode exact, sous le seuil déclaré de 0,4 %.

Chapitre 4

Le chemin des octets

D'où l'on part, ce que le chapitre établit, ce qui reste

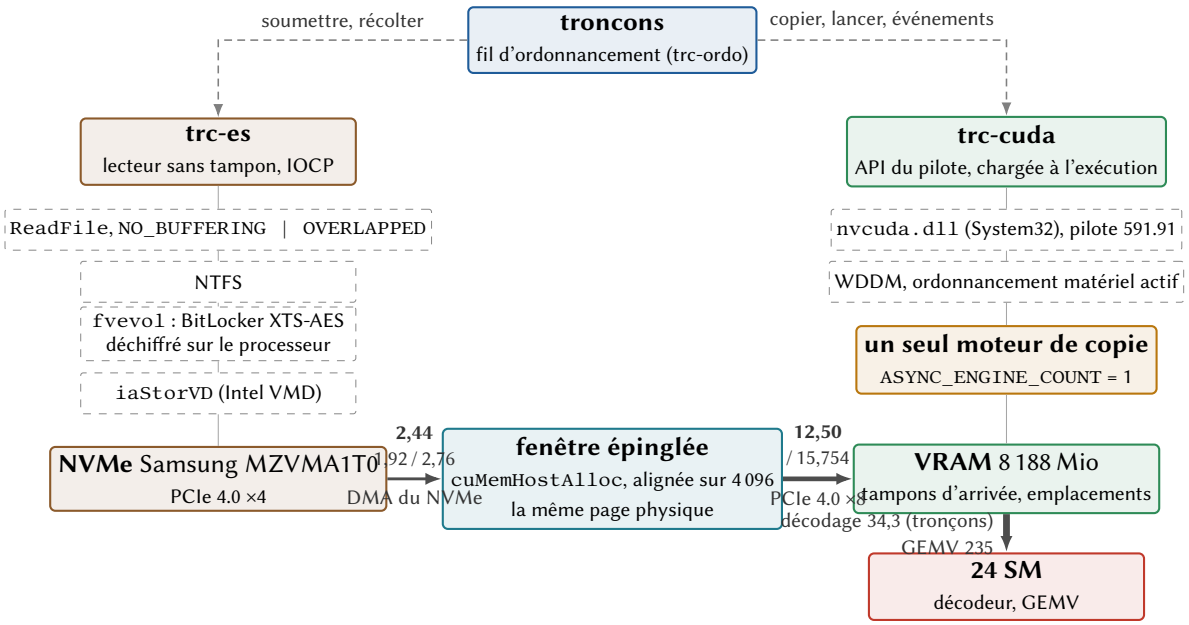
Au départ, la décision d'architecture tenait le chemin des octets pour réalisable sous CUDA sur la foi de trois mesures faites par d'autres moyens : une DMA à 12,2 Go/s depuis un tampon hôte de Vulkan, la mémoire épinglée plus rapide de 21 % et un recouvrement copie-calcul mesurés par le runtime CUDA de llama.cpp appelé depuis Python, un disque à 2,58 Go/s en blocs de 4 Mio. Trois choses n'étaient mesurées nulle part : épingler plusieurs gigaoctets sous WDDM sur une machine de 16 Go partagée ; lire le disque sans tampon *directement* dans la mémoire que lit la DMA ; et ce que coûte, à chaque couche, la lecture du routeur par l'hôte. **Ce chapitre établit** deux bibliothèques à nous, sans aucune bibliothèque de CUDA liée (`trc-cuda`, l'API du pilote chargée à l'exécution ; `trc-es`, la lecture sans tampon par port de complétion) ; les trois essais qui répondent aux trois questions, chacun avec son témoin, sa cassure et son seuil déclaré ; un fait de la machine que personne n'avait relevé (la carte n'a qu'un moteur de copie sous WDDM) ; le banc de bout en bout qui transporte les 768 experts de gpt-oss-20b et les 4 608 de gpt-oss-120b au bit près ; et une découverte : **sur cette machine, le débit d'une lecture sans tampon dépend de sa seule longueur**, en deux régimes, et toutes les longueurs de tronçon tombent dans le lent. Lire une longueur rapide fixe (12 Mio) rend ×1,335. **Il reste** la décision de l'arbitre sur la longueur des lectures, le mécanisme des deux régimes, l'épinglage de 11 Go au calme, et la carte des longueurs du fichier du 120b. Sources : `moteur/E-pour-le-memoire.md`, `moteur/REPRISE-E.md`, `mesures/moteur/E/journaux/`, `mesures/moteur/E/csv/` ; pour le moteur, `moteur/O-pour-le-memoire.md` et `mesures/moteur/O/csv/`.

4.1 Ce que le chemin doit faire

Un défaut de cache, dans le moteur, c'est un expert de gpt-oss qui manque en VRAM au moment où le routeur de la couche le demande. Il faut alors amener son *tronçon* (la partie codée de ses trois matrices et de leurs biais, 12 013 664 octets en moyenne sur le 20b) depuis le fichier `.trc` jusqu'à un emplacement de la carte, décodé, avant que les produits matrice-vecteur de la couche ne le lisent. Entre les deux, quatre étages traversés dans l'ordre : le disque, la mémoire hôte, le lien PCIe, la VRAM et ses unités de calcul (figure 4.1).

Le budget se lit dans l'équation du jeton du Tome I : dans le scénario LRU calé, le temps disque d'un jeton du 120b vaut 157 ms (07 § 3.3). Chaque étage doit donc soit se cacher derrière le disque, soit coûter peu devant lui. Trois questions n'avaient pas de réponse mesurée, et c'est par elles que commence le lot E :

1. **E0 (a)** : peut-on épingler plusieurs gigaoctets de mémoire hôte sous WDDM, sur une machine de 16 Go que cinq lots se partagent ?
2. **E0 (b)** : une lecture sans tampon (`FILE_FLAG_NO_BUFFERING`) peut-elle écrire directement dans la mémoire épinglée, puis la DMA la lire à pleine vitesse, octets égaux ?



Go/s, mesurés ici. Disque : lectures de 12 Mio à décalages tirés, QD 4, machine calme (E1) / tronçons entiers lus à leur longueur (E2, disque seul, régime lent) / lectures de 12 Mio aux décalages des tronçons (E2-V12, disque seul). Lien : DMA de 12 Mio depuis la mémoire épinglée (E0 b) / plafond calculé du PCIe 4.0 ×8. Décodage : un expert à la fois (E2, étages). GEMV : lot de 12 matrices MXFP4 (K2).

Figure 4.1 : La pile du chemin des octets, telle qu'elle est construite. À gauche la lecture (`trc-es`, puis les couches de Windows qu'une lecture sans tampon traverse : le système de fichiers, le filtre de BitLocker qui déchiffre chaque octet sur le processeur, le pilote du contrôleur VMD) ; à droite la carte (`trc-cuda`, le pilote, le moteur de copie unique). Au milieu, la fenêtre épinglée : la même page physique est écrite par la DMA du NVMe et lue par celle de la carte, sans copie par le processeur.

Données : `mesures/moteur/E/journaux/e1-2026-10-15-seance3/resume.json`, `e2-2026-10-15-seance1/resume.json`, `e2-2026-10-15-v12/resume.json`, `2026-10-15-seance3/resume.json` ; GEMV : `mesures/moteur/K/csv/k2-debit-passes.csv`.

3. **E0 (c)** : que coûte au jeton la lecture des quatre indices du routeur par l'hôte, une fois par couche, 36 fois par jeton sur le 120b ?

4.2 `trc-cuda` : l'API du pilote, sans bibliothèque liée

Le choix. Le moteur ne livre aucune DLL d'exécution de CUDA : le pilote installé (591.91, CUDA 13.1) suffit. `trc-cuda` charge `nvcuda.dll` à l'exécution, par `LoadLibraryExW` limité au dossier `System32` (aucune DLL déposée dans le dossier courant ne peut se faire passer pour le pilote), puis lit chaque point d'entrée par son nom exporté exact. Les noms sont ceux que `cuda.h` de CUDA 13.1 choisit par ses `#define` : `cuMemAlloc` est exporté sous `cuMemAlloc_v2`, `cuMemcpyHtoDAsync` sous `cuMemcpyHtoDAsync_v2` [13]. Un symbole absent fait échouer le chargement *avec son nom* : un pilote trop ancien se dit, il ne plante pas trois appels plus loin (listing 4.1).

Le contexte primaire. `cuCtxCreate` est devenu `cuCtxCreate_v4` en CUDA 13 ; le contexte primaire, lui, n'a pas changé d'interface [14]. `trc-cuda` retient donc le contexte primaire, et tout objet qu'il crée (flux, événement, mémoire de la carte, mémoire épinglée, module) tient une référence

Listing 4.1 : Le chargement de la table des points d'entrée (`moteur/crates/trc-cuda/src/api.rs`, lignes 60 à 76, commit `014bd11`). La macro `table!` engendre un champ par point d'entrée, typé selon `cuda.h` ; `symbole` rend une erreur qui nomme le symbole manquant.

```
            unsafe fn charger() -> Result<Api, String> {
                let nom_dll: Vec<u16> = "nvcuda.dll\0".encode_utf16().collect();
                // SAFETY : chaîne terminée par un zéro ; recherche limitée à System32, où le
                // pilote d'affichage installe nvcuda.dll (pas de DLL plantée dans le dossier courant).
                let module = unsafe {
                    LoadLibraryExW(nom_dll.as_ptr(), std::ptr::null_mut(), LOAD_LIBRARY_SEARCH_SYSTEM32)
                };
                if module.is_null() {
                    return Err("nvcuda.dll introuvable dans System32 : aucun pilote NVIDIA \
                                installé, ou pilote trop ancien".into());
                }
                let symbole = |nom: &str| -> Result<unsafe extern "system" fn() -> isize, String> {
                    let c = format!("{nom}\0");
                    // SAFETY : module valide, nom terminé par un zéro.
                    unsafe { GetProcAddress(module, c.as_ptr()) }
                        .ok_or_else(|| format!("symbole {nom} absent de nvcuda.dll (pilote trop ancien ?)"))
                };
```

comptée sur un objet interne qui ne relâche le contexte qu'après le dernier d'entre eux. Le contexte est rendu courant au besoin, fil par fil, et une *génération* distingue un contexte relâché puis repris d'un contexte resté courant. Le témoin `duree_de_vie_du_contexte` exerce cet ordre.

Un fait de la machine. L'attribut `CU_DEVICE_ATTRIBUTE_ASYNC_ENGINE_COUNT` vaut **1** sur la RTX 4060 Laptop sous WDDM [mes.] : la carte n'a qu'un moteur de copie pour les deux sens [15]. Le premier témoin d'ordre de `trc-cuda` l'a montré à son insu : il cherchait une course entre une copie aller et une copie retour posées sur deux flux sans événement, et ne l'a jamais vue en 20 essais. Les deux copies passaient par la même file. Refait avec un noyau qui lit la VRAM pendant la copie, il voit la course au premier essai : 66 393 069 octets sur 67 108 864 lus avant leur arrivée [mes.]. La section 4.5 en tire les conséquences.

4.3 `trc-es` : lire sans tampon, par port de complétion

Une lecture sans tampon ne passe pas par le cache de Windows : le contrôleur écrit directement dans le tampon de l'appelant. Elle exige un décalage, une longueur et une adresse multiples du secteur [6] ; le format `.trc` aligne tout sur 4 096 octets pour cette raison. Avec `FILE_FLAG_OVERLAPPED`, chaque lecture est confiée au système sans attendre, et son achèvement arrive comme un paquet dans un *port de complétion* (IOCP), qu'un fil récolte [16].

Table 4.1 : Une lecture de `trc-es` en mémoire. `Vol` est `#[repr(C)]`, alloué sur le tas, à adresse stable jusqu'à sa récolte : l'adresse de l'OVERLAPPED que rend le port est celle du `Vol`, et sert de clé à la table des lectures confiées.

structure	octets	champ	contenu
`Vol`	0–7	`Internal` (u64)	NTSTATUS de la lecture, écrit par le système
	8–15	`InternalHigh` (u64)	octets transférés
	16–19	`Offset` (u32)	décalage, 32 bits bas
	20–23	`OffsetHigh` (u32)	décalage, 32 bits hauts
	24–31	`hEvent`	nul (le port remplace l'événement)
	32–39	`jeton` (u64)	identifiant rendu à l'appelant
	40–	`depart`	instant de la remise au système
`Requete`		jeton, décalage, longueur, destination	u64, u64, u32, pointeur ; les trois derniers multiples de 4 096
`Achevement`		jeton, octets, erreur, durée	u64, u32, code Windows ou rien, u32 en µs (de la remise à la récolte)

4.3.1 Les structures, au bit près

Le lecteur tient au plus q lectures confiées au système (sa *profondeur*) ; au-delà, les requêtes attendent dans une file interne, et chaque récolte en confie de nouvelles. La figure 4.2 donne l'automate d'une requête, erreurs comprises.

4.3.2 L'exclusion, démontrée

Le danger d'un port de complétion est la course entre le fil qui confie une lecture et celui qui la récolte : si le paquet arrive avant que le lanceur ait inscrit la lecture dans sa table, la récolte cherche une clé absente.

Lemme 4.1 (Toute clé rendue par le port est dans la table)**.** *Dans `trc-es`, quand la récolte cherche une adresse d'OVERLAPPED rendue par `GetQueuedCompletionStatusEx`, cette adresse figure déjà dans la table des lectures confiées.*

Démonstration. L'appel à `ReadFile` et l'insertion de la clé dans la table se font sous le même verrou (`lancer` est appelée verrou tenu, par `soumettre` ou par `recolter`). La récolte attend le port *hors* du verrou, puis prend le verrou avant toute recherche. Soit un paquet qui arrive avant l'insertion de sa clé : à cet instant le lanceur tient le verrou (il est entre `ReadFile` et l'insertion) ; la récolte, réveillée, attend donc ce verrou, que le lanceur ne rend qu'après l'insertion. Dans tous les cas la recherche suit l'insertion. Une clé n'est retirée que par la récolte de son propre paquet, et un OVERLAPPED ne produit qu'un paquet. ■

Le même raisonnement fonde l'abandon : un OVERLAPPED appartient au système depuis l'appel à `ReadFile` jusqu'au retrait de son paquet [16]. Le destructeur annule donc tout (`CancelIoEx`), puis attend chaque paquet avant de libérer quoi que ce soit ; si aucun paquet ne vient en 10 s, il préfère *fuir* les OVERLAPPED que libérer une mémoire que le système pourrait encore écrire.

Listing 4.2 : `soumettre`, `lancer` et `recolter` (`moteur/crates/trc-es/src/lib.rs`, lignes 170 à 263, commit `014bd11`). Le verrou couvre `ReadFile` et l'insertion ; l'attente du port se fait hors du verrou (lemme 4.1).

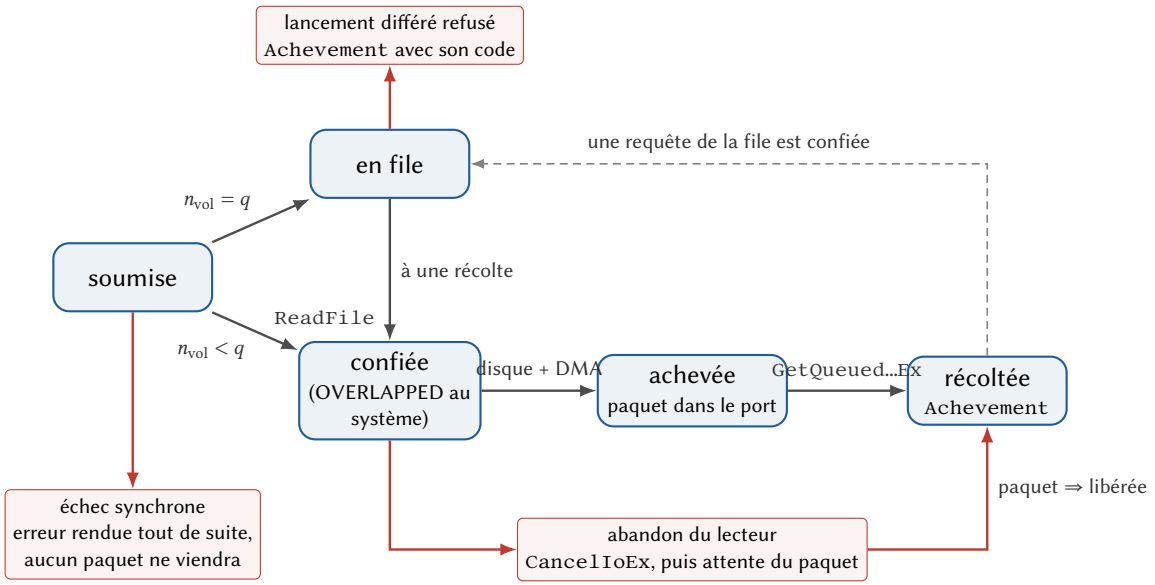


Figure 4.2 : L'automate d'une requête de `trc-es`. Un échec synchrone de `ReadFile` (code autre que `ERROR_IO_PENDING`) ne produira jamais de paquet : l'erreur est rendue tout de suite. Un succès immédiat produit quand même un paquet (`FILE_SKIP_COMPLETION_PORT_ON_SUCCESS` n'est pas posé), si bien que tout ce qui est confié finit dans le port. À l'abandon du lecteur, l'OVERLAPPED n'est libéré qu'après son paquet.

```rust
    /// Soumet une lecture. Refuse (sans rien lancer) un décalage, une longueur ou une destination
    /// qui ne sont pas multiples de 4 096, ou une longueur nulle.
    pub fn soumettre(&self, r: Requete) -> Result<()> {
        if r.decalage % ALIGNEMENT != 0 || r.longueur as u64 % ALIGNEMENT != 0 || r.longueur == 0 || r.destination as u64 % ALIGNEMENT != 0 {
            return Err(Erreur::Usage(format!(
                "requête {} non alignée sur 4 096 : décalage {}, longueur {}, destination {:p}",
                r.jeton, r.decalage, r.longueur, r.destination
            )));
        }
        let mut etat = self.etat.lock().unwrap();
        if etat.vols.len() < self.profondeur && etat.attente.is_empty() {
            self.lancer(&mut etat, r)
        } else {
            etat.attente.push_back(r);
            Ok(())
        }
    }

    /// Confie une requête au système. Sous le verrou.
    fn lancer(&self, etat: &mut Etat, r: Requete) -> Result<()> {
        let mut vol = Box::new(Vol { ov: unsafe { std::mem::zeroed() }, jeton: r.jeton, depart: Instant::now() });
        vol.ov.Anonymous.Anonymous.Offset = r.decalage as u32;
        vol.ov.Anonymous.Anonymous.OffsetHigh = (r.decalage >> 32) as u32;
        let cle = &mut vol.ov as *mut OVERLAPPED;
        // SAFETY : OVERLAPPED à adresse stable (boîte gardée jusqu'à la récolte) ; destination selon
        // le contrat de l'appelant.
```

```
        let ok = unsafe { ReadFile(self.fichier, r.destination, r.longueur, std::ptr::null_mut(), cle) };
        if ok == 0 {
            let code = unsafe { GetLastError() };
            if code != ERROR_IO_PENDING {
                // Échec immédiat : aucun paquet de complétion ne viendra.
                return Err(Erreur::Systeme { appel: "ReadFile", code });
            }
        }
        // Succès immédiat ou en attente : dans les deux cas un paquet arrivera sur le port (le
        // drapeau FILE_SKIP_COMPLETION_PORT_ON_SUCCESS n'est pas posé).
        etat.vols.insert(cle as usize, vol);
        Ok(())
    }

    /// Récolte les lectures achevées, en attendant au plus `attente_ms` la première (0 : sans
    /// attendre ; `u32::MAX` : sans limite). Ajoute les achèvements à `sortie` et relance la file.
    /// Rend tout de suite si rien n'est en vol.
    pub fn recolter(&self, attente_ms: u32, sortie: &mut Vec<Achevement>) -> Result<()> {
        {
            let mut etat = self.etat.lock().unwrap();
            sortie.append(&mut etat.refus);
            if etat.vols.is_empty() {
                return Ok(());
            }
        }
        let mut entrees: Vec<OVERLAPPED_ENTRY> = vec![unsafe { std::mem::zeroed() }; self.profondeur];
        let mut n: u32 = 0;
        // Attente hors du verrou : un autre fil peut soumettre pendant ce temps.
        let ok = unsafe { GetQueuedCompletionStatusEx(self.port, entrees.as_mut_ptr(), entrees.len() as u32, &mut n, attente_ms, 0) };
        if ok == 0 {
            let code = unsafe { GetLastError() };
            if code == WAIT_TIMEOUT {
                return Ok(());
            }
            return Err(Erreur::Systeme { appel: "GetQueuedCompletionStatusEx", code });
        }
        let mut etat = self.etat.lock().unwrap();
        for e in &entrees[..n as usize] {
            let cle = e.lpOverlapped as usize;
            let Some(vol) = etat.vols.remove(&cle) else {
                return Err(Erreur::Usage(format!("paquet de complétion inconnu ({cle:#x})")));
            };
            let duree_us = vol.depart.elapsed().as_micros().min(u32::MAX as u128) as u32;
            let mut erreur = None;
            // `Internal` porte le NTSTATUS de la lecture ; s'il n'est pas nul, on demande au
            // système le code Windows correspondant.
            if vol.ov.Internal != 0 {
```

```
                let mut lus: u32 = 0;
                let r = unsafe { GetOverlappedResult(self.fichier, &vol.ov, &mut lus, 0) };
                erreur = Some(if r == 0 { unsafe { GetLastError() } } else { vol.ov.Internal as u32 });
            }
            let octets = e.dwNumberOfBytesTransferred;
            etat.octets_lus += octets as u64;
            sortie.push(Achevement { jeton: vol.jeton, octets, erreur, duree_us });
        }
        while etat.vols.len() < self.profondeur {
            let Some(r) = etat.attente.pop_front() else { break };
            if let Err(e) = self.lancer(&mut etat, r) {
                let code = match e {
                    Erreur::Systeme { code, .. } => code,
                    Erreur::Usage(_) => u32::MAX,
                };
                sortie.push(Achevement { jeton: r.jeton, octets: 0, erreur: Some(code), duree_us: 0 });
            }
        }
        Ok(())
    }
```

Témoins. Dix tests tiennent : aller-retour hôte → carte → hôte sur quatre tailles non rondes, ordre entre deux flux par un événement, noyau PTX écrit à la main et chargé par l'API, douze lectures de 1 Mio comparées page par page, dernière page partielle, refus des requêtes non alignées, abandon en vol, soumission et récolte sur deux fils. Puis le *témoin des témoins* : chaque témoin est forcé à faux par une variable d'environnement (`TRC_E_CASSER`), et l'on vérifie qu'il tombe *sur son propre message* ; 7 cassures sur 7 tombent, la suite restaurée tient [mes.] (`mesures/moteur/E/journaux/e1-temoins/`).

4.4 Épingler : l'essai E0 (a)

`cuMemHostAlloc` verrouille des pages en mémoire physique et les rend visibles de la carte. Une épingle de 11 Go sur une machine de 16 Go aurait fait paginer les quatre autres lots ; le protocole déclare donc une marge : une taille n'est tentée que si la RAM disponible dépasse la taille plus 1,5 Go (réponse par délégation du 2026-10-15, réversible).

Table 4.2 : E0 (a), séance du 2026-10-15 à 16 h 46, sur secteur, carte libre [mes.]. La DMA est une copie de 64 Mio faite juste après l'allocation, sans chauffe.

taille	allocation	libération	RAM avant	DMA en fin	verdict
0,25 Go	96 ms	46 ms	8,57 Go	10,36 Go/s	réussie, 0 écart
1 Go	343 ms	133 ms	8,51 Go	10,93 Go/s	réussie, 0 écart
4 Go	1 331 ms	594 ms	8,47 Go	11,00 Go/s	réussie, 0 écart
11 Go	—	—	8,29 Go	—	non tentée (8,29 < 11 + 1,5)

Données : `mesures/moteur/E/csv/2026-10-15-seance3-a.csv`.

L'allocation coûte environ $1\,331/4 = 333$ ms par gigaoctet [calc.] : c'est un coût d'ouverture, pas

de chemin. Le critère d'abandon K3 de l'architecture (aucune mémoire épinglée importante) ne se déclenche pas. Une taille « non tentée » n'est pas une taille refusée par le pilote : la question des 11 Go attend une machine calme.

Proposition 4.2 (La fenêtre épinglée du mode A0)**.** *En mode A0 (aucun cache tiède en RAM), le moteur épingle $d \cdot k$ tampons d'arrivée de la taille du plus grand tronçon, plus une boîte aux lettres et une ligne d'enchâssement. Pour le 20b, $d = 3$ couches d'avance et $k = 4$:*

$$3 \times 4 \times 12\,144\,640 + 4\,096 + 8\,192 = 145\,747\,968 \text{ octets} \approx 0{,}146 \text{ Go}.$$

Démonstration. C'est la définition de la fenêtre (architecture § 5.8) : $3 \times 4 \times 12\,144\,640 = 145\,735\,680$ octets de tampons d'arrivée (12 144 640 est le plus grand tronçon du 20b), la boîte 4 096, la ligne d'enchâssement 8 192 (une ligne de 3 060 o lue dans une lecture alignée) [calc.]. ■

La fenêtre épinglée du mode A0 est donc 27 fois plus petite que la plus grande épingle réussie [calc.] : la mesure (a) laisse au mode A (cache tiède) toute la place qu'il pourrait demander jusqu'à 4 Go.

4.5 Le lien : l'essai E0 (b) et le moteur de copie unique

Justesse. 32 morceaux de 12 Mio lus sans tampon dans la mémoire épinglée, copiés vers la VRAM puis redescendus : 0 écart avec une lecture indépendante du même intervalle (`std::fs`, par le cache de Windows), 0 écart au retour ; la cassure (un octet faussé dans la mémoire épinglée après la lecture) donne exactement 1 écart [mes.]. La lecture sans tampon écrit donc bien, et seulement, dans la page que lit la DMA.

Débit. Seuil déclaré : 11,0 Go/s, soit le 12,2 de l'étude 01 moins 10 % de bruit. La série ABBA de 16 copies de 12 Mio (la taille d'un tronçon) et de 16 copies de 64 Mio (la taille de la mesure de 01) donne la figure 4.3.

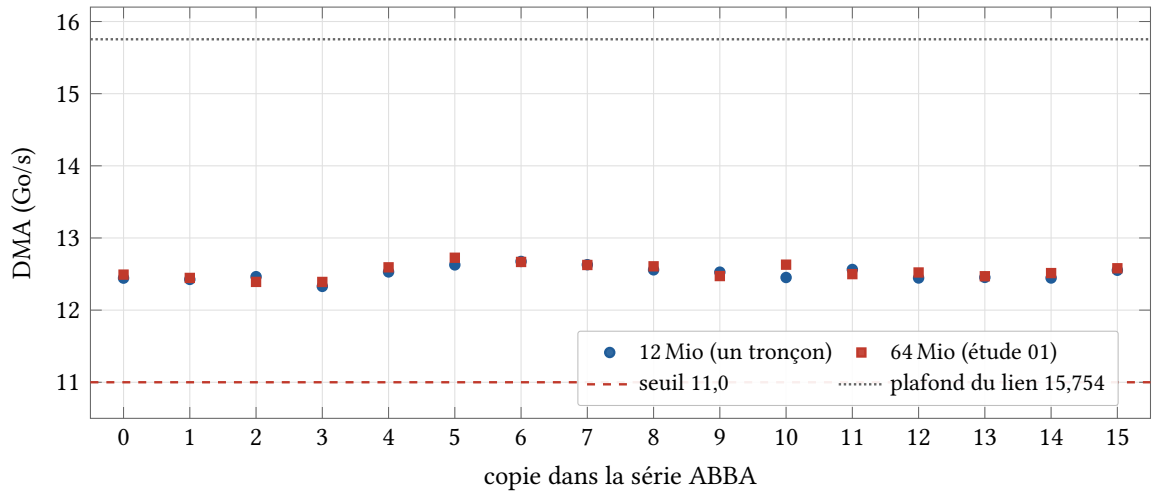


Figure 4.3 : E0 (b) : chaque copie hôte → carte de la série ABBA [mes.]. Médiane à 12 Mio : **12,495 Go/s** (12,33 à 12,67, bruit miroir 1,4 %) ; à 64 Mio : 12,517 (bruit 1,6 %). Les deux tailles ne diffèrent pas (règle de 06 § 7.5). Second chemin : l'horloge de l'hôte donne 12,45 et 12,51, jamais au-dessus des événements de la carte (0 sur 32). Le lien plafonne à 15,754 Go/s (PCIe 4.0 ×8) ; la DMA en tire 79 %.

Données : `mesures/moteur/E/csv/2026-10-15-seance3-b-dma.csv`, réordonné par `memoire/outils/tome2_chemin_noyaux.py` (`t2c-dma.csv`).

Proposition 4.3 (Le lien ne limite pas un tronçon)**.** *Soient B_d le débit du disque et B_c celui de la DMA, mesurés à la taille b d'un tronçon. Si $\rho = B_d/B_c < 1$, la copie d'un tronçon tient entière dans l'ombre de la lecture du tronçon suivant, et le moteur de copie reste libre une fraction $1-\rho$ du temps.*

Démonstration. Un tronçon occupe le disque b/B_d secondes et le moteur de copie b/B_c. En régime établi le disque livre un tronçon toutes les b/B_d secondes ; le moteur de copie le traite en $b/B_c = \rho \cdot b/B_d < b/B_d$, donc avant que le suivant n'arrive, et chôme le reste. ■

Avec les mesures de la séance, $\rho = 2{,}44/12{,}50 = 0{,}195$ [calc.] : la copie d'un tronçon de 12 Mio prend $12\,582\,912/12{,}50 \cdot 10^9 = 1{,}01$ ms, contre 5,2 ms de disque ; le moteur de copie chôme 80 % du temps. Ce n'est pas le lien qui fera la vitesse.

Proposition 4.4 (Le moteur de copie unique)**.** *Avec un seul moteur de copie, les copies des deux sens forment une seule file premier entré, premier sorti. Une copie carte → hôte soumise derrière k tronçons en vol attend au pire $k\,b/B_c$.*

Démonstration. Le moteur exécute une copie à la fois, dans l'ordre de soumission (c'est ce qu'a vu le témoin d'ordre : aucune course entre deux copies de sens opposés sur deux flux, en 20 essais). Une copie soumise derrière k copies de b octets ne commence qu'après elles, soit au plus $k\,b/B_c$ plus tard. ■

Pour $k = 4$ tronçons de 12 Mio à 11 Go/s, l'attente atteint $4 \times 12\,582\,912/11 \cdot 10^9 \approx 4{,}6$ ms [calc.]. Trois conséquences pour le moteur (figure 4.4) : (i) le mode témoin, qui redescend chaque expert décodé vers l'hôte, ralentit le chemin des tronçons et ne se chronomètre pas ; (ii) les logits d'un pas ($201\,088 \times 4$ o $\approx 0{,}8$ Mo, ≈ 73 µs de copie) attendent derrière les tronçons en vol ; (iii) la boîte aux lettres du routeur, écrite par un noyau directement dans la mémoire hôte projetée, *échappe* à la file : c'est elle qui porte la décision de chaque couche.

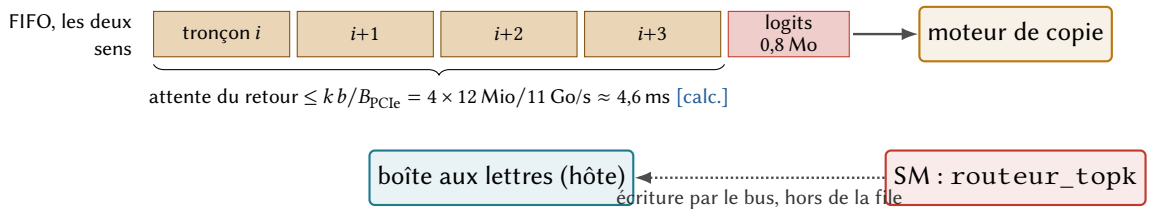


Figure 4.4 : Le moteur de copie unique. Toute copie, aller ou retour, passe par une seule file FIFO ; une écriture d'un noyau dans la mémoire hôte projetée (la boîte aux lettres du routeur) passe par le bus sans passer par cette file.

4.6 Lire le routeur, une fois par couche : l'essai E0 (c)

Dans un mélange d'experts, l'hôte ne sait quels tronçons lire qu'une fois le routeur de la couche calculé sur la carte. Il faut donc, à chaque couche, que l'hôte observe la fin du routeur, lise ses quatre indices, décide (succès ou défaut), et lance la suite. C'est une synchronisation de l'hôte par couche : 36 par jeton sur le 120b.

4.6.1 La boîte aux lettres, au bit près

Le routeur écrit sa décision dans une *boîte aux lettres* : une mémoire hôte épinglée et projetée (`cuMemHostAlloc` avec `DEVICEMAP`), où le noyau écrit par le bus et où l'hôte lit sans copie. Une case de 64 octets par couche, 36 cases (2 304 octets, dans une page de 4 096), figure 4.5.

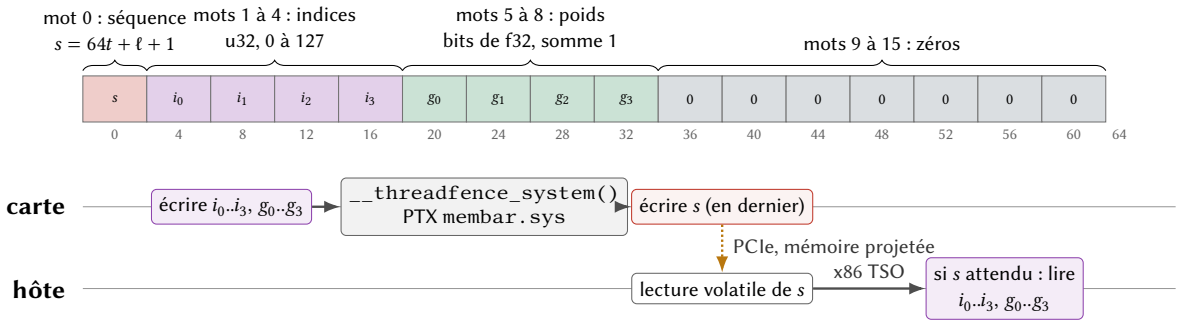


Figure 4.5 : La case de 64 octets d'une couche, et l'ordre des opérations. Le noyau écrit les indices et les poids, pose une barrière système, puis écrit le numéro de séquence $s = 64t + \ell + 1$ (t le jeton, compteur global ; ℓ la couche). L'hôte lit s par une lecture volatile ; s'il vaut la valeur attendue, il lit les indices. Le moteur du lot O reprend ce protocole tel quel (`routeur_topk`, chapitre 5).

Le numéro ne revient jamais pour un même jeton (le jeton est un compteur global) : une case lue avant son écriture porte toujours un numéro faux.

Proposition 4.5 (Un hôte qui lit le bon numéro lit les bons indices)**.** *Si le noyau écrit les mots 1 à 8, exécute `__threadfence_system()`, puis écrit le mot 0, et si l'hôte (x86-64) lit le mot 0 par une lecture volatile puis les mots 1 à 8, alors un hôte qui lit s attendu lit les indices et les poids écrits avec ce s.*

Démonstration. `__threadfence_system()` se traduit en PTX par `membar.sys` : toute écriture du fil faite avant elle est visible de tout observateur, l'hôte compris, avant toute écriture faite après elle [17]. Donc quand le mot 0 vaut s pour l'hôte, les mots 1 à 8 écrits avant portent déjà leurs valeurs pour lui. Sur x86-64 (ordre TSO), deux lectures ne se réordonnent pas entre elles : la lecture des indices, faite après celle du numéro dans le programme, se fait après elle. La lecture volatile empêche le compilateur de la déplacer ou de la supprimer. ■

Le témoin le vérifie (0 indice faux, 0 séquence absente dans toutes les variantes synchrones), et la cassure le montre nécessaire : lire la boîte sans attendre rend 3 141 indices faux sur la séance [mes.].

4.6.2 Le seuil, et le premier modèle

Le seuil (D1). On admet que la lecture du routeur coûte au plus 10 % du temps disque d'un jeton : $0{,}1 \times 157 = 15{,}7$ ms par jeton, soit $15{,}7/36 = 436$ µs par synchronisation [calc.]. Au-delà, ce coût cesse d'être négligeable et devient un étage de l'équation du jeton (critère d'abandon K4).

Proposition 4.6 (Modèle D2 : le coût d'une synchronisation)**.** *Soient, pour une couche, t_r la durée du routeur sur la carte et t_e celle des experts. Sans attente de l'hôte, les lancements sont en file d'avance et $T_{\text{async}} = L(t_r + t_e) + \varepsilon$. Avec une lecture du routeur par couche,*

$$T_{\text{sync}} = L\,(t_r + d_{\text{sig}} + d_{\text{hôte}} + d_{\text{lanc}} + t_e) + \varepsilon',$$

où d_{sig} est le délai entre la fin du noyau et le moment où l'hôte l'observe, $d_{\text{hôte}}$ le travail de l'hôte (lire quatre indices, préparer les arguments), d_{lanc} le délai entre `cuLaunchKernel` et le début du noyau. Le coût par jeton $T_{\text{sync}} - T_{\text{async}} = L\,s$, $s = d_{\text{sig}} + d_{\text{hôte}} + d_{\text{lanc}}$, ne dépend ni de t_r ni de t_e.

Démonstration. Sans attente, la carte enchaîne les $2L$ noyaux déjà en file : son temps est la somme de leurs durées, plus le premier lancement. Avec attente, l'expert de la couche ℓ ne peut commencer qu'après que l'hôte a vu la fin du routeur (d_{sig}), a travaillé ($d_{\text{hôte}}$) et a lancé (d_{lanc}) ; ces trois délais s'ajoutent en série à chaque couche, et rien d'autre ne change tant que la carte ne recouvre rien. ■

Listing 4.3 : Le routeur simulé (`moteur/bins/banc-chemin/noyaux/essais.cu`, lignes 45 à 64, commit `94ac4c4`). `occuper` attend en temps (registre `%globaltimer`), pas en cycles : la durée ne dépend pas de l'horloge du moment de la puce. Les indices attendus viennent d'un mélange de 32 bits que l'hôte recalcule [18].

```
// Le routeur simulé : occupe la carte `ns` nanosecondes (attention + routeur d'une couche), puis
// écrit les 4 indices et les 4 poids de la couche, une barrière système, et enfin le numéro de
// séquence. La barrière garantit qu'un hôte qui voit le numéro voit aussi les indices.
extern "C" __global__ void routeur_factice(uint32_t* boite, uint32_t jeton, uint32_t couche, uint64_t ns) {
    if (threadIdx.x != 0 || blockIdx.x != 0) return;
    occuper(ns);
    volatile uint32_t* c = boite + 16u * couche;
    float somme = 0.0f;
    float p[4];
    for (uint32_t i = 0; i < 4; ++i) {
        p[i] = 1.0f + (float)(indice_attendu(jeton, couche, i) & 7u);
        somme += p[i];
    }
    for (uint32_t i = 0; i < 4; ++i) {
        c[1 + i] = indice_attendu(jeton, couche, i);
        c[5 + i] = __float_as_uint(p[i] / somme);
    }
    __threadfence_system();
    c[0] = jeton * 64u + couche + 1u;
}
```

Le modèle porte en lui son témoin : mesuré à deux charges (0/0 et 50/250 µs), le coût doit être le même. La figure 4.6 le dessine à l'échelle.

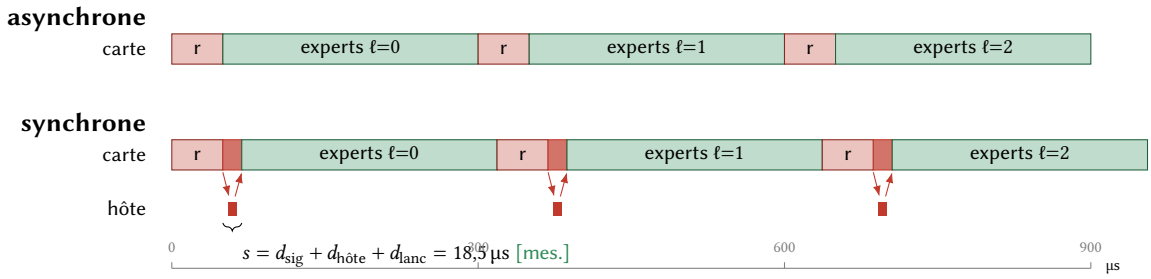


Figure 4.6 : Le modèle D2 pour trois couches, à l'échelle de l'essai sous charge (routeur 50 µs, experts 250 µs). En asynchrone, la carte enchaîne ; en synchrone, chaque couche montre le trou s entre la fin du routeur et le début des experts, pendant lequel l'hôte voit l'événement, lit la boîte et lance.

4.6.3 Ce que la mesure a changé au modèle

Cinq variantes sont mesurées : `evenement` (l'hôte attend un événement enregistré après le routeur, puis lit la boîte), `sondage` (il sonde l'événement), `boite` (il sonde la boîte elle-même), `boite_sans_vidage` (de même, sans pousser le lancement par `cuStreamQuery`), `bloquant` (événement à attente bloquante : le fil dort et le système le réveille).

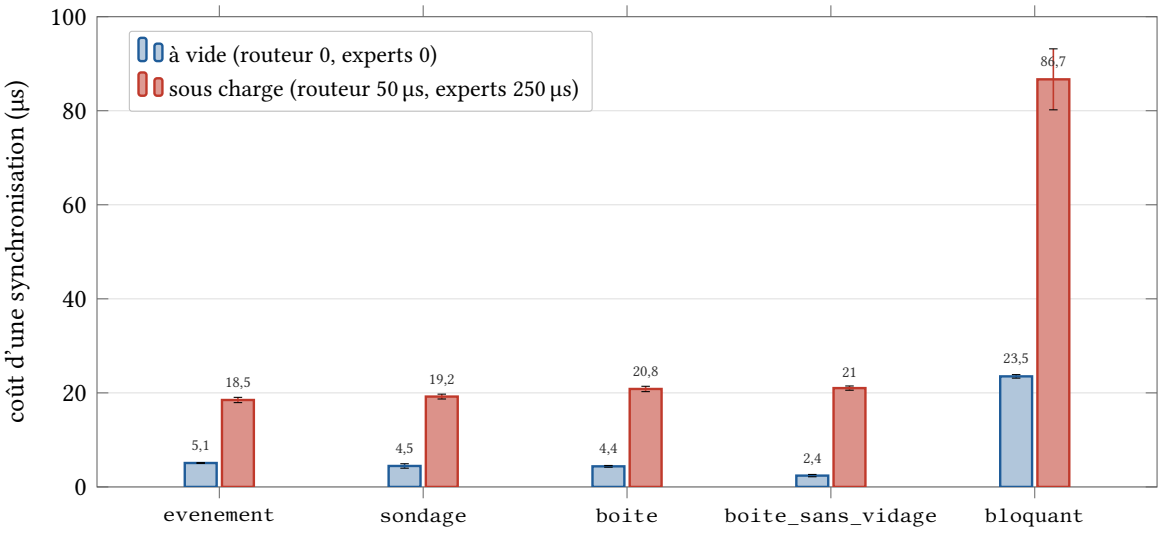


Figure 4.7 : E0 (c), séance du 2026-10-15 à 16 h 46 [mes.] : coût d'une synchronisation, $(T_{\text{sync}} - T_{\text{async}})/36$, par variante. Barres d'erreur : bruit des passes miroirs. Le seuil (436 µs) est hors du cadre. La variante retenue, `evenement`, coûte 18,5 µs sous charge, soit 0,666 ms par jeton, **23 fois sous le seuil**.

Données : `mesures/moteur/E/journaux/2026-10-15-seance3/resume.json` (`modele_D2`), réordonné dans `t2c-sync.csv`.

Le seuil tient largement (figure 4.7) : sous charge, `evenement` coûte 0,666 ms par jeton, `sondage` 0,692, `boite` 0,750, `bloquant` 3,12 ; avec un contexte en attente bloquante, `evenement` coûte 0,630 [mes.]. Ce coût est même au niveau du bruit de l'essai : la passe synchrone dure 11,779 ms contre 11,114 en asynchrone, et le résumé de la séance classe l'écart de `evenement` comme « non distinct du bruit ».

Mais le témoin du modèle tombe. Le coût sous charge vaut 3,6 à 4,8 fois le coût à vide pour quatre variantes (5,1 contre 18,5 µs pour `evenement`), et 8,7 fois pour `boite_sans_vidage` [mes.]. ~~rapports 3,6 à 4,8 selon la variante~~ La note du lot E écrivait « 3,6 à 4,8 selon la variante » : c'est vrai des quatre variantes retenues, pas de la cinquième (8,7, lue dans `modele_D2`).

Proposition 4.7 (Modèle D2 bis : le débit des lancements)**.** *Soit λ l'intervalle minimal entre deux lancements vus par la carte. La base asynchrone vaut $T_{\text{async}} = \max\big(L(t_r+t_e), 2L\lambda\big)+\varepsilon$. À vide ($t_r+t_e \ll 2\lambda$), la carte attend déjà les lancements, et la synchronisation n'ajoute que la part de $d_{\text{sig}} + d_{hôte}$ qui ne se cache pas derrière λ. Sous charge ($t_r + t_e \gg 2\lambda$), la carte est saturée en asynchrone, et la synchronisation expose la chaîne entière : $s \approx d_{\text{sig}} + d_{hôte} + d_{\text{lanc}}$, avec $d_{\text{lanc}} \geq \lambda$.*

Démonstration. Un lancement ne peut atteindre la carte plus souvent qu'une fois par λ : $2L$ noyaux durent au moins $2L\lambda$, d'où le maximum. À vide, le terme $2L\lambda$ domine les deux passes, et seule la part des délais de l'hôte qui excède λ allonge la passe synchrone. Sous charge, le premier terme domine la passe asynchrone, et chaque couche de la passe synchrone attend la chaîne complète, dont le lancement lui-même. ■

Le calcul. À vide, la passe asynchrone de 72 lancements dure 0,831 ms, d'où $\lambda \approx 831/72 = 11{,}5$ µs [mes.], bien au-dessus de $t_r + t_e \approx 0$. Sous charge, $L(t_r + t_e) = 36 \times 300 = 10{,}8$ ms et la passe asynchrone dure 11,114 ms ($\varepsilon \approx 0{,}31$ ms) : la carte est saturée. Estimation : $s_{\text{charge}} \approx s_{\text{vide}} + \lambda = 5{,}1 + 11{,}5 = 16{,}6$ µs, contre 18,5 mesurés (écart de 10 % ; d_{lanc} ne se mesure pas seul ici, c'est une hypothèse) [calc.]. **Le cas qui compte est le second** : un vrai expert lit 13 Mo en des centaines de microsecondes. C'est

pourquoi le seuil se juge sous charge. Leçon, gardée dans les recettes : mesurer une synchronisation à vide seulement donne un coût quatre fois trop bas.

4.7 Le disque seul : l'étape E1

4.7.1 Le seuil, et ce qu'est une passe propre

Seuil déclaré avant toute passe : la médiane des passes A (12 Mio, profondeur 4) doit atteindre $0{,}8 \times 2{,}58 = 2{,}064$ Go/s, où 2,58 Go/s est le débit de l'étude 01 en lectures aléatoires de 4 Mio et $0{,}8 = 1 - 2 \times 10\,\%$ de bruit. Mais la machine est partagée : un autre programme qui lit le disque pendant la passe lui prend une part de sa bande. Il faut savoir combien d'octets les autres ont lus pendant chaque passe.

Lemme 4.8 (Les octets étrangers par deux relevés bruts)**.** *Le compteur PDH `Disk Read Bytes/sec` de l'objet `PhysicalDisk(_Total)` est de type `PERF_COUNTER_BULK_COUNT`; sa donnée brute N est le nombre cumulé d'octets lus par toute la machine. Les octets lus pendant une passe valent exactement $N_1 - N_0$, et les octets étrangers $E = (N_1 - N_0) - b\,n$, où $b\,n$ sont les octets livrés par notre lecteur.*

Démonstration. Pour ce type de compteur, la valeur affichée est $(N_1 - N_0)/((D_1 - D_0)/F)$, avec D l'horodatage brut et F sa fréquence [19] : N est donc un cumul d'octets, et sa différence entre deux relevés compte exactement les octets de l'intervalle, sans intégrer un débit échantillonné [20]. Nos lectures sans tampon vont au disque ; elles sont comptées dans $N_1 - N_0$; le reste appartient aux autres. ∎

Le lemme donne un témoin gratuit : $N_1 - N_0 \geq b\,n$ à chaque passe (un compteur qui ne verrait pas le disque, ou une lecture servie par un cache, le ferait tomber) ; il tient à toutes les passes de toutes les séances [mes.]. Ce chemin est indépendant de celui du lot R, qui intègre le débit formaté.

Listing 4.4 : Le cumul brut (`moteur/bins/banc-chemin/src/disque.rs`, lignes 46 à 63, commit `1cca7f3`). Le compteur est ajouté par son nom anglais (`PdhAddEnglishCounterW`) : ce Windows est en français.

```
    /// Octets lus sur le disque par toute la machine depuis le démarrage.
    pub fn cumul(&self) -> Result<u64, String> {
        // SAFETY : requête et compteur ouverts par `ouvrir`.
        unsafe {
            let e = PdhCollectQueryData(self.requete);
            if e != 0 {
                return Err(format!("PdhCollectQueryData : {e:#x}"));
            }
            let mut brut: PDH_RAW_COUNTER = std::mem::zeroed();
            let mut sorte = 0u32;
            let e = PdhGetRawCounterValue(self.compteur, &mut sorte, &mut brut);
            if e != 0 || brut.CStatus != 0 {
                return Err(format!("PdhGetRawCounterValue : {e:#x}, statut {:#x}"
    , brut.CStatus));
            }
            Ok(brut.FirstValue as u64)
        }
    }
}
```

Définition 4.9 (Passe propre)**.** Règle 1 : une passe est propre si $E/T \leq 0{,}1 \times 2{,}58 = 0{,}26$ Go/s, la charge étrangère restant dans le bruit déjà admis. Règle 2, ajoutée après la séance 1 et avant la séance 2 : de plus, les autres programmes occupent au plus 40 % du processeur pendant la passe (source : 01 § 2.3, gros blocs reproductibles à quelques pour cent sous 35 à 40 % d'occupation). Un verdict ne se rend que sur au moins deux passes A propres ; sinon la série est « non jugée », jamais « tombée ».

Ce n'est pas une correction du débit (on ne retranche rien) : c'est un tri.

4.7.2 Une séance sous charge, une séance au calme

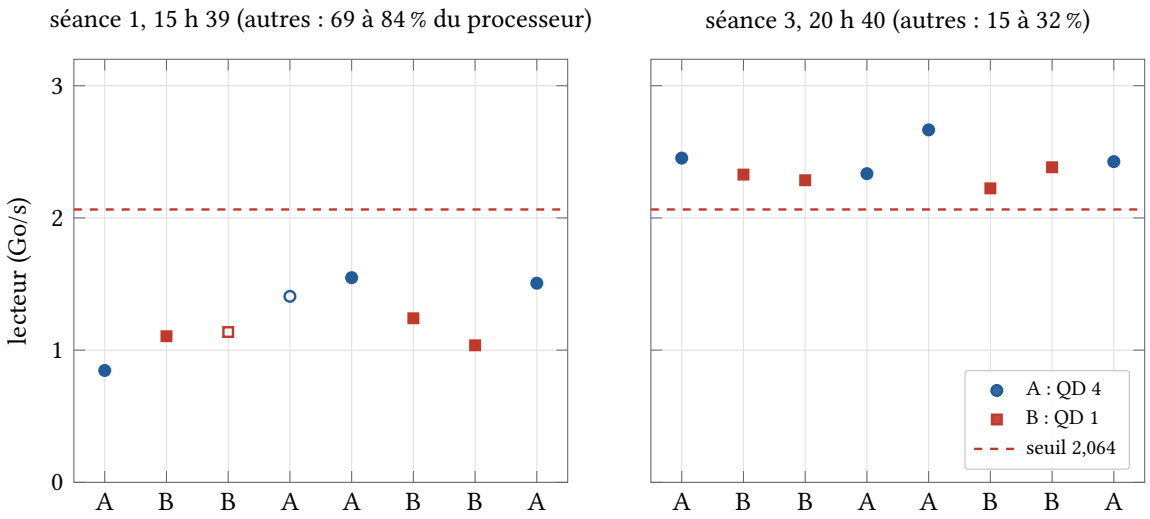


Figure 4.8 : E1, chaque passe du lecteur (128 lectures de 12 Mio à décalages tirés) [mes.]. Plein : passe propre ; creux : sale (règle 1). À gauche, la séance 1 (15 h 39) : médiane des A propres **1,506 Go/s**, sous le seuil ; le processeur était occupé à 69–84 % par d'autres lots (juge f64, traceur, antivirus), et BitLocker déchiffre chaque octet sur ce même processeur. À droite, la séance 3 (20 h 40, règle 2) : médiane **2,439 Go/s**, quatre passes A propres sur quatre : **E1 tient**.

Données : `mesures/moteur/E/csv/e1-2026-10-15-seance1-lecteur-passes.csv`, `e1-2026-10-15-seance3-lecteur-passes.csv` (`t2c-e1-s1.csv`, `t2c-e1-s3.csv`).

La séance 1 tombe sous la règle déclarée, et son verdict reste publié tel quel (figure 4.8) : il dit que *sous cette charge* le lecteur ne tient pas 2,06 Go/s. Ce que la règle 1 ne regardait pas, c'est le processeur. La séance 3, au calme, tient : 2,453 / 2,335 / 2,666 / 2,426 Go/s en QD 4, 2,328 / 2,285 / 2,224 / 2,383 en QD 1 [mes.]. Le bruit miroir des A vaut 13,4 %, celui des B 2,7 % ; QD 4 et QD 1 ne se distinguent pas ($5{,}6\,\% < 2 \times 13{,}4\,\%$).

4.7.3 La loi de Little, second chemin du débit

Théorème 4.10 (Little, 1961)**.** *Dans un système stable, le nombre moyen de clients présents $\bar{N}$, leur débit d'arrivée λ et leur temps de séjour moyen $\bar{R}$ vérifient $\bar{N} = \lambda \bar{R}$ [3].*

Corollaire 4.11 (Le débit d'un lecteur à profondeur fixe)**.** *Un lecteur qui garde en permanence q lectures de b octets en vol, chacune de durée moyenne $\bar{R}$ (de la remise au système à la récolte), lit $X = q\,b/\bar{R}$ octets par seconde.*

Démonstration. Les clients sont les lectures en vol : $\bar{N} = q$. Leur débit est $\lambda = X/b$ lectures par seconde. Le théorème 4.10 donne $q = (X/b)\,\bar{R}$. ∎

Le journal publie la latence *médiane* d'une lecture, non sa moyenne ; or une latence à queue longue a une moyenne au-dessus de sa médiane, et $q\,b/R_{\text{médiane}}$ surestime alors le débit. La figure 4.9 confronte les deux chemins.

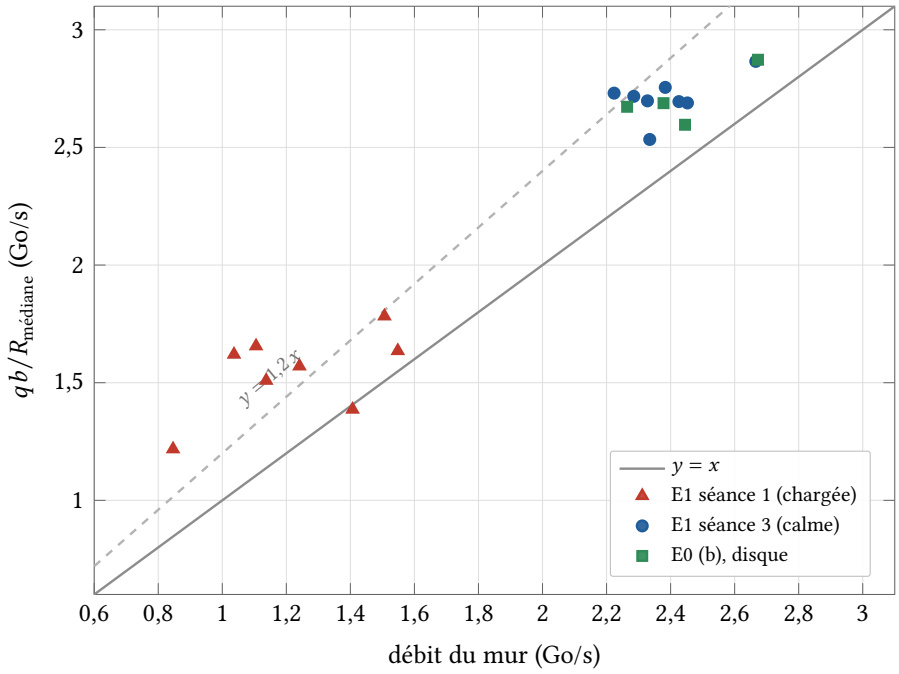


Figure 4.9 : Second chemin du débit du disque : $q\,b/R_{\text{médiane}}$ contre le débit du mur, chaque passe [mes.]. Au calme, Little surestime de 8 à 23 % (rapport 1,08 à 1,23 en séance 3) : la latence médiane n'est pas la moyenne, la queue est longue (p90 de 27,8 ms pour une médiane de 18,7). Sous charge (séance 1), la dispersion monte jusqu'à 1,56.

Données : `e1-*-lecteur-passes.csv` et `2026-10-15-seance3-b-disque.csv` (colonnes `go_s`, `go_s_little`), dans `t2c-little.csv`.

Ce que Little dit de plus : le disque sert une lecture de 12 Mio à la fois. En séance 3, la latence médiane vaut 18,7 ms à QD 4 (passes A : 18 717, 19 864, 17 563, 18 678 µs) et 4,62 ms à QD 1 (passes B : 4 664, 4 631, 4 608, 4 567 µs) [mes.] ; le rapport vaut 18,7/4,62 = 4,05 [calc.]. Quatre lectures en vol attendent donc chacune environ quatre fois plus longtemps qu'une seule : si le disque sert un débit μ quel que soit q, alors $\bar{R}(q) \approx q\,b/\mu$ et le corollaire donne $X = \mu$, indépendant de q. C'est ce qu'on observe (QD 4 et QD 1 indiscernables). [hyp.] Une lecture de 12 Mio occupe à elle seule tout le parallélisme interne du NVMe (ses canaux) ; ajouter des lectures en vol ne fait qu'allonger la file. Le chronogramme du moteur (section 4.10) montrera la même chose sur le chemin réel. Inversement, à QD 1 Little surestime de 16 à 23 % : entre deux lectures, le disque chôme pendant la récolte et la remise suivante.

4.8 De bout en bout : l'étape E2

E2 fait tout le chemin, sans modèle : chaque tronçon du fichier lu sans tampon en une requête de `longueur0` (sa partie de niveau 0, § 4.7 du format) dans la fenêtre épinglée, copié sur le flux de copie vers un tampon d'arrivée en VRAM, attendu par événement, décodé par les deux noyaux du lot K sur le flux de décodage dans un emplacement de disposition SoA (chapitre 5). Profondeur 4 (le k de gpt-oss), ordre : une permutation des 768 experts tirée d'une graine publiée, nouvelle à chaque passe.

4.8.1 Le seuil, posé avant la mesure

Proposition 4.12 (Le seuil d'E2 en experts servis)**.** *Si le disque livre B_d octets de tronçons par seconde et qu'un tronçon de b octets rend un expert de o octets d'origine, le débit d'experts servis, compté à la taille d'origine, vaut $X_{\text{servi}} = B_d \cdot o/\bar{b} = B_d \cdot r$, avec $r = o/\bar{b}$ le rapport de compression moyen.*

Démonstration. En une seconde le disque livre $B_d/\bar{b}$ tronçons, chacun décodé en un expert de o octets. ■

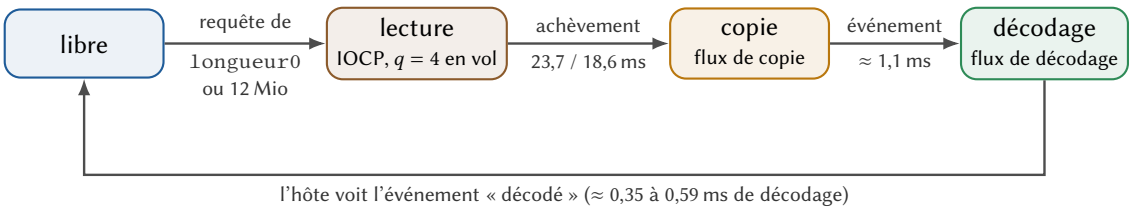
La spécification prend $B_d = 2{,}58$ Go/s, $r(F1) = 13\,253\,760/11\,990\,368 = 1{,}105367$ (§ 4.14) et la règle $0{,}8 = 1 - 2 \times 10\,\%$: seuil $0{,}8 \times 2{,}58 \times 1{,}105367 =$ **2,28148** Go/s d'experts servis [calc.]. Le fichier réel a $\bar{b} = 12\,013\,664$ o, soit $r = 1{,}103224$; le même seuil demande alors $2{,}28148/1{,}103224 = 2{,}068$ Go/s de tronçons, 0,2 % au-dessus du seuil d'E1 (2,064) : les deux étapes jugent presque le même disque, l'une seul, l'autre au bout du chemin.

Théorème 4.13 (Débit d'une chaîne d'étages et nombre d'emplacements)**.** *Soit une chaîne de trois étages que chaque tronçon traverse dans l'ordre : le disque (débit B_d à q lectures en vol), le moteur de copie (une copie à la fois, $t_c = b/B_c$), le flux de décodage (un expert à la fois, t_k). En régime établi :*

1. *$X \le \min(B_d, B_c, b/t_k)$ octets de tronçons par seconde ;*
2. *pour que le disque, s'il est l'étage le plus lent, travaille à q lectures en vol, il faut $S \ge q + \lceil N_{\text{hors}} \rceil$ emplacements, où $N_{\text{hors}} = (X/b)\,(t_c + t_k + \delta)$ et δ est le délai de l'hôte entre la fin d'un décodage et la remise en service de l'emplacement.*

Démonstration. (1) Un étage qui sert chaque objet ne peut en sortir plus vite que sa capacité ; la chaîne sort au rythme du plus lent. (2) Un emplacement est tenu de la soumission de sa lecture à la fin de son décodage plus δ. Hors du disque, il séjourne $t_c + t_k + \delta$ (plus une attente éventuelle) ; par le théorème 4.10 appliqué à cette partie du système, le nombre moyen d'emplacements occupés hors du disque vaut $N_{\text{hors}} = (X/b)(t_c + t_k + \delta)$. Le disque ne garde q lectures en vol que si un emplacement libre attend chaque achèvement : il faut $S - N_{\text{hors}} \ge q$ en moyenne. ■

Le calcul. Avec $X/b \approx 2{,}44{\cdot}10^9/1{,}2{\cdot}10^7 \approx 203$ tronçons par seconde, $t_c \approx 0{,}96$ ms, $t_k \approx 0{,}35$ à $0{,}40$ ms, et δ de 0,1 à 0,5 ms (sondage des événements, sommeil de 100 µs), $N_{\text{hors}} \approx 203 \times 1{,}8 \cdot 10^{-3} \approx 0{,}37$ [calc.]. Il faut donc $S \ge q + 1 = 5$ emplacements en moyenne ; on en prend $q + 2 = 6$ pour les fluctuations, afin qu'une lecture lente (le p90) ne prive pas les autres de place. Fenêtre épinglée : $6 \times 12\,144\,640 = 72{,}9$ Mo ; VRAM : $6 \times (12\,144\,640 + 13\,254\,144) = 152{,}4$ Mo (figure 4.10).



Un emplacement = tranche épinglée 12 144 640 o + tampon d'arrivée en VRAM 12 144 640 o + emplacement SoA 13 254 144 o + quatre événements. Six emplacements ($S = q + 2$) : épinglé $6 \times 12\,144\,640 = 72{,}9$ Mo ; VRAM $6 \times (12\,144\,640 + 13\,254\,144) = 152{,}4$ Mo [calc.]. Compteurs de contrôle : 768×16 o en VRAM, relus une fois en fin de passe.

Figure 4.10 : La fenêtre d'E2 et la vie d'un emplacement. Les durées sont les médianes mesurées en E2 (lecture de `longueur0` / de 12 Mio ; copie ; décodage) [mes.]. Dans la passe témoin, l'emplacement n'est libéré qu'après son retour vers l'hôte et ses deux empreintes.

Données : `mesures/moteur/E/csv/e2-2026-10-15-seance1-passes.csv`, `e2-2026-10-15-v12-passes.csv`.

Le point (1) du théorème 4.13 donne un témoin qu'aucune mesure juste ne peut violer : **la médiane des passes A ne doit pas dépasser le plus petit étage de la séance de plus de 10 %**. Au-delà, soit un étage est mal mesuré, soit la passe A ne fait pas ce qu'elle dit (lecture servie par un cache, horloge arrêtée trop tôt, expert compté sans décodage). Sa cassure naturelle est la passe B elle-même : même suite de lectures, même fenêtre, même lecteur, sans copie ni décodage.

4.8.2 Le transport exact, et ce que chaque contrôle voit

C2. Dans la passe témoin, chaque emplacement redescend vers l'hôte ; l'expert est remis dans la disposition d'origine et son empreinte comparée au répertoire ; l'empreinte des octets utiles arrivés dans la fenêtre épinglée est comparée à celle de la partie ; les compteurs de contrôle de la carte (voies hors de L, format, pointeurs) doivent être nuls. **768 experts sur 768**, à chacune des 8 passes témoin de la soirée ; et sur le 120b, 4 608 **sur** 4 608, aux deux passes témoin [mes.].

Proposition 4.14 (Ce que chaque contrôle peut voir d'une corruption (D14))**.** *Soit un bit retourné dans les octets utiles d'une partie. (i) L'empreinte64 des octets arrivés diffère de celle du répertoire, sauf collision du préfixe de 64 bits de SHA-256, de probabilité 2^{-64} par expert si SHA-256 se comporte comme une fonction aléatoire. (ii) L'empreinte de l'expert d'origine refaite depuis l'emplacement ne voit que les corruptions* qui changent la sortie *: un bit des flux bruts (médianes, biais) change exactement un octet d'échelle par bloc concerné ou un biais ; un bit d'un mot ou d'un état rANS change la suite des états, donc presque toujours les symboles ; mais un champ que le décodeur ne lit pas pour produire la sortie (couche et expert de l'en-tête de partie) ne change rien. (iii) Les compteurs de la carte, seul contrôle du mode de production, voient les fautes de format et les voies qui ne finissent pas sur L, mais ni les flux bruts ni l'identité de la partie.*

Démonstration. (i) est l'hypothèse sur SHA-256. (ii) : les médianes et les biais sont recopiés ou ajoutés sans transformation entropique (une médiane entre dans $e = (\text{méd} + \delta - 128) \bmod 256$ de chaque bloc de sa ligne) ; un état ou un mot faux fait diverger la récurrence $x \leftarrow f\,(x \gg 12) + b$; l'identité de l'en-tête n'est lue par aucun noyau. (iii) : les noyaux ne comptent que ce qu'ils vérifient (magie, sortes, en-têtes de flux, retour à L, pointeur final). ■

La cassure déclarée place un bit dans chaque famille, un expert par bit, dans une copie du fichier de 11,16 Go (graine 20261015). Le tableau 4.3 en est la table des détections.

La dernière ligne est un vrai trou du mode de production d'alors : un répertoire qui pointerait vers le tronçon d'un autre expert serait décodé sans erreur. La garde d'intégrité ajoutée par la ronde de corrections après l'épreuve indépendante le ferme : elle compare l'empreinte des octets utiles de chaque tronçon lu à celle du répertoire, et l'en-tête de partie (identité comprise) fait partie de ces octets [calc.] (`moteur/0-pour-le-memoire.md`, proposition 8).

La passe témoin ne se chronomètre pas (D15). Chaque emplacement de 13 254 144 o redescend par le moteur de copie unique (proposition 4.4), et l'hôte hache environ 25 Mo par expert par SHA-256 sur un fil, à 1,5 à 2 Go/s : 12 à 17 ms par expert, trois fois le temps disque d'un tronçon [calc.]. Son débit est publié pour mémoire, jamais jugé ; le seuil se juge sur les passes de production, où rien ne redescend que les compteurs, en fin de passe (12 288 o).

4.8.3 Le débit : le verdict tombe, et le témoin dit pourquoi

Telle que spécifiée, E2 **tombe** : passes A à 2,120 / 2,092 / 2,186 / 2,215 Go/s servis, médiane **2,153 < 2,2815**, 5,6 % sous le seuil ; 2,135 dans la série N V V N [mes.]. Les huit passes étaient propres (processeur des autres 6 à 12 %, octets étrangers ≤ 0,001 Go/s). Le verdict est celui-là.

Table 4.3 : E2 : la table des détections, contrôle par contrôle [mes.]. ● : le contrôle voit la corruption. Partie : empreinte des octets utiles arrivés dans la fenêtre ; origine : empreinte de l'expert refaite depuis l'emplacement décodé ; compteurs : compteurs de contrôle de la carte. **12 détectés sur 12, aucun en trop** (un expert est « détecté » si un contrôle du témoin tombe). Le mode de production en voit 9 : ni les médianes, ni les biais, ni l'identité de la partie, comme la proposition 4.14 le prédisait.

			passe témoin			
région du bit	couche	expert	partie	origine	compteurs	production
MEDIANES	23	4	●	●		
BIAIS	21	24	●	●		
DELTA, débuts de tuile	16	20	●	●	●	●
DELTA, états	14	28	●	●	●	●
DELTA, mots	18	30	●	●	●	●
CODES_CTX, débuts	4	25	●	●	●	●
CODES_CTX, états	11	20	●	●	●	●
CODES_CTX, mots	2	19	●	●	●	●
CODES_CTX, en-tête (P)	12	8	●	●	●	●
table des flux (sorte)	16	18	●	●	●	●
en-tête de partie, magie	19	24	●	●	●	●
en-tête de partie, identité	19	22	●			
total			12	11	9	9

Données : `mesures/moteur/E/csv/e2-2026-10-15-seance1-cassure.csv`, `journaux/e2-2026-10-15-seance1/resume.json`.

Mais le témoin de la chaîne tient, et il localise le défaut (figure 4.12) : la médiane des A vaut 1,951 Go/s de tronçons, le disque seul (B) 1,921, la DMA 12,66, le décodage 34,27 (0,350 ms par expert) ; $A/\min = 1{,}016 \leq 1{,}10$ [mes.]. A et B ne diffèrent pas (1,6 % pour un bruit de 1,6 %). **Le chemin ne coûte rien au disque ; c'est le disque lui-même qui lit ce fichier à 1,92 Go/s**, contre 2,44 pour le GGUF 34 minutes plus tôt. La DMA a 6,5 fois de marge, le décodage 18 fois.

La figure 4.13 montre le chemin vivre : chaque tronçon est une ligne, de sa soumission au disque à la fin de son décodage. La copie (1,0 ms environ) et le décodage (0,35 ms) tiennent dans l'ombre des quatre lectures en vol, et la pente de l'escalier n'est que celle du disque.

Little sur le chemin. À `longueur0`, $q\bar{b}/R = 4 \times 12\,013\,664/23{,}7 \cdot 10^{-3} = 2{,}03$ Go/s prédits contre 1,95 mesurés (rapport 1,04) ; en 12 Mio, $4 \times 12\,582\,912/18{,}6 \cdot 10^{-3} = 2{,}71$ contre 2,69 (1,01) [calc.]. Sur le chemin réel, la latence est plus serrée qu'au lecteur (p90 de 24 à 32 ms) et Little tombe presque juste.

4.9 La découverte : deux régimes selon la seule longueur

Le disque lisait le GGUF à 2,44 Go/s, et le même disque lisait le `.trc` à 1,92 au bout du chemin. Pourquoi ? Chaque hypothèse a été déclarée avant la mesure qui la départage, avec sa règle (deux fichiers ou deux réglages diffèrent si l'écart des médianes dépasse deux fois le plus grand bruit miroir et 3 %).

H3a est la seule qui tient : **pour moins d'octets, la lecture est 32 % plus lente**. Les décalages étant tirés sur 4 096 dans les deux cas, ni le début ni la fin n'ont à être alignés plus finement : c'est la longueur qui compte. Le premier balayage tue l'idée d'un grain (une longueur de 11 Mio rapide

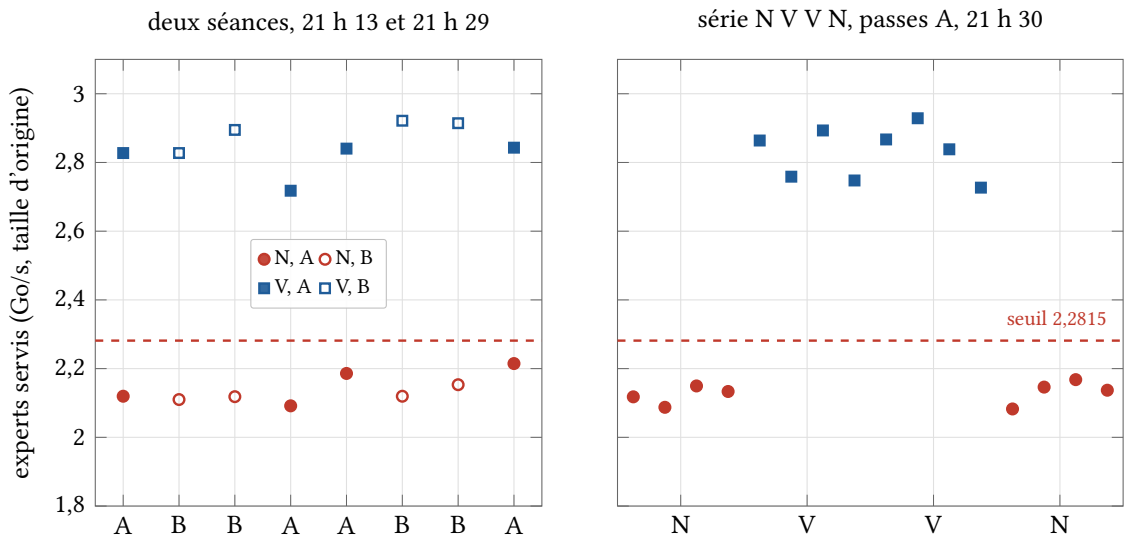


Figure 4.11 : E2, experts servis par passe (Go/s, comptés à la taille d'origine) [mes.]. À gauche, la séance de 21 h 13 telle que spécifiée (N : lecture de `longueur0`) et la variante de 21 h 29 (V : lectures de 12 Mio) ; A : production ; B : disque seul, mêmes lectures, compté de même. À droite, la série N V V N de 21 h 30, passes A : V / N = ×1,335 (médianes des huit passes A de chaque lettre : 2,851 et 2,135 Go/s), soit +33,5 % pour un bruit miroir de 1,5 % au plus.

Données : `mesures/moteur/E/csv/e2-2026-10-15-seance1-passes.csv`, `e2-2026-10-15-v12-passes.csv`, `e2-nvvn-2026-10-15-nvvn.csv` (`t2c-e2-n.csv`, `t2c-e2-v.csv`, `t2c-nvvn.csv`).

entre deux lentes) ; le second balayage, déclaré avant dans `balayage_longueurs.py`, mesure 23 longueurs de 8 à 13 Mio par pas de 0,25 Mio (plus 12 013 568 et 12 144 640), en deux répliques d'ordres tirés, quatre passes chacune (figure 4.14).

Deux faits rendent la carte crédible : les deux ordres tirés la dessinent pareil (une dérive dans le temps aurait dessiné deux courbes différentes), et la latence suit le débit (panneau du bas : 1,5 fois plus longue dans le régime lent). **Toutes les longueurs de tronçon tombent dans une bande lente ; 12 Mio est rapide** (3,07 Go/s aux deux répliques).

Ce que la mesure n'établit pas : le mécanisme. [hyp.] La découpe d'une requête en transferts par le pilote du contrôleur (VMD, `iaStorVD`) ou par le filtre de BitLocker (`fvevol`), dont la dernière part tomberait mal pour certaines longueurs. Rien ne se voit d'ici sans traçage du noyau (ETW). C'est un fait mesuré et inexpliqué, publié comme tel.

Proposition 4.15 (Experts servis avec une longueur de lecture fixe)**.** *Si chaque tronçon est lu en une requête de longueur fixe $\ell \geq b_{\max}$ (le plus grand tronçon), au débit $B(\ell)$ de cette longueur, alors le débit d'experts servis vaut*

$$X_{\text{servi}} = B(\ell) \cdot \frac{\bar{b}}{\ell} \cdot r = B(\ell) \cdot \frac{o}{\ell}.$$

Démonstration. Le disque livre $B(\ell)/\ell$ requêtes par seconde, chacune un tronçon entier (puisque $\ell \geq b_{\max}$) ; les octets en trop appartiennent au tronçon suivant, ne sont ni copiés ni décodés. Chaque requête rend un expert de o octets. ■

Pour $\ell = 12$ Mio, $o/\ell = 13\,253\,760/12\,582\,912 = 1{,}0533$; mesuré en E2-V12, $B(\ell) \approx 2{,}69$ Go/s aux décalages des tronçons, d'où 2,83 Go/s servis prédits, **2,834 et 2,851 mesurés** [calc.]. La lecture en trop

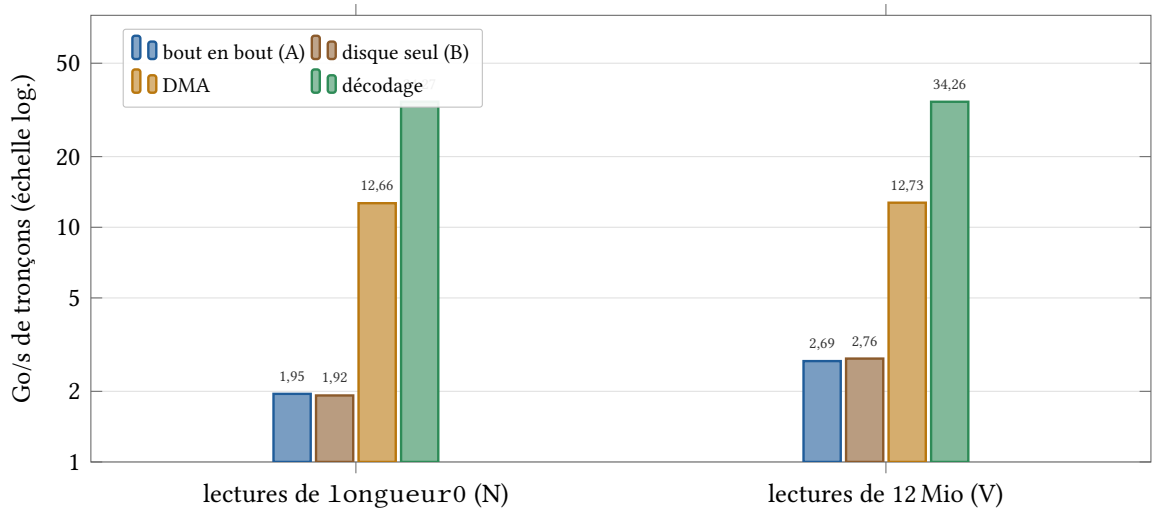


Figure 4.12 : E2 : le bout en bout et ses trois étages mesurés dans la même séance, en Go/s de tronçons lus (échelle logarithmique) [mes.]. Le disque est le goulot d'un ordre de grandeur ; A colle au disque seul, sous la borne du théorème 4.13.

Données : `e2-2026-10-15-seance1-etages.csv`, `e2-2026-10-15-v12-etages.csv` et les passes (`t2c-etages.csv`).

(au plus 0,75 Mio par tronçon, 4,7 % en moyenne) est payée, et rendue sept fois par le changement de régime. ~~prédiction : 3,07 × 12 013 664/12 582 912 × 1,103 ≈ 3,2 Go/s servis~~ La prédiction écrite avant la variante (3,2, avec le $B(\ell) = 3{,}07$ du lecteur) était fausse de 12 % : aux décalages des tronçons, le régime rapide ne donne que 2,7 Go/s ; cet écart-là n'est pas expliqué non plus. ~~lecture d'un tronçon en 12 Mio : ≈ 16 ms~~ La note du lot E dessinait la lecture d'un tronçon en 12 Mio à environ 16 ms ; c'est la latence du lecteur (S M M S), pas celle du chemin, où la médiane vaut 18,6 ms (`e2-2026-10-15-v12-passes.csv`).

Le gain, jugé par l'ABBA. Pour ne pas comparer deux séances à 14 minutes l'une de l'autre, la série N V V N a été déclarée avant : V / N = ×1,335, soit +33,5 % : c'est le rapport des médianes des huit passes A de chaque lettre, V 2,851 et N 2,135 Go/s servis [calc.] (`e2-nvvn-2026-10-15-nvvn.csv`) ; les médianes par passe, N 2,126 et 2,142, V 2,811 et 2,853, donnent par leurs moyennes 2,832/2,134 = 1,327, pas le chiffre publié. ~~Écart de 25 %~~ : c'est $1 - 1/1{,}335$, N sous V, non le gain de V. Le bruit miroir vaut au plus 1,5 % ; 16 passes A propres sur 16 ; C2 tient aux quatre [mes.]. Sur le 120b (une séance, non jugée) : N 2,063, V 2,877, ×1,39 ; décodage 0,353 ms par expert, DMA 11,07 Go/s ; la prédiction déclarée ($N \approx 2{,}1$, $V \approx 2{,}8$) tient [mes.] (`journaux/e2-120b-2026-10-15/`).

La proposition à l'arbitre. Arrondir chaque requête à une *longueur rapide calibrée sur la machine* (ici 12 Mio couvre tous les tronçons du 20b et du 120b) ; aucun octet du format ne change ; coût : une fenêtre épinglée de 12 Mio par tampon au lieu de 11,58 (+3,6 %) et au plus 0,75 Mio lus en trop par tronçon, déjà comptés dans le ×1,335. Autre voie, non retenue : bourrer les tronçons à une longueur rapide au codeur (+4,7 % de fichier, et la longueur rapide dépend de la machine). Le calibrage (`balayage_longueurs.py`) deviendrait un pas d'installation du moteur. La décision n'est pas prise à l'heure où ce chapitre s'écrit.

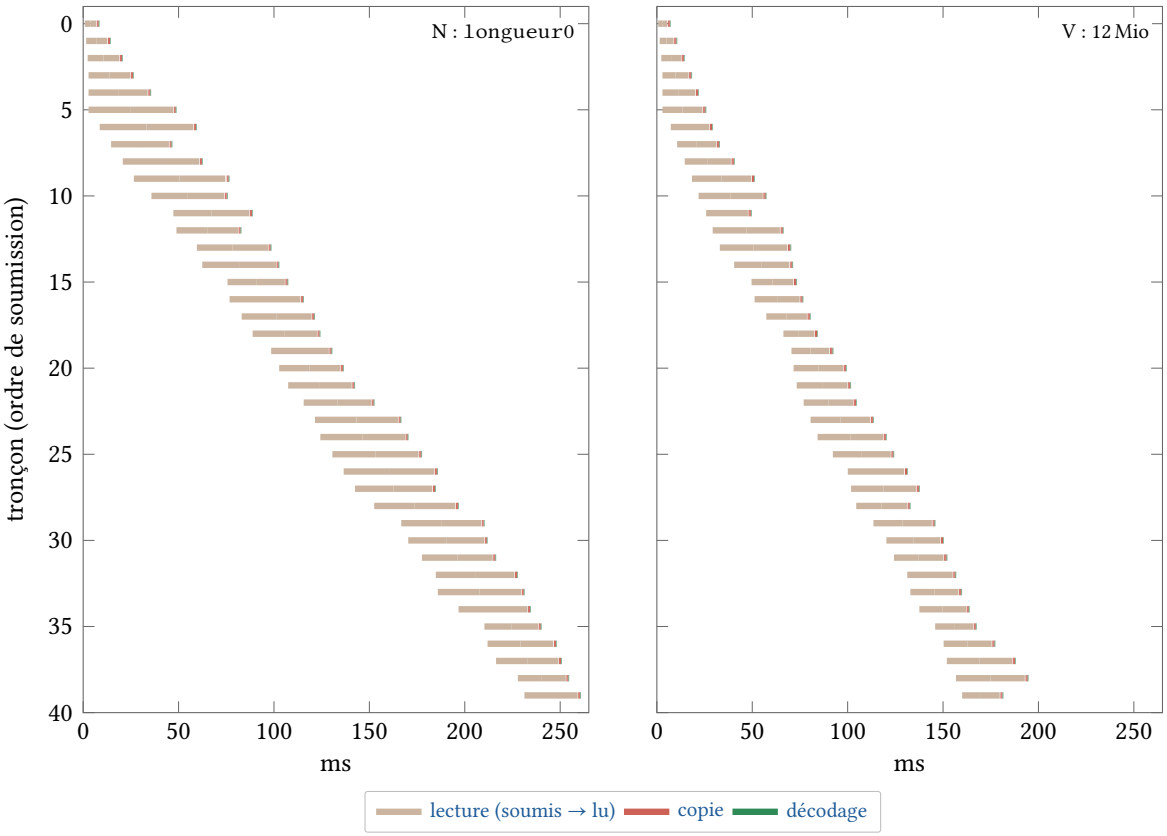


Figure 4.13 : Chronogramme des 40 premiers tronçons d'une passe A (second tour), même ordre tiré dans les deux séances [mes.]. La lecture d'un tronçon (soumis → lu) dure 23,5 ms en médiane à `longueur0`, 18,6 ms en 12 Mio ; les 40 tronçons finissent à 261 ms contre 182, les 768 à 4 657 ms contre 3 584. La copie et le décodage sont les petits segments au bout de chaque lecture.

Données : `mesures/moteur/E/journaux/e2-2026-10-15-seance1/chrono-1/chrono-A-t1-r0.csv`, `e2-2026-10-15-v12/chrono-v12582912-1/chrono-A-t1-r0.csv`.

4.10 Le chemin dans le moteur

Le moteur du lot O (`troncons`) emprunte ce chemin tel quel : même lecteur, même mémoire épinglée, même boîte aux lettres, même décodeur. Son option `--chronogramme` date chaque défaut (soumission, lecture, copie, décodage), le routeur et la fin de chaque couche.

La figure 4.15 retrouve sur le chemin réel ce que Little disait du lecteur (section 4.7) : quatre lectures confiées ensemble ne finissent pas ensemble, elles finissent à la file, espacées de 3 à 6 ms. Le moteur souffre en plus d'une file courte : la couche ne connaît ses défauts qu'après son routeur, et n'en a en moyenne que 1,6 (59 défauts par jeton sur 36 couches, séance O4) [mes.]. Le disque seul, en tronçons tirés à profondeur 4, lit 1,848 Go/s ; le moteur tire 0,707 Go en 457 ms par jeton, soit 1,55 Go/s utiles [calc.]. Ce qui limite J1, c'est la profondeur de file par couche, que le préchargement devait remplir (tué par la mesure, −11 % : 31 % des lectures anticipées étaient fausses sur un disque qui limite déjà ; voir le chapitre du moteur).

Les lectures de 12 Mio dans le moteur. La série L12 du lot O (A : J1 avec préremplissage par flux ; B : la même, `--longueur-lecture 12582912`), déclarée avant, donne la figure 4.16. En génération, B fait +30 % en médiane (2,179 et 2,182 contre 2,613 et 3,073 jetons/s), chaque passe B au-dessus de chaque passe A ; mais les deux passes B diffèrent de 0,46 jeton/s, soit un bruit $\beta = 0{,}46/2{,}843 = 16{,}2\,\%$

Table 4.4 : Les hypothèses sur la lenteur d'E2, dans l'ordre où elles ont été déclarées, le 2026-10-15 entre 21 h 16 et 21 h 22 [mes.]. G : le GGUF ; T : le `.trc` ; L : le lecteur d'E1 ; C : le chemin d'E2.

hypothèse	mesure	verdict
H1, le fichier (fragments)	lecteur G T T G : G 2,571 et 2,591 ; **T 3,039 et 3,023** Go/s ; G a 2 737 fragments, T en a 33	tombe, à l'envers
H2, le disque ralenti après la copie de 11 Go	G reste à 2,6 ≥ 2,064, 1 min 30 s après E2	tombe
H2 bis, encore ralenti à 21 h 14	L C C L : L 3,04 à 3,09 ; C, disque seul, 1,93 à 2,04, adjacent et propre	tombe
H3, la manière dont E2 lit	même fenêtre : C reste 36 % sous L	tient
H3a, la longueur des lectures	S M M S, mêmes décalages : **S (**12 582 912 **o) 3,00 à 3,09** Go/s, latence 16,3 ms ; **M (**12 013 568 **o) 2,06 à 2,11**, latence 23,4 ms	**tient** : −32 %
« un grain d'alignement »	12 Mio $-g$: rapides $g \leq 128$ Kio et $g = 1$ Mio ; lentes $g = 256$ Kio, 512 Kio, 2 Mio	tombe : pas de grain

Données : `mesures/moteur/E/journaux/e2-diag-2026-10-15/` (`lecteur-*`, `lccl-*`, `smms-*`, `grain-*`, `extents-*.txt`), `moteur/REPRISE-E.md`.

par la définition du définition 8.1 (~~21 %~~ selon le lot O, qui rapportait l'écart à la moyenne de A), et la règle déclarée (gain > 2× bruit +3 %, ici 2 × 16,2 + 3 = 35,4 % contre +30,4 % [calc.]) **n'est pas franchie** en quatre passes : L12 n'est ni établi ni réfuté. Les textes des quatre passes sont égaux (les logits ne dépendent pas de la longueur des lectures).

Une observation, calculée ici sur les passes publiées, non déclarée avant : au préremplissage par flux, qui lit chaque expert demandé d'une couche à la file, A vaut 27,44 et 27,39 jetons/s (écart 0,2 %), B 34,55 et 34,33 (0,6 %) : +25,6 %, très au-delà de 2 × 0,6 % + 3 % = 4,3 % [calc.]. Le préremplissage, qui garde le disque plein, montre le gain du régime rapide presque entier (×1,26, contre ×1,335 sur E2) ; la génération, à file courte, le montre noyé dans sa variance. Ce n'est pas un verdict (le critère n'était pas celui-là) ; c'est ce qu'une série plus longue devra confirmer.

La garde sur le chemin. Depuis la ronde de corrections, chaque lecture achevée est hachée (empreinte64 de ses octets utiles) par quatre fils en parallèle de la copie et du décodage, et le jeton n'est rendu qu'après la vérification de toutes les lectures du pas. Le débit de l'empreinte est mesuré à 1,07 à 1,10 Go/s par fil ; au chemin critique ne reste que le hachage de la dernière lecture, au plus 12,25 Mo/1,07 Go/s ≈ 11,4 ms moins le travail de carte qui la suit ; mesuré : 5,2 à 5,4 ms d'attente par jeton, −1,6 % à débit de disque égal [mes.] (`mesures/moteur/O/corrections/csv/vitesse-2026-10-16-passes.csv` ; la note du lot O, § 13.2, écrivait ~~5,2 à 5,3 ms~~ et ~~−1,7 %~~, section 8.5.5). Avec des lectures de 12 Mio, la garde ne hache que les octets utiles : les octets en trop ne lui coûtent rien. La combinaison L12 et garde n'a pas été mesurée.

4.11 Ce que le chapitre a barré

- ~~Ce qui reste : E2, le banc de bout en bout, qui attend le décodeur du lot K et le format du lot F.~~ E2 est fait (2026-10-15, 21 h 13 à 21 h 35).
- ~~Modèle D2 : coût indépendant de la charge.~~ Le témoin du modèle tombe ; D2 bis (proposition 4.7) le remplace.
- ~~rapports 3,6 à 4,8 selon la variante~~ : 3,6 à 4,8 pour quatre variantes, 8,7 pour `boite_sans_vidage`.

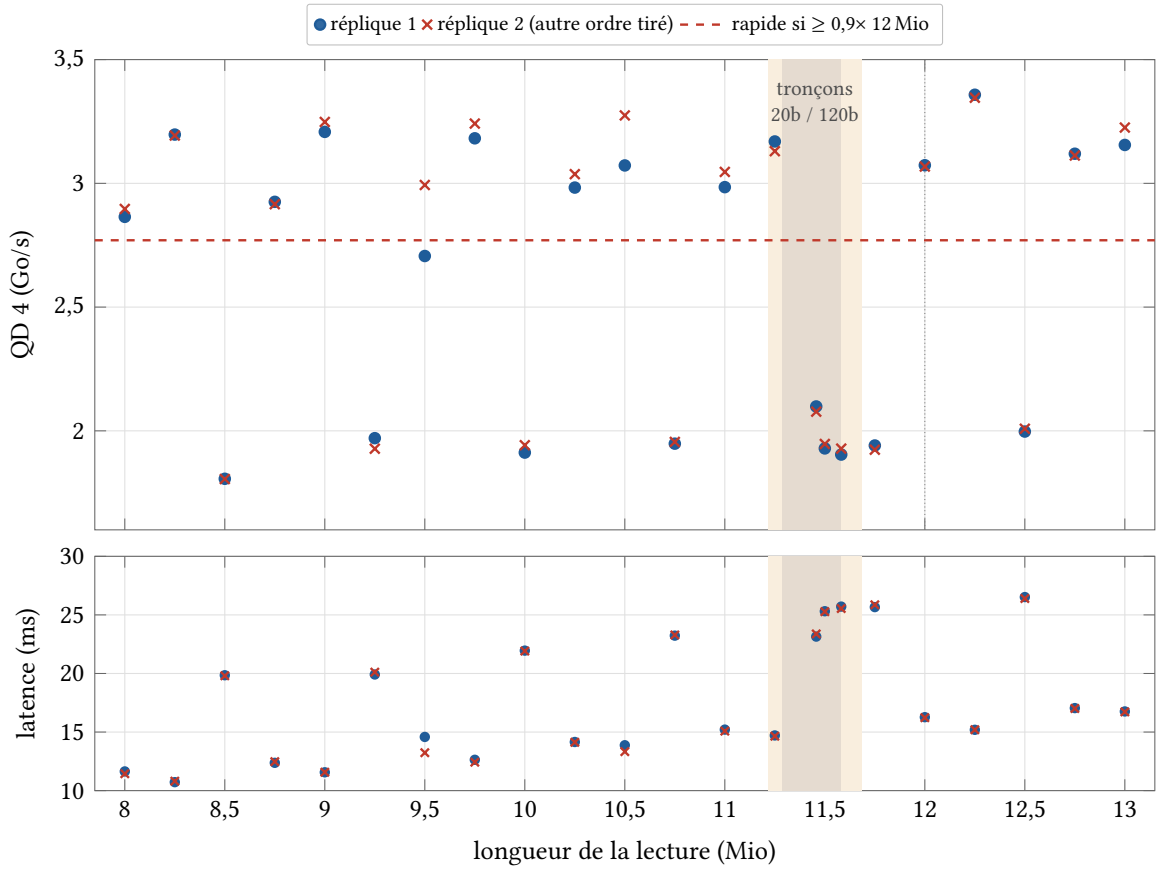


Figure 4.14 : Le débit d'une lecture sans tampon dépend de sa seule longueur, en deux régimes [mes.]. Lectures en QD 4 à décalages tirés sur le fichier du 20b ; deux répliques en ordres tirés. Rapides (2,87 à 3,36 Go/s) : 8 ; 8,25 ; 8,75 ; 9 ; 9,75 ; 10,25 ; 10,5 ; 11 ; 11,25 ; 12 ; 12,25 ; 12,75 ; 13 Mio. Lentes (1,80 à 2,10 Go/s, latence ×1,5) : 8,5 ; 9,25 ; 10 ; 10,75 ; 11,457 ; 11,5 ; 11,582 ; 11,75 ; 12,5 Mio. En bande : les longueurs des tronçons du 20b (11,29 à 11,58 Mio, foncée) et du 120b (11,22 à 11,68 Mio, claire). Les répliques s'accordent sur 22 longueurs sur 23 ; 9,5 Mio reste entre les deux (2,71 puis 2,99, pour un seuil de 2,77).

Données : `mesures/moteur/E/csv/e2-balayage-2026-10-15-longueurs.csv`, `journaux/e2-balayage-2026-10-15/`.

- ~~E2-V12 : ≈ 3,2 Go/s servis prédits~~ : 2,834 mesurés ; le régime rapide aux décalages des tronçons ne donne que 2,7 Go/s.
- ~~lecture d'un tronçon en 12 Mio : ≈ 16 ms~~ : 18,6 ms en médiane sur le chemin (16,3 ms est la latence du lecteur).

Ce qui pourrait être faux

- **La carte des longueurs est celle d'une machine, d'un fichier, d'une soirée** : 23 longueurs, deux répliques, le fichier du 20b. Le régime d'une longueur peut dépendre du pilote du contrôleur, de la version de BitLocker, du micrologiciel du NVMe, du remplissage du disque ; la carte ne se transporte pas, elle se recalibre. 9,5 Mio est resté entre les deux régimes ; une longueur rapide ici peut être lente sur le fichier du 120b (non mesuré).
- **Le seuil d'E2 repose sur 2,58 Go/s mesurés en blocs de 4 Mio**, une longueur que le balayage n'a pas couverte. Si 4 Mio est dans un régime rapide, le seuil compare un régime

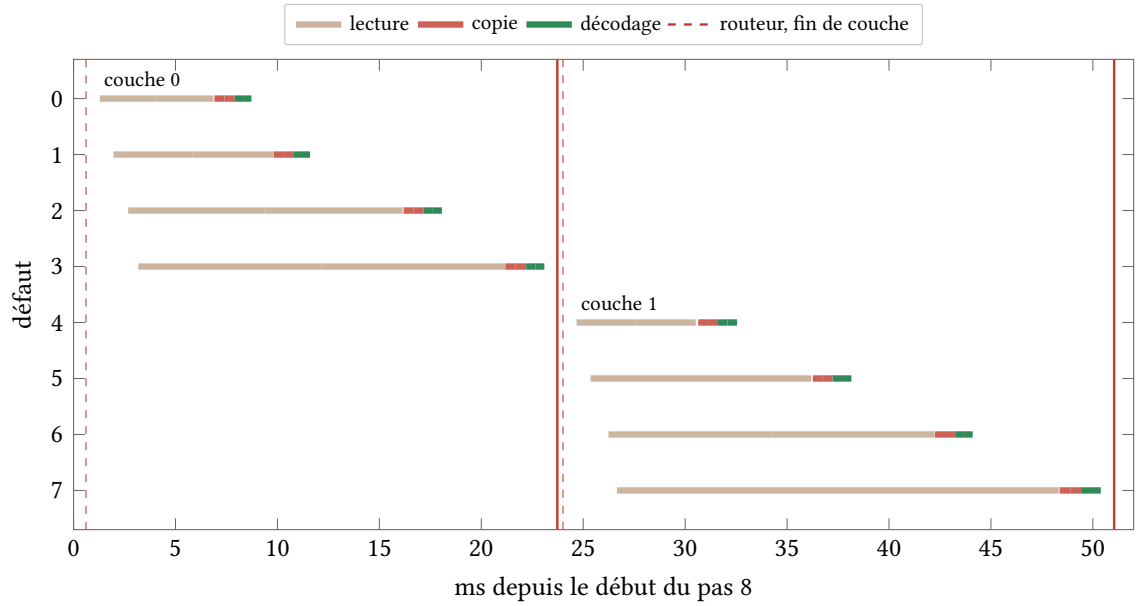


Figure 4.15 : Le moteur sur gpt-oss-120b, pas 8, couches 0 et 1, où les quatre experts manquent [mes.]. Couche 0 : le routeur finit à 0,61 ms ; les quatre lectures, soumises à 1,28 / 1,96 / 2,68 / 3,18 ms, finissent à 6,86 / 9,81 / 16,14 / 21,17 ms : **le disque les sert l'une après l'autre**, environ 5 ms pour 12 Mo chacune ; chaque copie (1,0 ms) et chaque décodage (0,85 ms) suivent leur lecture sans attendre les autres ; la couche finit à 23,73 ms.

Données : `mesures/moteur/O/csv/chronogramme-120b.csv` (8 pas, 1 048 défauts datés), extrait dans `t2c-couche.csv`.

rapide à une lecture dans un régime lent : le verdict « tombe » d'E2 dit d'abord cela. Il en va de même du plancher de C5 et des plafonds du Tome I.

- **E2-V12 lit des octets du tronçon suivant** : sans effet sur la justesse (C2 tient aux passes témoin en 12 Mio), mais un moteur qui lirait deux tronçons voisins lirait deux fois la frontière.
- **La passe témoin vérifie l'expert redescendu**, pas l'emplacement lu par la GEMV au moment du calcul : une écriture d'un autre noyau entre le décodage et la GEMV ne se verrait pas ici (le moteur l'exclut par l'épinglage du cache, lemme 4 du lot O).
- **Les essais E0 (c) simulent la couche**, ils ne la calculent pas : une carte occupée par une vraie GEMV pourrait retarder le lancement suivant plus qu'une attente active d'un seul fil. Les deux charges l'encadrent, sans le prouver pour toutes.
- **La machine est partagée** : le pilote refuse de mesurer quand un `llama-*` ou le traceur tourne et journalise l'état de la carte avant et après chaque passe, mais un programme qui n'est ni l'un ni l'autre échappe au test ; les octets étrangers et le processeur des autres le bornent.
- **E0 (a) ne tente que ce que la RAM disponible permet** : 11 Go « non tenté » n'est pas un refus du pilote.
- **La variante `boite_sans_vidage`** repose sur un comportement non documenté (l'ordonnancement matériel de WDDM, actif ici [21], soumet le lancement sans appel qui le pousse) ; elle est publiée, jamais retenue.
- **La lecture « le disque sert une lecture de 12 Mio à la fois »** est une interprétation de Little et du chronogramme, pas une mesure du contrôleur.

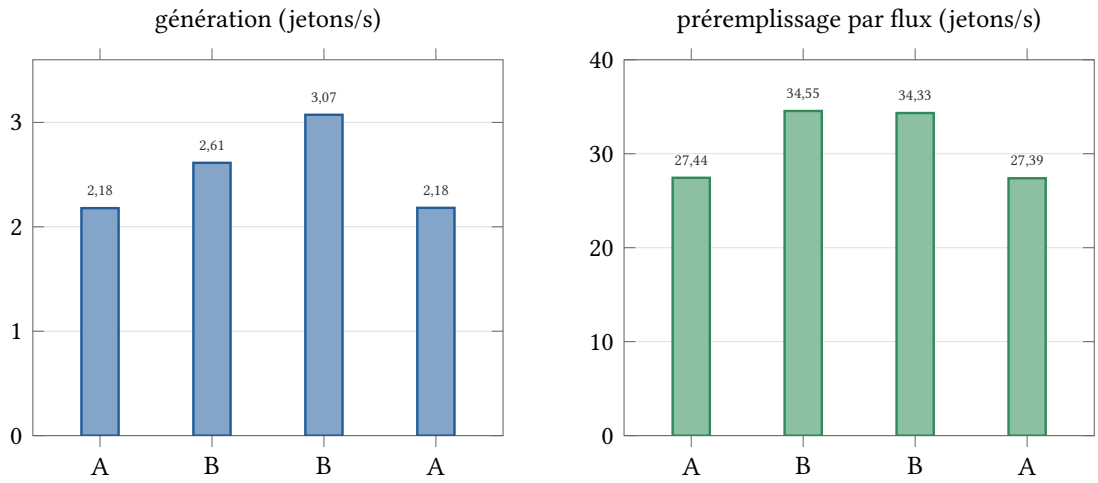


Figure 4.16 : Le moteur sur le 120b, série A B B A du 2026-10-16 (séance 2) [mes.]. A : J1 et préremplissage par flux ; B : la même avec des lectures de 12 Mio. À gauche, la génération, le critère déclaré : non franchi. À droite, le préremplissage par flux, qui n'était pas le critère déclaré.

Données : `mesures/moteur/O/csv/o-jetons-s.csv` (passes 2026-10-16-s2-15 à 18), dans `t2c-l12.csv`.

- **L'observation L12 au préremplissage** n'a pas été déclarée avant ; elle repose sur deux paires de passes.
- **`Lecteur::soumettre` et `copier_vers_hote` sont des fonctions sûres** qui confient au système ou au pilote une écriture asynchrone : le contrat d'ordre est documenté, pas vérifié par le type. `Contexte::memoire` rend (0, 0) sur une erreur du pilote.

D'où l'on part, où l'on va. Le chemin est construit et exact, de bout en bout, sur les deux modèles ; aucun de ses étages autres que le disque ne coûte quoi que ce soit au jeton (DMA ×6,5, décodage ×18, synchronisation ×23 sous leurs seuils). Ce qui reste est tout entier du côté du disque : la décision sur la longueur des lectures, son mécanisme, la carte du 120b, et une file plus profonde par couche, que seul un meilleur prédicteur que celui essayé pourra remplir.

Chapitre 5

Les noyaux sur la carte

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, l'étude 03 du Tome I avait un décodeur rANS entrelacé sur la carte (la v4, 80,8 et 82,1 Go/s sous CUDA en ABBA sur un grand flux) et un produit matrice-vecteur MXFP4 brut sous Vulkan (209,4 Go/s). La table choisie par voie, qui supprime le réassemblage des flux de classe, n'existait que sur le processeur (mesure F1 de l'arbitre). Aucun noyau de la passe avant de gpt-oss n'était écrit, et la référence f64 du lot R venait d'être validée contre llama.cpp. **Ce chapitre établit** le décodeur à table par voie sur la carte, démontré exact (le pas d'un warp rend exactement le décodage scalaire) et vérifié au bit sur les 4 608 experts de gpt-oss-120b et les 768 de gpt-oss-20b, par deux codeurs indépendants ; la GEMV MXFP4, exacte au bit sur entiers par construction et à $3 \cdot 10^{-7}$ d'une référence f64 sur poids réels, à 235 Go/s ; et les douze noyaux de la passe avant, chacun avec son algorithme, sa preuve d'exactitude ou sa borne d'erreur dérivée, son déterminisme au bit, son témoin et sa cassure. La chaîne entière de ces noyaux est à KL $3{,}0 \cdot 10^{-6}$ de la référence f64 quand le routage est imposé ; en routage libre, l'écart vient de bascules de routage aux marges sous 10^{-3}. Enfin, l'écart qui restait au moteur venait tout entier de l'arrondi f16 du cache KV. **Il reste** la décision de l'arbitre sur le repli du décodeur (DELTA en tuiles de 32 et décodage en lot), la cause de l'effet de première série de K1, la chaîne sur le 120b, l'attention en tuiles pour un long contexte, et le mode base du jalon J2. Sources : `moteur/K-pour-le-memoire.md`, `moteur/REPRISE-K.md`, `mesures/moteur/K/`, code `moteur/crates/trc-noyaux/` ; ronde de corrections : `moteur/O-pour-le-memoire.md` (partie III).

5.1 Ce que la carte doit faire

Sur la carte, un jeton traverse deux sortes de travail. Le *décodage* transforme un tronçon arrivé par DMA (chapitre 4) en un emplacement d'expert prêt à être lu ; le *calcul* enchaîne, couche après couche, les noyaux de la passe avant de gpt-oss. Les noyaux sont écrits en CUDA C++, compilés à la construction par `nvcc` 13.1 (`-arch=sm_89`, image cubin plus PTX), embarqués dans le binaire Rust et chargés par l'API du pilote : aucune bibliothèque de CUDA n'est livrée, aucune bibliothèque d'algèbre (pas de GEMM en J1).

Définition 5.1 (Règle de déterminisme, couture N)**.** Aucun noyau n'emploie d'addition atomique en virgule flottante ; toute réduction suit un arbre fixe qui ne dépend que des tailles (ni de la grille, ni de l'ordre d'exécution des blocs). Les seuls atomiques sont des compteurs entiers de contrôle.

Sans cette règle, le critère C3 du jalon (logits du fichier codé et du fichier brut égaux au bit) n'aurait pas de sens : deux exécutions du même programme sur les mêmes octets pourraient différer. Chaque section qui suit dit comment le noyau la tient.

Table 5.1 : Ressources des noyaux, relevées par `ptxas -v` pour `sm_89`, et occupation sur la RTX 4060 Laptop (Ada : 100 Kio de mémoire partagée, 48 warps et 64 K registres par SM [22]).

noyau	registres	partagée (o)	remarque
`decoder_codes`	57	49 152	trois LUT de 16 Kio ; 2 blocs de 8 warps par SM, 384 warps sur 24 SM
`decoder_echelles`	55	24 576	une LUT et 8 192 o de médianes
`decoder_*_lot`	61 / 57	idem	un expert par valeur de `blockIdx.z`
`gemv_mxfp4` / `_lot`	71 / 79	0	quatre lignes par warp
`copier_brut`	29	0	chemin du fichier brut (témoin C3)
`attention`	97	2 304 + 4/position	au plus 8 192 positions par appel
`routeur_topk`	—	144 (pile)	tableaux indexés

Données : `moteur/K-pour-le-memoire.md` (tableau T-K1), `moteur/REPRISE-K.md` (relevés de 15 h 06 et 20 h 50).

5.2 Le décodeur à table par voie (K1)

5.2.1 Le flux, la tuile, la table

Le format `.trc` (§ 4.9 de la spécification) code chaque flux par un rANS entrelacé sur 32 voies : probabilités sur 12 bits ($M = 4\,096$), état x sur 32 bits, borne basse $L = 2^{16}$, mots de 16 bits. Un *macro-pas* produit 128 octets contigus : la voie l du macro-pas m écrit le mot de sortie de 32 bits d'indice $w = 32m + l$, et ses quatre sous-pas $k = 0..3$ en écrivent les octets, du poids faible au poids fort. Une *tuile* enchaîne $N = 256$ macro-pas ; chaque tuile a son pointeur de début dans le flux de mots partagé et ses 32 états initiaux, si bien que les tuiles se décodent indépendamment.

Pour les codes MXFP4 (sorte CODES_CTX), un bloc de 32 poids porte 16 octets, soit 4 mots : les voies $4q$ à $4q + 3$ du macro-pas m écrivent le bloc $8m + q = \lfloor w/4 \rfloor$, et toutes les quatre choisissent la table du contexte κ de ce bloc, la *classe d'échelle* : $\kappa = 0$, 1 ou 2 selon que l'écart d'échelle δ du bloc est sous, à, ou au-dessus de la médiane de sa ligne (figure 5.1).

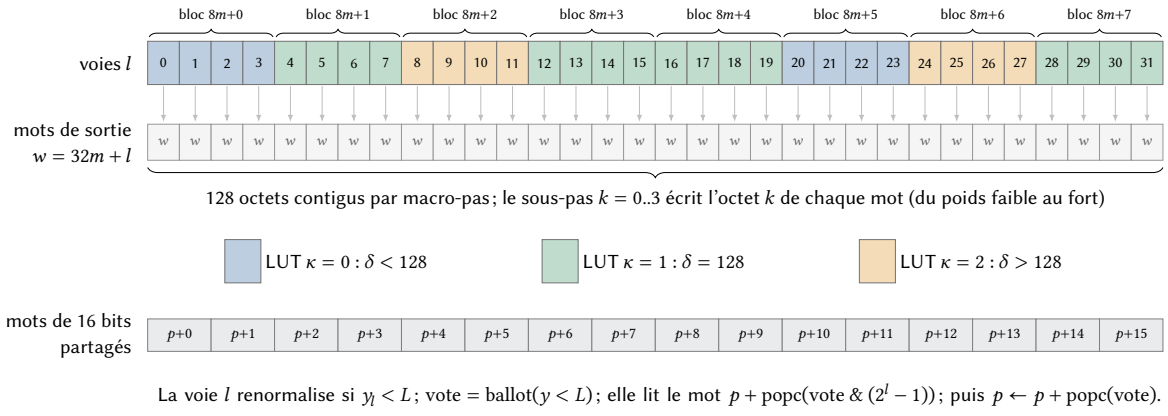


Figure 5.1 : Un macro-pas de la tuile : 32 voies, groupées par quatre sous la table de leur bloc (couleur : la classe d'échelle κ, exemple), au-dessus des 32 mots de sortie ; en bas, le flux de mots de 16 bits partagé par les voies, distribué par vote et comptage de bits.

Une table se monte en VRAM sous forme de LUT de 4 096 entrées de 32 bits, une par fente de probabilité (figure 5.2, en haut) : le symbole s dans les bits 0 à 7, $f - 1$ dans les bits 8 à 19, l'écart de la fente à la fréquence cumulée dans les bits 20 à 31. On range $f - 1$ et non f : un symbole seul

de sa table a $f = 4\,096 = 2^{12}$, qui ne tient pas en 12 bits (test `symbole_unique_a_frequence_pleine`).

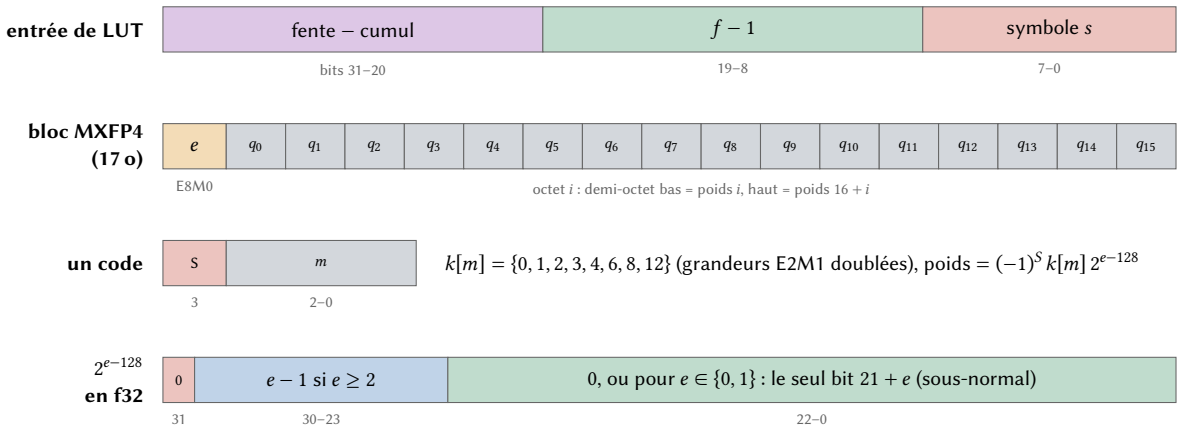


Figure 5.2 : Au bit près : l'entrée de LUT de géométrie 1 ; le bloc MXFP4 d'origine de 17 octets (une échelle E8M0, puis 16 octets de codes, l'octet i portant les poids i et $16 + i$, disposition de `block_mxfp4` de ggml [23]) ; un code de 4 bits, signe et grandeur E2M1 doublée [1] ; et l'échelle 2^{e-128} écrite directement en bits de f32 par le noyau.

5.2.2 Deux noyaux, un emplacement

La table de CODES_CTX dépend de κ, que donne DELTA : le décodeur décode toujours DELTA avant CODES_CTX (§ 4.9.3). Un expert se décode donc en deux lancements sur le même flux CUDA :

1. `decoder_echelles`, grille $(X, 3)$, 128 fils : pour chaque matrice j, décode DELTA_j (une seule table), reconstruit l'échelle de chaque bloc $e[\beta] = (\text{méd}[r] + \delta[\beta] - 128) \bmod 256$, et recopie bit à bit les biais (en entiers de 32 bits, pour qu'aucune conversion ne touche un NaN) ;
2. `decoder_codes`, grille $(X, 3)$, 256 fils, 48 Kio de mémoire partagée (les trois LUT de la matrice) : décode CODES_CTX_j, la voie qui produit le mot w choisissant la LUT du contexte $\kappa(\lfloor w/4 \rfloor)$, recalculé depuis l'échelle déjà écrite et la médiane.

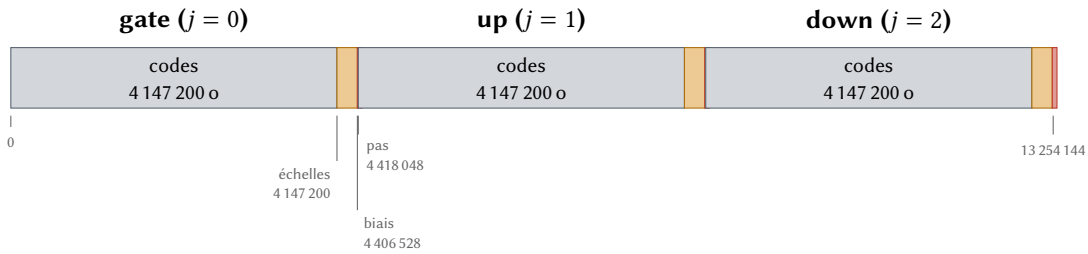
Les deux noyaux lisent la partie de tronçon telle qu'elle est arrivée par DMA (en-tête, table des flux, flux) : l'hôte n'a rien à analyser. Une tuile est confiée à un warp, et les warps parcourent les tuiles par pas de grille, si bien que toute grille est juste. La sortie est l'emplacement de disposition SoA que lit la GEMV (figure 5.3).

5.2.3 L'exactitude, démontrée

Le décodeur normatif du § 4.9.2 fait les 32 voies *une à une*, dans l'ordre croissant : la voie l lit son entrée de LUT, met à jour son état, et, si l'état passe sous L, lit le mot suivant du flux partagé. Le noyau fait les 32 voies *en même temps*. Il faut montrer qu'il produit les mêmes symboles.

Théorème 5.2 (Le pas d'un warp rend exactement le décodage scalaire)**.** *Soit une tuile codée par le codeur du § 4.9.2 (ordre inverse : macro-pas, sous-pas et voies décroissants ; chaque voie part de L ; pile de mots renversée). Le noyau, qui traite à chaque sous-pas les 32 voies simultanément, produit les mêmes symboles, dans les mêmes positions, que le décodeur scalaire qui traite les voies une à une dans l'ordre croissant ; et à chaque sous-pas, états et pointeur sont égaux dans les deux.*

Démonstration. Par récurrence sur les sous-pas de la tuile ; au départ, les deux ont $p = \text{début}[t]$ et les mêmes 32 états. (i) Le calcul de l'état d'une voie, $y \leftarrow f\,(x \gg 12) + b$ avec (s, f, b) lus dans la LUT à



Par matrice ($R = C = 2\,880$, $B = 90$) : codes $RB \cdot 16 = 4\,147\,200$ o dans l'ordre des blocs β ; échelles $RB = 259\,200$ o au décalage $\mathrm{al}_{256}(4\,147\,200) = 4\,147\,200$; biais $4R = 11\,520$ o au décalage $\mathrm{al}_{256}(4\,147\,200 + 259\,200) = 4\,406\,528$; pas $\mathrm{al}_{256}(4\,406\,528 + 11\,520) = 4\,418\,048$. Emplacement : $3 \times 4\,418\,048 = 13\,254\,144$ o (test `disposition_de_gpt_oss_egale_la_specification`).

Figure 5.3 : L'emplacement d'un expert de gpt-oss, à l'échelle (couture N, § 5.6) : pour chaque matrice, les codes, puis les échelles et les biais, chacun aligné sur 256 octets.

l'adresse x mod 4 096, ne dépend que de son propre état et de la table : il commute entre voies. (ii) Seul l'ordre de lecture des mots lie les voies. Dans le décodeur scalaire, au sous-pas courant, les voies qui renormalisent ($y < L$) lisent les mots $p, p+1, \ldots$ dans l'ordre croissant des voies. Le noyau calcule vote = ballot($y < L$) (un bit par voie) et donne à la voie l le mot $p + \mathrm{popc}(\mathrm{vote} \,\&\, (2^l - 1))$, c'est-à-dire p plus le nombre de voies d'indice inférieur qui renormalisent aussi : exactement la position scalaire. Puis $p \leftarrow p + \mathrm{popc}(\mathrm{vote})$, comme le scalaire après ses 32 voies. (iii) Une voie ne renormalise jamais deux fois au même sous-pas : si $x \geq L$, alors $x \gg 12 \geq 16$, donc $y \geq 16f \geq 16$, et pour $y < L$, $(y \ll 16) \mid \mathrm{mot} \geq 2^{16} = L$. (iv) Le codeur a émis ses mots dans l'ordre exactement inverse de ces lectures (le dernier émis est le premier lu, par le renversement de la pile) : chaque voie relit le mot qu'elle a écrit. États et pointeur étant égaux après le sous-pas, la récurrence se poursuit. ■

Les listings de ce chapitre sont des extraits verbatim des sources, à une exception près, dite ici : six caractères des commentaires que la police à chasse fixe du mémoire n'a pas (les crochets de partie entière et de plafond, le sigma de somme, l'exposant de transposée) y sont rendus par `floor(…)`, `ceil(…)`, `sum` et `^T` (`memoire/outils/tome2_chemin_noyaux.py`) ; le code n'en contient aucun.

Listing 5.1 : Le pas d'un warp (`moteur/crates/trc-noyaux/noyaux/commun.cuh`, lignes 197 à 227, commit `82a84e3`). Vote et comptage de bits donnent la position de chaque voie (théorème 5.2) ; deux échanges `__shfl_sync` distribuent les mots de la fenêtre (lemme 5.3) [17].

```
__device__ __forceinline__ uint32_t pas_octet(EtatVoie& v, const uint32_t*
    __restrict__ lut,
                                              const EnteteRans& h, uint32_t voie,
     uint32_t masque_inf) {
    const uint32_t e = lut[v.x & (M - 1u)];
    v.x = (((e >> 8) & 0xFFFu) + 1u) * (v.x >> PROBA_BITS) + (e >> 20);
    const bool besoin = v.x < L;
    const uint32_t vote = __ballot_sync(0xFFFFFFFFu, besoin);
    const uint32_t pos = v.ptr + __popc(vote & masque_inf);
    const uint32_t wi = (pos >> 1) - v.base;  // < 48 : voir la démonstration
    dans REPRISE-K
    const uint32_t a = __shfl_sync(0xFFFFFFFFu, v.cour, wi & 31u);
    const uint32_t b = __shfl_sync(0xFFFFFFFFu, v.suiv, wi & 31u);
    const uint32_t w32 = wi >= 32u ? b : a;
    const uint32_t w = (w32 >> ((pos & 1u) * 16u)) & 0xFFFFu;
    v.x = besoin ? ((v.x << 16) | w) : v.x;
    v.ptr += __popc(vote);
```

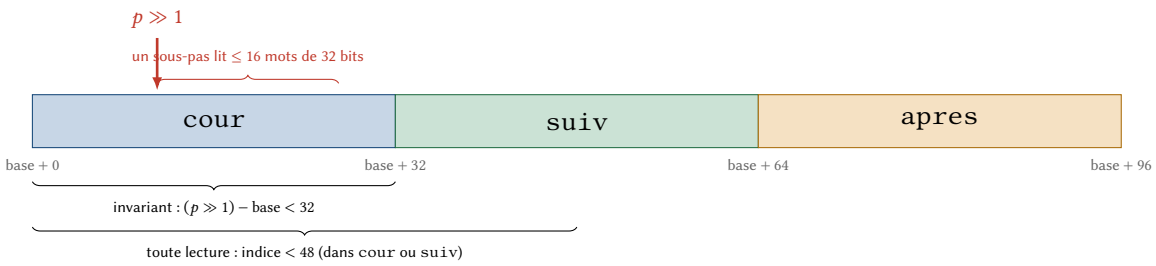
```
    if ((v.ptr >> 1) - v.base >= 32u) {
        v.base += 32u;
        v.cour = v.suiv;
        v.suiv = v.apres;
        v.apres = mot32(h, v.base + 64u + voie);
    }
    return e & 0xFFu;
}

__device__ __forceinline__ void debut_tuile(EtatVoie& v, const EnteteRans& h,
    uint32_t t, uint32_t voie) {
    v.x = h.etats[t * VOIES + voie];
    v.ptr = h.debut[t];
    v.base = v.ptr >> 1;
    v.cour = mot32(h, v.base + voie);
    v.suiv = mot32(h, v.base + 32u + voie);
    v.apres = mot32(h, v.base + 64u + voie);
}
```

Contrôles gratuits. En fin de tuile, chaque voie doit valoir L (l'état initial du codeur), et le pointeur doit égaler le début de la tuile suivante (ou W, le nombre de mots, pour la dernière). Le second contrôle, ajouté par le noyau et par le lot F indépendamment, voit un mot de trop que le premier ne voit pas. Aucun des deux ne garde la *table* : une fente de LUT faussée rend une sortie fausse sans sortir une voie de L (tableau 5.2) ; c'est pourquoi les tables portent une empreinte vérifiée à l'ouverture du fichier.

5.2.4 La fenêtre de mots

Lire un mot dans la mémoire globale à chaque renormalisation coûterait une latence entière par sous-pas. Le warp tient donc une fenêtre de 96 mots de 32 bits du flux (192 mots de 16) dans trois registres par voie : `cour` (le mot base$+l$), `suiv` (base$+32+l$) et `apres` (base$+64+l$), qu'il s'échange par `__shfl_sync` (figure 5.4).



Si $(p' \gg 1) - \text{base} \geq 32$: base $+= 32$, `cour` $\leftarrow$ `suiv`, `suiv` $\leftarrow$ `apres`, `apres` $\leftarrow$ `mot32`(base +64+ voie), une lecture globale qui ne sert qu'une avance plus tard.

Figure 5.4 : La fenêtre de mots du warp : trois registres par voie, 96 mots de 32 bits. L'invariant garantit qu'aucune lecture ne sort de `cour` et `suiv` ; `apres` est chargé une avance plus tôt, ce qui cache sa latence.

Lemme 5.3 (La fenêtre ne déborde jamais)**.** *Avant chaque sous-pas,* $(p \gg 1) - \text{base} < 32$. *Alors toute lecture du sous-pas tombe dans* `cour` *ou* `suiv`*, et après le sous-pas l'invariant est rétabli par au plus une avance de 32.*

Démonstration. Pendant un sous-pas, une voie lit au plus le mot $p+31$ (32 voies au plus renormalisent), d'indice de mot de 32 bits $(p+31) \gg 1 \leq (p \gg 1) + 16$; donc l'indice relatif $(\text{pos} \gg 1) - \text{base} <$

$32 + 16 = 48$: le mot est dans `cour` (moins de 32) ou dans `suiv`, jamais au-delà. Après le sous-pas, $p' \leq p + 32$, donc $(p' \gg 1) - \text{base} < 48$; s'il atteint 32, la fenêtre avance de 32, et il redevient inférieur à 16 : l'invariant tient. ■

Le calcul de la cadence. Une renormalisation apporte 16 bits, et un symbole en consomme son coût. Un octet de CODES_CTX coûte environ $2 \times 3{,}81 = 7{,}6$ bits (gpt-oss-120b, `bits_par_poids_entropie_tables`), d'où $7{,}6/16 = 0{,}48$ lecture par voie et par sous-pas, environ 15 mots de 16 bits par sous-pas pour le warp, et une avance de fenêtre (64 mots de 16 bits) tous les $64/15 \approx 4{,}2$ sous-pas [calc.]. Un octet de DELTA coûte environ $32 \times 0{,}034 = 1{,}1$ bit : une avance tous les 30 sous-pas environ. Avec deux registres, le mot chargé à l'avance sert dès que le pointeur a franchi 16 mots de 32 bits de plus, environ 2 sous-pas de CODES_CTX ; avec le troisième, il ne sert qu'après $4{,}2 + 2 \approx 6$ sous-pas, et sa latence est cachée : c'est la raison de la version 2 du noyau. Le commentaire du code écrit « tous les ≈ 8 pas » ; le calcul donne 4,2 pour CODES_CTX et 30 pour DELTA. L'écart est dit ici, non mesuré. Garde : `mot32` rend 0 au-delà de $\lceil W/2 \rceil$, si bien qu'un flux corrompu ne fait jamais lire hors de la partie.

5.2.5 La table par voie, et le déterminisme

Proposition 5.4 (La table par voie est juste et ne diverge pas)**.** *Le contexte du mot w est $\kappa(\lfloor w/4 \rfloor)$, connu du décodeur avant le décodage de CODES_CTX. Codeur et décodeur choisissent la même table, et le théorème 5.2 s'applique table par table. Le choix est une adresse, pas une branche : les 32 voies exécutent la même instruction.*

Démonstration. κ ne dépend que de δ, décodé par le premier noyau ; le codeur l'a calculé sur les mêmes δ. Dans le théorème 5.2, le point (i) vaut pour toute table pourvu que codeur et décodeur emploient la même au même symbole ; (ii) à (iv) ne regardent pas la table. Dans le noyau, la table est `lut[ctx]`, une base d'adresse en mémoire partagée ; seule l'adresse lue diffère d'une voie à l'autre. ■

Le noyau lit κ un macro-pas d'avance : l'échelle et la médiane du bloc suivant sont chargées pendant les quatre sous-pas du bloc courant, pour que leur latence se cache (listing 5.2).

Listing 5.2 : La boucle de `corps_codes` (`moteur/crates/trc-noyaux/noyaux/decodeur.cuh`, lignes 172 à 206, commit `a64740c`) : une tuile par warp, parcours par pas de grille ; la classe κ du bloc suivant lue d'avance ; la table choisie par adresse.

```
    for (uint32_t t = warp; t < h.T; t += nwarps) {
        EtatVoie v;
        debut_tuile(v, h, t, voie);
        const uint32_t m1 = min(t * h.N + h.N, nmacro);
        // Le mot w = m · 32 + voie appartient au bloc floor(w/4) = 8m + floor(
    voie/4) (16 octets = 4 mots).
        // Son contexte demande l'échelle du bloc et la médiane de sa ligne :
    deux lectures
        // globales, faites un macro-pas d'avance pour que leur latence se cache
    derrière les
        // quatre pas rANS du macro-pas courant (mesure du 2026-10-15). Ligne et
    reste suivis
        // sans division (le bloc avance de 8 par macro-pas).
        uint32_t bloc = (t * h.N * VOIES + voie) >> 2;
        uint32_t ligne = bloc / B, reste = bloc - ligne * B;
        bool valide = bloc < nblocs;
        uint32_t e_suiv = valide ? __ldg(ech + bloc) : 0u;
        uint32_t m_suiv = valide ? __ldg(med + ligne) : 0u;
```

```
        for (uint32_t m = t * h.N; m < m1; ++m) {
            // Au-delà des blocs réels (bourrage du flux), contexte 0 : cas que la
            // spécification ne dit pas (REPRISE-K), sans objet pour gpt-oss.
            const uint32_t ctx = valide ? classe_echelle(e_suiv, m_suiv) : 0u;
            const uint32_t w = m * VOIES + voie;
            bloc += 8u;
            reste += 8u;
            while (reste >= B) { reste -= B; ++ligne; }
            valide = (m + 1u < m1) && bloc < nblocs;
            if (valide) {
                e_suiv = __ldg(ech + bloc);
                m_suiv = __ldg(med + ligne);
            }
            const uint32_t* lt = lut[ctx];
            uint32_t mot = 0;
#pragma unroll
            for (uint32_t k = 0; k < 4; ++k) mot |= pas_octet(v, lt, h, voie, masque_inf) << (8u * k);
            if (w < mots_utiles) codes[w] = mot;
        }
        fin_tuile(v, h, t, voie, hors_l, pointeur);
    }
```

Proposition 5.5 (Déterminisme du décodeur)**.** *Toute grille rend les mêmes octets, et le décodage en lot rend les mêmes octets que l'expert seul.*

Démonstration. Le décodeur ne fait que des opérations entières ; ses seuls atomiques sont des compteurs entiers (commutatifs). La sortie de chaque voie ne dépend que de la tuile, pas de l'ordre d'exécution des warps ni de la grille (une tuile, un warp, parcours par pas de grille). Le lot et l'expert seul appellent les mêmes corps (`corps_echelles`, `corps_codes`). ■

Le test `decodeur_en_lot_egal_au_seul` le vérifie au bit (trois experts, un bit de flux retourné dans le deuxième : lui seul sort de L) ; sa cassure (descripteur 0 pour tous) tombe.

5.2.6 Témoins et cassures

Le fait à retenir est dans la première et la quatrième ligne : **une fente de LUT faussée rend 8 experts faux sans sortir une seule voie de** L. Le retour à L garde le flux et l'état, pas la table (03 § 3.3).

5.2.7 Pourquoi un expert seul est lent : le modèle de latence

Pour gpt-oss, une matrice de DELTA produit 259 200 octets, soit 2 025 macro-pas, soit 8 tuiles de 256 macro-pas ; une matrice de CODES_CTX, 4 147 200 octets, soit 32 400 macro-pas, soit 127 tuiles. Un expert compte 24 tuiles de DELTA et 381 de CODES_CTX. Chaque tuile est une chaîne de $256 \times 4 = 1\,024$ sous-pas *dépendants* : l'état x de chaque voie dépend du sous-pas précédent.

Proposition 5.6 (Modèle de latence du décodage)**.** *Soient τ_1 la durée d'un sous-pas quand un seul warp occupe son SM et τ_{16} sa durée quand 16 warps se partagent un SM. Le décodage d'un expert seul dure*

$$T_{\text{expert}} \approx 1\,024\,\tau_1 + 1\,024\,\tau_{16} + 2\,t_{\text{lancement}},$$

et un lot de n experts d'une même couche, par expert, $T_{\text{lot}}(n)/n \approx (1\,024\,\tau_1' + n \cdot 1\,024\,\tau_{16})/n$, qui tend vers $1\,024\,\tau_{16}$ quand n grandit.

Table 5.2 : K1 : les témoins du décodeur et leurs cassures [mes.]. « Hors L » : voies qui ne finissent pas sur L.

témoin	résultat	cassure	résultat de la cassure
120b, 4 608 experts, codeur indépendant du banc, noyaux v1	0 faux, 0 hors L	LUT $\kappa = 1$ faussée ; bit de flux	8 faux, 0 hors L ; 2 faux, 64 hors L, 2 pointeurs
120b, noyaux v2	0 faux sur 4 608	—	—
120b, noyaux finaux (deux passes)	0 faux sur 640 + 3 968	—	—
vrai fichier du lot F, 20b, 768 experts, contre `trc-codec` et le répertoire, seul et en lot	0 faux par les deux chemins	LUT ; flux	8 faux, 0 hors L ; 8 faux, 171 hors L, 7 pointeurs
vrai fichier du lot F, 120b, 4 608 experts, seul et en lot	0 faux, 0 hors L	LUT ; flux	8 faux, 0 hors L ; 8 faux, 163 hors L, 8 pointeurs
enveloppes Rust (`cargo test -p trc-noyaux`)	9 tests verts	fente, bit, magie, descripteur unique	tombent

Données : `mesures/moteur/K/k1-justesse-*.json`, `k1-trc-gptoss20b.json`, `k1-trc-gptoss120b.json`, `k1-*casse*.json` ; `mesures/moteur/K/csv/k1-justesse-par-couche.csv`, `k1-trc-par-couche.csv`.

Démonstration. Phase DELTA : 24 tuiles, donc 24 warps sur 24 SM, un par SM ; la phase dure une chaîne de 1 024 sous-pas à τ_1 (la carte est vide à 94 % : 24 warps sur 384 places). Phase CODES_CTX : 381 warps en une seule vague sur 384 places (16 par SM, limite de la mémoire partagée), une chaîne de 1 024 sous-pas à τ_{16}. En lot, les $24n$ warps de DELTA tournent ensemble (une chaîne, à $\tau_1' \approx \tau_1$ tant que la carte n'est pas saturée), puis les n vagues de CODES_CTX se suivent. ■

La confrontation, sans paramètre libre. Sur la passe propre de 16 h 46 (120b, couche 17), le temps de DELTA vaut 152 µs et celui de CODES_CTX 195 µs, avec 2 µs de trou entre les lancements [mes.] ; d'où $\tau_1 = 152/1\,024 = 148$ ns et $\tau_{16} = 195/1\,024 = 190$ ns par sous-pas, soit environ 400 et 510 cycles à 2,67 GHz [calc.] : le sous-pas d'un warp seul vaut une dizaine de latences dépendantes (LUT partagée, multiplication, vote, deux échanges, sélection, avance de fenêtre). Le modèle prédit $152 + 195 + 2 \approx 349$ µs pour un expert seul (mesuré : 343 µs, les deux lancements se chevauchant un peu) et, pour un lot de 4, $(152 + 4 \times 195)/4 = 233$ µs par expert : **mesuré 233,2 µs** (53,35 Go/s de codes) [calc.]. Ses deux temps sont mesurés sur A ; sa prédiction porte sur C.

La conséquence. Le décodage d'un expert seul est borné par la longueur de sa chaîne (1 024 sous-pas), non par la bande passante. Deux leviers seulement : moins de sous-pas par tuile (N plus petit au codage, un choix du codeur que le format porte déjà dans chaque flux, au prix de 0,4 % d'en-tête par flux de plus) ou plus de tuiles en vol (le lot). La borne du lot quand $n \to \infty$ vaut 12 441 600 o/195 µs = 63,8 Go/s de codes [calc.].

5.2.8 Les débits, et un verdict qui tombe de 0,9 et 1,1 %

Seuil déclaré avant la mesure : $0{,}8 \times 41{,}6 = 33{,}28$ Go/s de codes écrits, jugé sur la *pire* passe A d'une série, où 41,6 Go/s est le débit de la variante que K1 remplace (décodage puis réassemblage des trois flux de classe, publié par 03). Deux processus par variante, 5 séries A B C C B A chacun, carte verrouillée, secteur (figure 5.6).

Ce que la mesure dit, dans l'ordre :

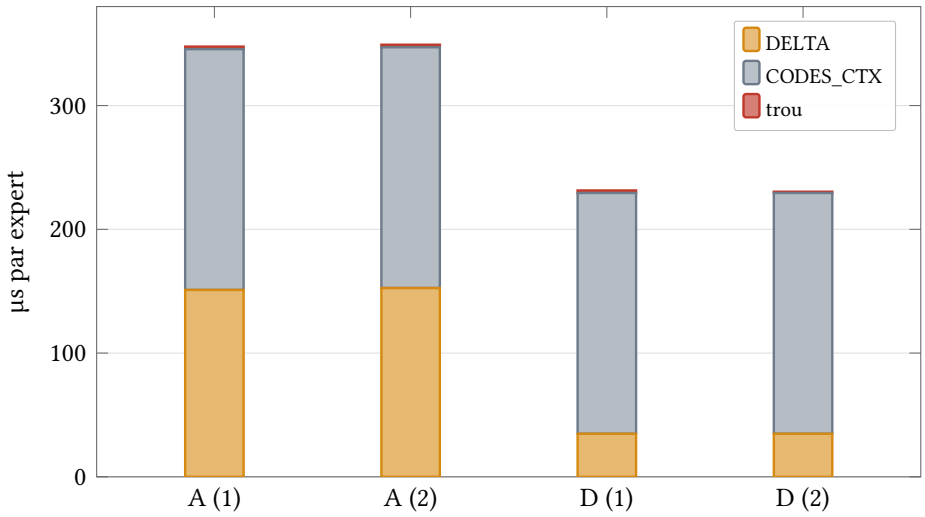


Figure 5.5 : K1 : décomposition du temps de décodage d'un expert, séance propre de 20 h 24 [mes.]. A : DELTA 151 et 153 µs, CODES_CTX 194,6 µs, trou 2 µs. D (DELTA codé en tuiles de 32 macro-pas) : DELTA 34,8 µs. La prédiction déclarée de D pour DELTA (19 à 22 µs) était fausse : à 8 warps par SM, le sous-pas passe de 148 à 272 ns ; le total tient dans ±10 % parce que CODES_CTX domine.

Données : `mesures/moteur/K/csv/k1-decomposition.csv`.

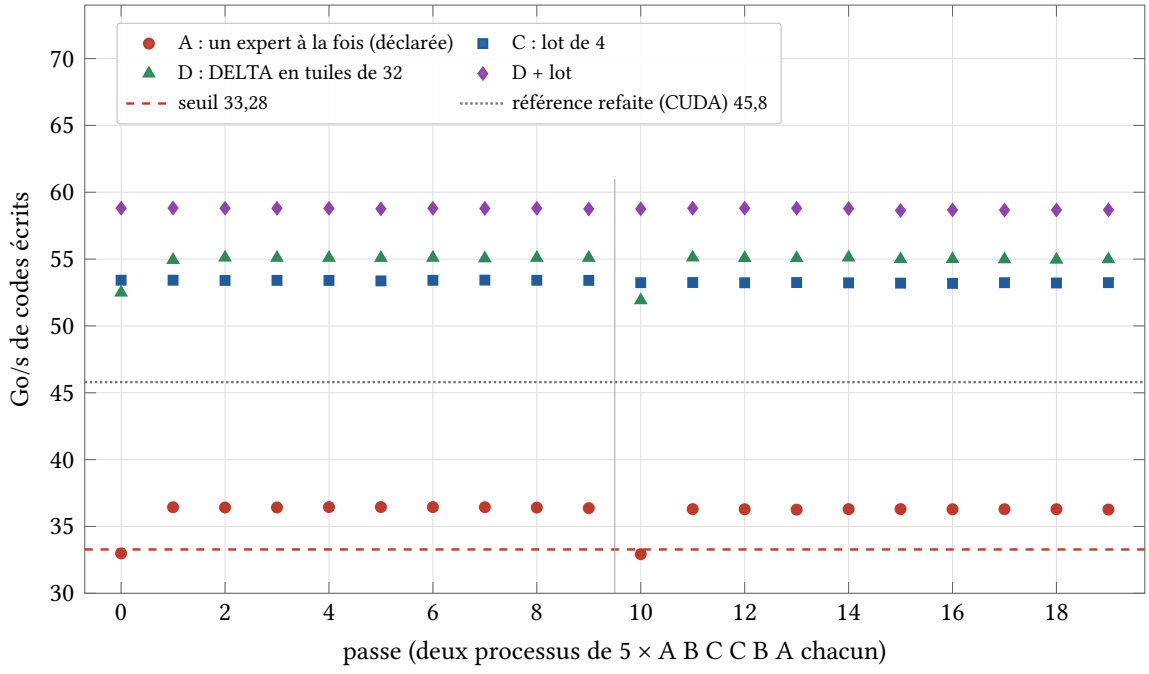


Figure 5.6 : K1, débit du décodeur par passe (Go/s de codes écrits ; le temps compte DELTA, échelles et biais), séance propre du 2026-10-15 à 20 h 24 [mes.]. La copie brute des mêmes octets (B) fait 105,4 Go/s, hors du cadre. Les deux pires passes de A sont la première série de chaque processus (33,0 et 32,9).

Données : `mesures/moteur/K/csv/k1-debit-passes.csv`, réordonné dans `t2n-k1.csv` ; médianes et pires dans `t2n-k1-resume.csv`.

Table 5.3 : K1 et K2 : débits (médianes et pires passes des deux processus) contre les seuils déclarés [mes.].

variante	médianes	pires	seuil	verdict
K1 A, un expert à la fois (déclarée)	36,43 / 36,29	32,99 / 32,92	33,28	**tombe de 0,9 et 1,1 %**
K1 C, lot de 4	53,41 / 53,23	53,37 / 53,18	33,28	tient
K1 D, DELTA en tuiles de 32	55,09 / 55,00	52,49 / 51,92	33,28	tient (prédit 57 ± 10 %)
K1 D + lot de 4	58,80 / 58,72	58,75 / 58,64	33,28	tient
référence refaite : v4 sous CUDA + réassemblage	45,79 / 45,75	—	—	~~41,6~~
copie brute (B)	105,45 / 105,45	—	—	—
K2, lot de 12 matrices	235,35 / 235,41	219,13 / 220,89	167,5	tient (+12 % sur Vulkan)
K2, une matrice par lancement	160,21 / 159,87	—	—	—

Données : `mesures/moteur/K/csv/k1-debit-passes.csv`, `k1-reference-03.csv`, `k2-debit-passes.csv`, `journaux/*-20261015-2024.*`.

1. **Le seuil déclaré tombe pour la variante déclarée**, de 0,9 et 1,1 % (une pire passe par processus : $1 - 32{,}99/33{,}28$ et $1 - 32{,}92/33{,}28$ [calc.] ; ~~de 0,9 %~~, repris de `moteur/K-pour-le-memoire.md`, ne comptait que le premier processus), sur un effet de première série (pire passe 9 % sous les autres, dans chaque processus ; les neuf autres de 36,3 à 36,5, écart 0,3 %). La règle était la pire passe : on ne retire pas la passe. La cause n'est pas cherchée ([hyp.] réveil de la carte ou du cache entre l'étalonnage et la série).
2. **La référence elle-même était sous-estimée.** ~~41,6 Go/s, le chemin que K1 remplace~~ 03 avait composé ce chiffre avec le v4 *sous Vulkan* (70 Go/s) ; refait sous CUDA dans la même séance (v4 82,69 / 82,57 Go/s, réassemblage 102,63 / 102,61), le chemin remplacé fait **45,8** Go/s. Contre lui ($0{,}8 \times 45{,}8 = 36{,}6$), la médiane de A tombe aussi.
3. **Le modèle de latence explique pourquoi** (proposition 5.6) et prédit les deux remèdes.
4. **Les deux remèdes tiennent contre les deux références** : D sans toucher au format (le codeur choisit $N = 32$ pour DELTA, +16,9 Ko par expert, +0,14 % du tronçon), C sans toucher au codeur ; ensemble, 58,8 Go/s.
5. La prédiction de D était juste sur le total (55,0 pour 57 ± 10 %) et fausse sur sa partie DELTA (34,8 µs pour 19 à 22) : le modèle ignore la contention à 8 warps par SM.

Proposition à l'arbitre (non tranchée à l'heure où ce chapitre s'écrit) : D et le décodage en lot des défauts d'une même couche ; le repli déclaré par la spécification (flux de classe, sorte 0x05) n'est pas nécessaire, il est plus lent (45,8) que D (55,0). Aucune conséquence sur J1 : 0,34 ms par expert contre 383 ms de disque par jeton mesurées au jalon (section 8.5.1 ; ~~157 ms~~, la prédiction du Tome I, n'est pas une mesure) ; l'élasticité du débit au décodeur est nulle tant que le disque limite.

5.3 Le produit matrice-vecteur MXFP4 (K2)

5.3.1 L'algorithme

$y = Wx + b$ sur l'emplacement SoA. Un warp traite 4 lignes ; la voie l prend les blocs $b = l, l+32, l+64, \ldots$ de ces lignes, charge une fois les 32 valeurs de x d'un bloc et les applique aux 4 lignes ; puis une réduction papillon sur les 32 voies (5 niveaux, `__shfl_xor_sync`). Un bloc de 256 fils couvre 32 lignes. Les codes et les échelles sont lus « en flux » (`__ldcs`) : les poids ne resservent pas. La valeur d'un poids est $k[c] \cdot 2^{e-128}$, avec $k = \{0, 1, 2, 3, 4, 6, 8, 12\}$ signé par le bit 3 : les grandeurs E2M1

doublées, d'où le -128 au lieu du -127 de l'E8M0 (figure 5.2).

Listing 5.3 : L'échelle E8M0 exacte, la valeur d'un code et le produit d'un bloc dans un ordre fixe (`moteur/crates/trc-noyaux/noyaux/gemv.cuh`, lignes 25 à 55, commit `4473050`). La constante `0xC8643210` range les huit grandeurs doublées, une par demi-octet.

```
// 2^(e - 128) en f32, exact pour tout e de 0 à 255 (e = 0 et 1 : sous-normaux).
__device__ __forceinline__ float echelle_e8m0(uint32_t e) {
    const uint32_t bits = e >= 2u ? (e - 1u) << 23 : (0x00200000u << e);
    return __uint_as_float(bits);
}

// Grandeur doublée d'un code E2M1 de 4 bits : les huit grandeurs sont rangées par
// demi-octet dans 0xC8643210 ; le bit 3 est le signe.
__device__ __forceinline__ float valeur_e2m1x2(uint32_t c) {
    const float g = float((0xC8643210u >> ((c & 7u) << 2)) & 15u);
    return (c & 8u) ? -g : g;
}

// Contribution d'un bloc : sum_i k[code_i] · x_i, dans un ordre fixe (mot t = 0..3,
// octet k = 0..3, demi-octet bas puis haut). xs[0..3] = x[0..15], xs[4..7] = x[16..31].
__device__ __forceinline__ float produit_bloc(const uint4 q, const float4 (&xs)[8]) {
    const uint32_t mots[4] = {q.x, q.y, q.z, q.w};
    float s = 0.0f;
#pragma unroll
    for (int t = 0; t < 4; ++t) {
        const uint32_t w = mots[t];
        const float xa[4] = {xs[t].x, xs[t].y, xs[t].z, xs[t].w};
        const float xb[4] = {xs[4 + t].x, xs[4 + t].y, xs[4 + t].z, xs[4 + t].w};
#pragma unroll
        for (int k = 0; k < 4; ++k) {
            s = fmaf(valeur_e2m1x2((w >> (8 * k)) & 15u), xa[k], s);
            s = fmaf(valeur_e2m1x2((w >> (8 * k + 4)) & 15u), xb[k], s);
        }
    }
    return s;
}
```

Lemme 5.7 (L'échelle E8M0 est exacte). *Pour tout $e \in [0, 255]$, `echelle_e8m0`(e) rend exactement 2^{e-128} en f32.*

Démonstration. Pour $e \geq 2$, les bits $(e-1) \ll 23$ donnent un f32 normal d'exposant biaisé $e-1$, de valeur $2^{e-1-127} = 2^{e-128}$, mantisse nulle ; $e - 1 \leq 254$ reste sous l'exposant réservé 255. Pour $e = 0$ et $e = 1$, 2^{-128} et 2^{-127} sont sous-normaux : un sous-normal de bits de mantisse m vaut $m \cdot 2^{-149}$, et $\texttt{0x00200000} = 2^{21}$, d'où $2^{21+e-149} = 2^{e-128}$. Les 256 valeurs sont en outre vérifiées une à une (`e8m0_256_valeurs_fausses` $= 0$). Le noyau est compilé sans `--use_fast_math`, qui ramènerait les sous-normaux à zéro. ∎

5.3.2 Exacte au bit sur entiers

Lemme 5.8 (Sommes exactes en f32). *Si tous les produits et toutes les sommes partielles d'un calcul, dans n'importe quel ordre, sont des multiples de 2^{-m} de valeur absolue inférieure à 2^{24-m}, alors le calcul en f32 (arrondi au plus proche) est exact quel que soit l'ordre.*

Démonstration. Un multiple de 2^{-m} de valeur absolue inférieure à 2^{24-m} s'écrit $a \cdot 2^{-m}$ avec $|a| < 2^{24}$ entier : il tient dans les 24 bits de mantisse d'un f32, il est représentable. Chaque opération (produit, somme, `fmaf`) a un résultat exact représentable ; l'arrondi IEEE 754 d'un nombre représentable le rend inchangé [24]. Par récurrence sur les opérations, aucune n'arrondit. ■

Le témoin, posé. Échelles $e \in [117, 123]$, entrées $x \in \{-2, \dots, 2\}$. Chaque terme $k \cdot x \cdot 2^{e-128}$ est un multiple de 2^{-11} (car $e - 128 \geq -11$ et k, x entiers), de valeur absolue au plus $12 \times 2 \times 2^{-5} = 0{,}75$; toute somme partielle d'une ligne de 2 880 termes est un multiple de 2^{-11} de valeur absolue au plus $2\,880 \times 0{,}75 = 2\,160 < 2^{12}$ [calc.] : elle tient en $12 + 11 = 23$ bits. Par le lemme 5.8, la carte doit égaler *au bit* la somme exacte. Mesuré : 0 ligne fausse sur 5 formes (dont $2\,880 \times 2\,880$, 37×96, 5×32) ; la cassure (référence ×1,01) fait tomber 8 639 lignes sur 8 640 [mes.]. La ligne qui tient sous la cassure n'a pas été examinée : [hyp.] une ligne de somme nulle, que ×1,01 ne change pas.

5.3.3 Sur poids réels : la borne d'erreur

Proposition 5.9 (Borne a priori de la GEMV)**.** *Pour une ligne, la somme est faite en f32 en trois étages : les 32 termes d'un bloc (ordre fixe), les $\lceil B/32 \rceil$ blocs d'une voie (3 au plus pour $B = 90$), puis l'arbre papillon sur 32 voies (5 niveaux). Le résultat vérifie*

$$|\hat{y} - y| \leq \gamma_{n_e} \sum |termes|, \qquad \gamma_n = \frac{nu}{1 - nu},\ u = 2^{-24},\ n_e = 32 + 3 + 5 = 40,$$

soit $\gamma_{40} \approx 2{,}4 \cdot 10^{-6}$ fois la somme des valeurs absolues.

Démonstration. Chaque terme traverse au plus n_e opérations en chaîne (une par terme du bloc, une par bloc de la voie, une par niveau de l'arbre), chacune d'erreur relative au plus u ; la borne classique d'une somme récursive s'applique à chaque chemin [25]. L'échelle est exacte (lemme 5.7) et multiplie sans erreur supplémentaire au-delà de la sienne, comptée dans la chaîne. ■

Le seuil déclaré (10^{-5} de $\max|y|$) suppose $\sum |\text{termes}| \leq 4 \max|y|$. Mesuré sur 24 matrices réelles du 120b (8 experts × 3) : $2{,}9 \cdot 10^{-7}$ **au pire**, 34 fois sous le seuil (figure 5.7) [mes.]. Le déterminisme suit de l'ordre fixe : lot et seul rendent les mêmes bits (`lot_different_du_seul` = 0).

5.3.4 Le toit

Une matrice $2\,880 \times 2\,880$ lit 4 406 400 octets de MXFP4 équivalents (codes et échelles), plus 11 520 de biais et 11 520 de x. La bande passante de la VRAM (bus de 128 bits, GDDR6) borne le débit ; le seuil de K2 est $0{,}8 \times 209{,}4 = 167{,}5$ Go/s (la GEMV MXFP4 brute de 03 sous Vulkan). Mesuré : **235,35 et 235,41 Go/s** pour un lot de 12 matrices (une couche de 4 experts), pires passes 219,1 et 220,9 ; une matrice par lancement fait 160 Go/s (figure 5.8) [mes.]. Une couche prend 230 µs ; les GEMV d'experts d'un jeton du 120b (36 couches), environ 8,3 ms [calc.], à comparer aux 383 ms de disque d'un jeton mesurées au jalon (157 prédites par le Tome I).

5.4 La passe avant : douze noyaux (K3)

5.4.1 Les noyaux et leurs structures

La passe avant de gpt-oss, telle que `trc-reference` (lot R) l'a écrite depuis la définition publiée par OpenAI et validée contre llama.cpp, se découpe en douze noyaux : `gemv_mxfp4` (K2), `gemv_q8_0`, `gemv_f32`, `rmsnorm` [26], `rope_yarn` [27, 28], `ecrire_kv`, `attention` (avec

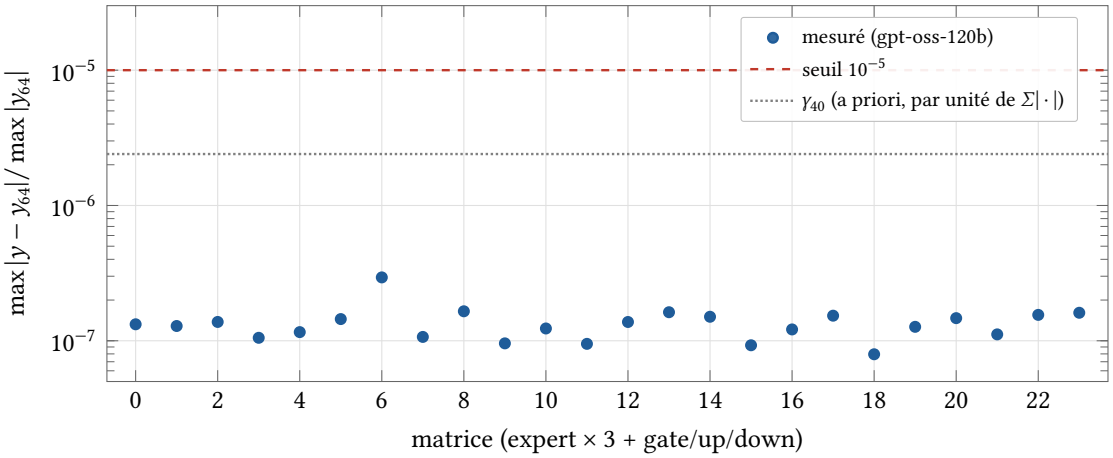


Figure 5.7 : K2 : écart relatif $\max|y - y_{64}| / \max|y_{64}|$ de la GEMV contre une référence f64, 24 matrices réelles de gpt-oss-120b [mes.]. En pointillé, γ_{40} : la borne a priori par unité de $\sum|\text{termes}|$ (proposition 5.9) ; les erreurs réelles se compensent, et l'écart mesuré reste un ordre de grandeur sous elle.

Données : `mesures/moteur/K/csv/k2-ecarts-reels.csv` (depuis `k2-justesse-gptoss120b.json`).

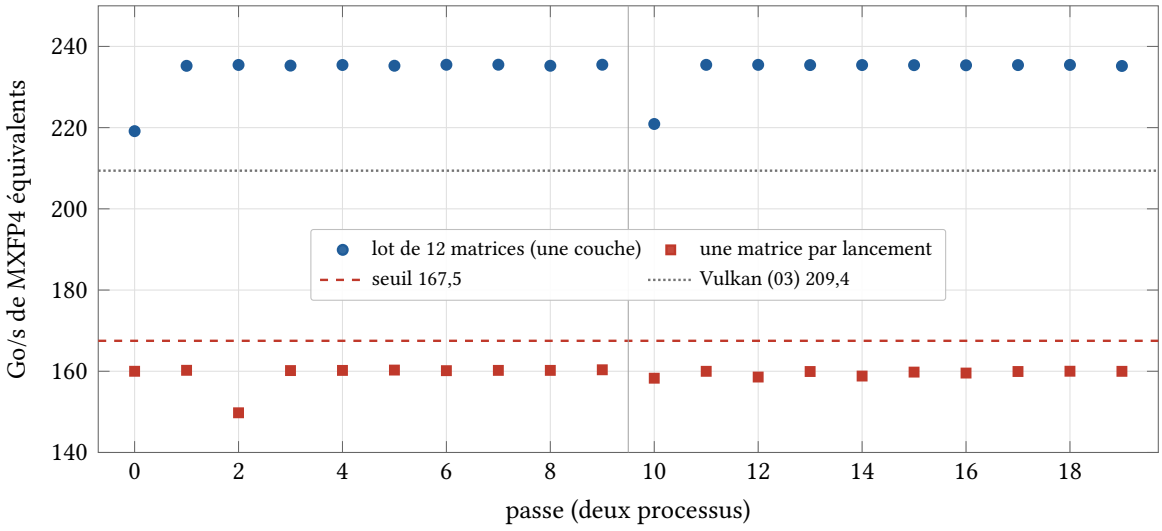


Figure 5.8 : K2, débit de la GEMV MXFP4 par passe (Go/s de MXFP4 équivalents), séance propre de 20 h 24 [mes.]. Le lot de 12 matrices dépasse de 12 % la référence Vulkan de 03 ; une matrice par lancement paie la queue de vague et le lancement.

Données : `mesures/moteur/K/csv/k2-debit-passes.csv`, réordonné dans `t2n-k2.csv`.

puits appris par tête [29]), `routeur_topk`, `swiglu_oai` [30], `combiner`, `ligne_q8_0` et `argmax`. La figure 5.9 les dispose dans l'ordre d'une couche.

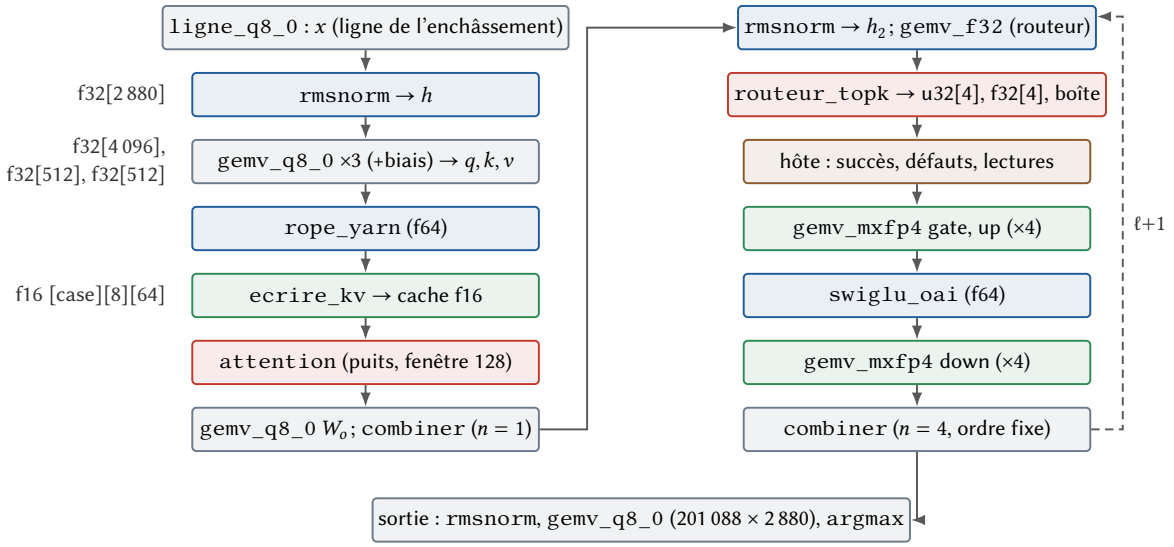


Figure 5.9 : La passe avant d'une couche de gpt-oss, noyau par noyau ($d = 2\,880$, 64 têtes de requête, 8 têtes de clé et de valeur de dimension 64). La moitié gauche précède le routeur ; la boîte aux lettres porte sa décision à l'hôte (chapitre 4), qui décide succès et défauts ; la moitié droite applique les quatre experts et les combine dans un ordre fixe.

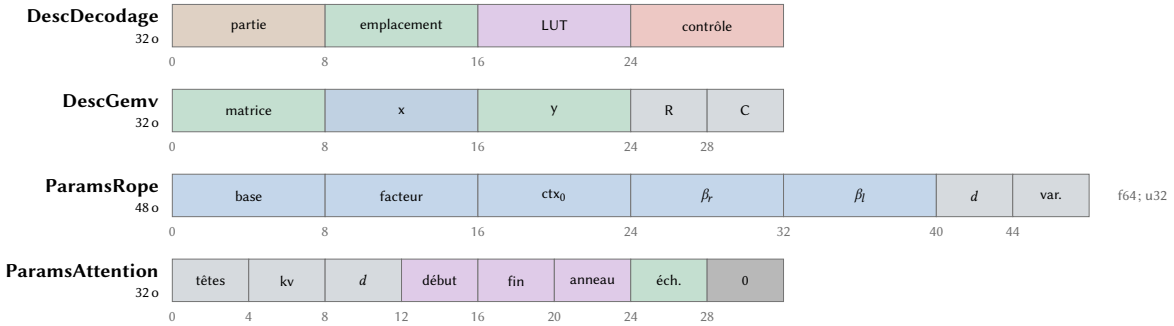


Figure 5.10 : Les structures passées aux noyaux, octet par octet (même disposition côté Rust, `#[repr(C)]`, `moteur/crates/trc-noyaux/src/abi.rs`). `DescDecodage` et `DescGemv` décrivent un élément d'un lot ; `ParamsRope` (alignement 8) porte YaRN en f64 (base 150 000, facteur 32, contexte d'origine 4 096, β rapide 32 et lent 1) et la variante (0 : OpenAI ; 1 : llama.cpp) ; `ParamsAttention` porte la plage lue et l'anneau (0, ou 128 sur une couche à fenêtre).

5.4.2 Le protocole du témoin

Chaque noyau est jugé *seul*, sur les poids **réels** lus dans le GGUF d'origine par le lecteur du lot R (chemin indépendant du `.trc`), aux couches 0 (à fenêtre) et 1 (pleine) de gpt-oss-20b puis de gpt-oss-120b. Son entrée est la sortie f64 de la référence arrondie en f32 (les erreurs ne s'enchaînent pas) ; sa référence est l'opération correspondante de `trc-reference`, appliquée en f64 *aux mêmes valeurs f32*. L'écart est $e = \max_i |y_i - y_i^{64}| / \max_i |y_i^{64}|$ sur le vecteur de sortie entier. Chaque appel est fait deux fois : les deux sorties doivent être égales au bit. Chaque cassure injecte une vraie faute plausible *dans la référence* ; seul le témoin visé doit tomber, les onze autres tenir.

Table 5.4 : Les tampons de la passe avant, au bit près.

structure	octets	champs
case de la boîte aux lettres	$4(2k+1) = 36$ utiles, dans 64	mot 0 : séquence, écrite en dernier après `__threadfence_system`; mots 1 à k : indices; mots $k+1$ à $2k$: poids (bits de f32). Protocole de l'essai E0 (c), chapitre 4
cache KV d'une couche	$2 \cdot$ cases $\cdot$ 512 par moitié	f16 `[case][tête_kv][64]`, pour K et pour V ; 128 cases sur une couche paire (anneau), sinon la longueur du contexte
bloc Q8_0 (ggml)	34	f16 d, puis 32× i8 ; l'adresse d'un bloc est paire, d'où la lecture par mots de 16 bits
activations	—	f32 contigus

5.4.3 Un seul arrondi : les noyaux en f64

Proposition 5.10 (Un seul arrondi tient à 2^{-24})**.** *Un noyau qui calcule en f64 sur la carte et arrondit une seule fois vers le f32 (`rmsnorm`, `rope_yarn`, `swiglu_oai`, les poids du routeur) vérifie* $e \leq 2^{-24} + O(2^{-50}) \approx 5{,}96 \cdot 10^{-8}$.

Démonstration. Soit y la valeur exacte et $\hat{y}_{64}$ le calcul en f64 : $|\hat{y}_{64} - y| \leq c \cdot 2^{-53}|y|$, avec c petit (une dizaine d'opérations, plus l'erreur de `pow`, `log`, `exp`, `sin`, `cos` de CUDA en double précision, 1 à 2 ulp). L'arrondi au plus proche ajoute $|\mathrm{fl}_{32}(\hat{y}_{64}) - \hat{y}_{64}| \leq 2^{-24}|\hat{y}_{64}|$. D'où $|\hat{y} - y| \leq (2^{-24} + c\,2^{-53})|y|(1 + O(2^{-24}))$ pour chaque composante, et e, qui normalise par le maximum, est borné de même. ∎

Mesuré (20b puis 120b) : `rmsnorm` 5,2 et 5,6·10^{-8} ; `rope_yarn` 5,3 et 4,6·10^{-8} ; `swiglu_oai` 4,9 et 3,5·10^{-8} ; poids du routeur 4,7 et 5,0·10^{-8} [mes.]. Tous sont sous la borne, et presque à son niveau : l'arrondi final est la seule erreur visible. Le seuil de la couture N (10^{-6}) laisse 17 fois de marge.

Pourquoi la rotation se calcule en f64, en un seul calcul. L'angle vaut $p\,\theta_i$, avec $\theta_0 = 1$. En f32, l'angle à la position p porte une erreur de l'ordre de $2^{-24}p$: $2{,}4 \cdot 10^{-4}$ rad à $p = 4\,096$, $7{,}8 \cdot 10^{-3}$ rad à $p = 131\,072$ [calc.]. Une erreur d'angle δ donne une erreur relative de l'ordre de δ sur la paire tournée ; le seuil de 10^{-6} serait manqué dès que $2^{-24}p > 10^{-6}$, soit $p > 10^{-6} \times 2^{24} \approx 17$ [calc.]. En f64, l'erreur d'angle vaut $2^{-53}p$, négligeable à toute position. Le noyau calcule donc en f64 la fréquence de YaRN (rampe entre les bornes β et concentration $0{,}1 \ln s + 1$), l'angle, le sinus et le cosinus, et la rotation (listing 5.4).

Listing 5.4 : `inv_freq_yarn` (`moteur/crates/trc-noyaux/noyaux/passe.cuh`, lignes 145 à 167, commit `c09551e`) : la fréquence de la paire i et la concentration, en f64. La variante 1 arrondit les bornes de la rampe comme llama.cpp ; la variante 0, celle du modèle, ne les arrondit pas.

```
// inv_freq de la paire i et facteur de concentration, en f64 (définition : trc-reference,
// ops::Rope::yarn, depuis la configuration publiée de gpt-oss).
__device__ inline double inv_freq_yarn(const ParamsRope& p, uint32_t i, double& concentration) {
    const uint32_t demi = p.dim_tete / 2u;
    const double freq = pow(p.base, double(2u * i) / double(p.dim_tete));
    if (p.facteur <= 1.0) {
        concentration = 1.0;
        return 1.0 / freq;
    }
    concentration = 0.1 * log(p.facteur) + 1.0;
```

```
    const double deux_pi = 6.28318530717958647692528676655900576839433879875021164194988918461563281257241799725606965068423413596429617302656461329418768921910116446345071405981613579;
    double bas = double(demi) * log(p.ctx_origine / (p.beta_rapide * deux_pi)) / log(p.base);
    double haut = double(demi) * log(p.ctx_origine / (p.beta_lent * deux_pi)) / log(p.base);
    if (p.variante == 1u) {
        bas = fmax(floor(bas), 0.0);
        haut = fmin(ceil(haut), double(p.dim_tete - 1u));
    }
    const double interpolation = 1.0 / (p.facteur * freq);
    const double extrapolation = 1.0 / freq;
    const double rampe = fmin(fmax((double(i) - bas) / (haut - bas), 0.0), 1.0);
    const double masque = 1.0 - rampe;
    return interpolation * (1.0 - masque) + extrapolation * masque;
}
```

5.4.4 Les noyaux exacts

Proposition 5.11 (L'enchâssement est exact)**.** *`ligne_q8_0` rend exactement $d \cdot q$ pour chaque poids d'une ligne Q8_0.*

Démonstration. d est un f16, à 11 bits de mantisse ; q est un i8, $|q| \leq 128$, à 7 bits significatifs au plus ($128 = 2^7$ n'en a qu'un). Le produit tient en 18 bits, sous les 24 d'un f32, et reste dans l'étendue des normaux ou des sous-normaux exacts ($|dq| \leq 65\,504 \times 128$, et $\geq 2^{-24}$ s'il n'est pas nul) : il est exact. ■

Mesuré : 0 différence sur 64 lignes, dont la première et la dernière [mes.]. Pour `gemv_q8_0`, `gemv_f32` et `gemv_mxfp4`, le lemme 5.8 donne des témoins exacts : Q8_0 avec $d = 2^e$, $e \in [-2, 1]$, $q \in [-127, 127]$, $x \in \{-1, 0, 1\}$, sommes multiples de 2^{-2} bornées par $128 \cdot 32 \cdot 127 \cdot 2 = 1\,040\,384 < 2^{22}$ pour $C = 4\,096$; f32 avec $W, x \in [-8, 8]$ entiers, somme au plus 184 320 ; MXFP4 avec $e \in [127, 129]$, $x \in [-2, 2]$, multiples de 2^{-1} bornés par 138 240 [calc.]. Mesuré : 0 différence sur 5 formes, jusqu'à $2\,880 \times 4\,096$ [mes.]. Sur poids réels, la borne de la proposition 5.9 vaut avec $n_e = 32 + 1 + \lceil B/32 \rceil + 5$: 41 pour $C = 2\,880$, 42 pour $C = 4\,096$ (`attn_output`), d'où $\gamma \approx 2{,}5 \cdot 10^{-6}$; mesuré : 1,6 à $2{,}6 \cdot 10^{-7}$ sur q, k, v, o, le routeur, les experts et la sortie à 201 088 lignes, 39 à 62 fois sous 10^{-5} [mes.].

Proposition 5.12 (Le combiner est égal au bit à l'hôte)**.** *`combiner` rend, pour chaque composante, exactement la valeur que calcule l'hôte en f32 pour $x + (((g_0 s_0 + g_1 s_1) + g_2 s_2) + g_3 s_3)$, dans cet ordre.*

Démonstration. IEEE 754 fixe le résultat de chaque addition et de chaque multiplication en arrondi au plus proche pair : mêmes opérandes, même ordre, même mode, même résultat [24]. Le noyau emploie `__fmul_rn` et `__fadd_rn`, que le compilateur ne contracte jamais en fma [31] ; l'hôte (Rust, x86-64, SSE) ne contracte jamais. Les deux suites d'opérations sont donc identiques, opération par opération. ■

Cassure faite dans la source : avec un `fmaf` à la place, le test tombe dès l'élément 649 (`tests/passe.rs`) [mes.]. Un arrondi de moins change des bits.

Listing 5.5 : `combiner` (`passe.cuh`, lignes 527 à 542, commit `c09551e`) : chaque produit et chaque somme arrondis séparément ; `poids` nul sert au résidu de l'attention ($n = 1$, poids 1, aucune multiplication).

```
// résidu[i] += ((₀w ₀s[i] + ₁w ₁s[i]) + ₂w ₂s[i]) + … dans l'ordre j = 0, 1, …, n − 1, chaque
// produit et chaque somme arrondis séparément (__fmul_rn, __fadd_rn : aucune contraction en fma),
```

```
// pour égaler au bit la même somme faite sur l'hôte en f32. s_j = sorties + j · pas. `poids` nul :
// tous les poids valent 1 (s_j pris tel quel ; sert à x ← x + a Wo^T + bo). Grille : ceil(d / 256) × 256.
extern "C" __global__ void __launch_bounds__(256)
combiner(float* __restrict__ residu, const float* __restrict__ sorties, uint32_t pas, const float* __restrict__ poids,
         uint32_t n, uint32_t d) {
    const uint32_t i = blockIdx.x * 256u + threadIdx.x;
    if (i >= d || n == 0) return;
    float s = poids != nullptr ? __fmul_rn(poids[0], sorties[i]) : sorties[i];
    for (uint32_t j = 1; j < n; ++j) {
        const float sj = sorties[uint64_t(j) * pas + i];
        s = __fadd_rn(s, poids != nullptr ? __fmul_rn(poids[j], sj) : sj);
    }
    residu[i] = __fadd_rn(residu[i], s);
}
```

5.4.5 L'attention, la plus petite marge

L'attention d'une position, une tête de requête par bloc de 256 fils, se fait en trois phases (listing 5.6) : (1) les scores $s_j = (q \cdot k_j)/8$, produit en f32 sur $d = 0..63$ dans l'ordre ; (2) le softmax avec le puits de la tête dans le dénominateur : $m = \max(\max_j s_j, \text{puits})$, $e_j = \exp(s_j - m)$, $z = \sum_j e_j + \exp(\text{puits} - m)$, les deux réductions par un arbre fixe sur 256 fils ; (3) la somme pondérée des valeurs en quatre groupes ($j \equiv g \bmod 4$, chacun dans l'ordre), combinés $((g_0 + g_1) + (g_2 + g_3))$. La case d'une position est la position, ou la position modulo 128 sur une couche à fenêtre.

Listing 5.6 : Le corps d'`attention` (`passe.cuh`, lignes 325 à 371, commit `c09551e`) : scores, softmax avec puits, somme en quatre groupes ; aucun atomique, deux arbres fixes.

```
    // 1. scores
    for (uint32_t j = t; j < n; j += ATTENTION_FILS) {
        const uint32_t pos = p.debut + j;
        const uint32_t c = p.anneau ? pos % p.anneau : pos;
        const uint4* kr = reinterpret_cast<const uint4*>(cache_k + uint64_t(c) * dkv + uint64_t(hk) * ATTENTION_DIM);
        float s = 0.0f;
#pragma unroll
        for (int b = 0; b < 8; ++b) {
            const uint4 w = kr[b];
            const uint32_t mots[4] = {w.x, w.y, w.z, w.w};
#pragma unroll
            for (int u = 0; u < 4; ++u) {
                s = fmaf(qs[8 * b + 2 * u], f16_vers_f32(mots[u] & 0xFFFFu), s);
                s = fmaf(qs[8 * b + 2 * u + 1], f16_vers_f32(mots[u] >> 16), s);
            }
        }
        scores[j] = s * p.echelle;
    }
    __syncthreads();

    // 2. softmax avec le puits dans le dénominateur
    float m = -INFINITY;
    for (uint32_t j = t; j < n; j += ATTENTION_FILS) m = fmaxf(m, scores[j]);
    const float puits_h = puits[h];
    m = fmaxf(reduire_bloc<true>(part, m), puits_h);
    float z = 0.0f;
```

```
    for (uint32_t j = t; j < n; j += ATTENTION_FILS) {
        const float e = expf(scores[j] - m);
        scores[j] = e;
        z += e;
    }
    z = reduire_bloc<false>(part, z) + expf(puits_h - m);

    // 3. somme pondérée des valeurs
    const uint32_t dd = t & (ATTENTION_DIM - 1u);
    const uint32_t g = t / ATTENTION_DIM;  // 0..3
    float a = 0.0f;
    for (uint32_t j = g; j < n; j += 4u) {
        const uint32_t pos = p.debut + j;
        const uint32_t c = p.anneau ? pos % p.anneau : pos;
        a = fmaf(scores[j], f16_vers_f32(cache_v[uint64_t(c) * dkv + uint64_t(hk)
     * ATTENTION_DIM + dd]), a);
    }
    groupes[g][dd] = a;
    __syncthreads();
    if (t < ATTENTION_DIM)
        sortie[uint64_t(h) * ATTENTION_DIM + t] = ((groupes[0][t] + groupes[1][t
    ]) + (groupes[2][t] + groupes[3][t])) / z;
}
```

Proposition 5.13 (Borne de l'attention). *L'erreur relative de la sortie est de l'ordre de* $\max_j |\delta s_j| + \gamma_{n_a}$, *avec* $|\delta s_j| \le \gamma_{64} \sum_t |q_t k_{j,t}|/8$, $\gamma_{64} \approx 3{,}8 \cdot 10^{-6}$, *et* $n_a \approx 1\,024$ *additions en chaîne pour* $n = 4\,096$ *positions* ($\gamma \approx 6 \cdot 10^{-5}$ *au pire*).

Démonstration. Le score est une somme de 64 produits en f32 (proposition 5.9 avec $n_e = 64$). Une erreur δs multiplie le poids par $\exp(\delta s) \approx 1 + \delta s$; la sortie, moyenne pondérée, hérite d'une erreur relative de l'ordre de $\max_j |\delta s_j|$. S'y ajoutent les sommes de e_j et de $e_j v_j$: au plus $n/4$ termes par groupe, puis un arbre de 4, soit environ 1 024 additions en chaîne pour $n = 4\,096$. Ce sont des bornes de pire cas ; les erreurs réelles se compensent. ■

Mesuré : 2,2 à 6,2·10^{-6} sur la couche à fenêtre, 0,46 à 1,9·10^{-6} sur la couche pleine (jusqu'à 4 096 positions), pour un seuil de 10^{-5} [mes.] : **la marge la plus petite, 1,6** sur le 20b (2,5 sur le 120b). Si C4 l'exigeait, des scores en f64 coûteraient environ 0,17 ms par couche (~~au plus 0,15 ms~~) [calc. : $3{,}36 \times 10^7$ opérations, soit 64 têtes ×4 096 × 64 × 2 opérations à environ 0,2 Tflop/s en f64], sans effet devant le disque.

5.4.6 Les sélections, et le déterminisme d'ensemble

Proposition 5.14 (Les sélections sont déterministes). `argmax` *et* `routeur_topk` *rendent le même élément quel que soit l'arbre de réduction ; les égalités se départagent au plus petit indice.*

Démonstration. Les deux noyaux prennent le minimum de la clé $(-v, i)$ sur les valeurs non NaN (`meilleur` : plus grande valeur, puis plus petit indice ; un NaN n'est jamais meilleur). C'est un ordre total ; le minimum d'un ensemble pour un ordre total est unique, et l'opération « meilleur de deux » est associative et commutative : toute réduction, quel que soit son arbre, rend cet élément. L'arbre est fixé quand même. ■

Vérifié sur des vecteurs à égalités (quatre maxima égaux, 128 zéros, maxima aux indices 64 et 127) ; la cassure « plus grand indice » ne fait tomber que ces cas [mes.]. `routeur_topk` calcule ensuite le

softmax des k valeurs retenues en f64 et écrit la case de la boîte aux lettres dans l'ordre du protocole de l'essai E0 (c) : indices et poids, barrière système, puis la séquence (listing 5.7).

Listing 5.7 : Le corps de `routeur_topk` (`passe.cuh`, lignes 461 à 514, commit `c09551e`) : k tours de réduction papillon sur l'ordre total, puis le softmax des k en f64 et la case de la boîte aux lettres, séquence en dernier.

```
    float v[8];
    bool pris[8];
#pragma unroll
    for (uint32_t i = 0; i < 8; ++i) {
        const uint32_t e = voie + 32u * i;
        v[i] = e < E ? logits[e] : -INFINITY;
        pris[i] = e >= E;
    }
    float choisi_v[8];
    uint32_t choisi_i[8];
    for (uint32_t r = 0; r < k; ++r) {
        float bv = -INFINITY;
        uint32_t bi = 0xFFFFFFFFu;
#pragma unroll
        for (uint32_t i = 0; i < 8; ++i)
            if (!pris[i] && meilleur(v[i], voie + 32u * i, bv, bi)) {
                bv = v[i];
                bi = voie + 32u * i;
            }
#pragma unroll
        for (int d = 16; d > 0; d >>= 1) {
            const float ov = __shfl_xor_sync(0xFFFFFFFFu, bv, d);
            const uint32_t oi = __shfl_xor_sync(0xFFFFFFFFu, bi, d);
            if (meilleur(ov, oi, bv, bi)) {
                bv = ov;
                bi = oi;
            }
        }
        if (bi != 0xFFFFFFFFu && (bi & 31u) == voie) pris[bi >> 5] = true;
        choisi_v[r] = bv;
        choisi_i[r] = bi;
    }
    if (voie != 0) return;
    double e[8];
    const double m = choisi_v[0];
    double z = 0.0;
    for (uint32_t r = 0; r < k; ++r) {
        e[r] = exp(double(choisi_v[r]) - m);
        z += e[r];
    }
    for (uint32_t r = 0; r < k; ++r) {
        const float w = float(e[r] / z);
        indices[r] = choisi_i[r];
        poids[r] = w;
        if (boite != nullptr) {
            reinterpret_cast<volatile uint32_t*>(boite)[1 + r] = choisi_i[r];
            reinterpret_cast<volatile uint32_t*>(boite)[1 + k + r] =
    __float_as_uint(w);
        }
    }
    if (boite != nullptr) {
        __threadfence_system();
```

```
        reinterpret_cast<volatile uint32_t*>(boite)[0] = sequence;
    }
}
```

Les réductions flottantes de `rmsnorm` et d'`attention` suivent un arbre fixe sur 256 fils, sans atomique ; celles des GEMV, un papillon fixe sur 32 voies. Chaque noyau est donc une fonction déterministe des octets qu'il lit (définition 5.1), et chaque appel des témoins, fait deux fois, rend les mêmes bits.

5.4.7 Résultats, noyau par noyau

Table 5.5 : K3 : chaque noyau contre `trc-reference` sur poids réels, et sa cassure [mes.]. Marge : seuil / pire écart. Les témoins comptent 143 cas (20b) et 116 (120b) ; chaque appel est déterministe au bit ; chaque cassure fait tomber son seul témoin.

noyau	seuil	pire 20b	pire 120b	marge 20b	cassure (dans la référence) : effet
`gemv_mxfp4`	10^{-5}	$2{,}4 \cdot 10^{-7}$	$1{,}6 \cdot 10^{-7}$	41	échelle 2^{e-127} : écart 0,66
`gemv_q8_0`	10^{-5}	$1{,}9 \cdot 10^{-7}$	$2{,}5 \cdot 10^{-7}$	53	biais omis : 0,70, sauf `attn_k` (biais nul)
`gemv_f32`	10^{-5}	$2{,}6 \cdot 10^{-7}$	$2{,}2 \cdot 10^{-7}$	39	biais omis : 0,28
`rmsnorm`	10^{-6}	$5{,}2 \cdot 10^{-8}$	$5{,}6 \cdot 10^{-8}$	19	moyenne sur $n-1$: $1{,}7 \cdot 10^{-4}$
`rope_yarn`	10^{-6}	$5{,}3 \cdot 10^{-8}$	$4{,}6 \cdot 10^{-8}$	19	sans la concentration : 0,35
`attention`	10^{-5}	$6{,}2 \cdot 10^{-6}$	$3{,}9 \cdot 10^{-6}$	1,6	puits omis : 1,0
`routeur_topk` (poids)	10^{-6}	$4{,}7 \cdot 10^{-8}$	$5{,}0 \cdot 10^{-8}$	21	égalité au plus grand indice : 18 indices différents
`swiglu_oai`	10^{-6}	$4{,}9 \cdot 10^{-8}$	$3{,}5 \cdot 10^{-8}$	20	$\alpha = 1$: 0,28
`combiner`	égalité	0	0	—	ordre inversé : 5 762 différences
`ecrire_kv`	égalité	0	0	—	arrondi vers zéro : 392 654 différences
`ligne_q8_0`	égalité	0	0	—	ligne voisine : 176 448 différences
`argmax`, indices du routeur	égalité	0	0	—	égalité au plus grand indice : 3 différences

Données : `mesures/moteur/K/csv/k3-par-noyau.csv` (lignes `temoin` et `casse:*`), `mesures/moteur/K/k3/`.

5.4.8 La chaîne de bout en bout

Les noyaux justes un à un ne font pas encore une passe juste : il faut les enchaîner. Le mode `chaine` du témoin fait la passe avant *entière* de gpt-oss-20b sur la carte, par les seuls noyaux de K1 à K3 (aucun arrondi de l'hôte entre deux noyaux), sur les 160 premiers jetons réels de la fenêtre 0 de wikitext-2, contre la référence f64, couche par couche (chaque expert chargé une fois par couche).

Proposition 5.15 (Ordre couche par couche = ordre jeton par jeton)**.** *Si les deux ordres lancent les mêmes noyaux avec les mêmes arguments numériques, l'ordre couche par couche (toutes les positions d'une couche, puis la couche suivante) et l'ordre jeton par jeton (toutes les couches d'une position, puis la suivante) rendent les mêmes bits.*

Démonstration. Soit x_p^ℓ l'état de la position p après la couche ℓ. Par causalité, x_p^ℓ ne dépend que de $x_p^{\ell-1}$ et des clés et valeurs des positions $q \le p$ à la couche ℓ, elles-mêmes fonctions de $x_q^{\ell-1}$. Par récurrence sur (ℓ, p) dans l'ordre produit, chaque x_p^ℓ sort des mêmes noyaux, sur les mêmes octets, dans le même ordre interne (définition 5.1). Sur l'anneau d'une couche à fenêtre, la position p est lue juste après son écriture, et les 128 cases portent alors exactement les positions $p-127 \dots p$ dans les deux ordres. ■

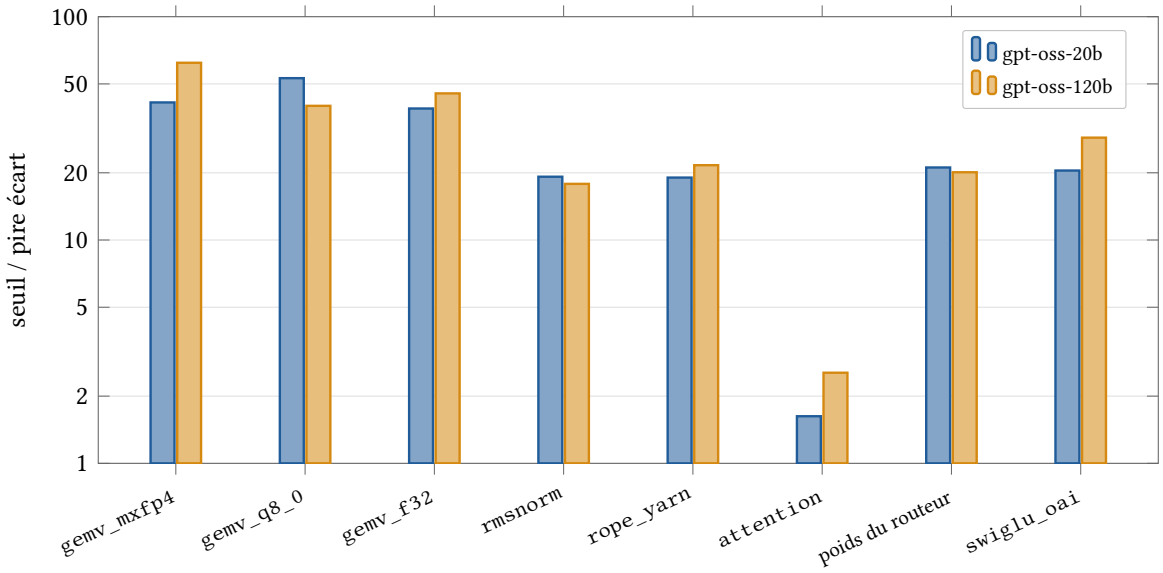


Figure 5.11 : K3 : la marge de chaque noyau à seuil (seuil / pire écart, échelle logarithmique), 20b et 120b [mes.]. Les noyaux en f64 tiennent à 2^{-24} (marge 17 à 29 pour un seuil de 10^{-6}) ; les GEMV, 39 à 62 fois sous 10^{-5} ; l'attention est la plus serrée.

Données : `mesures/moteur/K/csv/k3-par-noyau.csv`, dans `t2n-marges.csv`.

C'est une identité de programme, non une mesure ; le moteur du lot O l'a vérifiée ensuite (160 lignes sur 160 égales entre la chaîne et le moteur, témoin O2).

Table 5.6 : K3 : la chaîne de bout en bout, gpt-oss-20b, 24 couches, 160 jetons [mes.]. Seuil déclaré : KL moyenne $\leq 7 \cdot 10^{-5}$.

essai	KL moyenne	KL max	médiane	premier choix	verdict
routage libre (tel que déclaré)	$7{,}28 \cdot 10^{-3}$	1,02	$2{,}5 \cdot 10^{-6}$	158 / 160	tombe
D2, routage imposé	$3{,}0 \cdot 10^{-6}$	$1{,}4 \cdot 10^{-4}$	$1{,}8 \cdot 10^{-7}$	160 / 160	tient
D2, cassure : fenêtre sur les couches impaires	$1{,}6 \cdot 10^{-2}$	1,36	—	156 / 160	tombe
D2, cassure : poids inversés	0,52	2,27	—	77 / 160	tombe

Données : `mesures/moteur/K/k3/k3-chaine-*.json`, `mesures/moteur/K/csv/k3-chaine-kl.csv`.

Telle que déclarée, la chaîne **tombe** (tableau 5.6) : KL moyenne $7{,}28 \cdot 10^{-3}$, cent fois le seuil. Mais la médiane vaut $2{,}5 \cdot 10^{-6}$, et les six positions qui portent 99,3 % de la KL sont exactement celles des six premières bascules de routage (67, 87, 114, 129, 136, 153 ; figure 5.12). Le diagnostic, déclaré après ce premier résultat et avant son code (c'est écrit dans la note du lot K, 21 h 22), impose à la chaîne les experts de la référence (D2) : la KL tombe à $3{,}0 \cdot 10^{-6}$, 160 premiers choix sur 160.

Proposition 5.16 (L'amplificateur de routage)**.** *Soit μ la marge du routeur de la référence à une décision (l'écart entre ses 4e et 5e logits) et η l'erreur de la chaîne f32 sur cette marge. La décision bascule si $\eta > \mu$ (avec le bon signe). Une bascule remplace un expert de poids g par un autre, et l'état de la position change de l'ordre de $g\,\|s_e - s_{e'}\|$ au lieu de 10^{-7} : c'est un saut, non un arrondi.*

Démonstration. Le routeur garde les 4 plus grands logits ; une perturbation η plus grande que l'écart entre le 4e et le 5e, dans le sens qui les inverse, échange les deux experts. La sortie de la couche pour cette position change alors de $g\,(s_{e'} - s_e)$ (au renormalisé du softmax près), sans commune mesure

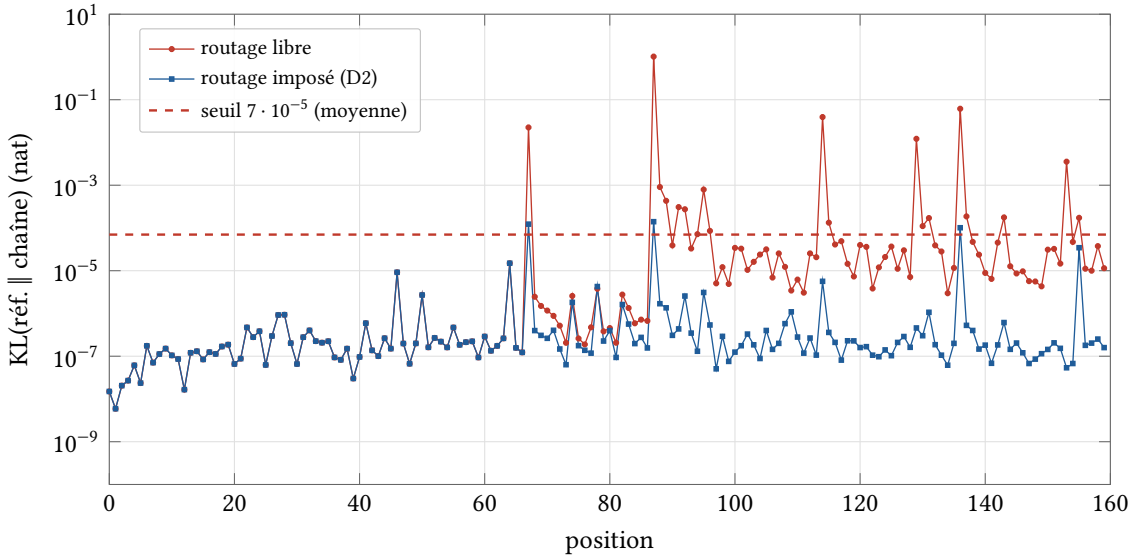


Figure 5.12 : K3, la chaîne : KL(référence ‖ chaîne) par position, gpt-oss-20b [mes.]. En routage libre, six pics (positions 67, 87, 114, 129, 136, 153) portent 99,3 % de la KL ; sous routage imposé, ils disparaissent et la KL reste entre 10^{-9} et 10^{-4}.

Données : `mesures/moteur/K/csv/k3-chaine-kl.csv`.

avec l'erreur d'arrondi des noyaux. L'écart se propage aux couches suivantes de la position, et un peu aux positions plus tardives, par leur attention. ■

Mesuré (figure 5.13) : sur 3 840 décisions, 334 ont une marge de référence sous 10^{-2}, 34 sous 10^{-3}, 4 sous 10^{-4} ; en routage libre, 22 décisions changent d'ensemble (et 15 seulement d'ordre). Sous routage imposé, la carte aurait basculé 3 décisions, de marges $1{,}4 \cdot 10^{-4}$, $3{,}3 \cdot 10^{-4}$ et $7{,}9 \cdot 10^{-4}$; η est donc de l'ordre de 10^{-4} à 10^{-3}, et quatre décisions de marge plus petite n'ont pas basculé : le signe de η décide [mes.].

La conséquence pour le critère C4. La KL de bout en bout d'un moteur f32 contre la référence f64 ne mesure pas la précision de ses noyaux ; elle compte les décisions dont la marge est sous le η du moteur, chacune au coût d'une bascule. troncons (activations f32, poids exacts) doit avoir un η plus petit que llama.cpp, qui ramène ses activations en Q8_0. C4 se jouera sur ce terme. Proposition du lot K à l'arbitre : juger toute comparaison de bout en bout aussi sous routage imposé, et compter les bascules à part, avec la marge de chacune.

Trouvé en chemin.

- **Le biais des clés de gpt-oss vaut exactement 0**, dans les deux GGUF : 512 valeurs sur 512, aux couches 0, 1 et 5 (`k3_biais_attention.py`), alors que les biais de q et de v sont pleins ($|b|$ jusqu'à 3,9) [mes.]. Sans puits, un biais de clé ajouterait $q \cdot b_k$ à tous les scores d'une tête et le softmax l'effacerait ; avec le puits dans le dénominateur, il ne s'effacerait pas. Sa nullité est une donnée du modèle, non une évidence ; la cassure « biais omis » l'a fait voir en ne tombant pas sur `attn_k`.
- **La fenêtre ne se voit qu'après 128 positions** (figure 5.14) : sous la cassure « fenêtre », la KL des positions 0 à 127 est égale au bit à celle de la chaîne saine ; sur les positions 128 à 159, elle vaut $8{,}2 \cdot 10^{-2}$. Un témoin de bout en bout plus court que la fenêtre ne voit pas cette faute.

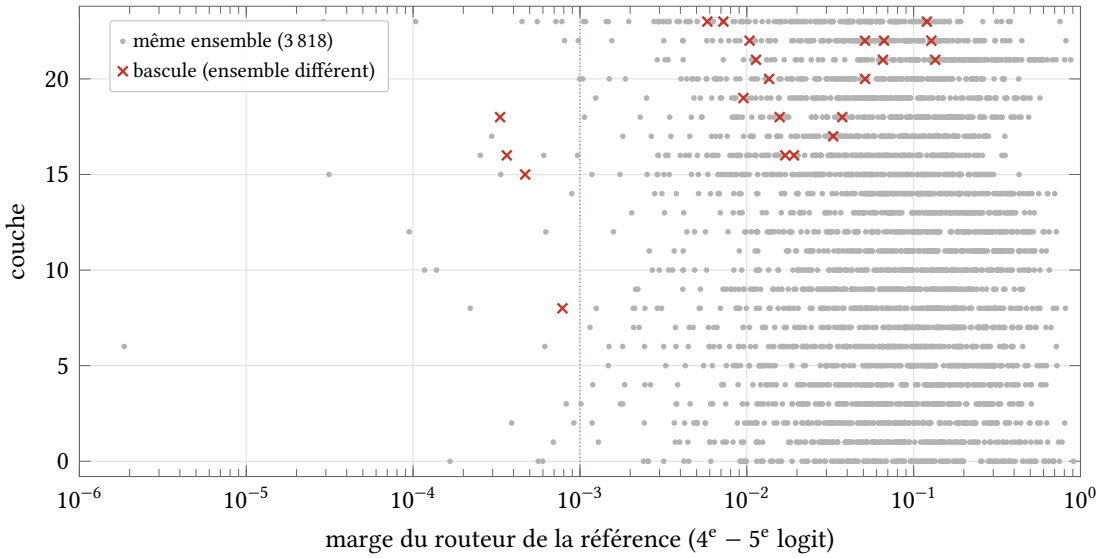


Figure 5.13 : K3, diagnostic D1 : chaque décision de routage de la chaîne en routage libre (24 couches × 160 positions), selon la marge du routeur de la référence [mes.]. Croix : la chaîne choisit un autre ensemble d'experts. Les bascules se concentrent aux petites marges ; la ligne marque 10^{-3}.

Données : `mesures/moteur/K/k3/k3-chaine-d1-gptoss20b-marges.csv`, dans `t2n-routage.csv`.

– **Le fma change des bits** : un `fmaf` remis dans `combiner` fait tomber l'égalité avec l'hôte (proposition 5.12).

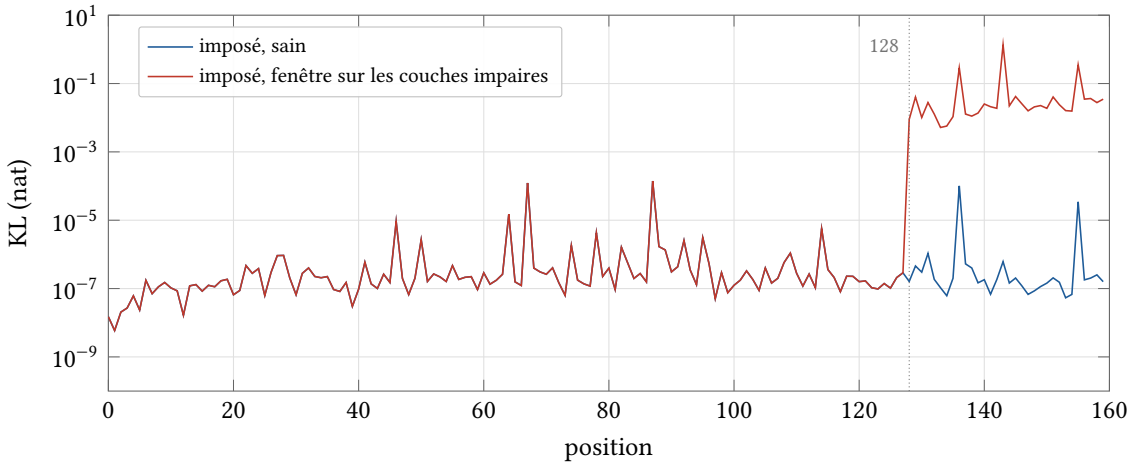


Figure 5.14 : K3 : la chaîne sous routage imposé, saine et sous la cassure « fenêtre posée sur les couches impaires » [mes.]. Les deux courbes sont égales au bit jusqu'à la position 127 ; la faute n'apparaît qu'au moment où l'anneau de 128 cases commence à oublier.

Données : `mesures/moteur/K/csv/k3-chaine-kl.csv` (colonnes `kl_impose`, `kl_impose_casse_fenetre`).

5.5 La ronde : le cache KV en f32

L'épreuve indépendante a trouvé que le premier jeton du texte de C12 différait de celui de la vérité f64, et que l'écart venait du cache KV. La ronde de corrections a ajouté deux noyaux, `ecrire_kv_f32` et `attention_kvf32`, qui font les mêmes calculs dans le même ordre que `ecrire_kv` et `attention`, à ceci près que les clés et les valeurs sont gardées en f32, sans toucher aux noyaux de la couture N (option `--kv-f32`). Témoin : égaux au bit au chemin f16 sur des valeurs représentables en f16 (cache plein et anneau) ; cassure : des valeurs non représentables font différer [mes.].

Proposition 5.17 (Ce que coûte l'arrondi f16 du cache)**.** *L'arrondi f16 d'une clé,* $\tilde{k} = k(1+\delta)$, $|\delta| \le u_{16} = 2^{-11} \approx 4{,}9\cdot10^{-4}$, *perturbe le score* $s = c\sum_d q_d k_d$ ($c = 1/8$, *64 termes*) *d'au plus* $c\,u_{16}\sum_d |q_d k_d|$ *au premier ordre, contre* $\gamma_{64} \approx 3{,}8\cdot10^{-6}$ *pour le produit f32 lui-même. En f32, la seule erreur qui reste est celle des noyaux,* $u_{32}/u_{16} = 2^{-13} \approx 1/8\,192$ *fois plus petite. La KL d'une petite perturbation* Δz *des logits est du second ordre,* $\mathrm{KL} \approx \frac{1}{2}\mathrm{Var}_p(\Delta z)$.

Démonstration. Le premier point est le développement au premier ordre de s en δ. Pour le second, soit $q_i \propto p_i e^{-\Delta z_i}$: $\mathrm{KL}(p\|q) = \mathbb{E}_p[\Delta z] + \ln \mathbb{E}_p[e^{-\Delta z}]$; le développement de $\ln \mathbb{E}_p[e^{-\Delta z}] = -\mathbb{E}_p[\Delta z] + \frac{1}{2}\mathrm{Var}_p(\Delta z) + O(\Delta z^3)$ donne le résultat. ∎

~~Divisée par $8\,192^2 \approx 6{,}7\cdot10^7$, la KL du f16 ($3{,}08\cdot10^{-3}$ sur le 120b) tomberait vers $5\cdot10^{-11}$ [… les bascules de routage ne le sont pas, mais elles disparaissent avec elle]~~ : estimation reprise du lot O, barrée par le théorème 6.9 (deux régimes de la KL d'un mélange d'experts). Le terme sans bascule, en u^2, est divisé par $8\,192^2 \approx 6{,}7\cdot10^7$; celui des bascules, linéaire en u, ne disparaît pas avec l'erreur : il est divisé par 8 192 seulement. Partant de la KL du f16 ($3{,}08\cdot10^{-3}$ sur la fenêtre 0 du 120b), la KL en f32 est donc au plus de l'ordre de $0{,}00308/8\,192 \approx 3{,}8\cdot10^{-7}$ nat [calc.] (section 6.6.3), encore bien sous le pas du format à 16 bits par logit ($\approx 2{,}4\cdot10^{-4}$ nat). Le seul juge reste la mesure : −0,00048, le zéro de l'instrument (section 6.6.4).

Table 5.7 : Le cache KV en f16 contre f32, contre la vérité f64 du juge [mes.]. La KL négative est celle de la référence f64 du lot R contre la même vérité ($-4{,}8\cdot10^{-4}$) : un biais commun du plancher du format, qui fixe le zéro de l'instrument.

cas	KL moyenne	premier choix	KL > 0,1	écart max des log p
120b, fenêtre 0, KV f16	0,00308	95,7 %	0,39 %	3,07 nat
120b, fenêtre 0, KV f32	**−0,00048**	**100 %**	0	**0,69 pas**
20b, fenêtre 0, KV f16	0,00174	96,5 %	0,39 %	2,32 nat
20b, fenêtre 0, KV f32	**−0,00048**	**100 %**	0	2,04 pas
120b, invite, positions 512 à 706, f16	0,00963	—	2,05 %	—
120b, invite, positions 512 à 706, f32	**−0,00054**	—	0	—

Données : `mesures/moteur/O/corrections/journaux/verdicts-kv-2026-10-16.json`, `m5-ecart-logp-*.json` ; `moteur/O-pour-le-memoire.md`, § 13.3.

Sur l'invite de 708 jetons, les argmax sont égaux à la vérité 708 sur 708 (f16 : 687), le routage égal sur toutes les (couche, position), et le premier jeton de C12 devient celui de la vérité [mes.]. **L'arrondi f16 du cache KV portait tout l'écart mesurable entre troncons et les mathématiques exactes du modèle.** Le coût : $18 \times (4\,096 + 128) \times 2 \times 512 \times 2\,\mathrm{o} = 155{,}7$ Mo en f16, le double en f32, soit $155{,}7/13{,}25 = 11{,}8$ emplacements d'experts de moins [calc.] ; mesuré : 326 → 314 emplacements [mes.]. Le choix de la couture N (f16 ou f32) revient à l'arbitre.

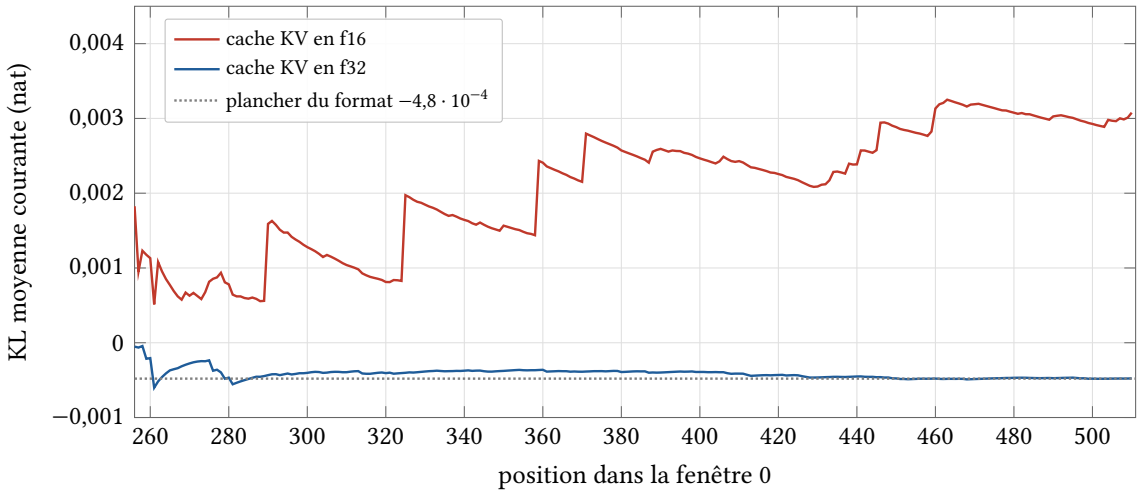


Figure 5.15 : KL moyenne courante contre la vérité f64, fenêtre 0 de gpt-oss-120b, positions notées 256 à 510 [mes.]. En f16, la KL monte par sauts (bascules de routage) jusqu'à 0,00308 ; en f32, elle reste au plancher du format.

Données : `mesures/moteur/0/corrections/csv/kv-f16-f32-par-position.csv` (cas `f0-120b`), moyenne courante dans `t2n-kv-120b.csv`.

5.6 Ce que le chapitre a barré

- ~~Référence de K1 : 41,6 Go/s~~ : composée par 03 avec le v4 sous Vulkan ; refaite sous CUDA, 45,8 Go/s.
- ~~Ce qui reste : K3 (la passe avant), K4~~ : K3 est fait (les douze noyaux tiennent sur les deux modèles) ; reste K4.
- ~~Le témoin 120b repose sur le codeur du banc : le second codeur n'est prouvé que sur le 20b~~ : prouvé aussi sur le 120b depuis 20 h 52 (4 608 / 4 608 contre le fichier du lot F).
- ~~Prédiction de D : DELTA en 19 à 22 µs~~ : 34,8 µs mesurés (contention à 8 warps par SM) ; le total, lui, tient.

Ce qui pourrait être faux

- **Le seuil de K1 compare deux grandeurs différentes** : 41,6 Go/s est un débit sur un grand flux (des milliers de tuiles en vol), celui de K1 un débit par expert (381 tuiles, une vague, plus 24 tuiles de DELTA seules). La comparaison est prudente pour nous, mais ce n'est pas la même grandeur ; C s'en rapproche.
- **L'effet de première série n'est pas expliqué** ; il a décidé du verdict de K1.
- **Le modèle de latence** ignore la queue de vague, le recouvrement de deux lancements et la contention de plusieurs warps de DELTA sur un SM : il a prédit juste le lot et faux la partie DELTA de D.
- **Les témoins par noyau ne voient que les couches 0 et 1**, et l'entrée de la couche 1 est l'enchâssement normé, pas l'état réel après la couche 0. Une couche aux scores d'attention plus pointus pourrait rapprocher l'attention de son seuil (marge 1,6). La chaîne passe par les 24 couches du 20b et tient sous routage imposé : cela borne l'effet sur le 20b, pas sur le 120b.
- **La chaîne n'a tourné que sur le 20b**, sur 160 positions d'une seule fenêtre ; sur le 120b, elle demande de l'ordre de 120 Go de lectures.
- **D2 impose les experts mais garde les poids de la carte** : il ne sépare pas une erreur sur

les poids du routeur d'une erreur sur les experts ; les deux sont bornées par les témoins par noyau.
- **Le seuil de la chaîne ($7 \cdot 10^{-5}$) était mal posé pour un MoE qui amplifie** ; il est resté tel quel et son verdict est publié ; le diagnostic a été déclaré après ce premier résultat.
- **L'attention lit au plus** 8 192 **positions par appel** (4 octets de mémoire partagée par position) : assez pour J1 (4 096), pas pour un long contexte.
- **La cadence de la fenêtre de mots** (4,2 sous-pas par avance pour CODES_CTX) est calculée sur un taux moyen ; le commentaire du code dit 8 ; aucune des deux n'est mesurée.
- **« KV f32 = vérité »** est mesuré sur une fenêtre de 512 jetons et une invite de 708 (120b) et une fenêtre du 20b, pas sur les 10 fenêtres.
- **Les débits dépendent de la fréquence de la carte d'un portable** ; l'état de la carte avant et après chaque passe est journalisé.

D'où l'on part, où l'on va. Le décodeur et les douze noyaux sont écrits, démontrés là où une preuve existe, bornés là où il n'y a qu'une borne, et vérifiés au bit ou sous leur seuil sur les poids réels des deux modèles ; ils coûtent au jeton deux ordres de grandeur de moins que le disque. Ce qui reste n'est plus de la justesse : c'est le choix du repli du décodeur, celui de la précision du cache KV, et, pour la suite, une attention en tuiles et le mode base de J2, où la GEMV lira une table de centres au lieu des grandeurs E2M1.

Deuxième partie

La vérité et la mémoire

Chapitre 6

La référence exacte et le juge

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, l'étude des modèles avait montré que deux moteurs justes ne rendent pas les mêmes logits : sur Qwen3.5-4B, le chemin du processeur et celui de la carte de llama.cpp s'écartaient d'une divergence de Kullback-Leibler (KL) de 0,0055, quand le même moteur lancé deux fois ne s'écartait de rien (au plus $6{,}8 \cdot 10^{-5}$, l'arrondi du format) [mes.]. L'architecture en a tiré une règle (critère C4) : troncons ne se juge pas contre un autre moteur, il se juge contre *les mathématiques exactes du modèle*, calculées en double précision. **Ce chapitre établit** cette vérité et l'instrument qui la compare : une passe avant de gpt-oss en f64 qui voit les poids du fichier sans aucun arrondi, deux chemins de calcul égaux au bit, la rampe de YaRN telle qu'OpenAI la définit (et non telle que llama.cpp l'arrondit) ; la théorie de la KL dont le juge a besoin, et les deux défauts de l'instrument que ses témoins ont trouvés ; puis la découverte du lot : **un mélange d'experts amplifie les plus petites erreurs**, parce que le choix des experts est une fonction discontinue de l'entrée ; 88 % de la KL disparaît quand on impose le routage ; l'arrondi du cache KV en f16 porte 70 à 77 % des bascules de routage du 120b ; et un cache KV en f32 rend troncons égal à la vérité f64 au plancher du format. **Il reste** la référence f64 du 120b sur les dix fenêtres (une seule fenêtre et une invite sont jugées), la décision de l'arbitre sur la couture N (le cache KV en f32), un critère qui juge les distributions de KL et non leur seule moyenne, et l'identification du second amplificateur (les 12 % qui survivent au routage imposé). Sources : `moteur/R-pour-le-memoire.md`, `moteur/REPRISE-R.md`, `moteur/O-pour-le-memoire.md` (partie III), `mesures/epreuve-J1.md`, et les journaux sous `mesures/moteur/R/`, `mesures/moteur/O/corrections/`, `mesures/moteur/epreuve-J1/`.

En bref

- La référence calcule exactement sur les poids exacts : un poids Q8_0 tient sur 18 bits significatifs, un poids MXFP4 sur 4, un f64 en offre 53 (lemme 6.1).
- Ses deux chemins (jeton par jeton, couche par couche) rendent les mêmes 4 826 112 logits au bit (proposition 6.2) ; le pavage qui la rend 1 900 fois plus économe en trafic mémoire ne change aucun bit (proposition 6.3).
- La KL d'une perturbation Δz des logits vaut exactement $\log \mathbb{E}_p e^{\Delta z} - \mathbb{E}_p \Delta z$ (proposition 6.6) : elle est du second ordre tant que le calcul est lisse, et du premier ordre dès qu'un expert bascule (théorème 6.9).
- Mesuré sur gpt-oss-20b : changer la seule rampe de YaRN (un écart de phase de 10^{-4} rad par position) donne une KL de 0,0145 ; imposer le routage la fait tomber à 0,00168 ; 1 157 positions qui basculent elles-mêmes portent 96,5 % de la KL.
- Sur le 120b, avec un cache KV en f32, troncons égale la vérité : KL −0,00048 (le zéro de l'instrument), 100 % de premiers choix, routage égal sur 36 couches × 708 positions ; en f16 :

> KL 0,00308.

6.1 Pourquoi juger contre une vérité mathématique

Un moteur d'inférence sans perte devrait rendre « les bons logits ». Mais lesquels ? Deux moteurs qui lisent les mêmes octets et qui sont tous deux corrects ne font pas les mêmes sommes dans le même ordre, ne gardent pas les activations dans le même format, n'arrondissent pas le cache des clés et des valeurs de la même façon. Leurs logits diffèrent, et la mesure qui le dit a un nom : G_{moteurs}, la KL entre deux moteurs justes. Juger troncons contre llama.cpp reviendrait à exiger qu'il reproduise les approximations de llama.cpp, ce qui n'a pas de sens ; le juger « à G_{moteurs} près » laisserait passer toute erreur plus petite que l'écart entre deux moteurs.

La réponse de l'architecture (réponse par délégation du 2026-10-15, réversible ; `decisions/ARCHITECTURE.md` § 12, point 9) est de juger chaque moteur contre **la fonction que le modèle définit**, calculée assez exactement pour que son erreur soit invisible à l'instrument. Le critère C4 s'écrit alors : la KL de la vérité à troncons ne dépasse pas celle de la vérité à llama.cpp, et troncons choisit au moins aussi souvent le même premier jeton que la vérité. La vérité ne diffère de rien.

Encore faut-il dire ce qu'est « le modèle ». Le fichier servi est le GGUF de ggml-org, où l'attention est quantifiée en Q8_0 alors que le modèle d'OpenAI la porte en bf16. La référence juge donc *le modèle tel que stocké dans le fichier qu'on sert*, pas le point de contrôle d'OpenAI : c'est le bon juge de troncons, qui sert ces mêmes octets, et ce n'est pas un juge de la quantification de ggml-org (`moteur/R-pour-le-memoire.md` § 6). Pour la forme du calcul (attention à puits, fenêtre glissante une couche sur deux, YaRN, SwiGLU bornée, routeur), la référence suit l'implémentation publiée par OpenAI et sa configuration [32, 33], et non llama.cpp [34] ; le section 6.2.4 montre que la différence n'est pas académique.

6.2 La passe avant en double précision

6.2.1 Des poids exacts

Le premier geste de la référence est de lire les poids *sans les arrondir*. Ce n'est pas une précaution, c'est un fait arithmétique qu'on démontre.

Lemme 6.1 (La déquantification est exacte en f64)**.** *Tout poids d'un tenseur Q8_0 ou MXFP4 de gpt-oss, et tout tenseur F16 ou F32, est représenté exactement par un nombre f64.*

Démonstration. Q8_0. Un bloc de 34 octets porte une échelle d en f16 puis 32 entiers $q \in [-128, 127]$; le poids vaut $w = d\,q$. Un f16 normal a 11 bits significatifs (10 stockés et le bit implicite) et un exposant de -14 à 15 (de -24 pour les sous-normaux) ; $|q| \le 128$ a au plus 7 bits significatifs ($128 = 2^7$ n'en a qu'un ; ~~$|q|$ tient sur 7 bits plus le signe~~, faux pour $q = -128$, qui demande 8 bits). Le produit d'entiers de 11 et de 7 bits significatifs tient sur au plus 18 bits significatifs, son exposant reste dans $[-24-7, 15+7]$, bien dans l'étendue normale du f64 ($[-1022, 1023]$) : il est exact, puisque le f64 offre 53 bits significatifs. *MXFP4.* Un bloc de 17 octets porte un exposant $e \in [0, 255]$ (E8M0) puis 16 octets, deux codes de 4 bits par octet ; le poids vaut $w = K[c] \cdot 2^{e-128}$, avec $K = \{0, \pm1, \pm2, \pm3, \pm4, \pm6, \pm8, \pm12\}$, les valeurs E2M1 doublées. $K[c]$ tient sur au plus 4 bits significatifs ($12 = 1100_2$) et 2^{e-128} est une puissance de deux d'exposant dans $[-128, 127]$: leur produit est exact. *F16, F32.* La conversion vers un format plus large est exacte. ∎

Deux détails du format, que la référence a dû trouver et que ses tests gardent (`moteur/crates/trc-reference/src/poids.rs`) : l'exposant est $e - 128$ et non $e - 127$, parce que K porte les grandeurs E2M1 doublées ($0{,}5 \to 1$) ; et l'octet j d'un bloc porte l'élément j dans son demi-octet faible et l'élément $j + 16$ dans son demi-octet fort (test `mxfp4_ordre_des_demi_octets`). Tableau 6.1 donne les deux blocs au bit près.

Table 6.1 : Les deux blocs quantifiés que lit la référence, champ par champ, et la valeur exacte qu'elle en tire.

Bloc	Octets	Champ	Valeur en f64
Q8_0 (34 o, 32 poids)	0–1	d, f16 petit-boutiste	exacte
	2–33	$q_0 \dots q_{31}$, i8	$w_j = d\, q_j$ (18 bits significatifs au plus)
MXFP4 (17 o, 32 poids)	0	e, E8M0	2^{e-128}
	$1 + j$, $j < 16$	bits 0–3 : c_j ; bits 4–7 : c_{j+16}	$w_j = K[c_j]\, 2^{e-128}$, $w_{j+16} = K[c_{j+16}]\, 2^{e-128}$

Conséquence : **toute l'erreur de la référence vient des sommes et des fonctions transcendantes**, en f64, de l'ordre de 10^{-16} en relatif par opération. Elle est invisible à un instrument qui code chaque logit sur 16 bits (section 6.4).

6.2.2 Deux chemins, une fonction

La référence offre deux chemins. Le premier calcule un jeton à la fois, toutes les couches, avec un cache KV, comme un moteur. Le second calcule une couche à la fois pour toutes les positions d'un lot de fenêtres : il lit chaque expert au plus une fois par couche. Ce qu'il économise se pose en une ligne [calc.] : jeton par jeton, dix fenêtres de 511 positions du 120b relisent $10 \times 511 \times 36 \times 4 \times 13{,}254$ Mo $\approx 9{,}75$ To d'experts ; couche par couche, au plus $36 \times 128 \times 13{,}254$ Mo $\approx 61{,}1$ Go, 160 fois moins, et le lot tient en mémoire ($5\,110 \times 2\,880 \times 8$ o $= 118$ Mo par tableau d'activations).

Proposition 6.2 (Les deux chemins calculent la même fonction, au bit)**.** *Si chaque scalaire est calculé par la même suite d'opérations dans les deux ordres, la passe couche par couche et la passe jeton par jeton rendent, position par position, des logits égaux au bit.*

Démonstration. Avec un masque causal, la sortie de la position t à la couche ℓ ne dépend que des entrées des positions $\leq t$ à la couche ℓ. Récurrence sur ℓ : si les entrées de la couche ℓ sont égales au bit dans les deux ordres pour toutes les positions, chaque sortie de la couche ℓ est une suite d'opérations IEEE 754 [25] sur des opérandes égaux, donc égale ; c'est vrai ici parce que le produit scalaire ne dépend que de ses deux vecteurs (jamais du nombre de positions ni du découpage en fils), que l'attention lit les mêmes clés dans le même ordre, et que la somme des quatre experts suit l'ordre $j = 0, 1, 2, 3$. À la couche 0, les entrées sont les lignes d'enchâssement, égales. ∎

Le témoin W1 en fait une égalité exacte, pas une tolérance : sur 24 jetons du 20b, **4 826 112 logits, 0 différent** ; la cassure (le chemin jeton par jeton avec l'autre rampe de YaRN) en fait différer 4 625 024, d'un écart maximal de 0,124 dès 24 jetons [mes.] (`mesures/moteur/R/r1/W1-chemins-20b.log`, `W1-chemins-casse-20b.log`).

6.2.3 Le pavage du produit, et pourquoi il ne change aucun bit

La première version du produit d'une matrice par un lot, $y[p, r] = \langle W_r, x_p \rangle$, parcourait les rangées et, pour chacune, relisait les n vecteurs d'entrée. Dès que ceux-ci ne tiennent plus dans le cache de dernier niveau (24 Mo), le trafic vers la mémoire vive vaut $T = R\,n\,C \cdot 8$ octets. Posons les nombres de

R1 [calc.] : $n = 10 \times 511 = 5\,110$ positions, $C = 2\,880$, donc $nC \cdot 8 = 117{,}7$ Mo ; pour la seule matrice de sortie ($R = 201\,088$ rangées), $T = 201\,088 \times 117{,}7$ Mo $\approx 23{,}7$ To, soit à 60 Go/s (ordre de grandeur de la DDR5, hypothèse) environ $3{,}9 \cdot 10^5$ s : plus de quatre jours pour une matrice. Le témoin W1, à 24 positions (553 Ko, tenus en cache), ne pouvait pas le voir (`moteur/R-pour-le-memoire.md` § 2.6).

Le pavage (cadre : l'anatomie du produit matriciel de Goto et van de Geijn [35]) range le calcul en quatre niveaux : les positions par paquets de 256 ($256 \times 2\,880 \times 8$ o $= 5{,}9$ Mo, tenus en L3) ; pour chaque paquet, une tâche par bloc de 32 rangées déquantifiées une fois (737 Ko, en L2) ; dans la tâche, des tuiles de 8 positions (184 Ko) ; dans la tuile, quatre produits à la fois (2 rangées × 2 positions), soit huit chaînes de FMA indépendantes. Le trafic vers la mémoire vive tombe aux octets quantifiés de W une fois par paquet, $\lceil n/256 \rceil\, RC\, b/8$ ($b = 8{,}5$ bits par poids pour Q8_0) : pour la sortie, $20 \times 0{,}615 = 12{,}3$ Go, environ **1 900 fois moins** [calc.].

Proposition 6.3 (Le pavage est égal au bit au produit scalaire seul)**.** *Soit $S(a, b)$ la suite d'opérations du produit scalaire de référence : pour $i = 0, 8, 16, \ldots < C_8 = 8\lfloor C/8 \rfloor$, $\ell \leftarrow \mathrm{fma}(a_{i..i+4}, b_{i..i+4}, \ell)$ et $h \leftarrow \mathrm{fma}(a_{i+4..i+8}, b_{i+4..i+8}, h)$ sur deux registres de quatre accumulateurs ; puis $s = ((\ell_0 + h_0) + (\ell_2 + h_2)) + ((\ell_1 + h_1) + (\ell_3 + h_3))$; puis $s \leftarrow \mathrm{fma}(a_j, b_j, s)$ pour $j = C_8 \ldots C - 1$. Alors, pour toute taille de paquet, de bloc, de tuile et tout nombre de fils, le produit pavé rend $y[p, r] = S(W_r, x_p)$ au bit.*

Démonstration. Le noyau 2×2 exécute, pour chacun des quatre couples (a_u, b_v), la suite $S(a_u, b_v)$ sur ses propres accumulateurs (ℓ_{uv}, h_{uv}), entrelacée avec les trois autres, et la même fonction de réduction, écrite une seule fois (listing 6.1). Une FMA IEEE 754 rend l'arrondi correct de $a \cdot b + c$, fonction de ses trois opérandes seulement [25] ; deux suites d'opérations identiques sur des opérandes identiques rendent donc des résultats identiques, quel que soit l'entrelacement. Le pavage ne change que *quels* produits sont calculés *quand* ; aucun ne change d'ordre interne. ■

Le témoin et ses deux cassures. Le test `produit_egal_scalaire_au_bit` (réels non dyadiques, bords partout) tient. La première cassure essayée, échanger ℓ et h pour le seul couple (a_1, b_1), *tient encore* : l'échange change $(\ell_k + h_k)$ en $(h_k + \ell_k)$, égal par commutativité exacte de l'addition IEEE. C'est une leçon de méthode que ce mémoire garde : **une cassure doit changer la fonction calculée, pas seulement le texte du code**. La seconde cassure (les huit termes de (a_1, b_1) sur la seule chaîne ℓ) tombe : 304 sorties sur $71 \times 19 = 1\,349$. W1 rejoué après le pavage : 4 826 112 logits, 0 différent, et la cassure de W1 rend exactement les mêmes 4 625 024 écarts qu'avant [mes.] (`mesures/moteur/R/r1/W1-chemins-20b-pavage.log`).

6.2.4 La rampe de YaRN : la définition d'OpenAI et la forme de llama.cpp

gpt-oss tourne ses requêtes et ses clés par paires (RoPE), avec des fréquences étirées par YaRN pour porter un contexte 32 fois plus long que celui de l'entraînement [28]. Les fréquences d'origine sont $\theta_i = b^{-2i/d}$, $i = 0 \ldots d/2 - 1$, avec $b = 150\,000$ et $d = 64$. YaRN « NTK par parties » mélange l'extrapolation θ_i et l'interpolation θ_i / s ($s = 32$) selon une rampe sur l'indice de paire :

$$r_i = \mathrm{clamp}\Big(\frac{i - \alpha}{\beta - \alpha}, 0, 1\Big), \qquad \omega_i = \theta_i\Big((1 - r_i) + \frac{r_i}{s}\Big). \tag{6.1}$$

Les bornes sont les paires dont la longueur d'onde $2\pi/\theta_i$ fait respectivement $\beta_r = 32$ et $\beta_l = 1$ tours sur le contexte d'origine $L_0 = 4\,096$:

$$2\pi\, b^{2i/d} = \frac{L_0}{\beta} \iff i(\beta) = \frac{d}{2} \cdot \frac{\ln\big(L_0/(2\pi\beta)\big)}{\ln b}. \tag{6.2}$$

Listing 6.1 : Le noyau 2 × 2 de la référence (extrait de `trc-reference/src/poids.rs` ; commentaires du lot R : « quatre produits scalaires à la fois, chacun par exactement la suite d'opérations de scalaire_avx2 : huit chaînes de FMA » ; « une seule écriture de l'ordre de réduction, donc une seule fonction calculée »). Chaque produit garde ses deux accumulateurs et la réduction commune ; seules les quatre suites sont entrelacées.

```
// Quatre produits scalaires à la fois, (a0.b0, a0.b1, a1.b0, a1.b1), chacun par
// exactement la suite d'opérations de scalaire_avx2 : huit chaînes de FMA.
while i < n8 {
    let (x0l, x0h) = (_mm256_loadu_pd(pa0.add(i)), _mm256_loadu_pd(pa0.add(i + 4)
    ));
    let (x1l, x1h) = (_mm256_loadu_pd(pa1.add(i)), _mm256_loadu_pd(pa1.add(i + 4)
    ));
    let (y0l, y0h) = (_mm256_loadu_pd(pb0.add(i)), _mm256_loadu_pd(pb0.add(i + 4)
    ));
    let (y1l, y1h) = (_mm256_loadu_pd(pb1.add(i)), _mm256_loadu_pd(pb1.add(i + 4)
    ));
    lo00 = _mm256_fmadd_pd(x0l, y0l, lo00);  hi00 = _mm256_fmadd_pd(x0h, y0h,
    hi00);
    lo01 = _mm256_fmadd_pd(x0l, y1l, lo01);  hi01 = _mm256_fmadd_pd(x0h, y1h,
    hi01);
    lo10 = _mm256_fmadd_pd(x1l, y0l, lo10);  hi10 = _mm256_fmadd_pd(x1h, y0h,
    hi10);
    lo11 = _mm256_fmadd_pd(x1l, y1l, lo11);  hi11 = _mm256_fmadd_pd(x1h, y1h,
    hi11);
    i += 8;
}
// Une seule écriture de l'ordre de réduction, donc une seule fonction calculée.
[reduire_avx2(lo00, hi00, a0, b0, n8), reduire_avx2(lo01, hi01, a0, b1, n8),
 reduire_avx2(lo10, hi10, a1, b0, n8), reduire_avx2(lo11, hi11, a1, b1, n8)]
```

Calcul posé [calc.] : $\ln b = \ln 150\,000 = 11{,}9184$; $L_0/(2\pi \cdot 32) = 20{,}372$, de logarithme 3,0141 ; $L_0/(2\pi) = 651{,}90$, de logarithme 6,4798. Donc

$$\alpha = 32 \times \frac{3{,}0141}{11{,}9184} = \mathbf{8{,}0928}, \qquad \beta = 32 \times \frac{6{,}4798}{11{,}9184} = \mathbf{17{,}398}.$$

L'implémentation de référence d'OpenAI garde ces bornes réelles, et sa configuration le dit : la clé `truncate` y vaut `false` ; llama.cpp b11515 prend $\lfloor\alpha\rfloor = 8$ et $\lceil\beta\rceil = 18$ (fonction `ggml_rope_yarn_corr_dims`). La référence suit OpenAI et garde la variante de llama.cpp pour en chiffrer l'écart (listing 6.2). Les deux rampes ne diffèrent que sur quelques paires (figure 6.1) : ~~pour $8 \le i \le 17$~~, corrigé par le second chemin (`rampe_yarn.py`, `mesures/moteur/R/r1/rampe-yarn-temoin.json`) : **pour** $9 \le i \le 17$, car à $i = 8$ les deux rampes valent 0 ($8 < 8{,}0928$ d'un côté, $(8-8)/10 = 0$ de l'autre) et à $i = 18$ les deux valent 1. Le second chemin retrouve les fréquences à moins d'un ulp (0,94 au plus) ; sa cassure (la rampe arrondie prise pour celle d'OpenAI) fait tomber les paires 9 à 17, et elles seules.

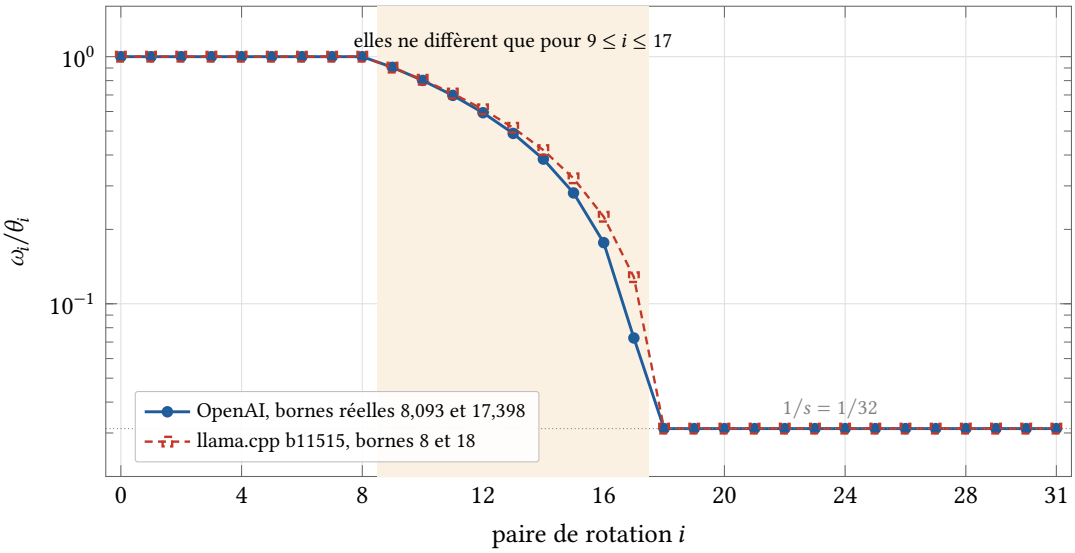


Figure 6.1 : Le rapport ω_i/θ_i de la fréquence étirée à la fréquence d'origine, pour les 32 paires d'une tête de gpt-oss. Au-dessous de la rampe, extrapolation (rapport 1) ; au-dessus, interpolation (rapport 1/32). Les deux définitions ne s'écartent que sur la bande colorée.

Données : `mesures/moteur/R/r1/csv/rampe-yarn.csv` (écrit par `juge rampe`, le code de la référence ; refait par `rampe_yarn.py`).

Ordre de grandeur de l'écart. À la paire $i = 17$: $\theta_{17} = b^{-34/64} = 1{,}78 \cdot 10^{-3}$; chez OpenAI, $r = 0{,}957$, donc $\omega/\theta = 0{,}043 + 0{,}957/32 = 0{,}0729$; chez llama.cpp, $r = 0{,}9$, $\omega/\theta = 0{,}1 + 0{,}9/32 = 0{,}128$. D'où $\Delta\omega = 1{,}78 \cdot 10^{-3} \times (0{,}128 - 0{,}0729) = 9{,}8 \cdot 10^{-5}$ rad par position, soit 0,05 rad à la position 500 [calc.]. À la paire 9 : $\theta_9 = b^{-18/64} = 0{,}0350$; chez OpenAI, $r = (9 - 8{,}0928)/(17{,}3980 - 8{,}0928) = 0{,}0975$, donc $\omega/\theta = 0{,}9025 + 0{,}0975/32 = 0{,}90555$; chez llama.cpp, $r = 0{,}1$, $\omega/\theta = 0{,}9 + 0{,}1/32 = 0{,}90313$; d'où $\Delta\omega = 0{,}0350 \times 0{,}00243 \approx 8{,}5 \cdot 10^{-5}$ rad par position [calc.]. La note du lot R (`moteur/R-pour-le-memoire.md`) écrivait ~~$8{,}8 \cdot 10^{-5}$~~, repris ici sans être refait jusqu'au jugement du Tome II. Petit, mais non nul, et l'écart croît avec la position : à 131 072 jetons, 12,8 rad, une phase sans rapport. Ce que ce petit écart coûte en KL est l'objet du section 6.5, et la réponse surprend.

Le facteur de concentration. YaRN multiplie aussi le cosinus et le sinus par $0{,}1 \ln s + 1 = 1{,}3466$, donc le produit $q \cdot k$ par $1{,}3466^2 = 1{,}8133$. llama.cpp obtient la même valeur par un autre chemin

(`yarn_attn_factor` ramené à 1, puis `mscale`). L'omettre est l'erreur injectée de la cassure de R1 (section 6.4.3).

Listing 6.2 : Les fréquences de YaRN dans la référence (`trc-reference/src/ops.rs`). La variante de llama.cpp arrondit les bornes ; celle d'OpenAI les garde réelles.

```
let concentration = 0.1 * facteur.ln() + 1.0;
// Dimension de correction : la paire dont la longueur d'onde 2 pi / inv_freq
    fait beta tours sur L0.
let corr = |beta: f64| (demi as f64) * (ctx_origine / (beta * 2.0 * PI)).ln() /
    base.ln();
let (mut bas, mut haut) = (corr(beta_rapide), corr(beta_lent));
if v == VarianteYarn::LlamaCpp {
    bas = bas.floor().max(0.0);
    haut = haut.ceil().min((d - 1) as f64);
}
let inv_freq = (0..demi).map(|i| {
    let interpolation = 1.0 / (facteur * freq[i]);
    let extrapolation = 1.0 / freq[i];
    let rampe = ((i as f64 - bas) / (haut - bas)).clamp(0.0, 1.0);
    let masque = 1.0 - rampe;
    interpolation * (1.0 - masque) + extrapolation * masque
}).collect();
```

6.3 La divergence de Kullback-Leibler, et ce qu'elle mesure

6.3.1 Définition et premières propriétés

Soit p et q deux lois sur le vocabulaire V ($|V| = 201\,088$ pour gpt-oss). La divergence de Kullback-Leibler [2] de q à p est

$$\mathrm{KL}(p \,\|\, q) = \sum_{i \in V} p_i \log \frac{p_i}{q_i}. \tag{6.3}$$

Elle se lit comme le nombre moyen de nats perdus à coder des jetons tirés selon p avec un code optimal pour q [36]. Deux faits la rendent propre à notre usage.

Théorème 6.4 (Inégalité de Gibbs)**.** $\mathrm{KL}(p \,\|\, q) \geq 0$*, avec égalité si et seulement si* $p = q$*.*

Démonstration. Sur le support de p, $-\log$ est strictement convexe ; par l'inégalité de Jensen,

$$\sum_i p_i \log \frac{q_i}{p_i} \leq \log \sum_{i \,:\, p_i > 0} q_i \leq \log 1 = 0,$$

avec égalité si et seulement si q_i / p_i est constant sur le support de p et que ce support porte toute la masse de q, c'est-à-dire $q = p$. ∎

Proposition 6.5 (Entropie croisée, perplexité)**.** *Pour la loi vraie* p *du jeton suivant, l'entropie croisée d'un moteur* q *se décompose en* $H(p, q) = H(p) + \mathrm{KL}(p \,\|\, q)$*. La perte moyenne du moteur sur un texte tiré selon* p *dépasse donc celle du modèle exact d'exactement sa KL moyenne, et le rapport des perplexités vaut* $\exp(\overline{\mathrm{KL}})$*.*

Démonstration. $H(p, q) = -\sum_i p_i \log q_i = -\sum_i p_i \log p_i + \sum_i p_i \log(p_i / q_i)$. La perplexité est l'exponentielle de la perte moyenne. ∎

Le sens de la KL compte. Le juge écrit KL(vérité ‖ moteur), la vérité en base : la somme est pondérée par ce que *le modèle exact* tient pour probable, et c'est là qu'une erreur coûte (proposition 6.5). Ce choix est celui du critère R1 ; le critère C4 l'écrivait dans l'autre sens, et l'ingénieur du lot R l'a aligné sur R1 pour qu'un seul instrument juge les deux (réponse par délégation du 2026-10-15, réversible ; `moteur/REPRISE-R.md`, besoin 2). Le juge relève d'ailleurs les deux sens.

6.3.2 La KL d'une petite perturbation des logits

Un moteur approché ne rend pas une loi quelconque : il rend les logits de la vérité, perturbés. Soit $z \in \mathbb{R}^V$ les logits exacts, $p = \mathrm{softmax}(z)$, et $q = \mathrm{softmax}(z + \Delta z)$.

Proposition 6.6 (La KL est une fonction génératrice des cumulants)**.**

$$\mathrm{KL}(p \,\|\, q) = \log \mathbb{E}_p\big[e^{\Delta z}\big] - \mathbb{E}_p[\Delta z] = \tfrac{1}{2}\,\mathrm{Var}_p(\Delta z) + \tfrac{1}{6}\kappa_3 + \cdots \tag{6.4}$$

où $\mathbb{E}_p$ *et* Var_p *portent sur l'indice* i *tiré selon* p*, et* κ_3 *est le troisième cumulant de* Δz *sous* p*. En particulier, la KL ne dépend pas d'un décalage commun des logits, et elle est* du second ordre *en* Δz*.*

Démonstration. $\log q_i = z_i + \Delta z_i - \log \sum_j e^{z_j + \Delta z_j}$ et $\log p_i = z_i - \log \sum_j e^{z_j}$. Donc $\log p_i - \log q_i = -\Delta z_i + \log\big(\sum_j e^{z_j} e^{\Delta z_j} / \sum_j e^{z_j}\big) = -\Delta z_i + \log \mathbb{E}_p[e^{\Delta z}]$. On moyenne sous p. Le développement est celui de la fonction génératrice des cumulants $K(t) = \log \mathbb{E}_p e^{t\Delta z} = t\kappa_1 + t^2\kappa_2/2 + t^3\kappa_3/6 + \cdots$ en $t = 1$, avec $\kappa_1 = \mathbb{E}_p \Delta z$ et $\kappa_2 = \mathrm{Var}_p\, \Delta z$. ∎

Cette identité est exacte ; elle servira deux fois. Elle dit d'abord que la KL d'un arrondi est minuscule : si chaque logit est perturbé d'un relatif u, $\mathrm{Var}_p(\Delta z) = O(u^2)$ et la KL est en u^2. Elle dira ensuite pourquoi un mélange d'experts échappe à ce régime (théorème 6.9).

6.3.3 Le premier choix, et ses intervalles

Le juge relève aussi, position par position, si le moteur et la vérité donnent le même jeton le plus probable (« premier choix », égalité tranchée par le plus petit indice). Une proportion $\hat\pi = k/n$ porte l'intervalle de score de Wilson à 95 % [37], avec $z = 1{,}96$:

$$\frac{\hat\pi + \frac{z^2}{2n}}{1 + \frac{z^2}{n}} \pm \frac{z}{1 + \frac{z^2}{n}} \sqrt{\frac{\hat\pi(1-\hat\pi)}{n} + \frac{z^2}{4n^2}}. \tag{6.5}$$

Une moyenne de KL porte l'incertitude $\sqrt{(\langle x^2\rangle - \langle x\rangle^2)/(n-1)}$, comme llama.cpp (`moteur/crates/trc-juge/src/stats.rs`).

6.3.4 Juger deux moteurs sur les mêmes positions

C4 compare deux KL mesurées *aux mêmes positions* contre la même vérité V : $x_i = \mathrm{KL}(V_i \,\|\, A_i)$ pour llama.cpp, $y_i = \mathrm{KL}(V_i \,\|\, B_i)$ pour troncons, et $d_i = y_i - x_i$. Alors

$$\mathrm{Var}(\bar d) = \frac{\sigma_x^2 + \sigma_y^2 - 2\rho\,\sigma_x\sigma_y}{n}, \qquad \hat\sigma(\bar d) = \sqrt{\frac{\langle d^2\rangle - \langle d\rangle^2}{n-1}}. \tag{6.6}$$

Deux moteurs qui se trompent aux mêmes positions difficiles ont ρ proche de 1, et la comparaison appariée voit une différence que celle de deux moyennes indépendantes noierait. Le lot R propose à l'arbitre un verdict statistique, $\bar d \le 2\hat\sigma(\bar d)$ et $\bar e \ge -2\hat\sigma(\bar e)$ (où $e_i \in \{-1, 0, 1\}$ compare les premiers choix), à côté du verdict littéral (les deux moyennes).

Le défaut du verdict littéral, observé. Essayé sur deux moteurs réels contre la vérité (A = llama.cpp carte, B = llama.cpp processeur ; `mesures/moteur/R/r2/comparer-ref-carte-processeur-20b.json`) : KL 0,022606 contre 0,026470 ; premiers choix 89,61 % contre 90,39 % ; $\bar{d} = +0{,}00386 \pm 0{,}00236$ ($1{,}6\,\sigma$), $\bar{e} = +0{,}0078 \pm 0{,}0055$ [mes.]. Le verdict littéral sur la KL tombe, celui sur le premier choix tient ; les deux verdicts statistiques tiennent. Deux moteurs que rien ne distingue en justesse reçoivent des verdicts littéraux contraires selon la mesure. L'écart-type apparié (0,00236) n'est guère plus petit que $\sqrt{0{,}0017^2 + 0{,}0025^2} = 0{,}0030$: les deux moteurs ne se trompent pas aux mêmes positions, et le section 6.5 dira pourquoi. Cassure (B = la référence sans échelle de YaRN) : les quatre verdicts tombent ($\bar{d} = 0{,}220 \pm 0{,}008$, $\bar{e} = -0{,}315 \pm 0{,}010$).

6.4 L'instrument et ses témoins

6.4.1 Le fichier de logits et son plancher

Pour que tous les moteurs passent par le même instrument, le juge lit et écrit le format de llama-perplexity (`--kl-divergence-base` ; relevé dans `tools/perplexity` de b11515, rien n'est copié). Par position notée, il stocke deux réels et un code de 16 bits par jeton du vocabulaire (figure 6.2).

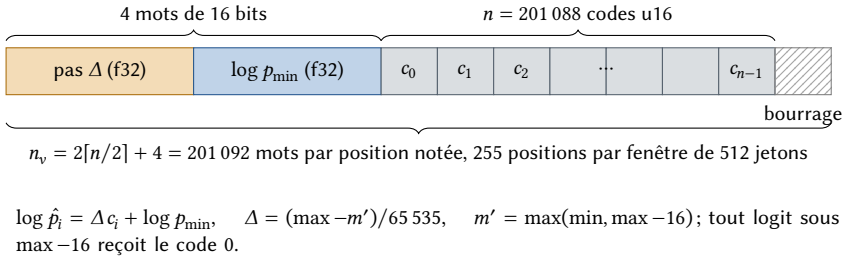


Figure 6.2 : Une entrée du fichier `.kld`, par position notée (positions 256 à 510 de chaque fenêtre de 512 jetons). Le jeton i reçoit le code c_i ; sa log-probabilité se relit par $\Delta c_i + \log p_{\min}$.

Données : `moteur/crates/trc-juge/src/kld.rs`.

Lemme 6.7 (Le plancher de l'instrument)**.** *L'erreur de codage d'une log-probabilité au-dessus de la coupe vérifie $|\delta_i| \le \Delta/2 \le 16/131\,070 = 1{,}22 \cdot 10^{-4}$ nat. Une distribution jugée contre son propre codage a une KL d'au plus $\Delta/2$ au premier ordre, et non $O(\Delta^2)$.*

Démonstration. Le code est l'arrondi au plus proche de $(\log p_i - \log p_{\min})/\Delta$, d'où $|\delta_i| \le \Delta/2$, et $\Delta = (\max - m')/65\,535 \le 16/65\,535$. Contre soi-même, $\log \hat{p}_i = \log p_i + \delta_i$ et la KL vaut $\sum_i \hat{p}_i(\log \hat{p}_i - \log p_i) \approx \sum_i p_i \delta_i$, bornée par $\max|\delta_i|$; elle ne descend pas à l'ordre 2 parce que les $\hat{p}$ décodés ne sont pas renormalisés (ils ne somment pas à 1). ∎

C'est le plancher mesuré par l'étude des modèles (au plus $6{,}8 \cdot 10^{-5}$, soit $\Delta/2$ pour un écart max − min d'environ 9) et l'origine du seuil de $7 \cdot 10^{-5}$. Le premier test unitaire du juge avait un seuil posé au jugé, ~~$|\mathrm{KL}| \le 10^{-7}$~~ ; il est tombé à $5{,}7 \cdot 10^{-6}$, et a été remplacé par la borne dérivée, $|\mathrm{KL}| \le \Delta$ (`moteur/REPRISE-R.md`).

6.4.2 Le défaut que le témoin d'identité a trouvé : le jugé codé ne se renormalise pas

Quand le jugé Q est lui-même un fichier `.kld` (le second chemin de R0 et R1, et C4, où troncons passe par le même codage), ses valeurs décodées sont *déjà* des log-probabilités, normalisées par le

log-somme-exp des logits exacts au moment du codage. La première version du juge les renormalisait. Or le codage écrase tous les jetons sous max -16 sur le code 0, donc sur $\log p_{\min}$, au lieu de leur vraie valeur, bien plus petite. Renormaliser ajoute alors à chaque position

$$\log \sum_i \hat{q}_i = \log\Big(\sum_{c_i>0} \hat{q}_i + N_0\, e^{\log p_{\min}}\Big) \approx N_0\, e^{\log p_{\min}}, \tag{6.7}$$

où N_0 est le nombre de jetons écrasés. Pour gpt-oss, presque tous les jetons sont sous la coupe : $N_0\, e^{-16} \approx 2 \cdot 10^5 \times 1{,}1 \cdot 10^{-7} \approx 2 \cdot 10^{-2}$ au pire [calc.] ; **mesuré :** $1{,}11 \cdot 10^{-3}$ **en moyenne** sur 2 550 positions, pour un fichier jugé contre lui-même (`mesures/moteur/R/r2/comparer-temoin-identite-renormalise-AVANT-CORRECTION.json`), seize fois le seuil de $7 \cdot 10^{-5}$ [mes.]. Le second chemin de R0, de R1 et le jugement C4 auraient été aveugles sous 10^{-3}. Corrigé (listing 6.3) : le même fichier contre lui-même donne **0 exactement** (`comparer-temoin-identite.json`), et le test qui le garde tombe quand on remet la renormalisation ($> 10^{-4}$). Ce que la correction laisse, dit : un jeton que P tient pour probable et que Q a écrasé reçoit $\log p_{\min}$, une borne haute de sa vraie log-probabilité ; le terme est sous-estimé, jamais infini.

Listing 6.3 : La KL d'une position quand le jugé est une entrée décodée (`trc-juge/src/stats.rs`) : on ne renormalise pas. Les commentaires disent : le codage écrase tous les jetons sous max − 16 sur log_p_min ; renormaliser ajoute environ $N_0 e^{\log p_{\min}}$ (1,11e-3 mesuré sur gpt-oss-20b) ; sans renormalisation, un fichier contre lui-même donne exactement 0.

```rust
/// Le codage écrase tous les jetons sous max - 16 sur log_p_min ; renormaliser l'entrée
/// décodée ajoute log(somme des p décodés) ~ N0 * exp(log_p_min) (1,11e-3 mesuré, gpt-oss-20b).
/// Sans renormalisation, un fichier contre lui-même donne 0 exactement.
pub fn position_decodee(p: &Entree, q: &Entree, n: usize, suivant: usize) -> Position {
    position_avec(p, |i| q.log_p(i) as f64, n, suivant, false)
}
// … dans position_avec : seuls les termes de la base au-dessus de la coupe comptent.
if lp > -16.0 {
    kl += lp.exp() * (lp - (q(i) - lse_q));   // lse_q = 0 sans renormalisation
}
```

La KL négative contre un fichier. Une vérité jugée contre un fichier qui code un calcul identique ne donne pas zéro mais une petite valeur négative. Soit $\tilde{q}$ la loi relue, non normalisée : ses jetons sous la coupe reçoivent $\log q_{\min} > \log q_i$, et $\sum \tilde{q} = e^{\delta}$, avec δ mesuré à 0,0020 en moyenne (`mesures/moteur/epreuve-J1/journaux/normalisation-kld-20b-f0.json`). La KL tend alors vers $-\sum_{i \text{ sous la coupe}} p_i(\log q_{\min} - \log p_i) < 0$, mesurée à $-4{,}8 \cdot 10^{-4}$ (`mesures/epreuve-J1.md` § 3.3) [mes.]. Ce biais est commun à tous les fichiers jugés ; il fixe le zéro de l'instrument et ne change aucun ordre. On le retrouvera au section 6.6.

6.4.3 Les témoins : ce que l'instrument voit, et ce qu'il ne voit pas

Les passes de llama.cpp sur la carte pour R0 et R1 ont tourné le 2026-10-15 de 19 h 59 à 20 h 20, sur secteur, carte libre (celles de la référence, de 15 h 35 à 17 h 35), sur dix fenêtres de 512 jetons de wikitext-2 [38], soit 2 550 positions notées, par deux chemins indépendants : llama-perplexity, et `juge kl` (tableau 6.2).

Table 6.2 : L'écart entre moteurs (R0) et la validation de la référence (R1) sur gpt-oss-20b, dix fenêtres, 2 550 positions [mes.]. « Base ‖ jugé » ; chemin 1 : llama-perplexity ; chemin 2 : `juge kl`.

Mesure	KL (ch. 1)	médiane	même 1er	ch. 2	verdict
T1 plancher : carte ‖ carte	−0,000000	−0,000000	100 %	0	tient
G_{cfg} : `--cpu-moe` ‖ `--fit`	0	0	100 %	0	publié : 0
G_{moteurs} : carte ‖ processeur	0,022741	0,0024	91,57 %	0,022735	T3 tient
T2 cassure : carte ‖ $k = 2$ experts	0,4513	0,290	47,96 %	—	tient
R1 : vérité ‖ llama.cpp carte	**0,022612**	0,0029	89,61 %	0,022606	tient, de 0,6 %
R1 cassure : vérité sans échelle ‖ llama.cpp	0,2644	0,131	57,80 %	0,26372	tombe
mesure Y : réf. à rampe llama.cpp ‖ llama.cpp	0,022603	0,0027	89,96 %	0,022581	non jugée

Données : `moteur/R-pour-le-memoire.md` § 2.11 ; `mesures/moteur/R/r0/r0-resume.json`, `mesures/moteur/R/r1/r1-resume-20b.json` ; chemins 2 refaits ici par `memoire/outils/tome2_reference_cache.py`.

Ce qu'ils disent :

- **L'instrument voit** : enlever deux experts sur quatre (T2) fait une KL de 0,45 ; ôter le facteur de concentration de YaRN à la référence (cassure de R1) la fait tomber d'un facteur 12.
- **Les moteurs sont déterministes** : les deux passes de chaque configuration écrivent des fichiers identiques octet pour octet ; la référence rejouée aussi (expérience D, 1 Go identique).
- $G_{\textbf{cfg}} = 0$ **exactement**, en évaluation par lots de 512 : les répartitions `--fit` et `--cpu-moe` sont identiques au bit. Interprétation, non vérifiée dans le code : sur un lot de 512, llama.cpp calcule sur la carte les produits dont les poids sont en mémoire de l'hôte, en les y copiant ; quelle que soit la répartition, ce sont les mêmes noyaux sur les mêmes octets. G_{cfg} n'est donc nul qu'en évaluation, pas en génération.
- $G_{\textbf{moteurs}}$ **de gpt-oss vaut 0,0227**, quatre fois le chiffre de Qwen3.5-4B (0,0055) que le seuil Q1 du jalon J2 reprenait : c'est à l'arbitre de le barrer et de le remplacer (`moteur/REPRISE-R.md`, besoin 6).
- **R1 tient, de 0,6 %** : la référence n'a pas d'erreur grossière. Mais la variante à rampe llama.cpp est aussi près de llama.cpp que la référence (0,022603 contre 0,022612) : à 0,02 près, tout calcul juste ou presque juste est à la même distance de llama.cpp. R1 ne départage donc qu'une erreur grossière d'une référence juste, et la justesse fine de la référence repose sur ses propres témoins (W1, D, les tests de chaque opération contre sa définition, le second chemin de la rampe), et sur le troisième calcul qui suit.

Un troisième calcul. L'épreuve indépendante a écrit sa propre passe avant de gpt-oss en numpy f64, depuis la forme publiée par OpenAI, sans rien lire du lot R (`mesures/moteur/epreuve-J1/gptoss_numpy.py`). Elle retrouve les bornes de la rampe (8,09278 et 17,39802). Sur les 255 positions de la fenêtre 0 du 20b et ses 201 088 jetons chacune, la référence du lot R l'égale à **0,000123 nat, soit 0,504 pas de quantification** du format (pas médian $2{,}44 \cdot 10^{-4}$) : l'arrondi à 16 bits seul, plus 0,004 pas d'arrondi f32 de Δ et de $\log p_{\min}$; 255 positions sur 255 sous un pas ; 100 % de premiers choix identiques. La rampe de llama.cpp, elle, s'en écarte jusqu'à 6,5 nat [mes.] (`mesures/moteur/epreuve-J1/journaux/ecart-logp-20b-f0.json`). Figure 6.3 résume d'où vient chaque distribution que le juge compare, et ce qui relie chaque paire.

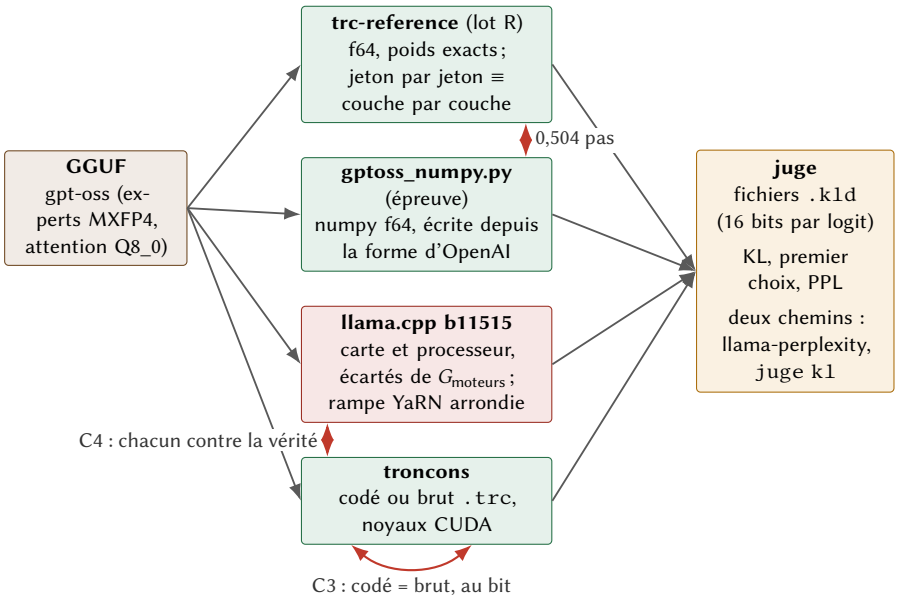


Figure 6.3 : Les chemins de la vérité. Quatre calculs lisent le même GGUF ; le juge compare leurs fichiers `.kld` par deux chemins. Sur chaque lien rouge, la mesure qui relie deux calculs : la référence et la passe numpy de l'épreuve à 0,504 pas du format ; troncons codé et brut au bit (C3) ; troncons et llama.cpp, chacun contre la vérité (C4).

6.5 La découverte : un mélange d'experts amplifie les plus petites erreurs

6.5.1 Les nombres

R1 a été accompagné d'une série d'expériences, chacune déclarée avant sa passe (`moteur/REPRISE-R.md`), qui isolent une seule différence entre deux calculs en f64 (tableau 6.3).

Trois faits sautent aux yeux, et figure 6.4 les montre.

1. **La KL n'est pas un bruit réparti, c'est une queue.** La médiane est dix fois sous la moyenne, le 99e centile cent fois au-dessus : la plupart des positions sont presque justes, et une minorité diverge franchement.
2. **Une perturbation minuscule coûte autant qu'un autre moteur.** Changer la seule rampe, dont l'effet direct est une phase de l'ordre de 10^{-4} rad par position sur neuf paires (section 6.2.4), donne déjà 0,0145 ; ajouter seulement des arrondis de l'ordre de 10^{-3} (Q8_0 sur les activations, f16 sur le cache KV) donne 0,029, plus que l'écart entre la référence et llama.cpp lui-même.
3. **Imiter les approximations ne rapproche presque pas** : la référence munie des arrondis de llama.cpp n'est guère plus près de lui (0,0220 contre 0,0273). Dans un régime qui amplifie, deux calculs perturbés différemment sont aussi loin l'un de l'autre que de l'exact.

L'hypothèse I, déclarée avant la passe des imitations (seuil $2{,}5 \cdot 10^{-3}$), supposait un régime linéaire ; elle est tombée, et c'est le régime qu'elle a révélé.

6.5.2 Le mécanisme : la bascule de routage

À chaque couche, le routeur de gpt-oss calcule un logit par expert (32 sur le 20b, 128 sur le 120b), garde les quatre plus grands et normalise leurs poids par un softmax sur les quatre. La sortie de la

Table 6.3 : Ce qu'une petite différence de calcul coûte en KL sur gpt-oss-20b (dix fenêtres, 2 550 positions) [mes.]. Les quatre dernières lignes ne comparent que deux passes de la référence f64.

Base ‖ jugé	KL moyenne	médiane	99e centile	même 1er
llama.cpp processeur ‖ référence (rampe OpenAI)	0,0255	0,0027	0,47	90,4 %
llama.cpp processeur ‖ référence (rampe llama.cpp)	0,0273	0,0027	—	90,4 %
llama.cpp processeur ‖ réf. (rampe llama.cpp, Q8_0 et KV f16 imités)	0,0220	0,0025	0,40	91,6 %
réf. OpenAI ‖ réf. llama.cpp (*la rampe seule change*)	0,0145	$9{,}8 \cdot 10^{-4}$	0,29	92,5 %
réf. llama.cpp ‖ la même avec imitations (*seuls des arrondis changent*)	0,0291	0,0030	0,54	90,3 %
réf. OpenAI ‖ réf. llama.cpp, *routage imposé*	0,00168	$2{,}2 \cdot 10^{-4}$	0,031	97,2 %
llama.cpp processeur ‖ réf. sans échelle (cassure)	0,249	—	—	57,3 %

Données : `moteur/R-pour-le-memoire.md` § 2.8 et § 2.9 ; `mesures/moteur/R/r1/juge-kl-*.csv`. La ligne des imitations était publiée à 0,029 ; refaite ici à 0,02905 (`t2-verif-reference-cache.json`).

couche est

$$f(h) = \sum_{j \in \mathrm{top}_4(z(h))} g_j(h)\, E_j(h), \qquad g = \mathrm{softmax}\left(z_{\mathrm{top}_4}\right).$$

Cette fonction est **discontinue** partout où le 4e et le 5e logit sont égaux (figure 6.5).

Proposition 6.8 (Le saut d'une bascule ne dépend pas de la perturbation)**.** *Soit $m = z_{(4)} - z_{(5)} > 0$ la marge du routeur en une entrée h, et une perturbation qui change les logits de δz. Si $\delta z_{(5)} - \delta z_{(4)} > m$, l'expert 4 sort, l'expert 5 entre, et la sortie change de*

$$\Delta f = g_4 \left(E_4(h) - E_5(h)\right) + O(\|\delta z\|),$$

dont la norme ne tend pas vers 0 quand $\|\delta z\| \to 0$ (à la limite de la bascule, $g_4 \to g_5 > 0$). Sinon, $\Delta f = O(\|\delta z\| + \|\delta h\|)$.

Démonstration. Sans bascule, l'ensemble des experts est le même et f est dérivable en h (softmax, experts lisses à la SwiGLU bornée près), d'où l'ordre 1. Avec bascule, la contribution de l'expert 4, de poids $g_4 > 0$, est remplacée par celle de l'expert 5, de poids voisin ; la différence des deux sorties d'experts sur la même entrée est une grandeur de l'ordre des sorties elles-mêmes. ■

Le saut entre dans le résidu de la position, puis, par les clés et les valeurs de cette couche et des suivantes, dans toutes les positions qui suivent. Avec 24 couches et 5 110 positions, même une faible probabilité de bascule par (couche, position) touche une part notable des positions. L'identité exacte de la proposition 6.6 dit maintenant ce que cela coûte.

Théorème 6.9 (Deux régimes de la KL d'un mélange d'experts)**.** *Soit une perturbation de taille relative u des calculs (arrondi, phase), qui déplace les marges du routeur de δ_u, avec $\mathbb{E}|\delta_u| = c\,u$ au premier ordre ; soit f_m la densité des marges, continue et non nulle en 0. Alors, pour u petit, la KL moyenne se décompose en*

$$\overline{\mathrm{KL}}(u) \;\approx\; \underbrace{a\,u^2}_{\text{sans bascule}} \;+\; \underbrace{f_m(0)\,c\,u \cdot \overline{\mathrm{KL}}_{saut}}_{\text{bascules}}, \tag{6.8}$$

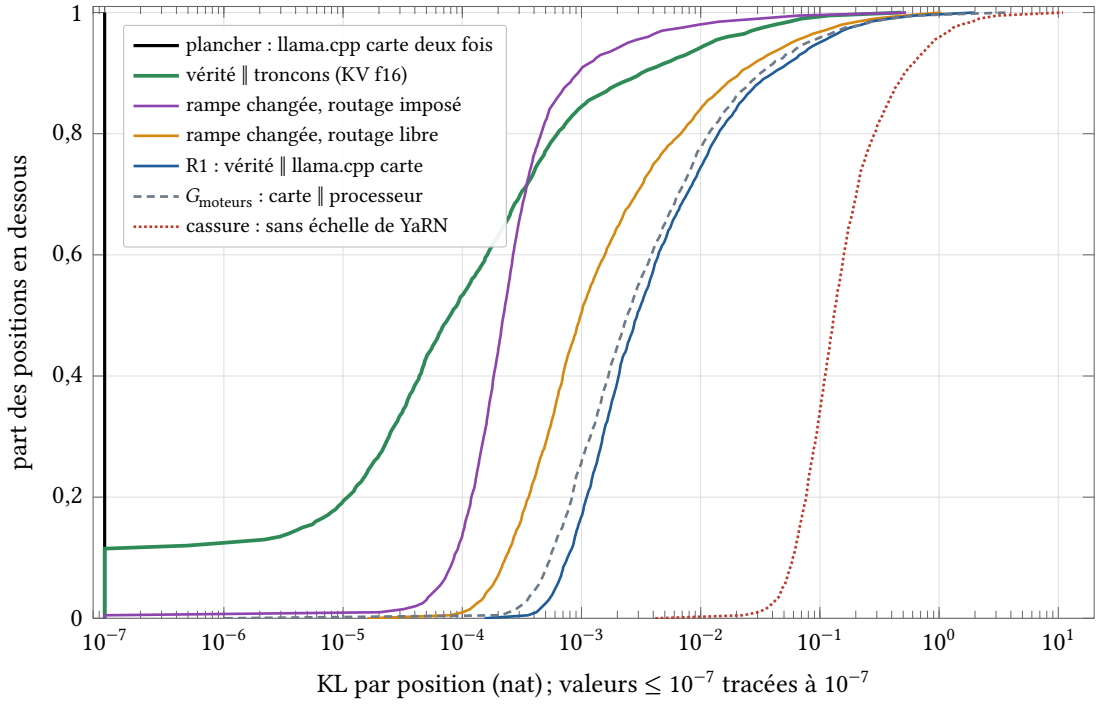


Figure 6.4 : Répartition de la KL par position sur gpt-oss-20b (2 550 positions, échelle logarithmique). Toutes les courbes approchées se croisent sous la médiane et se séparent au-delà du 90e centile. Imposer le routage (violet) resserre la queue d'un ordre de grandeur. troncons (vert, cache KV f16) a 12 % de positions à KL ≤ 10^{-7} (le zéro de l'instrument est légèrement négatif, section 6.4.2).

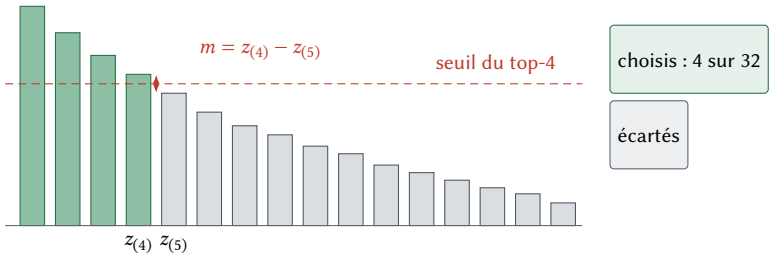
Données : `memoire/figures/tome2/donnees/t2r-kl-repartition.csv`, tiré de `mesures/moteur/R/r0/juge-kl-*.csv`, `r1/juge-kl-*.csv` et `mesures/moteur/O/csv/o-c4-20b.csv`.

où $\overline{\mathrm{KL}}_{saut}$ *est la KL moyenne d'une position qui bascule, d'ordre 1 en u. Le second terme,* linéaire, *domine le premier dès que* $u < f_m(0)\, c\, \overline{\mathrm{KL}}_{saut}/a$ *: plus le calcul est précis, plus la part des bascules dans la KL est grande.*

Démonstration. Hors bascule, les logits de sortie sont perturbés de $\Delta z = O(u)$ et la proposition 6.6 donne une KL $\frac{1}{2}\operatorname{Var}_p \Delta z = O(u^2)$. Une bascule se produit en une (couche, position) quand $|\delta_u| > m$; sa probabilité est $\int_0^\infty f_m(x)\Pr(|\delta_u| > x)\,dx \approx f_m(0)\,\mathbb{E}|\delta_u| = f_m(0)\,c\,u$ tant que f_m est à peu près constante sur l'étendue de $|\delta_u|$. Une position qui bascule voit, par la proposition 6.8, un saut d'ordre 1 de son résidu, donc de ses logits, donc une KL d'ordre 1 qui ne dépend pas de u. La somme des deux contributions donne (6.8). (Les bascules propagées par le cache KV ajoutent un terme du même ordre en u ; elles sont comptées dans $\overline{\mathrm{KL}}_{\text{saut}}$ par position touchée.) ■

Le théorème est une analyse d'ordre, sous hypothèses (densité des marges continue en 0, déplacement des marges proportionnel à u avant toute cascade) ; ses conséquences se vérifient sur les mesures qui suivent.

Un ordre de grandeur qui se tient. Les marges de toutes les (couche, position) du 20b ont pour 1er centile $1{,}14 \cdot 10^{-3}$ et pour 10e centile $1{,}24 \cdot 10^{-2}$ (`mesures/moteur/R/r1/bascules-openai-llamacpp-20b.json`) : la densité en 0 vaut donc $f_m(0) \approx 0{,}01/1{,}14 \cdot 10^{-3} = 8{,}8$ (ou $0{,}1/1{,}24 \cdot 10^{-2} = 8{,}1$) [calc.]. La part mesurée des bascules entre les deux rampes est 4,15 % ; le théorème en tire un déplacement



Si une perturbation δz des logits fait $|\delta z_{(4)} - \delta z_{(5)}| > m$, l'expert 4 sort et l'expert 5 entre. La sortie de la couche saute de $g_4\,(E_4(h) - E_5(h))$, une grandeur d'ordre 1, quelle que soit la taille de δz ; le saut entre dans le résidu, puis dans les clés et valeurs de toutes les positions suivantes.

Figure 6.5 : La bascule de routage. Les logits du routeur d'une position, triés ; le top-4 garde les quatre premiers. La marge m entre le 4e et le 5e est la distance à la discontinuité.

moyen des marges $\mathbb{E}|\delta| \approx 0{,}0415/8{,}5 \approx 5 \cdot 10^{-3}$, l'ordre de grandeur attendu d'un écart de phase de 10^{-4} rad par position, accumulé sur quelques centaines de positions et sommé dans un produit $q \cdot k$ [calc.] sur [mes.]. Et la médiane des marges des seules (couche, position) qui basculent, $9{,}5 \cdot 10^{-3}$, est bien de cet ordre.

6.5.3 L'expérience du routage imposé

L'expérience B (déclarée avant sa passe) impose à la passe de rampe llama.cpp les experts que la passe de rampe OpenAI a choisis à chaque (couche, position), puis compare les deux : seule la rampe diffère, et aucune bascule n'est possible. Seuil déclaré : si la KL tombe sous 10^{-3}, les bascules portent l'écart.

Mesuré : la KL passe de 0,0145 à 0,00168, la médiane de $9{,}8 \cdot 10^{-4}$ à $2{,}2 \cdot 10^{-4}$, le premier choix de 92,5 % à 97,2 % [mes.]. Le seuil tombe ($1{,}68 \cdot 10^{-3} > 10^{-3}$), mais ce que l'expérience établit est net : les bascules portent

$$1 - \frac{0{,}00168}{0{,}0145} = \mathbf{88}\,\%$$

de la KL [calc.] (refait : 0,884). Le reste garde une queue (maximum 0,53) : un second amplificateur, non identifié, ou une sensibilité propre à quelques positions. Figure 6.6 range toutes ces expériences.

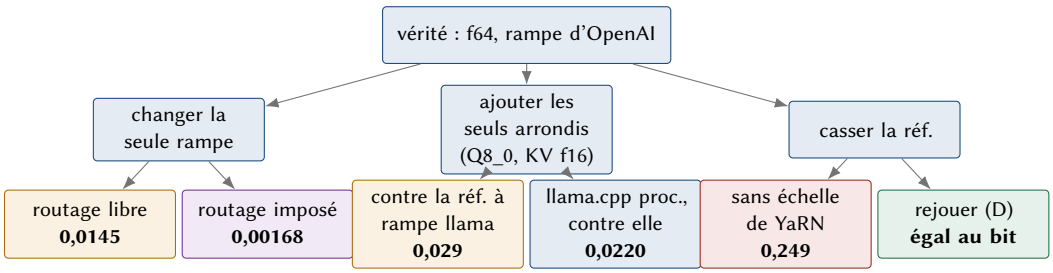


Figure 6.6 : Les expériences de R1, chacune déclarée avant sa passe, avec la KL moyenne mesurée à chaque feuille (gpt-oss-20b, dix fenêtres). La feuille « rejouer (D) » est le témoin du déterminisme de la référence.

Données : `moteur/R-pour-le-memoire.md` § 2.8, § 2.9.

6.5.4 Les bascules, comptées

Les journaux de routage des deux passes (rampe OpenAI, rampe llama.cpp ; la seconde rejouée identique au bit, D tient) permettent de compter les bascules une à une (`bascules.py`, `mesures/moteur/R/r1/bascules-openai-llamacpp-20b.json`) [mes.] :

- **5 087 bascules sur 122 640 (couche, position), soit 4,15 %** ;
- aucune des 2 550 positions notées n'est sans bascule en amont dans sa fenêtre : à partir de la position 256, il y a toujours eu une bascule plus tôt ;
- **1 157 positions basculent elles-mêmes (45 %) et portent 96,5 % de la KL** (moyenne 0,0309) ; les 1 393 autres, touchées seulement par le cache KV, ont une KL moyenne de $9{,}4 \cdot 10^{-4}$, 33 fois moins ;
- la marge du routeur vaut 0,083 en médiane sur toutes les (couche, position), et **0,0095 sur celles qui basculent** ; 1 % des bascules ont une marge sous $9{,}6 \cdot 10^{-5}$.

La bascule est locale avant d'être propagée ; la propagation par le cache KV existe, mais pèse peu (figure 6.7).

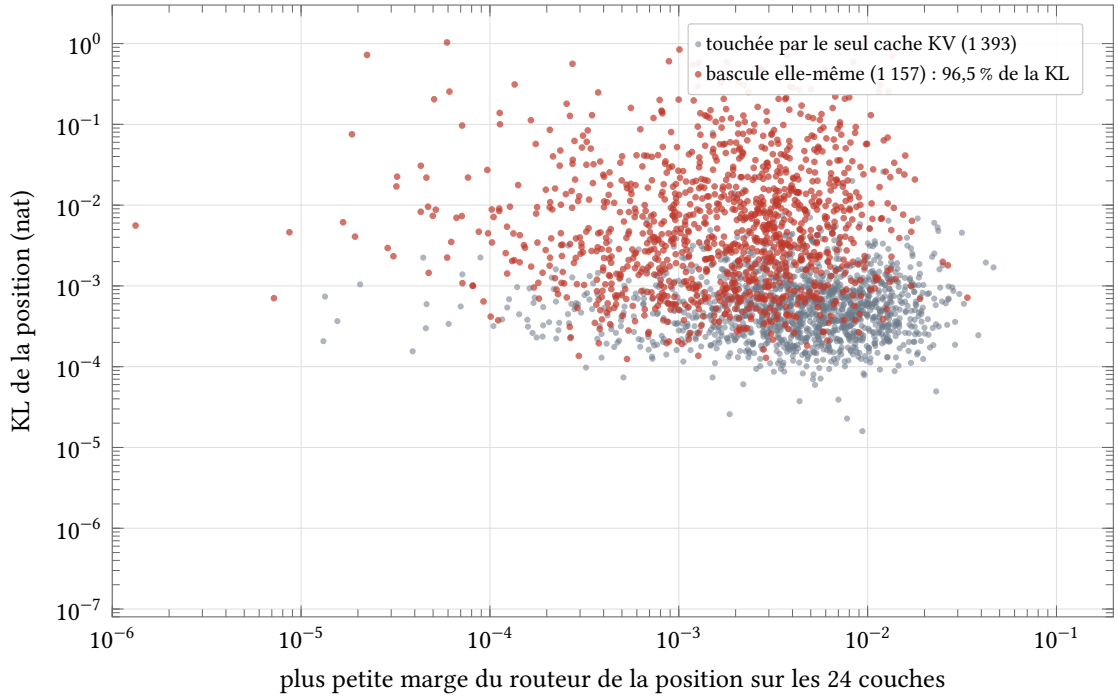


Figure 6.7 : KL de chaque position notée contre la plus petite marge de son routeur sur les 24 couches (rampe changée, routage libre ; gpt-oss-20b). Les positions qui basculent elles-mêmes (rouge) forment le haut du nuage.

Données : `memoire/figures/tome2/donnees/t2r-bascules-*.csv`, tirés de `mesures/moteur/R/r1/csv/bascules-openai-llamacpp-20b.csv`.

6.5.5 Ce que cela fait aux critères

Le critère R1 de l'architecture, « KL(référence ‖ llama.cpp carte) ≤ max(G_{moteurs}, $7{\cdot}10^{-5}$) », supposait qu'une référence juste est plus près de llama.cpp que deux moteurs llama.cpp entre eux. Dans un régime qui amplifie, la distance de tout calcul approché à l'exact est du même ordre que la distance entre deux calculs approchés : la référence peut se trouver un peu au-delà de G_{moteurs} sans aucun défaut, et le critère ne départage plus une référence juste d'une référence fausse d'une petite erreur (la

rampe seule fait 0,0145) ; il ne départage qu'une erreur grossière (0,249). Ce qui départage vraiment : le routage imposé, qui retrouve le régime linéaire, et un critère qui compare des distributions de distances plutôt qu'une moyenne. Les deux sont portés à l'arbitre (`moteur/REPRISE-R.md`, besoins 4 et 5), qui n'a pas tranché à l'heure où ce chapitre s'écrit ; le critère littéral reste appliqué et publié tel quel. La même amplification touche le seuil Q1 du jalon J2.

6.6 Le cache KV en f16 porte les bascules, le f32 rejoint la vérité

6.6.1 Le premier jeton de C12

L'épreuve indépendante a passé les 708 jetons de l'invite du texte C12 à sa passe numpy f64 du 120b, avec l'argmax et la trace de routage de chaque position, et le moteur a rejoué les mêmes jetons (`mesures/epreuve-J1.md` § 6.1). À la position 707, celle du premier jeton produit, la vérité f64 choisit le jeton 1008 (« ... », $\log p = -1{,}46$) ; troncons choisit 313 (« am »), auquel la vérité donne $\log p = -10{,}03$; llama.cpp, lui, choisit 1008 [mes.]. Ce n'est pas une rupture à 512 (les 512 premiers jetons sont la fenêtre 0, égale au bit, et `--contexte 1024` comme `--emplacements 200` laissent les 708 empreintes inchangées). C'est une **cascade de bascules** : à la position 707, 20 couches sur 36 routent autrement que la vérité, à partir de la couche 12 ; à la position 706, aucune.

L'expérience déclarée avant sa passe (`mesures/moteur/epreuve-J1/seuils-declares.md`) remplace, dans la vérité f64, le cache des clés et des valeurs par sa version arrondie en f16, et rien d'autre. Cette vérité-là **choisit 313 à la position 707, avec la même cascade de 20 couches**. Le seul arrondi f16 du cache KV, que la couture N de l'architecture fixe pour troncons (et que llama.cpp partage), est donc une cause suffisante de l'erreur.

6.6.2 La part des bascules que porte l'arrondi du cache

troncons route comme la vérité à 96,9 % des (couche, position) avant 512 et 93,1 % après ; il route comme la vérité *munie du KV f16* à 99,3 % et 97,9 % (`mesures/epreuve-J1.md` § 6.1). La part des bascules de troncons que l'arrondi du cache explique se pose ainsi [calc.] :

$$\text{avant } 512 : 1 - \frac{1-0{,}993}{1-0{,}969} = 1 - \frac{0{,}7}{3{,}1} = \mathbf{77}\,\%, \qquad \text{après} : 1 - \frac{1-0{,}979}{1-0{,}931} = 1 - \frac{2{,}1}{6{,}9} = \mathbf{70}\,\%.$$

(Refait par `memoire/outils/tome2_reference_cache.py`, dont le témoin tombe quand on fausse l'un des taux.) L'accord de routage décroît avec la profondeur sur le 120b, pas sur le 20b (figure 6.8) : 128 experts par couche laissent des marges plus serrées que 32, et une bascule d'une couche basse se propage aux suivantes.

6.6.3 Pourquoi le f32 doit rejoindre la vérité

Un f16 garde 11 bits significatifs : l'arrondi au plus proche donne $\tilde{k} = k(1+\delta)$ avec $|\delta| \le u_{16} = 2^{-11} \approx 4{,}9 \cdot 10^{-4}$. Pour un score d'attention $s = c\sum_{d=1}^{64} q_d k_d$ ($c = 1/8$), l'erreur au premier ordre vérifie

$$|\tilde{s} - s| \le c\, u_{16} \sum_{d} |q_d k_d|,$$

et le produit fait en f32 n'ajoute que $\gamma_{64} = 64u_{32}/(1-64u_{32}) \approx 3{,}8 \cdot 10^{-6}$ ($u_{32} = 2^{-24}$; Higham [25]). La sortie de l'attention hérite d'une erreur relative du même ordre, par les poids du softmax et par $\tilde{v}$. En f32, la seule erreur qui reste est celle des noyaux, $u_{32}/u_{16} = 2^{-13} = 1/8\,192$ fois plus petite [calc.].

Le théorème 6.9 dit ce que devient la KL quand u est divisé par 8 192. Le terme sans bascule, en u^2, est divisé par $8\,192^2 \approx 6{,}7 \cdot 10^7$; le terme des bascules, linéaire, par 8 192. Partant des 0,00308 mesurés

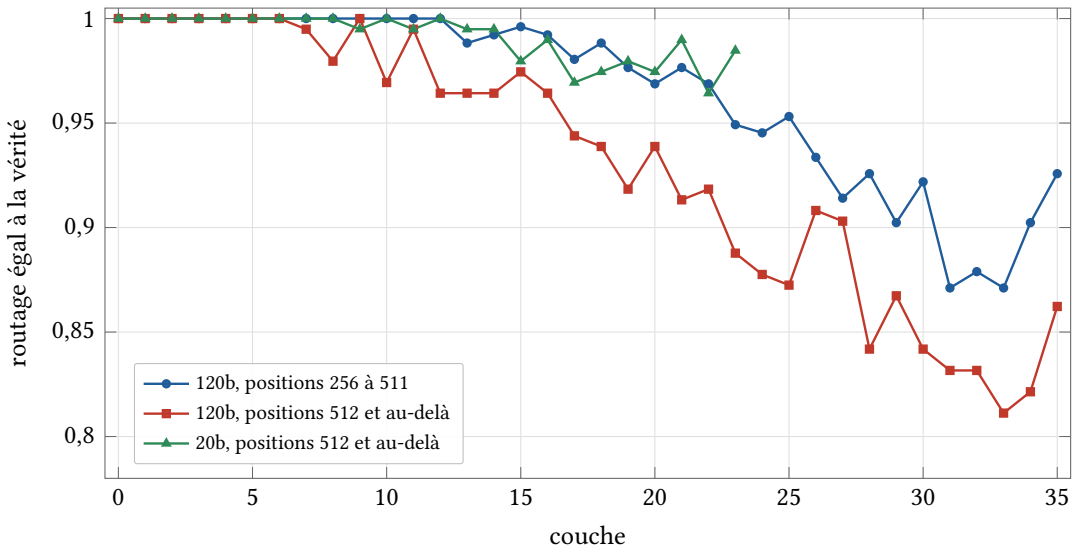


Figure 6.8 : Part des positions dont le routage de troncons (cache KV f16) égale celui de la vérité f64, par couche, sur l'invite de 708 jetons. Le 120b décroche avec la profondeur, davantage au-delà de 512 ; le 20b reste au-dessus de 0,96.

Données : `mesures/moteur/epreuve-J1/csv/routage-par-couche-invite.csv`.

en f16 sur la fenêtre 0 du 120b, la KL en f32 serait au plus de l'ordre de $0{,}00308/8\,192 \approx 3{,}8 \cdot 10^{-7}$ nat [calc.], bien sous le pas du format ($\approx 2{,}4 \cdot 10^{-4}$ nat) : invisible. ~~La KL du f16 tomberait vers $5 \cdot 10^{-11}$~~ : c'est l'estimation de l'ingénieur du lot O (`moteur/O-pour-le-memoire.md` § 13.3), qui supposait le régime quadratique pour toute la KL ; avec les bascules, qui ne le sont pas, la borne honnête est l'ordre de $4 \cdot 10^{-7}$. La conclusion, elle, ne change pas : en f32, l'écart du moteur à la vérité passe sous le plancher de l'instrument.

6.6.4 Mesuré

L'ingénieur du lot O a ajouté au moteur un cache KV en f32 sur option (`--kv-f32`, noyaux `ecrire_kv_f32` et `attention_kvf32` ajoutés sans toucher aux noyaux f16, donc hors couture N). Témoin : sur des valeurs représentables en f16, le chemin f32 est égal au bit au chemin f16 (cache plein et anneau) ; cassure : des valeurs non représentables font différer (`mesures/moteur/O/corrections/journaux/m5-invariants.txt`). Les résultats contre la vérité de l'épreuve sont au tableau 6.4 et à la figure 6.9.

Sur l'invite de 708 jetons, en f32, les argmax égalent la vérité **708 fois sur 708** (f16 : 687) ; le routage est égal sur toutes les (couche, position), 36 × 708 (f16 : 96,9 et 93,1 %) ; et **le premier jeton de C12 devient 1008, celui de la vérité** (f16 : 313) [mes.]. La KL négative est exactement celle de la référence f64 du lot R contre la même vérité ($-4{,}8 \cdot 10^{-4}$) : le biais commun du plancher du format, le zéro de l'instrument.

> **Ce que cela établit**
>
> **L'arrondi f16 du cache KV portait tout l'écart mesurable entre troncons et les mathématiques exactes du modèle.** Avec un cache KV en f32, le moteur, ses noyaux, son décodeur et son cache d'experts rendent la vérité f64 au plancher du format à 16 bits par logit, sur le 120b comme sur le 20b. L'épreuve l'avait borné par en dessous (70 à 77 % des bascules) ; la mesure du

Table 6.4 : KL(vérité f64 ‖ troncons) selon le format du cache KV [mes.]. « Écart max des log p » : au-dessus de la coupe du format, en nat ou en pas de quantification.

Cas	KL moyenne	1er choix	KL > 0,1	KL p99	écart max des log p
120b, fenêtre 0, KV f16	0,00308	95,7 %	0,39 %	0,0747	3,07 nat
120b, fenêtre 0, KV f32	**−0,00048**	**100 %**	0	$2{,}3 \cdot 10^{-6}$	**0,69 pas**
20b, fenêtre 0, KV f16	0,00174	96,5 %	0,39 %	0,0529	2,32 nat
20b, fenêtre 0, KV f32	**−0,00048**	**100 %**	0	≈ 0	2,04 pas
120b, invite, positions 512 à 706, f16	0,00963	—	2,05 %	0,196	—
120b, invite, positions 512 à 706, f32	**−0,00054**	—	0	$3{,}6 \cdot 10^{-6}$	—

Données : `moteur/O-pour-le-memoire.md` § 13.3 ; `mesures/moteur/O/corrections/journaux/verdicts-kv-2026-10-16.json`, `m5-ecart-logp-*.json` ; moyennes refaites ici depuis `mesures/moteur/O/corrections/csv/kv-f16-f32-par-position.csv`.

> lot O le ferme.

Ce que cela coûte. Le cache KV tient 18 couches pleines de 4 096 positions et 18 anneaux de 128, chacun de 512 valeurs pour k et autant pour v : $18 \times (4\,096 + 128) \times 2 \times 512 \times 2$ o = 155,7 Mo en f16, le double en f32. Les 155,7 Mo de plus valent 155,7/13,25 = 11,8 emplacements d'experts [calc.] ; **mesuré : 326 → 314 emplacements**. Sur la trace de l'invite, le simulateur du cache donne alors un taux de succès de 0,459 → 0,442 et 77,9 → 80,4 défauts par jeton, soit +3,2 % d'octets lus, l'ordre du coût en vitesse quand le disque limite [calc.] sur trace mesurée (`mesures/moteur/O/corrections/journaux/m5-rejouer-invite-326-314.jsonl`). La vitesse de génération mesurée en f32 (1,43 et 1,56 jeton/s, `mesures/moteur/O/corrections/csv/vitesse-2026-10-16-passes.csv`) *n'est pas comparable* à celle du f16 : le texte change (le f32 suit la vérité, puis des appels d'outil simulés, comme la sortie gloutonne de llama.cpp ; le f16 tombe dans une boucle « Here is a summary » qui relève son taux de succès à 0,590 contre 0,435) ; l'écart mesure l'effet du texte, pas le coût du f32. Le choix du format du cache KV appartient à l'arbitre (couture N) ; il n'est pas tranché.

« Six fois plus près », requalifié. Le jugement C4 sur le 20b donnait KL(vérité ‖ troncons) = 0,003679 contre 0,022606 pour llama.cpp carte, et 97,18 % de premiers choix contre 89,61 % (`mesures/moteur/O/csv/o-c4-20b.csv` ; refait ici : rapport 6,15). L'ingénieur du lot O l'avait résumé par « six fois plus près des mathématiques exactes du modèle que llama.cpp ». C'est vrai, **dont la rampe de YaRN** : 0,0145 des 0,0226 de llama.cpp viennent de sa rampe arrondie, un choix de modélisation, et non de la précision de ses noyaux (constat m5 de l'épreuve ; sur la fenêtre 0, 0,0143 des 0,0261). Et l'écart qui restait à troncons venait tout entier de son cache KV en f16. Sur le 120b, l'épreuve a jugé C4 pour la première fois, sur une fenêtre : 0,00308 et 95,7 % pour troncons, 0,0607 et 80,4 % pour llama.cpp, différence appariée $-0{,}0576 \pm 0{,}0120$, intervalles de Wilson disjoints [mes.] (`mesures/epreuve-J1.md` § 3.3).

6.7 Ce qui reste

- La référence f64 du 120b sur les dix fenêtres (environ 61 Go d'experts lus par passe, de l'ordre de quatre à six heures estimées) : C4 sur le 120b ne repose aujourd'hui que sur une fenêtre et une invite.

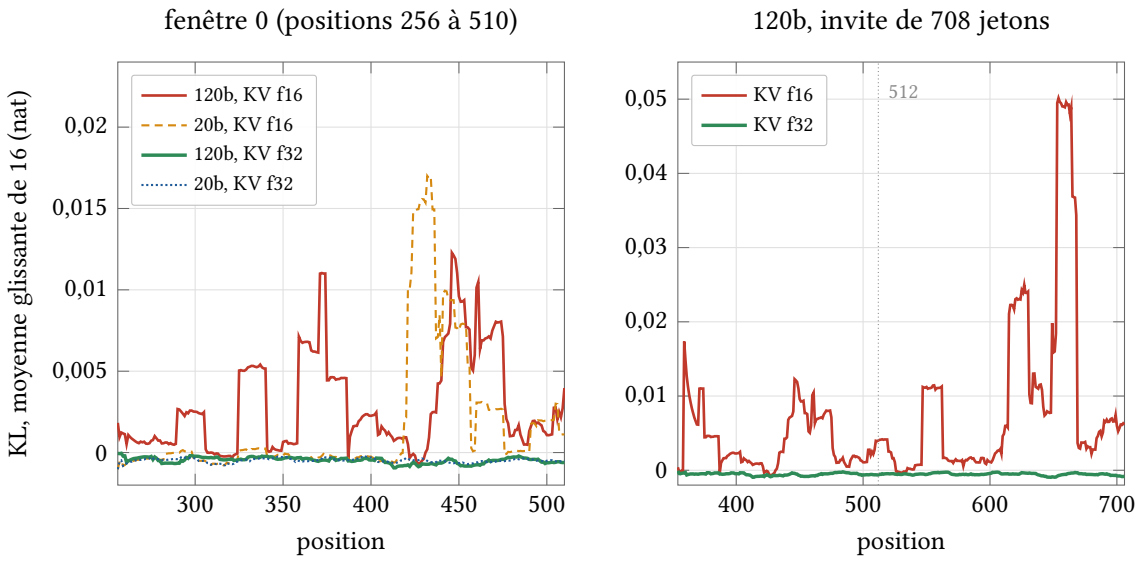


Figure 6.9 : KL(vérité f64 ‖ troncons) par position, en moyenne glissante de 16 positions. En f16, la KL vient par bouffées (les bascules) et monte avec la position sur l'invite du 120b ; en f32, elle reste au zéro de l'instrument ($\approx -5 \cdot 10^{-4}$, section 6.4.2).

Données : `memoire/figures/tome2/donnees/t2r-kv-glissant.csv`, `t2r-kv-invite-glissant.csv`, tirés de `mesures/moteur/O/corrections/csv/kv-f16-f32-par-position.csv`.

- La décision de l'arbitre sur la couture N (KV f32 : +156 Mo, 12 emplacements, de l'ordre de 3 % d'octets lus de plus), et sur un critère C4 « de queue » (part des positions de KL $> 0{,}1$, 99e centile, jugés aussi au-delà de 512 positions).
- Un critère R1 qui départage une petite erreur : la référence jugée contre llama.cpp à routage imposé, avec ses imitations, en régime linéaire (R1').
- Le second amplificateur : les 12 % de la KL qui survivent au routage imposé. Hypothèses non testées : les coudes de la SwiGLU bornée ($\min(g, 7)$), les positions dont la distribution est presque plate, une attention presque égale entre deux clés.

Ce qui pourrait être faux

- **La référence suppose que le GGUF de ggml-org est le modèle.** Elle juge le modèle tel que stocké (attention en Q8_0), pas le point de contrôle d'OpenAI en bf16.
- **La rampe d'OpenAI est tenue pour le modèle** sur la foi de deux sources primaires concordantes (l'implémentation de référence et `truncate: false`) ; l'entraînement lui-même n'est pas observable. La passe numpy de l'épreuve et la référence suivent la même lecture du modèle et lisent le même GGUF : une erreur de convention commune passerait. Elles se valident l'une l'autre sur l'arithmétique et l'ordre des opérations, pas sur la convention.
- **Le théorème des deux régimes est une analyse d'ordre** ; ses hypothèses (densité des marges continue et non nulle en 0, déplacement des marges proportionnel à u) ne sont vérifiées que par l'ordre de grandeur du section 6.5.2. La borne de $4 \cdot 10^{-7}$ pour le f32 en dépend ; la mesure, elle, n'en dépend pas.
- **Le régime qui amplifie est mesuré sur gpt-oss-20b**, dix fenêtres de wikitext-2. Sur le 120b, les marges sont plus serrées et la part des bascules peut être plus grande ; les chiffres de R1 ne

se transportent pas sans être remesurés.
- **« KV f32 = vérité »** est mesuré sur une fenêtre de 512 jetons et une invite de 708 pour le 120b, une fenêtre pour le 20b ; pas sur les dix fenêtres.
- **Les imitations** reproduisent la quantification Q8_0 des activations et le cache KV f16 tels que lus dans le code de b11515 ; pas les sommes en f32, ni les noyaux répétés des poids Q8_0, ni le chemin de la carte.
- $G_{\mathbf{cfg}} = 0$ ne vaut que pour l'évaluation par lots de 512 ; l'interprétation (déport des produits sur la carte) n'est pas vérifiée dans le code de llama.cpp.
- **R1 tient de 0,6 %**, dans le bruit (±0,0017 et ±0,0025) : une autre graine de fenêtres pourrait le faire tomber sans que la référence change.
- **Le pavage** est prouvé égal au bit par construction et par deux témoins ; il n'a pas été comparé passe contre passe à l'ancienne version sur dix fenêtres (elle aurait pris des jours).
- **Les passes de la référence** (15 h 35 à 17 h 35 le 2026-10-15) ont tourné pendant la série 120b de l'agent de mesure gpt-oss : sans effet sur les logits (D tient), possible sur les temps.

Chapitre 7

Le cache, l'ordonnanceur et le moteur

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, l'architecture avait fixé l'interface du cache d'experts (couture A) et un témoin : le simulateur du moteur doit égaler, à l'unité, ceux des études de chargement et de mesure. Le plafond de vitesse de troncons se lisait dans une seule grandeur, le taux de succès h du cache, que les études antérieures calculaient sous un routage uniforme faute de trace réelle. **Ce chapitre établit** la théorie et la mesure du cache et de ce qui l'entoure : l'optimalité de l'algorithme de Belady, démontrée ; LRU, sa propriété de pile, sa borne compétitive et son pire cas ; le cache de troncons, ses règles et deux lemmes qui le rendent égal aux simulateurs ; **la falaise du LRU partagé** : aucun succès sous $(L-1)k$ emplacements, exactement la réutilisation adjacente à Lk, démontrée et vérifiée au succès près sur 17 traces réelles ; la localité mesurée de gpt-oss-20b et 120b ; l'ordonnanceur à la demande et les lemmes qui le rendent sûr, exact et égal à son simulateur ; le préremplissage par flux, gardé ; le préchargement, essayé puis abandonné sur son bilan chiffré ; et le moteur contre la base la mieux réglée qu'on ait mesurée : **2,22 fois llama.cpp**, garde d'intégrité comprise. **Il reste** une politique consciente du cycle des couches, un prédicteur plus juste que le routeur de la couche suivante, la longueur des lectures (12 Mio), et une base llama.cpp balayée au-delà de 16 fils. Sources : `moteur/O-pour-le-memoire.md`, `moteur/REPRISE-O.md`, `mesures/gpt-oss.md`, `mesures/epreuve-J1.md`, et les journaux sous `mesures/moteur/O/` et `mesures/localite-gpt-oss/`.

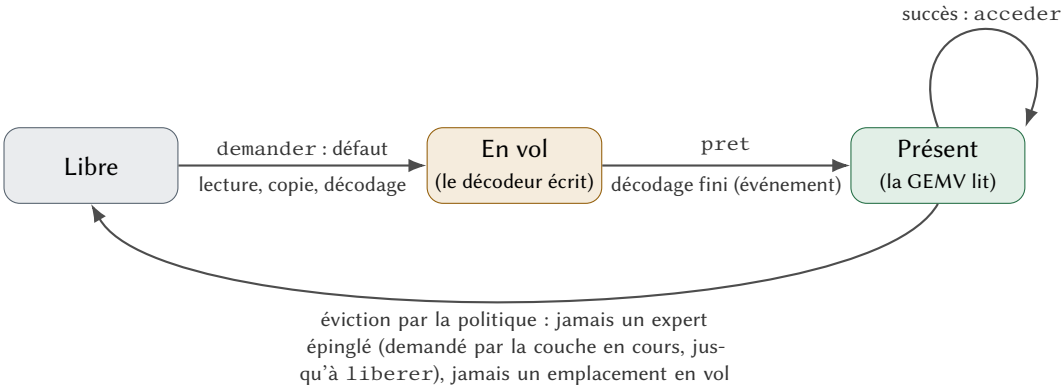
En bref

- Belady (évincer ce qui servira le plus tard) est optimal parmi toutes les politiques (théorème 7.2) ; LRU fait au plus N fois ses défauts, plus N (théorème 7.4), et son pire cas est un cycle sur $N+1$ clés.
- Le flux des experts d'un MoE *est* ce cycle : un LRU partagé entre les L couches n'a aucun succès sous $(L-1)k$ emplacements, quel que soit le routage, et vaut exactement la réutilisation adjacente à Lk (proposition 7.7). Sur gpt-oss-20b : 0 succès sur 1 024 224 accès à 92 emplacements, quand Belady en sert 66,4 %.
- gpt-oss est très local : un jeton reprend 50 % (20b) et 31 % (120b) des experts du précédent, contre 12,5 et 3,1 % au hasard.
- L'ordonnanceur est sûr (lemme 7.9), calcule au bit ce que les noyaux ont validé (proposition 7.10) et compte ses défauts comme son simulateur, à l'unité (454 210 succès, 37 310 défauts des deux côtés).
- Le préchargement par le routeur de la couche suivante coûte 11 % : 69 % de ses lectures servent, les autres ajoutent 34 % d'octets à un disque qui limite déjà.
- troncons : 1,965 jeton/s sur le 120b, garde comprise, contre 0,884 pour la meilleure configuration mesurée de llama.cpp (`fit -t 16`) ; ~~×3,85~~, ~~×2,8~~, ×2,22.

7.1 Le problème du cache d'experts

Un mélange d'experts de L couches, E experts par couche, k experts actifs par jeton (gpt-oss-120b : $L = 36$, $E = 128$, $k = 4$; gpt-oss-20b : $L = 24$, $E = 32$, $k = 4$) demande, à chaque jeton et à chaque couche, un **groupe** de k identifiants distincts (couche, expert). Le flux des demandes est la suite des groupes dans l'ordre du calcul : $(t, 0), (t, 1), \ldots, (t, L-1), (t+1, 0), \ldots$ La carte tient N **emplacements** d'experts décodés (13 254 144 octets chacun pour gpt-oss) ; un expert présent est un **succès** ; un expert absent est un **défaut**, qui coûte une lecture d'environ 12 Mo sur le disque, une copie vers la carte et un décodage. La part h des accès servis par le cache décide de tout : à chaque jeton, le disque doit livrer $(1 - h)$ fois les octets routés, $R = k\,L\,e = 1{,}909$ Go pour le 120b à la taille d'origine (`mesures/gpt-oss.md` § 1).

Une **politique** choisit, à chaque défaut d'un cache plein, la victime à évincer. Elle est *en ligne* si elle ne voit que le passé, *hors ligne* si elle connaît toute la suite. Deux règles de troncons s'ajoutent (`moteur/crates/trc-cache/src/lib.rs`) : un expert demandé par la couche en cours est **épinglé** jusqu'à la fin de ses produits (on ne l'évince pas pendant qu'il sert), et un emplacement **en vol** (que le décodeur écrit) n'est jamais évincé. La figure 7.1 donne l'automate d'un emplacement.



Étage RAM (cache tiède, mode A) : une entrée ne naît que d'une lecture du disque (on ne refait pas un tronçon codé depuis un expert décodé) ; tout accès à un expert qu'il tient la touche, succès VRAM compris (étage inclusif). Un défaut VRAM trouvé en RAM coûte une copie et un décodage, pas une lecture.

Figure 7.1 : L'automate d'un emplacement VRAM du cache de troncons (schéma M5), et l'étage RAM du mode A. En J2, les états porteront le niveau (base ou exact) de l'expert présent.

Données : `moteur/crates/trc-cache/src/lib.rs` (en-tête).

7.2 L'optimum hors ligne : Belady

Commençons par la borne que nulle politique ne peut dépasser. On se place d'abord dans le modèle classique : une suite de demandes isolées $\sigma_1, \sigma_2, \ldots$ sur un cache de N places, une politique « à la demande » (qui ne charge qu'au défaut, en évinçant une seule clé).

Définition 7.1 (MIN, ou Belady)**.** À chaque défaut d'un cache plein, MIN évince la clé présente dont la *prochaine demande est la plus lointaine* (jamais : l'infini ; égalité : la plus petite clé) [39].

Théorème 7.2 (Optimalité de MIN)**.** *Pour toute suite σ et toute politique à la demande A partant du même contenu, MIN ne fait pas plus de défauts que A.*

Démonstration. On construit une suite de politiques $A = A_0, A_1, \dots, A_n = \text{MIN}$ telle que A_i prend les mêmes décisions que MIN sur les i premières demandes et ne fait pas plus de défauts que A_{i-1}. Supposons A_{i-1} construite ; elle suit MIN jusqu'à la demande $i-1$, donc les deux ont le même contenu avant la demande i. Si A_{i-1} prend à la demande i la décision de MIN, on pose $A_i = A_{i-1}$. Sinon, à un défaut, A_{i-1} évince y quand MIN évince x, avec $n_x \geq n_y$, où n_z est l'instant de la prochaine demande de z. On définit A_i : à la demande i, elle évince x ; ensuite elle imite A_{i-1}, en suivant trois phases.

Phase 1 : A_i tient $D \cup \{y\}$ et A_{i-1} tient $D \cup \{x\}$, avec autant de défauts. Sur toute demande d'une clé de D ou d'une clé absente des deux, A_i fait comme A_{i-1} (même succès, ou même défaut et même victime), sauf si A_{i-1} évince x : alors A_i évince y, et les deux contenus deviennent égaux. Avant n_x arrive $n_y \leq n_x$: à la demande de y, A_i fait un succès et A_{i-1} un défaut, qui évince une clé w. Si $w = x$, les contenus sont égaux et A_i a un défaut de moins. Sinon on passe en phase 2 avec la paire (w, x).

Phase 2 : A_i tient $D' \cup \{a\}$, A_{i-1} tient $D' \cup \{x\}$, et A_i a *au moins un défaut de moins*. Une demande de a : succès pour A_i, défaut pour A_{i-1}, qui évince w' ; si $w' = x$ les contenus deviennent égaux, sinon la paire devient (w', x) et l'avance de A_i grandit. Une demande de x : défaut pour A_i, qui évince a et charge x ; les contenus deviennent égaux et A_i a perdu au plus l'avance qu'il avait, donc il n'a toujours pas plus de défauts. Si A_{i-1} évince x sur un autre défaut, A_i évince a : contenus égaux. Toute autre demande est traitée de la même façon par les deux.

Phase 0 : contenus égaux ; A_i imite exactement A_{i-1}.

Dans tous les cas, A_i ne fait pas plus de défauts que A_{i-1} jusqu'à la fin, et suit MIN jusqu'à la demande i. Elle n'évince qu'au défaut : c'est une politique à la demande. Par récurrence, $\text{MIN} = A_n$ ne fait pas plus de défauts que A. ■

Ce que le théorème couvre, et ce qu'il ne couvre pas. Le moteur sert des *groupes* de k experts épinglés, pas des demandes isolées. Le simulateur de troncons applique MIN parmi les clés non épinglées (`moteur/crates/trc-cache/src/politiques.rs`) ; son optimalité dans ce modèle à épinglage n'est pas démontrée ici. Elle est vérifiée : sur chaque trace réelle et chaque budget, Belady sert au moins autant que LRU, LFU et l'épinglage statique (témoin G3 de la mesure gpt-oss, aucune violation ; l'outil s'arrête sinon). Belady lit l'avenir de la trace : ses chiffres sont des **bornes**, pas une politique du moteur.

Un défaut que le témoin a trouvé. La première version de la politique Belady cherchait le prochain usage d'un expert à partir d'un « maintenant » mis à jour par chaque accès. Or le cache choisit sa victime *avant* d'insérer le défaut : quand le premier expert d'un groupe manque, « maintenant » vaut encore l'instant du groupe précédent, et un expert servi à ce groupe-là semblait redemandé tout de suite, donc n'était jamais évincé. Le témoin O1 l'a vu : de 10 à 6 646 succès de moins que les simulateurs de référence, quand LRU était exact partout [mes.] (`mesures/moteur/O/journaux/temoin-o1-passe1-belady-faux.json`, gardé). Le prochain usage se calcule désormais au moment de l'accès (listing 7.1) : un présent non épinglé n'a pas été servi depuis son dernier accès, donc ce chiffre reste juste.

7.3 LRU : pile, borne compétitive, pire cas

LRU évince la clé dont le dernier usage est le plus ancien. C'est la politique en ligne la plus répandue, celle du cache d'experts de llama.cpp, et celle de troncons en J1. Trois propriétés la décrivent.

Proposition 7.3 (Propriété de pile)**.** *À tout instant, le contenu d'un cache LRU de N places est l'ensemble des N clés distinctes les plus récemment demandées. En particulier, le contenu à la capacité N est inclus*

Listing 7.1 : Le prochain usage dans la politique Belady de `trc-cache` (`politiques.rs`) : calculé au moment de l'accès, par recherche dichotomique dans les instants où l'expert est demandé (t = jeton $\cdot L$ + couche).

```
/// Le premier usage de `id` strictement après `t` ; u64::MAX s'il n'y en a plus.
fn prochain_apres(&self, id: Id, t: u64) -> u64 {
    match self.usages.get(&id) {
        None => u64::MAX,
        Some(v) => {
            let i = v.partition_point(|&x| x <= t);
            v.get(i).copied().unwrap_or(u64::MAX)
        }
    }
}
fn acceder(&mut self, id: Id, t: u64) {        // à chaque succès
    let p = self.prochain_apres(id, t);
    self.prochain.insert(id, p);
}
// victime : le présent non épinglé de plus grand prochain usage ; égalité : le
    plus petit id.
```

dans le contenu à toute capacité $N' > N$, sur la même suite [40].

Démonstration. Récurrence sur les demandes. Une demande de z fait de z la plus récente ; si z était parmi les N plus récentes, l'ensemble ne change pas ; sinon c'est un défaut, et LRU évince la plus ancienne des présentes, qui est la N-ième plus récente avant la demande et devient la $(N{+}1)$-ième après : le contenu reste l'ensemble des N plus récentes. L'inclusion suit : les N plus récentes sont parmi les N' plus récentes. ■

Théorème 7.4 (LRU est N-compétitif)**.** *Pour toute suite σ, $\mathrm{LRU}_N(\sigma) \le N\,\big(\mathrm{OPT}_N(\sigma) + 1\big)$, où OPT_N est le nombre de défauts de MIN [41].*

Démonstration. Découpons σ en *phases* : chaque phase est la plus longue suite de demandes, à partir de la fin de la précédente, qui ne contient que N clés distinctes. (i) LRU fait au plus N défauts par phase : une clé chargée pendant une phase n'est pas évincée avant la fin de la phase, car il faudrait pour cela N autres clés distinctes demandées après son dernier usage, et la phase n'en contient que $N-1$ autres. (ii) MIN fait au moins un défaut par phase, sauf peut-être la première : soit p la première demande de la phase j ; la suite qui va de la deuxième demande de la phase j à la première de la phase $j+1$ contient les $N-1$ autres clés de la phase j et la clé nouvelle qui ouvre la phase $j+1$, soit N clés distinctes, toutes différentes de p. Juste après avoir servi p, MIN tient p et $N-1$ autres clés ; servir N clés distinctes de p lui coûte au moins un défaut. Ces suites sont disjointes d'une phase à l'autre. Avec P phases, $\mathrm{LRU} \le NP$ et $\mathrm{OPT} \ge P - 1$. ■

Le pire cas. La borne est atteinte : sur la suite cyclique $1, 2, \ldots, N+1, 1, 2, \ldots$, LRU évince toujours la clé qui sera demandée juste après, et fait un défaut à chaque demande ; MIN n'en fait qu'un toutes les N demandes. La section suivante montre que **le flux des experts d'un MoE, servi par un cache unique pour toutes les couches, est exactement de cette forme**.

7.4 Le cache de troncons, et pourquoi il égale ses simulateurs

7.4.1 L'interface

Le crate `trc-cache` (Rust, sans dépendance) porte le cache à deux étages et sa politique (tableau 7.1) : la VRAM tient des experts décodés, l'étage RAM tiède (mode A) des tronçons codés en mémoire épinglée. Le simulateur `rejouer` est *le code même* du moteur, appelé sur une trace de routage, ce qui rend possible le témoin « compte du moteur égal au compte du simulateur, à l'unité ».

Table 7.1 : L'interface du cache de troncons (couture A), et ce que l'ordonnanceur fait de chaque décision.

Élément	Rôle
`demander(couche, experts, t)`	épingle le groupe, puis rend une décision par expert, dans l'ordre du routeur
`Succes {emplacement}`	l'expert est (ou sera, s'il est en vol) dans cet emplacement : la GEMV le lit
`DefautRam {emplacement, tiede}`	le tronçon codé est déjà en mémoire épinglée : copie, décodage
`DefautDisque {emplacement, tiede}`	lecture du disque (dans l'entrée tiède ou un tampon d'arrivée), copie, décodage
`pret(emplacement)`	le décodage est fini : En vol → Présent
`liberer(couche)`	fin des GEMV de la couche : ses experts sont désépinglés
`Compteurs`	`succes_vram + succes_ram + defauts_disque` = accès ; octets, évictions
`rejouer(trace, N, tièdes, ...)`	le simulateur : même code, appelé sur une trace

7.4.2 Deux règles d'éviction, un seul résultat

Les simulateurs de référence n'évincent pas au même moment que le moteur. Celui de la mesure gpt-oss (règle « B », comme le cache de llama.cpp) sert tout le groupe, puis évince jusqu'à revenir à N ; celui de troncons et de l'étude de chargement (règle « P ») épingle le groupe et évince à chaque défaut. Il faut montrer qu'ils comptent la même chose. Notations : un groupe $G = (x_1, \dots, x_k)$ de clés distinctes servies dans cet ordre ; S le contenu, ordonné du plus ancien au plus récent ; $N \geq k$.

Lemme 7.5 (Deux règles, un seul résultat)**.** *Partant du même contenu S_0, les règles B et P donnent les mêmes succès, dans le même ordre, et le même contenu final, dans le même ordre.*

Démonstration. Succès. En B, rien ne sort pendant le service : x_i est un succès si et seulement si $x_i \in S_0$. En P, aucun élément de G ne sort (épinglé) et rien n'entre sauf des éléments de G ; les x_i étant distincts, x_i n'a pas été ajouté avant le pas i ; donc $x_i \in S$ au pas i si et seulement si $x_i \in S_0$. *Ordre des autres.* Seuls les éléments de G bougent ; ceux de $S_0 \setminus G$ gardent leur ordre dans les deux règles. *Évictions de P.* Une fois S plein, il le reste ; chaque défaut à S plein retire le plus ancien de $S \setminus G$: P retire donc les m plus anciens de $S_0 \setminus G$, avec $m = \max(0, |S_0| + d - N)$ et d le nombre de défauts. Il existe toujours un candidat : si S était plein et inclus dans G au moment d'un défaut, on aurait $N = |S| \leq k - 1$ (le défaut courant n'est pas encore dans S), contraire à $N \geq k$. *Évictions de B.* Après le service, tous les éléments de G sont en fin ; $|S| = |S_0| + d$; les plus anciens sont ceux de $S_0 \setminus G$ dans leur ordre ; B en retire $m = |S_0| + d - N \leq |S_0| + d - k = |S_0 \setminus G|$ (car $d = k - |G \cap S_0|$) : exactement les m plus anciens de $S_0 \setminus G$. *Ordre de G.* Dans les deux règles, les éléments de G finissent en queue, dans l'ordre de service. ■

La même démonstration vaut pour Belady en remplaçant « le plus ancien » par « le plus grand prochain usage » : la clé d'un élément de $S_0 \setminus G$ ne change pas pendant le groupe (il n'est pas servi). **Corollaire** : à chaque frontière de groupe, un LRU à règle P contient les N identifiants distincts les plus récemment servis.

7.4.3 L'étage RAM inclusif

Lemme 7.6 (L'étage RAM égale un cache unique de sa taille)**.** *Soit un étage VRAM de V emplacements qui décide seul, et un étage RAM de $R \geq V$ entrées qui reçoit* `acceder` *pour tout accès à une clé qu'il contient (succès VRAM compris) et* `inserer` *pour toute lecture du disque, et qui épingle le groupe. Avec LRU : les succès VRAM sont ceux d'un LRU seul de V emplacements, et les succès VRAM plus RAM ceux d'un LRU seul de R emplacements.*

Démonstration. La VRAM évolue comme un cache seul (ses décisions ne lisent pas la RAM). Pour la RAM, il suffit de montrer qu'elle reçoit les mêmes appels qu'un LRU seul de R emplacements, ce qui ne peut manquer que sur un succès VRAM dont la clé serait absente de la RAM. Récurrence sur les groupes : si la RAM a évolué comme un LRU seul jusqu'à la frontière, le corollaire du lemme 7.5 donne VRAM_0 = les V plus récents $\subseteq$ les R plus récents = RAM_0. Pendant le groupe, un succès VRAM x_i est dans $\mathrm{VRAM}_0 \subseteq \mathrm{RAM}_0$, et la RAM n'évince pas les éléments du groupe (épinglés) : x_i y est encore, et la RAM fait `acceder` comme le ferait le LRU seul. Un défaut VRAM est, pour la RAM, un succès ou un défaut, comme pour le LRU seul. ■

C'est exactement le modèle « deux étages » des plafonds (figure 7.2) : $h_v = h(V)$, $h_r = h(R) - h(V)$, défauts $1 - h(R)$; le moteur se comporte comme l'équation qui a fixé ses seuils. Pour Belady, la même conclusion suit de la propriété de pile de MIN [40] ; avec la règle de groupe et l'épinglage, elle n'est pas démontrée ici, seulement vérifiée sur 20 configurations (graines 100 à 103, $(V, R) \in \{(8,8), (8,20), (16,40), (30,90), (12,160)\}$).

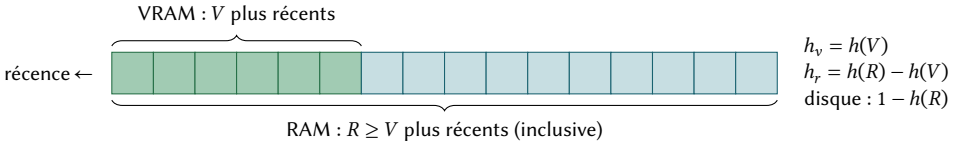


Figure 7.2 : Les deux étages du cache sur le même flux : à chaque frontière de groupe, la VRAM tient les V clés les plus récentes, la RAM les R plus récentes, et la première est incluse dans la seconde (lemme 7.6).

7.4.4 Le témoin O1

Quatre chemins, chacun comparé *à l'unité* (succès entiers ; un taux publié par un outil Python est reconverti en entier, et l'outil vérifie que n/accès redonne le flottant publié au bit près) : tableau 7.2.

Le témoin compare des *règles*, pas des vitesses : il prouve que `rejouer` et les simulateurs Python appliquent la même règle. Il ne dit pas que cette règle est celle de llama.cpp, qui en diffère par l'ordre des défauts d'un même groupe (4 et 5 succès d'écart, mesurés par la réplique `g_lru_llama`, section 7.6). Neuf cassures des témoins unitaires tombent toutes, et la copie restaurée tient (`mesures/moteur/O/journaux/tests-casses.txt`).

Table 7.2 : Le témoin O1 : le simulateur de troncons contre les simulateurs de référence [mes.]. Traces synthétiques : $T = 4\,000$ jetons, $L = 4$, $E = 128$, $k = 8$, graine 20261015.

Chemin	Contre	comparaisons	écarts	sous cassure
P1	`cache_lru`, `cache_belady` de l'étude de chargement, couche par couche ; uniforme, figée, collante ; $C \in \{8, 16, 32, 64\}$	96	0	48
P2	`g_lru`, `g_belady` (cache global) ; $N \in \{8, 31, 32, 51, 128, 205\}$	36	0	22
P3	`g_lru` à V et à R (deux étages) ; $(V, R) \in \{(8, 32), (16, 64), (32, 32), (51, 205)\}$	24	0	12
P4	taux publiés sur 12 traces réelles de gpt-oss-20b, budgets 5 à 40 %	352	0	308
	total	**508**	**0**	**390**

Données : `moteur/O-pour-le-memoire.md`, partie I ; `mesures/moteur/O/journaux/temoin-o1.json`, `temoin-o1-casse.json`.

7.5 La falaise du LRU partagé

Le cache de llama.cpp, comme la plupart des propositions publiées, partage un seul LRU entre toutes les couches. Ce choix a une conséquence qu'on démontre, et qui ne dépend pas du texte.

Proposition 7.7 (Le seuil et le plateau du LRU partagé)**.** *Un cache LRU de N emplacements sert le flux des groupes d'un MoE à L couches et k experts distincts par couche, les accès d'un groupe étant jugés contre le contenu avant toute éviction. Pour tout routage :*

1. *si $N \leq (L-1)\,k$, le LRU partagé n'a **aucun succès** ;*
2. *si $N \geq L\,k$, toute réutilisation adjacente (un expert demandé au jeton t et au jeton $t+1$, même couche) est un succès, donc le taux de succès sur les jetons $t \geq 1$ est au moins la réutilisation adjacente moyenne $\bar{r}$;*
3. *si $N = L\,k$ exactement, un succès est une réutilisation adjacente et seulement elle : le taux vaut $\bar{r}$ exactement sur les jetons $t \geq 1$, soit $\bar{r}\,(T-1)/T$ sur tous les accès de T jetons.*

Démonstration. Soit la clé (ℓ, e) demandée au jeton $t+1$. Le rang de récence d'une clé est sa place dans l'ordre des derniers usages (1 = la plus récente) ; par la proposition 7.3 et le corollaire du lemme 7.5, au moment de juger un groupe, le cache tient exactement les clés de rang $\leq N$. (i) Le dernier usage de (ℓ, e) date au plus tard du jeton t, à la couche ℓ. Entre ce dernier usage et le jugement du groupe $(t+1, \ell)$, le flux a servi au moins les groupes $(t, \ell+1), \ldots, (t, L-1), (t+1, 0), \ldots, (t+1, \ell-1) : L-1$ groupes de k clés distinctes, toutes d'autres couches que ℓ, donc distinctes entre elles et de (ℓ, e). Son rang est au moins $(L-1)k+1 > N$: elle n'est plus dans le cache, et c'est un défaut. (ii) Si (ℓ, e) a servi au jeton t, les clés plus récentes qu'elle au moment du jugement sont au plus les $(L-1)k$ clés ci-dessus et au plus $k-1$ clés de son propre groupe (t, ℓ) : rang $\leq Lk \leq N$, elle est dans le cache. (iii) Si (ℓ, e) n'a pas servi au jeton t, son dernier usage date au plus tard du jeton $t-1$; entre lui et le jugement de $(t+1, \ell)$, le flux a servi tout le jeton t : L groupes, Lk clés distinctes, toutes différentes de (ℓ, e) (celles de la couche ℓ au jeton t ne la contiennent pas, celles des autres couches ont une autre couche). Son rang est au moins $Lk+1 > N$: défaut. Avec (ii), à $N = Lk$, succès équivaut à réutilisation adjacente. Rapporté à tous les accès (le jeton 0 n'a que des défauts), la valeur exacte est $\bar{r}\,(T-1)/T$. ∎

Corollaire 7.8 (Le seuil en fraction des experts). *Avec $f = N/(LE)$, le LRU partagé est nul pour $f \le \frac{L-1}{L} \cdot \frac{k}{E}$ et vaut au moins $\bar{r}$ pour $f \ge k/E$. gpt-oss-20b : nul jusqu'à 11,98 %, au moins $\bar{r}$ dès 12,5 % ; gpt-oss-120b : nul jusqu'à 3,04 %, au moins $\bar{r}$ dès 3,125 %.*

La figure 7.3 dessine la démonstration : un jeton est un tour d'un anneau de L secteurs de k accès ; entre deux demandes possibles d'un même expert passent $(L-1)k$ accès distincts. C'est la suite cyclique, pire cas de LRU (section 7.3), que le calcul d'un MoE produit de lui-même. Le point (i) a été démontré deux fois le même jour, indépendamment : par l'ingénieur du lot O (son lemme 3, « la falaise », avec son témoin unitaire) et par l'agent de mesure gpt-oss (sa proposition 1, qui ajoute (ii) et (iii)).

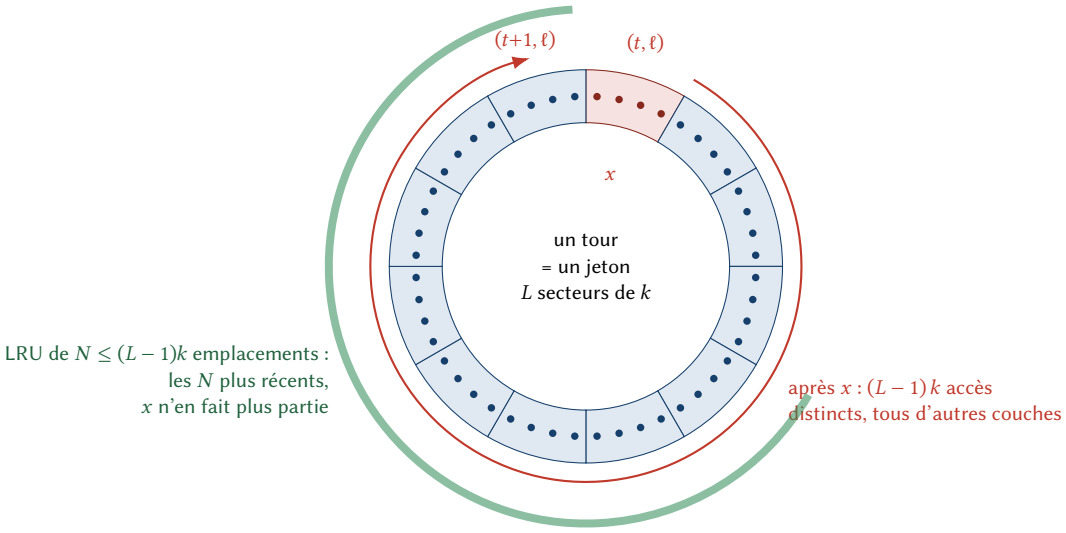


Figure 7.3 : Le flux des accès comme un anneau : un tour est un jeton, chaque secteur une couche de k accès. L'expert x servi en (t, ℓ) ne peut revenir qu'en $(t+1, \ell)$, après $(L-1)k$ accès distincts ; un LRU de $N \le (L-1)k$ emplacements, qui ne garde que les N plus récents, l'a déjà évincé.

Mesuré. Sur les quatre lectures de gpt-oss-20b ($L = 24$, $k = 4$: $(L-1)k = 92$, $Lk = 96$), LRU rend **0 succès sur 1 024 224 accès jusqu'à $N = 92$ inclus**, 9,3 % à 93, 34,8 % à 95, **50,26 % à 96** ; Belady, à 92, en sert 66,4 % [mes.] (figure 7.4). Le plateau qui suit est la valeur exacte de (iii) : $\bar{r}\,(T-1)/T = 0{,}5026$. Le témoin aux capacités exactes du seuil (déclaré avant le calcul ; l'outil s'arrête s'il tombe) vérifie, sur les **17 traces réelles des deux modèles**, le zéro à $(L-1)k$ et l'égalité (iii) au succès près (écart inférieur à 10^{-10} accès, l'arrondi de deux ordres de sommation ; égalité au bit sur 11 traces) [mes.] (figure 7.5). Sa cassure (le zéro attendu à $(L-1)k + 1$) le fait tomber : LRU(93) = 0,0972.

Au-delà de la falaise, un plateau. De $N = 96$ à 120, LRU ne gagne que 0,12 point (50,26 → 50,37 %), Belady 6,8 points. La raison est celle de (iii) : un expert de la couche ℓ absent du jeton t a encore devant lui, à son jugement, tout le jeton t et les clés des autres couches servies depuis son dernier usage ; les emplacements en plus de Lk gardent les experts non réutilisés des couches servies depuis le plus longtemps, qui sortent avant que leur couche ne revienne.

Ce que cela dit au moteur. Pour gpt-oss-120b, $(L-1)k = 140$ emplacements de 13 254 144 o, soit **1 855 580 160 o (1,73 Gio)** de VRAM au-dessous desquels LRU ne sert rien [calc.]. La VRAM de J1 (326 emplacements, environ 4,3 Go) est bien au-dessus ; mais toute « fenêtre minimale » qui ferait descendre le cache sous 1,86 Go avec LRU paierait $h = 0$, là où l'optimum hors ligne en garde encore

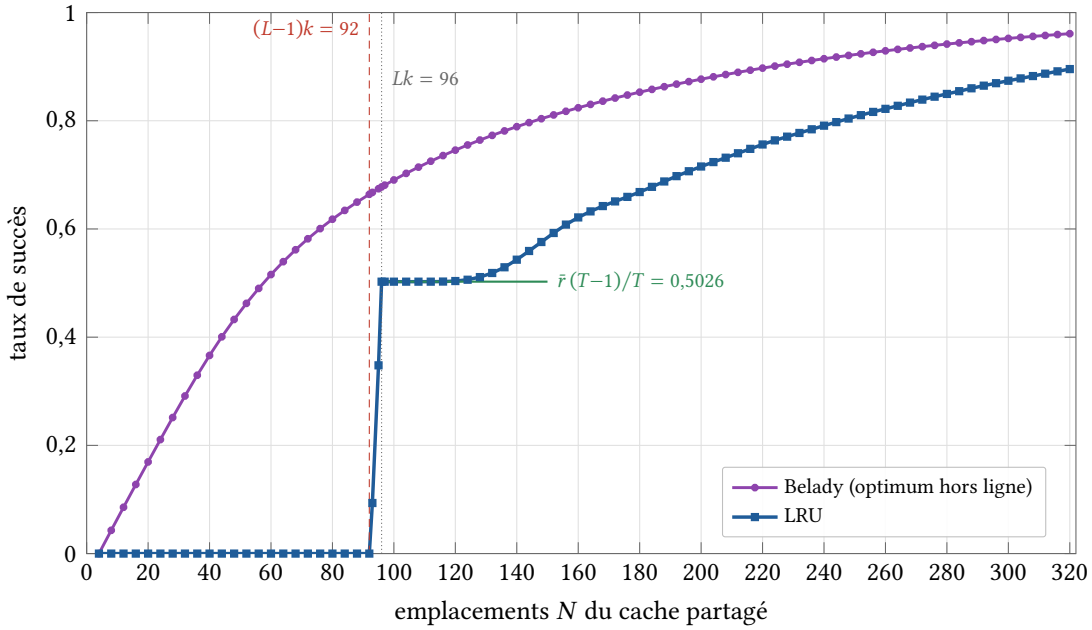


Figure 7.4 : Taux de succès d'un cache partagé selon le nombre d'emplacements, LRU et Belady, sur les quatre lectures de gpt-oss-20b (10 669 jetons). LRU est nul jusqu'à 92, saute à la réutilisation adjacente à 96, puis reste sur un plateau.

Données : `mesures/moteur/O/csv/falaise-20b-dense.csv` (trace `t20b.tous.lecture`).

les deux tiers. Et être au-dessus du seuil ne suffit pas : sur la génération réelle du 120b, au budget du cache de llama.cpp (323 emplacements), un ensemble fixe d'experts bat son LRU (0,500 contre 0,392) et Belady atteint 0,685 (`mesures/localite-gpt-oss/cache-base-contre-optimum.json`) : 29 points de succès à prendre par une meilleure politique avant tout gain de format.

7.6 La localité de gpt-oss

Les traces sont celles de la mesure gpt-oss : 10 669 jetons de lecture par modèle, en quatre genres (prose anglaise et française, dialogue, code), relevés par le même moteur que la base llama.cpp (`mesures/gpt-oss.md` § 4). Deux témoins les garantissent : un second chemin du top-k (notre tri sur les scores du routeur) ne trouve aucun désaccord sur 345 336 choix (20b) et 444 276 (120b) ; et la réplique de la règle du cache de llama.cpp (`g_lru_llama` : succès touchés dans l'ordre du score, défauts triés par numéro, emplacement le moins récemment utilisé) retrouve **exactement** le bilan que llama.cpp publie dans la même exécution : 20 373 succès sur 24 576 et 88 443 sur 98 304 (20b), 14 446 sur 36 864 (120b) ; avec 322 emplacements au lieu de 323, le témoin tombe (témoin G5) [mes.].

7.6.1 Réutilisation : popularité et récence

La réutilisation se décompose en deux termes (tableau 7.3). La **popularité** (certains experts servent plus que d'autres) la porte de 0,125 à 0,31 sur le 20b ; la **récence** (le jeton suivant reprend les experts du précédent), de 0,31 à 0,50. Le témoin G4 le prouve : mélanger l'ordre des jetons ramène la réutilisation au hasard à popularité réelle (0,309 contre 0,310), donc le second terme tient à l'ordre du texte. Sur le 120b, dix fois le hasard uniforme. Par couche, la réutilisation croît avec la profondeur : de 0,14 (couche 0) à 0,53 (couche 35) sur le 120b, de 0,23 à 0,66 sur le 20b.

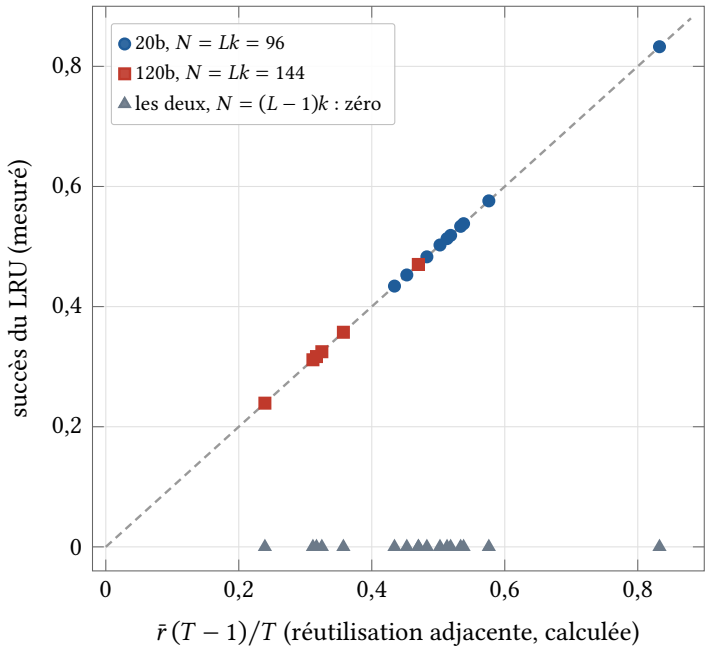


Figure 7.5 : La proposition 7.7 sur les 17 traces réelles (10 du 20b, 7 du 120b ; lecture et génération) : à $N = Lk$, le succès mesuré tombe sur la diagonale $\bar{r}\,(T-1)/T$; à $N = (L-1)k$, il est nul partout.

Données : `memoire/figures/tome2/donnees/t2o-prop1.csv`, tiré de `mesures/localite-gpt-oss/20b/localite-gpt-oss-20b.json` et `120b/localite-gpt-oss-120b.json` (champs `seuil_lru_partage`, `seuil_borne_ii`).

Table 7.3 : Part des k experts d'un jeton déjà choisis par le jeton précédent, même couche, moyenne des couches, quatre genres en lecture [mes.]. Le « hasard à popularité réelle » tire chaque jeton indépendamment, avec la fréquence mesurée de chaque expert.

Modèle	jetons	$\bar{r}$ mesurée	hasard uniforme k/E	hasard à popularité réelle	écart en σ
gpt-oss-20b	10 669	**0,503**	0,125	0,310	+422
gpt-oss-120b	10 669	**0,312**	0,031	0,131	+664

Données : `mesures/gpt-oss.md` § 6.1, § 6.2 ; `mesures/localite-gpt-oss/figures/reutilisation-gpt-oss-*.csv`.

7.6.2 Les politiques

Cinq faits ressortent (figure 7.6 ; `mesures/gpt-oss.md` § 6) :

1. **La localité est forte** : à 20 % des experts, Belady sert 81 % (20b) et 90 % (120b) des accès, LRU 60 et 78 %, quand un routage sans localité en servirait 20 %. L'hypothèse uniforme des études antérieures sous-estimait le cache d'un facteur 3 à 4 sur la part servie.
2. **L'écart à Belady est grand** (0,60 contre 0,81 à 20 % sur le 20b) : c'est la marge d'un prédicteur parfait.
3. **Partager le budget ne profite qu'à une bonne politique** : sur le 120b, Belady partagé bat Belady par couche à tout budget (0,632 contre 0,508 à 5 %), mais LRU partagé reste sous LRU par couche jusqu'à 15 % (0,333 contre 0,384 à 5 %) : le flux cyclique lui coûte même au-dessus du seuil.
4. **Un ensemble choisi hors ligne ne voyage pas** : l'épinglage calibré sur le début d'une trace s'effondre quand le genre change (0,147 à 10 % sur le 120b, contre 0,37 à 0,64 dans chaque genre

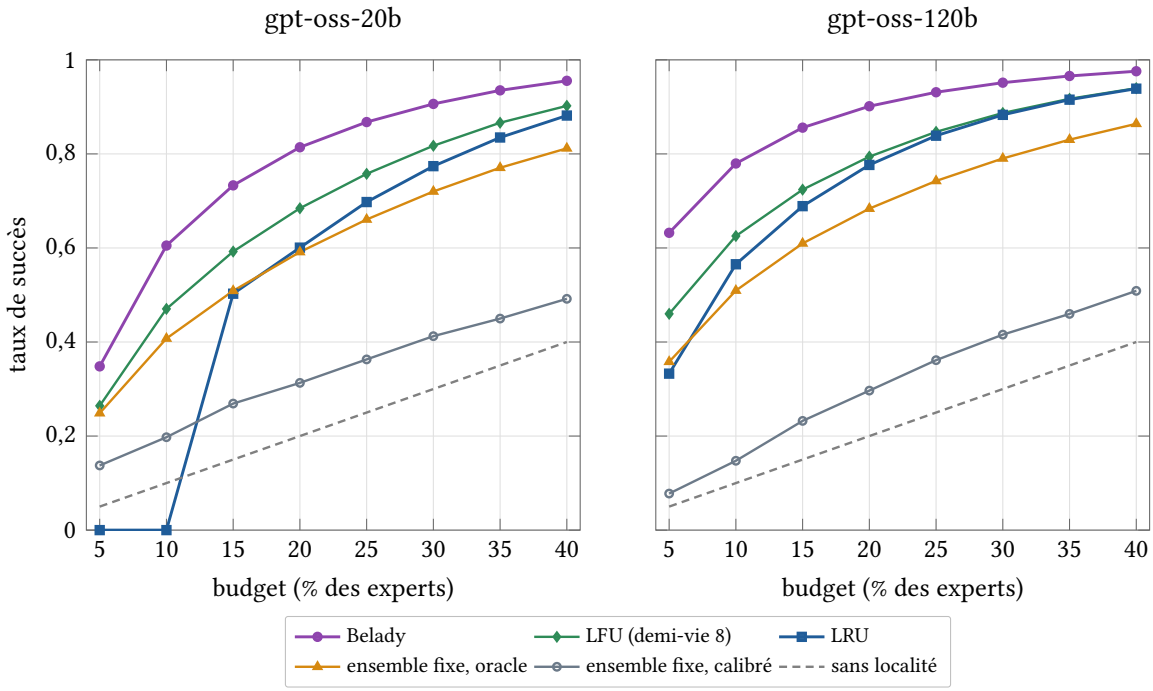


Figure 7.6 : Taux de succès d'un cache partagé selon son budget (part des experts), quatre genres en lecture. Sur le 20b, LRU est nul à 5 et 10 % (sous la falaise, 11,98 %) ; sur le 120b, la falaise est à 3,04 %, sous le premier budget tracé.

Données : `mesures/localite-gpt-oss/figures/succes-gpt-oss-20b-lecture.csv` et `succes-gpt-oss-120b-lecture.csv`.

pris seul).

5. **Le routage dépend du chemin de calcul** : sur le 120b, carte contre processeur, 81,1 % de choix identiques seulement, et le premier écart arrive au 3e jeton. Une trace vaut pour son chemin, et la justesse d'un prédicteur d'experts a un plafond fixé par ce bruit.

Ce que la localité vaut en jetons par seconde. Avec les débits mesurés de la machine (VRAM 251,3 Go/s, PCIe 12,2 Go/s, disque 2,82 Go/s en blocs de 16 Mio), un expert valant les octets d'origine, et le modèle à deux étages du lemme 7.6 (VRAM à 7 % des experts, $h_v = 0{,}449$; budget total 20 %, LRU, $h = 0{,}777$), le temps d'un jeton recouvert se pose ainsi [calc.] :

$$\text{VRAM} : \frac{1{,}686 + 0{,}449 \times 1{,}909}{251{,}3} = 10{,}1\ \text{ms}, \qquad \text{PCIe} : \frac{0{,}551 \times 1{,}909}{12{,}2} = 86{,}2\ \text{ms},$$
$$\text{disque} : \frac{0{,}223 \times 1{,}909}{2{,}82} = 151\ \text{ms}.$$

Le maximum, 151 ms, donne 6,6 jetons/s ; le disque borne. Tout facteur de compression c des experts *en transit* divise les termes du disque et de la PCIe par c : à $c = 2$, $\max(10{,}1 ; 43{,}1 ; 75{,}5) = 75{,}5$ ms, soit 13,2 jetons/s, tant que le décodage sur la carte suit (`mesures/gpt-oss.md` § 8.3). C'est le levier de troncons, chiffré.

7.7 L'ordonnanceur à la demande

7.7.1 Le chemin d'un jeton

Le moteur de J1 (`trc-ordo`) sert chaque couche à la demande, sans préchargement (figure 7.7). Trois flux CUDA : **calcul** (priorité haute), **copie**, **décodage**. Pour chaque couche ℓ, le flux de calcul fait l'attention puis le routeur, qui écrit les k indices et leurs poids dans une **boîte aux lettres** en mémoire projetée (mot 0 : un numéro de séquence écrit en dernier, après une barrière ; mots 1 à k : les indices ; $k+1$ à $2k$: les poids). L'hôte attend l'événement « routeur ℓ », vérifie la séquence, lit les indices et appelle `demander`. Chaque défaut devient une lecture sans tampon par IOCP dans un tampon d'arrivée épinglé ; dès qu'elle est finie, la copie part sur le flux de copie, puis le décodage sur le flux de décodage, qui écrit l'emplacement ; le flux de calcul attend l'événement « prêt » de chaque défaut, puis lance les quatre GEMV et la combinaison, dans l'ordre du routeur.

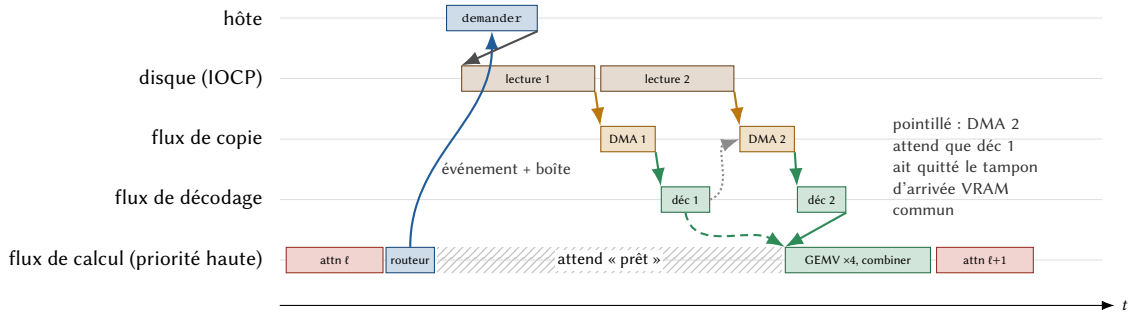


Figure 7.7 : Le chemin d'une couche dont deux experts manquent (schéma M1, tel que construit) : les flux ne se rencontrent que par des événements ; la boîte aux lettres porte les indices du routeur vers l'hôte.

Données : `moteur/crates/trc-ordo/src/lib.rs` (en-tête et `Moteur::pas`).

Listing 7.2 : La décision d'une couche dans `Moteur::pas` (`trc-ordo/src/lib.rs`, extrait) : attendre le routeur, vérifier la boîte, demander au cache.

```
self.ev_routeur.enregistrer(&self.calcul)?;
// Variante « evenement » de l'essai E0 c (18,5 us par synchronisation sous charge) :
// on attend l'événement, puis on lit la boîte.
self.ev_routeur.attendre()?;
let lu = self.boite.lire_volatil_u32(0);
if lu != seq {
    bail!("couche {l} : boîte aux lettres à la séquence {lu}, {seq} attendue");
}
for i in 0..k {
    let x = self.boite.lire_volatil_u32(4 * (1 + i));
    if x as usize >= h.experts { bail!("couche {l} : expert {x} hors de [0, {})", h.experts); }
    experts[i] = x as u16;
}
self.trace.extend_from_slice(&experts[..k]);          // la trace que rejouer relira
let decisions = self.cache.demander(l as u16, &experts[..k], pas * l_n as u64 + l as u64);
```

7.7.2 L'ordre est sûr

Lemme 7.9 (L'ordre des trois flux est sûr)**.** *Dans* `Moteur::pas`*, aucun emplacement d'expert n'est lu par une GEMV pendant qu'un décodage l'écrit, et aucun tampon d'arrivée (épinglé ou en VRAM) n'est récrit avant que la copie ou le décodage qui le lisait soit fini.*

Hypothèses. (H1) Un flux CUDA exécute ses opérations dans l'ordre de leur mise en file, chacune après la fin de la précédente. (H2) `cuStreamWaitEvent(f, e)` fait attendre à toute opération mise ensuite sur f la fin de tout ce que e a capturé [42]. (H3) L'hôte ne décide la couche ℓ (demander, soumettre, mettre en file) qu'après avoir vu fini l'événement « routeur ℓ », enregistré sur le flux de calcul juste après le routeur. (H4) Un expert demandé par la couche ℓ est épinglé jusqu'à `liberer(`ℓ`)`, appelé après la mise en file de la fin de la couche ℓ ; la victime d'une éviction n'est jamais épinglée ni en vol.

Démonstration. (a) *Emplacements.* Soit s un emplacement décodé pour la couche ℓ' et évincé par une demande de la couche ℓ, plus tard dans le flux. Par (H4), l'éviction suit `liberer(`ℓ'`)`, donc la mise en file de la fin de ℓ' sur le flux de calcul ; le nouveau décodage dans s est mis en file après la décision de ℓ, donc, par (H3), après que l'hôte a vu fini le routeur de ℓ. Comme ce routeur est mis en file après la fin de ℓ' sur le même flux, (H1) donne : toutes les GEMV qui lisaient s sont finies quand le décodage commence. Réciproquement, les GEMV de ℓ qui lisent s sont mises en file après l'attente de l'événement de son décodage (H2) : elles ne commencent qu'après sa fin. (b) *Tampons d'arrivée.* Le tampon i sert au plus une fois par couche ; sa réutilisation à la couche ℓ suit la décision de ℓ, donc (H3, H1) la fin des couches précédentes, qui ont attendu (H2) les décodages de leurs défauts, qui ont attendu leurs copies. Les attentes explicites (copie après le décodage précédent du même tampon, lecture après la copie du même tampon épinglé) rendent ce point vrai sans (H3), ce qui le garde si un préchargement avance les défauts. ■

Coût : une synchronisation de l'hôte par couche, 36 par jeton sur le 120b, 18,5 µs chacune sous charge (essai E0 c du lot E), soit 0,67 ms par jeton, 0,15 % des 457 ms d'un jeton [calc.].

7.7.3 Le moteur calcule ce que les noyaux ont validé, au bit

Proposition 7.10 (Jeton par jeton égale couche par couche)**.** *Pour une suite de jetons et des poids donnés, les logits que rend le moteur (un jeton à la fois, toutes les couches) sont égaux au bit à ceux de la chaîne de validation des noyaux (une couche à la fois, toutes les positions), si les deux lancent les mêmes noyaux avec les mêmes arguments numériques.*

Démonstration. Par la règle de déterminisme de l'architecture (aucune addition atomique flottante, réductions à arbre fixe ne dépendant que des tailles), chaque noyau est une fonction déterministe des octets qu'il lit et de ses paramètres. On ordonne les calculs par (couche ℓ, position p). Hypothèse de récurrence : pour tout $(\ell', p') < (\ell, p)$ dans l'ordre produit, le résidu en entrée de (ℓ', p') et la case p' du cache KV de ℓ' sont égaux au bit dans les deux ordres. En (ℓ, p), les entrées sont : le résidu, sortie de $(\ell - 1, p)$, égal par récurrence ; les cases $q \le p$ du cache KV de ℓ, écrites par (ℓ, q), égales par récurrence, et la case p elle-même ; pour une couche à fenêtre, l'anneau de 128 contient au moment de la lecture exactement les positions $\max(0, p - 127) \dots p$ dans les deux ordres ; les experts choisis sont ceux du routeur de (ℓ, p), calculé sur des entrées égales, et leurs emplacements portent les mêmes octets (format et transport exacts). Les mêmes noyaux rendent les mêmes bits. ■

Témoin O2 : 160 lignes sur 160 égales à la chaîne du lot K (KL, premier choix, écart relatif), KL moyenne $7{,}2794050122285915 \cdot 10^{-3}$ au chiffre près ; le chemin qui pose les experts du GGUF sans

décodeur, 16 sur 16. Les cassures (position décalée de 1 ; experts 0 et 1 échangés à la couche 5) font différer toutes les lignes [mes.] (`mesures/moteur/O/journaux/o2/`). Le témoin ne juge pas la justesse du modèle (c'est C4, au chapitre de la référence) : il juge que le moteur calcule *exactement* ce que le lot K a validé.

Proposition 7.11 (C3 par construction : le cache n'entre pas dans le calcul)**.** *Si deux fichiers (codé et brut) donnent, pour chaque expert, des emplacements égaux octet pour octet, le moteur rend les mêmes logits au bit sur toute suite de jetons, quels que soient le nombre d'emplacements, la politique du cache et l'ordre d'arrivée des lectures.*

Démonstration. Les GEMV d'un expert lisent son emplacement et le vecteur d'entrée ; l'emplacement porte, au moment de la lecture, les octets décodés de cet expert (lemme 7.9), égaux par hypothèse dans les deux fichiers. Le cache ne choisit que *où* est un expert, jamais *quels octets* il porte ; l'ordre de la combinaison est celui du routeur, pas celui des emplacements. Les autres entrées ne dépendent pas du fichier d'experts. La proposition 7.10 conclut. ■

Mesuré : sur le 20b, 5 120 pas, 0 empreinte de logits différente, fichiers `.kld` égaux octet pour octet, perplexité égale (461,978277060145) ; les deux passes n'avaient pas le même nombre d'emplacements (362 et 361) ni donc la même suite de défauts : la partie « quel que soit le cache » est observée, pas seulement déduite. Sur le 120b, 5 120 pas, 0 différence, perplexité 422,20558522791305 des deux côtés, avec 325 et 326 emplacements [mes.].

Proposition 7.12 (Le compte du moteur égale le simulateur)**.** *Le moteur appelle* `trc-cache` *dans l'ordre du simulateur : pour chaque jeton t et chaque couche ℓ,* `demander(`ℓ`,` $G(t,\ell)$`,` $tL+\ell$`)`*,* `pret` *sur chaque défaut dans l'ordre des décisions, puis* `liberer(`ℓ`)`*, et rien d'autre ne touche le cache en J1.* `rejouer` *fait la même suite sur la trace écrite par le moteur. Mêmes appels sur le même état initial : mêmes compteurs.*

Mesuré : sur le 20b (5 120 pas, 362 emplacements), **454 210 succès, 37 310 défauts, 36 948 évictions des deux côtés** ($h = 0{,}924$) ; cassure (LRU faussé) : 427 984 / 63 536 / 63 174. Sur le 120b (326 emplacements) : 334 608 succès, 402 672 défauts, 402 346 évictions des deux côtés ($h = 0{,}454$) ; cassure : 289 788 / 447 492 [mes.] (`moteur/REPRISE-O.md`, O3).

7.7.4 Le plancher en série, et ce que le moteur en fait

L'équation du jeton pose le temps d'un jeton servi en série comme la somme des étages. Avec les quantités mesurées dans la séance O4 (génération, 120b, mode A0, 326 emplacements, $h = 0{,}591$), le critère C5 se calcule pas à pas (tableau 7.4).

Mesuré : 457 ms par jeton, 2,186 et 2,182 jetons/s, au-dessus du plancher *en série* : les copies et décodages d'une couche se recouvrent avec les lectures des défauts suivants de la même couche. Le chronogramme d'une couche dont les quatre experts manquent (figure 7.8) montre ce qui limite : le routeur finit à 0,61 ms ; les quatre lectures, soumises à 1,28, 1,96, 2,68 et 3,18 ms, finissent à 6,86, 9,81, 16,14 et 21,17 ms. **Le disque les sert l'une après l'autre** (environ 5 ms pour 12 Mo, 2,4 Go/s chacune) ; la profondeur 4 ne les fait pas avancer ensemble. Chaque copie (environ 1,0 ms) et chaque décodage (0,85 ms) suivent la lecture de leur tronçon sans attendre les autres, et la fin de la couche (23,73 ms) suit le dernier décodage de 0,6 ms. Avec 59 défauts sur 36 couches, 1,6 par couche en moyenne, le moteur tire 0,707 Go en 457 ms, 1,55 Go/s utiles, contre 1,85 Go/s pour le disque seul à profondeur 4 : **ce qui limite J1 est la profondeur de file par couche** ($\leq k$), que seul un préchargement pourrait remplir.

Table 7.4 : Le plancher en série du 120b, étage par étage, dans la séance O4 [calc.] sur [mes.].

Étage	Formule	Valeur
disque	octets de tronçons lus par jeton / débit du disque seul	0,7072/1,848 Go/s = **382,7 ms**
PCIe	temps de carte des copies par jeton (59 défauts, 1,28 ms chacun)	**75,8 ms**
décodage	temps de carte des décodages par jeton (0,92 ms par expert)	**54,2 ms**
GEMV	$(A + R)/235$ Go/s, $A = 1{,}686$ Go, $R = 144 \times 13{,}25$ Mo	**15,3 ms**
T_0	coût fixe	3,5 ms
somme		**531,5 ms → 1,881 jeton/s**
seuil C5	0,8× plancher	1,505 jeton/s

Données : `moteur/O-pour-le-memoire.md` § 6.5 ; `mesures/moteur/O/journaux/o4/verdicts-2026-10-15-s1.json`.

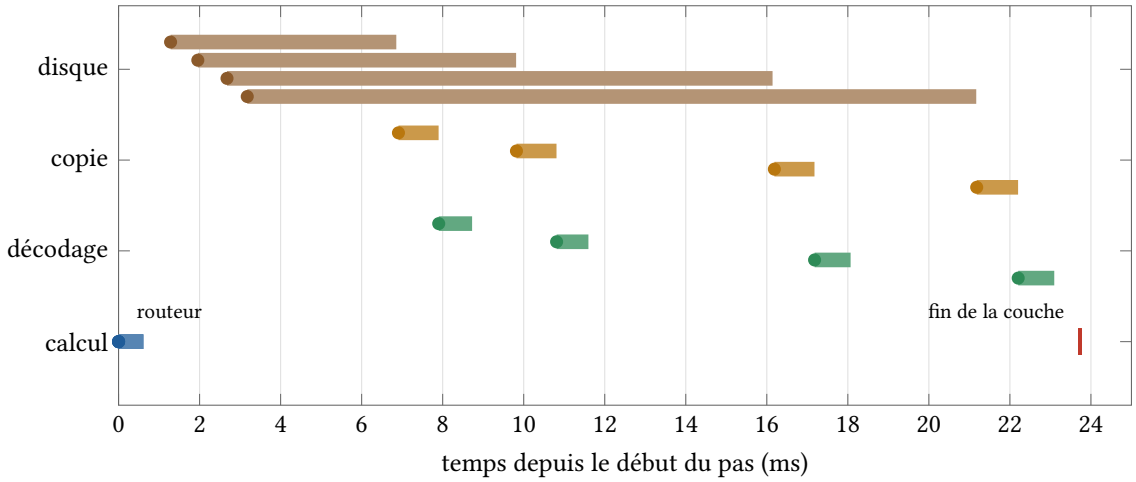


Figure 7.8 : Le chronogramme mesuré d'une couche du 120b dont les quatre experts manquent (pas 8, couche 0) : de la soumission à la fin de chaque lecture, puis sa copie et son décodage ; en bas, le routeur et la fin de la couche.

Données : `memoire/figures/tome2/donnees/t2o-chrono-couche.csv`, tiré de `mesures/moteur/O/csv/chronogramme-120b.csv` (8 jetons, 1 048 défauts datés).

7.8 Le préremplissage par flux

Dans la boucle de pas, une invite de P positions lit $B_{\text{boucle}} = \sum_p \sum_\ell (1 - h_{p,\ell})\, k\, t$ octets, avec $t \approx 12{,}03$ Mo le tronçon. Par flux, chaque couche passe toutes les positions par l'attention et le routeur, puis lit une fois chaque expert demandé par au moins une position : $B_{\text{flux}} = \sum_\ell |U_\ell \setminus V_\ell|\, t$, avec U_ℓ l'ensemble des experts choisis et V_ℓ ceux déjà en VRAM. Sous un routage uniforme, $\mathbb{E}|U_\ell| = E\,(1 - (1 - k/E)^P)$; pour $E = 128$, $k = 4$, $P = 708$, $(1 - 1/32)^{708} = e^{708 \ln 0{,}96875} = e^{-22{,}5}$: U_ℓ est tout l'ensemble, et $B_{\text{flux}} \le 36 \times 128 \times 12{,}03$ Mo $= 55{,}4$ Go, contre $B_{\text{boucle}} \approx 708 \times 144 \times 0{,}55 \times 12{,}03$ Mo ≈ 675 Go [calc.] : un rapport d'environ 12. **Mesuré** : 26 s au lieu de 7 min 36 s, ×17,5, aux débits moyens des passes `s2-11` à `s2-14` (27,15 contre 1,554 jetons/s, `mesures/moteur/O/csv/o-jetons-s.csv` ; la note du lot O écrivait ~~7 min 50 s~~, qui ne vient pas de ces passes) ; en plus des octets, la boucle paie une synchronisation par couche et par position, que le flux fait une fois par couche. L'exactitude suit de la proposition 7.10 (l'ordre couche par couche est celui de la chaîne des noyaux) ; témoin F : 5 120 empreintes égales à la boucle sur le 20b, et, refait par l'épreuve indépendante sur le 120b, des fichiers

`.kld` égaux octet pour octet d'une construction à l'autre [mes.].

Un défaut démontré, puis corrigé. Le préremplissage par flux confie au système jusqu'à huit lectures à la fois, dans un tourniquet de 12 tampons d'arrivée. Sa première règle prenait le tampon $s \bmod 12$ pour la s-ième lecture. Contre-exemple : que la lecture $s-12$ reste dans le système (les achèvements arrivent dans le désordre) pendant que $s-11, \dots, s-1$ passent et sont récoltées ; la liste des lectures en vol ne contient plus que $s-12$, et s est soumise dans le tampon $s \bmod 12 = (s-12) \bmod 12$, que le disque écrit encore. Il suffit d'un retard d'environ $11 \times 6{,}6\ \text{ms}/3 \approx 24\ \text{ms}$ d'une lecture sur les autres [calc.], et l'épreuve a relevé un 90e centile de 172 ms pour une lecture à froid.

Lemme 7.13 (Les tampons d'arrivée du préremplissage par flux)**.** *Avec la règle corrigée (le tampon du tourniquet s'il ne porte aucune lecture en vol, sinon le suivant qui n'en porte pas), aucune lecture n'écrit dans un tampon d'arrivée tant qu'une autre lecture y écrit, que la copie qui le lisait n'est pas finie, ou qu'un hachage de la garde le lit encore.*

Démonstration. Existence : au moment du choix, au plus $\min(dk, 2 \cdot \text{profondeur}) - 1 = 7 < 12$ lectures sont en vol, donc un tampon libre existe. La copie : on attend l'événement de la copie précédente de ce tampon avant de soumettre (elle a été mise en file à sa récolte, donc avant). La garde : on attend la fin de son hachage avant toute soumission. Le tampon VRAM n'est récrit qu'après l'attente du décodage qui le lisait. ■

Sous la charge de l'épreuve (sa passe numpy f64 du 120b en même temps), les dix fenêtres du 120b brut finissent en 361 s avec 0 conflit évité : le cas ne s'est pas produit dans cette passe. La règle est démontrée ; que ce défaut soit la cause du blocage observé par l'épreuve (constat G1) reste une hypothèse, l'ancien binaire ayant lui aussi fini sous la même charge (`moteur/O-pour-le-memoire.md` § 14.1).

7.9 Le préchargement, essayé et abandonné

Le mécanisme. Après l'attention de la couche ℓ, le moteur applique au résidu courant la norme et le routeur de $\ell + 1$; les experts prédits absents de la VRAM sont lus d'avance dans $2k$ tampons épinglés, hors du cache (dont le compte reste celui de `rejouer` ; les quatre textes de la série sont égaux). L'étude de chargement citait 96 % de prédictions justes une couche d'avance, *littérature, non reproduite.*

Le bilan, posé avant la mesure. Une prédiction juste retire une lecture du chemin critique ; une fausse ajoute une lecture sur un disque qui limite déjà. Soit p la part des prédictions servies et f la part du temps d'un jeton que le recouvrement peut cacher (le calcul hors disque) ; le gain relatif vaut au premier ordre

$$\text{gain} \approx f - \frac{1-p}{p} \cdot (\text{part du disque}). \tag{7.1}$$

La prédiction déclarée (`moteur/REPRISE-O.md`) était d'au plus +19 % (les $457 - 383 = 74$ ms de calcul hors disque, rapportées aux 383 ms du disque) plus ce que la file plus profonde rendrait au disque ; cette prédiction ne comptait pas les lectures fausses.

Mesuré (séance `2026-10-16-s3`, A B B A, 120b, secteur) : 69,2 % des lectures anticipées servent (11 529 sur 16 670) ; les autres ajoutent 34 % d'octets lus (953 contre 710 Mo par jeton) ; le calcul qu'elles peuvent recouvrir vaut environ 80 ms par jeton, $f \approx 80/460 = 0{,}17$. Avec $p = 0{,}692$ et un disque qui

porte environ 80 % du temps, le terme négatif de (7.1) vaut $(0{,}308/0{,}692) \times 0{,}8 \approx 0{,}36$ et l'emporte sur f : le premier ordre prédit une perte, de l'ordre de $0{,}17 - 0{,}36 \approx -0{,}19$ [calc.]. **Génération : A 2,158 et 2,178, B 1,935 et 1,924 jetons/s, soit** −11,0 %, pour un bruit de 0,9 % [mes.] (refait ici depuis `mesures/moteur/O/csv/o-jetons-s.csv`). L'écart passe la règle de décision ($2 \times 0{,}9\,\% + 3\,\%$) : il est réel, et dans le mauvais sens, moindre que le premier ordre (la file plus profonde rend au disque une partie de ce que les lectures fausses lui prennent, ce que (7.1) ne compte pas).

> **Ce qui est tué, et ce qui ne l'est pas**
>
> Le préchargement par le routeur de la couche suivante est **abandonné pour ce prédicteur et ce disque** (chiffré par BitLocker, 1,8 Go/s), pas en général. Par (7.1), il ne reviendrait qu'avec p au-dessus d'environ 0,9, ou un disque qui ne limite pas. Un défaut a été trouvé en chemin et corrigé (l'achèvement de la couche en cours récolté pendant le vidage des préchargements).

Les lectures de 12 Mio, non établies. Lire chaque tronçon en une requête de 12 Mio (au lieu de sa longueur exacte, de l'ordre de 12,0 Mo) donne +30 % en médiane (A 2,179 et 2,182, B 2,613 et 3,073 jetons/s), chaque passe B au-dessus de chaque passe A ; mais le bruit de B, $\beta = 16{,}2\,\%$ (~~21 %~~ dans la note du lot O, écart rapporté à la moyenne de A), porte le seuil de la règle à $2 \times 16{,}2 + 3 = 35{,}4\,\%$, au-dessus du gain : la règle ne conclut pas en quatre passes. Le mécanisme supposé est la carte des deux régimes du disque relevée par le lot E ; la décision revient à l'arbitre.

7.10 La garde d'intégrité, et ce qu'elle coûte

L'épreuve indépendante a montré qu'en production le moteur ne vérifiait pas les octets bruts d'un tronçon (médianes et biais, 0,36 % des octets, non codés par rANS) : une médiane corrompue donnait un expert faux sans erreur, et le texte produit était même identique (constat M1). La ronde de corrections a ajouté une **garde** : chaque lecture achevée est hachée par quatre fils (empreinte64, le préfixe de 64 bits de SHA-256 [43]) et comparée au répertoire avant que le jeton ne sorte ; chaque ligne d'enchâssement lue est comparée à une table bâtie à l'ouverture.

Proposition 7.14 (Aucune sortie ne dépend d'un octet non vérifié)**.** *Garde active, le jeton et les logits que rend un pas ne dépendent que d'octets lus dont l'empreinte égale celle du répertoire, à une collision du préfixe de 64 bits près (probabilité 2^{-64} par vérification, si SHA-256 se comporte comme une fonction aléatoire).*

Démonstration. Le décodeur et la copie ne lisent d'une partie que ses octets utiles ; chaque lecture achevée est confiée à la garde avant toute autre opération sur son tampon ; aucune lecture n'écrit ce tampon avant la fin du hachage (lemme 7.13), donc l'empreinte porte sur les octets que la copie a lus ; le pas attend tous les hachages avant de lire le jeton ou les logits, et une empreinte différente est une erreur fatale nommée. Une entrée tiède a été hachée à sa lecture ; une corruption de la RAM après coup n'est pas couverte. ∎

Le coût, posé puis mesuré. Un pas du 120b lit environ 0,71 Go de tronçons ; l'empreinte va à 1,07 à 1,10 Go/s par fil, mesuré (`mesures/moteur/O/corrections/journaux/debit-empreinte.jsonl` ; ~~2 Go/s par cœur~~, estimé par l'épreuve). Il faut donc environ 0,65 s de cœur par pas, étalées sur quatre fils (0,17 s de mur, sous les 0,5 s d'un pas) ; au chemin critique ne reste que le hachage de la dernière lecture, au plus 12,25 Mo/1,07 Go/s ≈ 11,4 ms, moins le travail de carte qui la suit [calc.]. **Mesuré** : 5,2 à 5,4 ms d'attente par jeton sur les quatre passes gardées ; à débit de disque égal (passes

04 et 05, à 1,715 et 1,708 Go/s), 1,900152/1,931729 − 1 = −1,6 % [mes.] (`mesures/moteur/O/corrections/csv/vitesse-2026-10-16-passes.csv`). La note du lot O écrivait ~~5,2 à 5,3 ms~~ (la passe 07, à 5,42 ms, n'y était pas) et ~~−1,7 %~~, calculé sur les valeurs arrondies 1,900 et 1,932 (section 8.5.5). La série déclarée dit −12,3 % sous un seuil de 26,8 % (bruit 11,9 %), non réel : le disque a varié de 1,51 à 1,95 Go/s pendant la séance. L'ouverture coûte 0,95 s de plus (la table des lignes : 0,615 Go lus et hachés), l'ensemble de travail 1,9 Mo, le même pour les deux modèles.

7.11 Le moteur contre la base : le critère C6

C6 demande que troncons batte la meilleure configuration mesurée de llama.cpp sur le 120b, même invite, 256 jetons gloutons, de plus de deux fois le plus grand bruit et de 3 % (règle de l'étude des modèles). Toute la difficulté est dans « la meilleure ». Son histoire se lit en barres (figure 7.9).

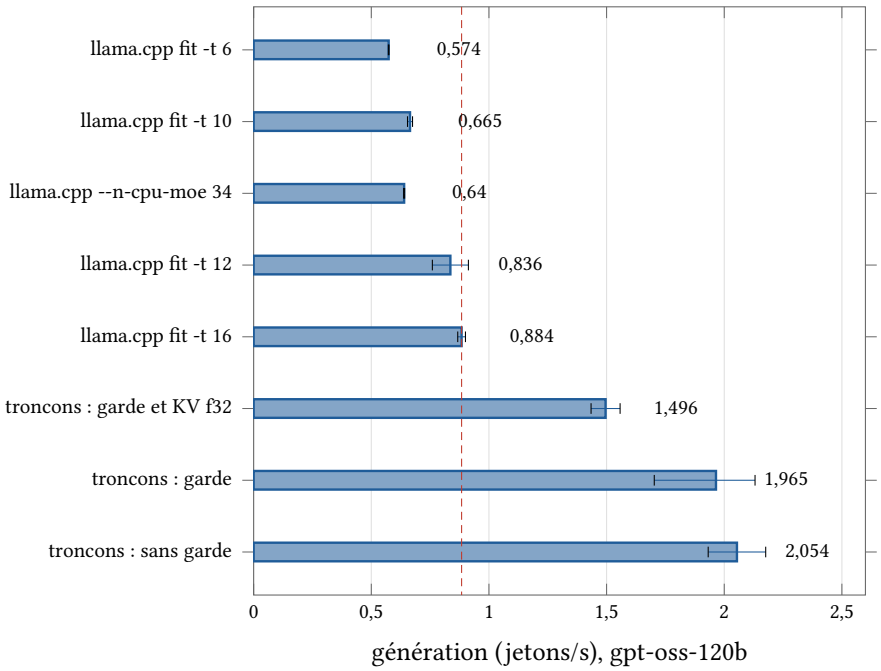


Figure 7.9 : Génération sur gpt-oss-120b (256 jetons gloutons, invite wikitext de 708 jetons) : la base llama.cpp balayée par la ronde de corrections et troncons dans la séance de vitesse de la même ronde, le même jour, sur secteur. Moyenne des passes, barres de la plus petite à la plus grande ; trait pointillé : la meilleure base, `fit -t 16`.

Données : `memoire/figures/tome2/donnees/t2o-c6.csv`, tiré de `mesures/moteur/O/corrections/csv/base-2026-10-16-passes.csv` et `vitesse-2026-10-16-passes.csv`.

- ~~×3,85~~ : J1 (2,184 jetons/s) contre `fit` à 6 fils (0,568), la base publiée par la mesure gpt-oss. Barré : la base était sous-réglée.
- ~~×2,79 à 2,89~~ : l'épreuve indépendante, contre `fit` à 10 fils (0,702 dans sa séance). Barré à son tour : l'épreuve n'avait pas essayé au-delà de 10 fils.
- ×2,22 : la ronde de corrections a balayé la base (séance du 2026-10-16 de 8 h 13 à 10 h 06, palindrome, une passe d'échauffement jetée, secteur à chaque passe, aucun processus étranger) : `fit -t 6` 0,574 ; `-t 10` 0,665 ; `-t 12` 0,836 (bruit 18,3 %) ; **`-t 16` 0,884** (0,900 et 0,867) ; `-ngl 99 --n-cpu-moe 34 --fit off -t 10` 0,640. La meilleure, `fit -t 16`, bat `-t 6` de 54 % : la base

publiée sous-estimait llama.cpp de 54 %. Le mécanisme est celui que l'épreuve avait déclaré : sous mmap, chaque fil en faute de page est une lecture en vol ; 16 fils, c'est une file plus profonde sur le disque. troncons, **garde d'intégrité comprise**, fait 1,965 jeton/s (quatre passes de la séance de vitesse, bruit 10,97 %) : 1,965/0,884 = **2,22**, au-delà du seuil 2 × 10,97 % + 3 % = 24,9 % [mes.] (refait ici : 1,9649 et 0,8838). **C6 tient.**

Sans garde, troncons fait 2,054 jetons/s dans la même séance (deux passes) ; avec la garde et un cache KV en f32, 1,496, mais avec un autre texte (chapitre de la référence), donc sans comparaison possible. Le préremplissage par flux fait 26,6 jetons/s, contre 3,1 à 4,0 pour llama.cpp (par lots) : un facteur 7. Et la mémoire de pointe du moteur est de 384 Mo, contre 10,9 Go pour llama.cpp sur le même modèle (`moteur/O-pour-le-memoire.md` § 8). **La base peut encore être meilleure** : ni plus de 16 fils, ni `--mlock`, ni d'autres `--n-cpu-moe` n'ont été balayés.

7.12 Ce qui reste

- **Une politique consciente du cycle des couches.** La falaise et le plateau viennent de ce que LRU ignore que le flux revient à chaque couche une fois par jeton ; un LRU par couche, un SLRU, ou un LFU (0,47 à 10 % sur le 20b, là où LRU fait 0) font mieux sous le seuil, et il reste 29 points entre le LRU de llama.cpp et Belady sur la génération du 120b.
- **Un prédicteur plus juste** : le préchargement ne revient qu'au-dessus de $p \approx 0{,}9$; le routage dépendant du chemin de calcul (81,1 % d'accord carte contre processeur) borne ce qu'on peut espérer d'un prédicteur appris ailleurs.
- **La longueur des lectures** (12 Mio, +30 % en médiane, non établi) et la profondeur de file par couche, ce qui limite vraiment J1.
- **La base** au-delà de 16 fils et avec `--mlock`.

Ce qui pourrait être faux

- **L'optimalité de Belady avec épinglage de groupe** n'est pas démontrée, seulement vérifiée (G3) ; de même le lemme 7.6 pour Belady. Belady est un oracle : ses chiffres sont des bornes.
- **Le témoin O1 compare des règles**, pas des chiffres de vitesse ; la règle du moteur diffère de celle de llama.cpp par l'ordre des défauts d'un groupe (4 et 5 succès d'écart).
- **La falaise dépend de l'épinglage du groupe et de** $N \geq k$; un cache qui servirait les experts d'un jeton dans un autre ordre (préchargement) déplace le seuil.
- **La localité vient de 10 669 jetons de quatre textes**, sans gabarit de conversation ; les générations relevées sont souvent dégénérées et gonflent la récence. Les traces sont celles du chemin de calcul de llama.cpp ; le moteur produit les siennes.
- **Le plancher de C5 compte la GEMV au débit d'une autre séance** (235 Go/s) ; si la GEMV réelle est plus lente, le plancher est optimiste et le seuil trop haut : le verdict reste valable dans ce sens.
- **Le bilan du préchargement** (7.1) est un premier ordre ; la mesure (A B B A, une séance) est ce qui tranche, et elle ne vaut que pour ce disque.
- **C6 compare deux séances du même jour** (base et vitesse) ; la séance de vitesse a vu le disque varier de 1,51 à 1,95 Go/s, et le bruit de troncons (11 %) est grand. La base peut être meilleure que 0,884.
- **La cause du blocage G1** n'est pas établie ; le défaut des tampons d'arrivée est démontré possible, pas observé (0 conflit sous la charge rejouée).

> – **La garde suppose SHA-256 sans collision utile** sur 64 bits ; elle ne couvre ni la RAM tiède après sa lecture, ni la VRAM. Son coût (−1,6 %) repose sur une paire de passes à disque égal.

Troisième partie

Le jugement

Chapitre 8

Résultats du jalon J1

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, un programme construit par cinq lots, chacun prouvé seul par son témoin, et douze critères déclarés le 15 octobre 2026 avant la première ligne de code (`decisions/ARCHITECTURE.md`, § 7). Le Tome I avait prédit ce que ce programme pourrait faire ; il ne l'avait jamais vu tourner. **Ce chapitre établit** ce que troncons fait vraiment sur la machine de Stone, avec gpt-oss-120b : des logits identiques au bit entre le fichier codé et le fichier brut sur 5 120 pas ; 384 Mo de mémoire de pointe pour un modèle de 63,4 Go, la même à 3,7 Mo près que pour un modèle cinq fois plus petit ; 2,18 jetons par seconde en génération, 2,22 fois la meilleure base llama.cpp mesurée une fois la garde d'intégrité comprise ; et, avec un cache KV en f32, des sorties égales aux mathématiques exactes du modèle au format près. Chaque chiffre qu'une mesure plus récente a démenti est barré à sa place, avec ce qui l'a remplacé. **Il reste** les décisions de l'arbitre que la ronde de corrections lui a remises (cache KV par défaut, critères C4, C7 et C8), et ce que l'épreuve n'a pas pu refaire (chapitre 9). Sources : `moteur/O-pour-le-memoire.md` (parties II et III), `moteur/REPRISE-O.md`, `mesures/epreuve-J1.md`, `mesures/gpt-oss.md`, et les CSV et journaux de `mesures/moteur/O/` et `mesures/moteur/epreuve-J1/`.

8.1 Ce qui se mesure, et comment

Le modèle et la tâche. gpt-oss-120b d'OpenAI dans son format d'origine MXFP4 : un fichier GGUF de 63 387 346 208 octets, 36 couches, 128 experts par couche dont 4 servent chaque jeton, un expert de 13,25 Mo ; 4 608 experts en tout, 61,07 Go, et 1,69 Go de poids lus à chaque jeton (attention, routeurs, tête de sortie) [mes.] (`mesures/gpt-oss.md`, § 1). Son jumeau de développement, gpt-oss-20b (12 109 566 624 octets, 24 couches, 32 experts), est déjà plus gros que la carte. La tâche de vitesse est fixe depuis le Tome I : l'invite wikitext de 708 jetons (SHA-256 `eadbe87e`…), puis 256 jetons gloutons, en mode A0 (aucun cache d'experts en RAM : tout défaut est lu du disque).

Les séances. Six séances ont produit les chiffres de ce chapitre ; toutes sur secteur, relevé avant et après chaque passe (`Win32_Battery`, état 2), carte libre et verrou de la carte pris (tableau 8.1).

La règle de décision. Elle est celle du Tome I, déclarée avant toute mesure (étude 06, § 7.5) et reprise telle quelle par l'architecture (§ 7.2).

Définition 8.1 (Bruit, différence réelle)**.** Soit une configuration mesurée par deux passes X_1, X_2 d'un même palindrome. Son *bruit* est $\beta = |X_1 - X_2|/\bar{X}$; quand une configuration a plusieurs paires, son bruit est le plus grand bruit d'une paire. Une configuration B diffère *réellement* d'une configuration A si

$$\left|\frac{\bar{X}_B}{\bar{X}_A} - 1\right| \;>\; 2\,\max(\beta_A, \beta_B) + 3\,\%.$$

Table 8.1 : Les séances de mesure du jalon J1, de l'épreuve et de la ronde de corrections.

Séance	Date	Contenu et fichier
O4 `s1` (lot O)	15 oct., 22 h 40–23 h 03	A120 D Y20 Y20 D A120 : C5, C7, C8, C10 ; `mesures/moteur/O/csv/o4-2026-10-15-s1-passes.csv`
O3 120b (lot O)	15–16 oct., 23 h 03–00 h 53	C3, C11, compte du cache sur les 10 fenêtres
J1+ `s2`, `s3` (lot O)	16 oct., 00 h 54–02 h 08	préremplissage par flux, préchargement, lectures de 12 Mio ; `csv/o-jetons-s.csv`
épreuve `s1`	16 oct., à partir de 02 h 27	12 passes en palindrome : troncons, base à 6 et 10 fils, 20b, cache tiède ; `mesures/moteur/epreuve-J1/csv/s1-2026-10-16-passes.csv`
ronde, vitesse	16 oct., 06 h 41–07 h 19	garde, sans garde, KV f32, en ABBA ; `mesures/moteur/O/corrections/csv/vitesse-2026-10-16-passes.csv`
ronde, base	16 oct., 08 h 13–10 h 06	llama.cpp balayé : 5 configurations en palindrome ; `corrections/csv/base-2026-10-16-passes.csv`

Le bruit d'un programme contre lui-même sert d'étalon : une différence plus petite que deux fois l'écart d'un programme à lui-même n'est pas une différence. La marge de 3 % protège contre une série trop courte pour voir son propre bruit. L'épreuve l'écrivait « deux fois le bruit et 3 % » ; les verdicts de la ronde l'appliquent sous la forme additive ci-dessus, plus sévère. Aucun verdict de ce chapitre ne change d'une lecture à l'autre [calc.] (`memoire/outils/tome2_resultats.py`).

Un second chemin pour chaque chiffre. Les nombres de ce chapitre ne sont pas recopiés des rapports : un outil commité (`memoire/outils/tome2_resultats.py`) relit les passes elles-mêmes, recalcule moyennes, bruits, rapports, intervalles et planchers, les compare aux verdicts publiés par les lots et s'arrête en erreur au premier écart. Il en a trouvé un, qui n'était pas une faute des lots mais une ambiguïté de lecture (le coût de la garde, section 8.5.5).

8.2 Les critères, d'un coup d'œil

Tableau 8.2 suit chaque critère de J1 à travers les trois juges successifs : le lot qui l'a mesuré, l'épreuve indépendante qui l'a refait, et la ronde de corrections qui a répondu à l'épreuve.

> **En bref**
>
> ~~Dix critères tiennent sur douze~~ : c'étaient dix lignes sur douze, C7 y comptant deux fois et C9 manquant. Des douze critères déclarés, onze sont jugés et neuf tiennent entiers ; C9, le rendement physique, est publié sans être jugé (0,84 pour J1, section 8.5.1). C7 tient sur son plafond, avec une marge de cinq, mais son terme de tolérance (0,5 Mo) était mal posé : la mémoire du moteur gagne 2,9 Mo à 3,7 Mo quand le modèle grossit de 51 Go, et la borne qui nomme chaque terme n'en explique que 2,05 Mo. C8 mesurait la machine plutôt que le moteur ; l'instrument du processus montre que le moteur compte ses octets à 20 Mo près. Les deux critères attendent une décision de l'arbitre. Aucune des deux chutes ne touche la thèse : la mémoire ne suit pas la taille du modèle.

Table 8.2 : Le jalon J1, critère par critère (gpt-oss-120b sauf mention). Les seuils sont ceux de l'architecture, § 7.1, déclarés le 15 octobre avant toute construction.

Critère	Seuil	Lot (15 oct.)	Épreuve et ronde (16 oct.)	Verdict
C1 format	0 faux sur 4 608	0 faux (lot F, deux chemins)	40 tronçons tirés + 13 nommés, 0 faux, par un 3e décodeur	**tient**
C2 transport	0 écart ; 100 % des corruptions vues	0 sur 4 608 (lot E)	3 394 décodages égaux à l'empreinte ; en production, flux bruts non gardés (M1), puis garde construite	**tient** (garde)
C3 exact	0 bit sur 5 120 pas	0 (20b et 120b)	0, autre construction, flux = boucle octet pour octet	**tient**
C4 juste	KL et premier choix ≥ llama.cpp	20b : 0,003 68 contre 0,022 6	120b : 0,003 08 contre 0,060 7 ; KV f32 : au plancher du format	**tient**
C5 vitesse	≥ 0,8× plancher en série	2,186 et 2,182 ≥ 1,505	1,70 à 2,18 ≥ 1,43 à 1,52, garde comprise	**tient**
C6 (J1+)	$> 2\beta + 3$ % sur la meilleure base	~~×3,85~~	~~×2,79~~ ; ×2,22 contre `fit -t 16`	**tient**
C7 plafond	≤ 1,9 Go	384,2 Mo	384,4 Mo ; 386,8 Mo garde comprise	**tient**
C7 écart	$\leq 2\beta + 0{,}5$ Mo	3,70 Mo	2,94 Mo ; borne à termes nommés 2,05 Mo	**tombe**
C8 octets	PDH ≤ repos × durée + 100 Mo	tombe sur 1 passe sur 4	PDH tombe ; compteurs du processus : 19,92 Mo constants	instrument à changer
C9 rendement (J1+)	publié, pas jugé	non calculé	0,84 (J1) ; 0,64 à 0,82 (ronde), calculé par la reprise du jugement	publié
C10 VRAM	≤ 7 279 Mio	6 520 à 6 537	6 574 ; 6 811 (ronde)	**tient**
C11 sans perte	PPL égales	422,205 585 227 913 05	la même, codé et brut	**tient**
C12 texte	256 jetons égaux	`e4c6dbac`… sur 17 passes	le même ; mais 1er jeton ≠ vérité f64 (KV f16)	**tient**

Données : `moteur/O-pour-le-memoire.md` § 8 et § 14, `mesures/epreuve-J1.md` § 3.1, `mesures/moteur/O/corrections/journaux/verdicts-*-2026-10-16.json`.

8.3 Exactitude : codé égale brut, au bit, sur 5 120 pas

Ce que l'on compare. Le lot F écrit deux fichiers du même modèle : le `.trc` *codé*, dont chaque expert est compressé sans perte par le rANS à table par voie, et le `.trc` *brut*, aux mêmes octets d'experts recopiés tels quels. Le moteur lit l'un en décodant sur la carte, l'autre en copiant. Si le codage est vraiment sans perte et si le moteur est déterministe, les deux doivent rendre les mêmes logits, au bit près ; la moindre différence serait une faute.

Proposition 8.2 (C3 par construction ; le cache n'entre pas dans le calcul)**.** *Si les fichiers codé et brut donnent, pour chaque expert, un emplacement de VRAM égal octet pour octet, le moteur rend les mêmes logits au bit sur toute suite de jetons, quels que soient le nombre d'emplacements, la politique du cache et l'ordre d'arrivée des lectures.*

Démonstration. Les produits d'un expert lisent son emplacement et l'activation h_2. Au moment de la lecture, l'emplacement porte les octets décodés de cet expert : l'ordre des trois flux de la carte l'assure (lemme 4 du lot O, `moteur/O-pour-le-memoire.md` § 6.1), et ces octets sont égaux dans les deux fichiers par hypothèse (C1 pour le codé, une copie pour le brut, C2 pour le transport). Le cache choisit seulement *où* se trouve un expert, jamais *quels octets* il porte ; la somme pondérée des k experts se fait dans l'ordre $i = 0, \dots, k-1$ du routeur, pas dans celui des emplacements. Les autres entrées (poids résidents, cache KV) ne dépendent pas du fichier d'experts. Chaque noyau étant une fonction déterministe des octets qu'il lit (aucune addition atomique flottante, réductions à arbre fixe), les logits sont égaux. ■

Comment comparer 5 120 distributions sans les garder. Une distribution de sortie compte 201 088 logits f32 : les garder pour les 10 fenêtres de 512 positions demanderait $5\,120 \times 201\,088 \times 4 =$ 4,12 Go [calc.]. Le moteur garde à la place l'empreinte de 64 bits (préfixe de SHA-256) des logits de chaque pas, soit 40 960 octets ; deux pas égaux au bit ont la même empreinte, deux pas différents en ont une autre sauf collision, de probabilité 2^{-64} par paire. Un second chemin, indépendant de l'empreinte, compare les fichiers `.kld` écrits par les deux passes octet pour octet.

Table 8.3 : C3 et C11 : le fichier codé contre le fichier brut, sur les 10 fenêtres de 512 jetons (5 120 pas) [mes.].

	Pas différents	`.kld`	PPL des positions notées	Emplacements
20b, lot O	0 / 5 120	égaux octet pour octet	461,978 277 060 145 (les deux)	362 et 361
120b, lot O (boucle de pas)	0 / 5 120	SHA-256 `5b212fe9`… (les deux)	422,205 585 227 913 05 (les deux)	326 et 325
120b, épreuve (flux, autre construction)	0 / 5 120	`5b212fe9`…, = lot O	422,205 585 227 913 05	—

Données : `mesures/moteur/O/journaux/o3/`, `mesures/moteur/epreuve-J1/journaux/c3-120b-10fenetres.json`.

Trois faits rendent cette égalité plus forte qu'un simple « 0 écart ». (i) Les passes codée et brute n'avaient pas le même nombre d'emplacements (362 et 361 sur le 20b, 326 et 325 sur le 120b, parce que les tampons d'arrivée du brut sont plus grands) : elles n'ont donc pas servi les mêmes défauts dans le même ordre, et la partie « quel que soit le cache » de la proposition 8.2 est observée, pas seulement déduite. (ii) L'épreuve a refait les deux passes depuis un clone propre, avec sa propre construction, par le préremplissage par flux (une couche à la fois pour toutes les positions) au lieu de la boucle

de pas (un jeton à la fois pour toutes les couches) : ses `.kld` égalent ceux du lot O octet pour octet. Deux constructions, deux ordres de calcul, deux jours, un seul résultat au bit. (iii) Le texte glouton de 256 jetons (C12) a la même empreinte, `e4c6dbacb63a`…, sur les 17 passes du lot O (codé, brut, flux, lectures de 12 Mio, préchargement), sur les quatre passes de l'épreuve et sur les six passes à KV f16 de la ronde, garde comprise ou non.

8.4 Mémoire : 384 Mo pour 63 Go

Le chiffre et sa pente. En mode A0, l'ensemble de travail de pointe du processus (mémoire épinglée comprise, lu par le système sur la poignée après la sortie) vaut 384,19 Mo pour gpt-oss-120b et 380,50 Mo pour gpt-oss-20b [mes.] (`o4-2026-10-15-s1-passes.csv`). llama.cpp, dans sa meilleure configuration publiée, tient 10,93 Go et 9,42 Go [mes.] (`mesures/base-gpt-oss/*/resume.csv`). Sur le 120b, la base de C6, `fit -t 16`, balayée par la ronde, tient davantage : 11 355 et 11 483 Mo, 11,42 Go en moyenne [mes.] (`base-2026-10-16-passes.csv`, colonne `pointe_et_mo`). Les pentes ci-dessous gardent la base publiée, mesurée sur les deux modèles ; elle est la plus favorable à llama.cpp. La figure 8.1 trace les deux droites.

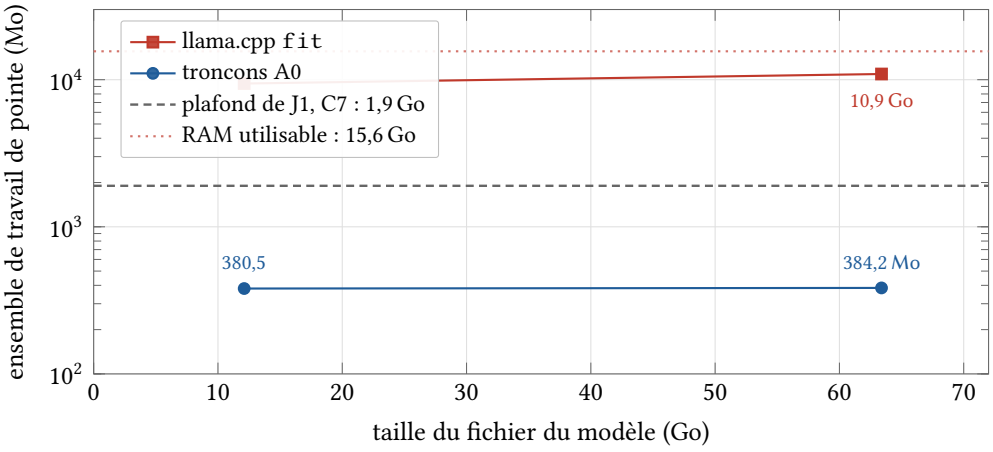


Figure 8.1 : Mémoire de pointe contre taille du fichier du modèle (échelle logarithmique). troncons gagne 3,7 Mo quand le modèle gagne 51,3 Go ; llama.cpp en gagne 1,5 Go et approche la RAM de la machine. Le plafond déclaré de J1 (C7) est en tirets.

Données : `memoire/figures/tome2/donnees/res-memoire.csv`, tiré de `o4-2026-10-15-s1-passes.csv` et `mesures/base-gpt-oss/\{20b,120b\}/resume.csv`.

Posons les pentes [calc.] :

$$\frac{\Delta M_{\text{troncons}}}{\Delta P} = \frac{384\,192\,512 - 380\,502\,016}{63\,387\,346\,208 - 12\,109\,566\,624} = \frac{3\,690\,496}{51\,277\,779\,584} \approx 7{,}2 \times 10^{-5},$$
$$\frac{\Delta M_{\text{llama}}}{\Delta P} = \frac{(10{,}932\,8 - 9{,}418\,5) \times 10^{9}}{51\,277\,779\,584} \approx 0{,}029\,5.$$

Un octet de plus dans le modèle coûte à troncons $7{,}2 \times 10^{-5}$ octet de mémoire, à llama.cpp 0,03, quatre cents fois plus. Le modèle du 120b est 165 fois plus gros que la fenêtre de troncons, et llama.cpp tient 28,5 fois plus de mémoire pour le servir (29,7 fois dans la configuration de C6). C'est, mesurée, la lecture de « sans RAM ni rien » que le Tome I avait retenue : *la mémoire occupée ne dépend plus de la taille du modèle.*

De quoi est faite la fenêtre. La proposition A.1 du partisan de A écrivait la mémoire de l'hôte du moteur

$$M = M_{\text{fixe}} + d \cdot k \cdot \ell_{\max} + M_{\text{tiède}} + M_{\text{répertoire}},$$

où $d \cdot k = 12$ tampons d'arrivée épinglés de la taille $\ell_{\max}$ du plus grand tronçon, $M_{\text{tiède}} = 0$ en mode A0, et seul le répertoire croît avec le modèle. Le diagnostic du lot O, qui relève l'ensemble de travail après chaque étape de l'ouverture, a montré que la proposition oubliait des termes (tableau 8.4).

Table 8.4 : Les termes de la mémoire du moteur qui croissent avec le modèle : comptes du 120b moins ceux du 20b, et coûts unitaires mesurés [calc.] sur [mes.].

Terme	Δ compte × coût unitaire	Δ octets
fenêtre : $d \cdot k$ tampons à la taille du plus grand tronçon	12 × (12 251 136 − 12 144 640)	1 277 952
événements horodatés (4 par défaut possible, 2 par couche)	216 × 657 o à 660 o	≈ 142 000
allocations de carte (résidents, caches KV)	180 × ≤ 290 o	≤ 52 000
répertoire tenu deux fois (fichier, tailles du cache)	2 × 3 840 × 56 o	430 080
table des résidents, tables rANS	156 × 88 ; 144 × ≈ 560	≈ 94 000
trace de routage (256 pas)	96 o par pas	≤ 49 000
somme des termes nommés		≈ 2,05 Mo
mesuré (épreuve ; lot O)		2,94 Mo ; 3,30 Mo à 3,70 Mo

Données : `moteur/O-pour-le-memoire.md` § 13.6, `mesures/moteur/O/corrections/journaux/m4-couts-unitaires.jsonl`.

Le premier terme disparaît si la fenêtre a une taille fixe : avec des lectures de 12 582 912 octets pour les deux modèles, l'écart 120b − 20b tombe de 3,30 Mo à 0,99 Mo [mes.]. Les autres sont proportionnels au nombre d'experts $L \cdot E$ et restent sous 1 Mo jusqu'à environ 10 000 experts [calc.]. Il reste 0,9 Mo que ces termes n'expliquent pas (granularité des pages des petites allocations, tas) : la borne à termes nommés n'est pas fermée, et même avec elle, $2{,}05 + 2 \times 0{,}14 + 0{,}5 = 2{,}83$ Mo laisse la mesure de l'épreuve tomber de 0,11 Mo. Le verdict de C7-écart reste donc « tombe » ; ce qui était faux, c'est le terme de tolérance que l'architecture avait posé, pas la propriété qu'il voulait juger.

Ce que la machine perd, et la mémoire engagée. Deux mesures neuves de l'épreuve complètent C7 (figure 9.2 au chapitre suivant). La mémoire physique disponible de *toute* la machine baisse de 0,46 Go à 0,73 Go pendant une passe de troncons, contre 6,3 Go à 11,6 Go pendant une passe de llama.cpp [mes.]. En revanche, la mémoire privée *engagée* de troncons vaut 6,93 Go : c'est l'ordre de ses allocations de VRAM et de sa mémoire épinglée, réservées contre la limite d'engagement de Windows (RAM plus fichier d'échange) sans occuper de RAM physique. Elle n'est pas jugée ; elle est publiée (constat m3).

La fenêtre est visible. Donner 1 Go de cache tiède au moteur fait monter sa pointe de 994,2 Mo [mes.] : l'instrument voit bien la fenêtre, et C7 ne mesure pas un plancher qu'il ne saurait pas quitter.

VRAM (C10). La pointe relevée par `nvidia-smi` toutes les 250 ms vaut 6 520 à 6 537 Mio au lot O, 6 574 à l'épreuve, 6 811 dans la séance de la ronde (même occupation par le moteur, 6 317 303 600

octets ; la différence est ailleurs sur la carte), toujours sous le seuil de 7 279 Mio (8 188 − 909, la VRAM moins ce que l'affichage et les applications de Stone occupent au repos).

8.5 Vitesse

8.5.1 Le plancher en série (C5)

La formule. Un jeton servi *sans aucun recouvrement* paierait l'un après l'autre chaque étage de son chemin. Son temps est alors la somme

$$t_{\text{série}} = t_{\text{disque}} + t_{\text{PCIe}} + t_{\text{déc}} + t_{\text{GEMV}} + T_0,$$

avec t_{disque} les octets de tronçons lus par jeton (compteur du moteur) divisés par le débit d'une passe disque seule de la même séance, t_{PCIe} et $t_{\text{déc}}$ les temps de carte des copies et des décodages par jeton (événements CUDA du moteur), $t_{\text{GEMV}} = (A + R)/235\,\text{Go/s}$ avec A les poids résidents lus et $R = k \cdot L$ emplacements d'experts, et $T_0 = 3{,}5\,\text{ms}$ le coût fixe du Tome I. C5 exige au moins $0{,}8/t_{\text{série}}$ jetons par seconde ; $0{,}8 = 1 - 2 \times 10\,\%$, le bruit d'une lecture de 4 Mio en profondeur 4 mesuré au Tome I.

Le calcul, pas à pas (séance O4, passe 1, [mes.] pour chaque terme) :

$$t_{\text{disque}} = \frac{0{,}707\,3\,\text{Go}}{1{,}848\,\text{Go/s}} = 382{,}7\,\text{ms}, \qquad t_{\text{PCIe}} = 59\ \text{défauts} \times 1{,}28\,\text{ms} = 75{,}8\,\text{ms},$$

$$t_{\text{déc}} = 59 \times 0{,}92\,\text{ms} = 54{,}2\,\text{ms}, \qquad t_{\text{GEMV}} = \frac{1{,}686\,\text{Go} + 144 \times 13{,}25\,\text{Mo}}{235\,\text{Go/s}} = 15{,}3\,\text{ms},$$

d'où $t_{\text{série}} = 382{,}7 + 75{,}8 + 54{,}2 + 15{,}3 + 3{,}5 = 531{,}5\,\text{ms}$, un plancher de 1,881 jeton/s et un seuil de 1,505. Le moteur mesure 457 ms par jeton, 2,186 **jetons/s** : il passe *sous* la somme, parce que les copies et les décodages d'une couche recouvrent les lectures des défauts suivants de la même couche. La figure 8.2 refait ce calcul pour chaque passe de la ronde.

Le rendement physique (C9). L'architecture (§ 7.2) déclarait un critère publié et non jugé : le débit mesuré divisé par le plafond calculé au h mesuré. Ni le lot O ni l'épreuve ne l'avaient calculé ; la reprise du jugement le fait ici, sur les termes du plancher [calc.] sur [mes.]. Le plafond est l'équation du jeton du Tome I sans spéculation ni latence restante ($\mathscr{E} = 1$, $t_{\text{lat}} = 0$), tous les étages recouverts :

$$t_{\text{plafond}} = \max\left(t_{\text{disque}}, t_{\text{PCIe}}, t_{\text{déc}} + t_{\text{GEMV}}\right) + T_0.$$

Passe 1 de J1 : $\max(382{,}7;\ 75{,}8;\ 54{,}2 + 15{,}3) + 3{,}5 = 386{,}2\,\text{ms}$, un plafond de 2,589 jetons/s ; mesuré 457,4 ms, d'où un rendement de $386{,}2/457{,}4 = \mathbf{0{,}84}$ (passe 6 : 0,84). Sur les six passes à KV f16 de la ronde, de 0,64 (passe 02, la plus lente) à 0,82 (passe 03). Le moteur tient donc 84 % du débit qu'il aurait si le lien et la carte se cachaient entièrement derrière le disque ; les 16 % qui manquent sont le terme t_{lat} de l'équation, la latence que la file courte du disque laisse voir, celle que le préchargement visait (section 8.5.6). Refait par `memoire/outils/tome2_reprise.py` depuis `res-plancher.csv`.

Ce qui limite J1, et ce que le Tome I avait prédit. Le disque lit en moyenne 1,6 tronçon par couche (59 défauts sur 36 couches) : la file a rarement ses quatre requêtes en vol, et le moteur ne tire du disque que 1,55 Go/s utiles quand une passe disque seule à profondeur 4 en tire 1,85 [mes.]. Le chronogramme d'une couche le montre : quatre lectures soumises à 1,28, 1,96, 2,68 et 3,18 ms finissent à 6,86, 9,81, 16,14 et 21,17 ms, servies l'une après l'autre (`mesures/moteur/O/csv/chronogramme-120b.csv`). Le

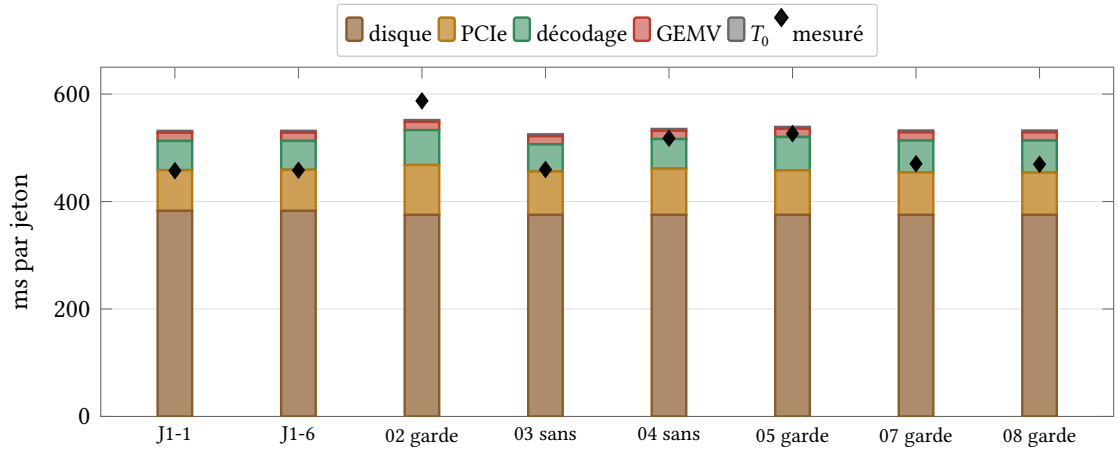


Figure 8.2 : Le plancher en série, étage par étage (barres empilées), contre le temps mesuré par jeton (losanges), gpt-oss-120b en mode A0 [calc.] sur [mes.]. Le disque prend 68 % à 72 % de la somme. Le losange passe sous la barre dès que la file du disque recouvre la carte ; la passe 02 de la ronde, au-dessus, est la plus lente de la séance (1,70 jeton/s), garde comprise.

Données : `memoire/figures/tome2/donnees/res-plancher.csv`, tiré de `o4-2026-10-15-s1-passes.csv` et `vitesse-2026-10-16-passes.csv` ; chaque plancher recalculé égale celui du journal à 10^{-6} près.

Tome I avait calé un scénario de localité « LRU » sur un seul point publié ; il y prédisait, pour un cache de 4,8 Go en VRAM, un succès $h_v = 0{,}590$ et, en mode A0, un repère de vitesse de 2,15 jetons/s (les 326 emplacements de J1 font 4,3 Go). Le moteur compte $h = 0{,}591$ sur ses 326 emplacements et mesure 2,18 [mes.]. La concordance est à prendre pour ce qu'elle est : un scénario calé sur un point, que la mesure de localité du 20b avait déjà contredit dans sa forme, tombe juste au point de fonctionnement de J1.

8.5.2 La base, et l'histoire de ses chiffres

C6 compare troncons à la *meilleure* configuration mesurée de llama.cpp sur le même modèle, la même invite et les mêmes 256 jetons. Le chiffre de la base a changé deux fois ; les deux premiers rapports sont barrés là où ils étaient écrits, et rassemblés ici.

1. **15 octobre** (`mesures/gpt-oss.md`, § 3.2) : trois configurations, toutes à 6 fils ; la meilleure, `fit`, fait 0,568 jeton/s. Contre le J1 du lot O : ~~×3,85~~.
2. **16 octobre, épreuve** : le juge ajoute une configuration de son choix, `fit -t 10`, par un argument déclaré d'avance (sous `mmap`, chaque fil en faute de page est une lecture en vol) : 0,702 contre 0,627 à 6 fils dans la même séance, écart réel. Contre troncons : ~~×2,79 à 2,89~~.
3. **16 octobre, ronde** : la base est balayée (tableau 8.5). La meilleure est `fit -t 16`, 0,884 **jeton/s**, 54 % au-dessus de la base publiée.

Pourquoi le nombre de fils change tout. llama.cpp projette le fichier en mémoire (`mmap`) ; un expert absent du cache de pages est lu par fautes de page, et un fil en faute attend sa page avant de continuer. Avec n fils de calcul, il y a donc au plus n lectures en vol. La loi de Little [3] relie le débit X d'un système à son nombre moyen de requêtes en vol N et à leur temps de séjour R : $X = N/R$. Tant que le disque n'est pas saturé, R varie peu et le débit croît avec N ; le Tome I a mesuré cette croissance du débit du disque avec la profondeur de file. De 6 à 16 fils, la base gagne 54 % ; de 12 à 16, rien de

Table 8.5 : La base llama.cpp b11515 balayée par la ronde (gpt-oss-120b, 256 jetons gloutons ; palindrome t6 t10 t12 t16 m m t16 t12 t10 t6 après une passe d'échauffement jetée) [mes.]. « m » : `-ngl 99 --n-cpu-moe 34 --fit off -t 10`.

Configuration	Passes (j/s)	Moyenne	Bruit β	Écart de `t16`	Réel ? (seuil)
`fit -t 6` (publiée)	0,575 ; 0,573	0,574	0,4 %	+54,0 %	oui (10,4 %)
`fit -t 10` (épreuve)	0,675 ; 0,654	0,665	3,2 %	+33,0 %	oui (10,4 %)
m (`--n-cpu-moe 34`)	0,642 ; 0,638	0,640	0,6 %	+38,2 %	oui (10,4 %)
`fit -t 12`	0,913 ; 0,760	0,836	18,3 %	+5,7 %	non (39,6 %)
`fit -t 16`	0,900 ; 0,867	0,884	3,7 %	—	—

Données : `mesures/moteur/O/corrections/csv/base-2026-10-16-passes.csv`, `journaux/verdicts-base-2026-10-16.json` ; recalculé par `tome2_resultats.py`.

discernable : le plateau commence. Cette lecture est une hypothèse cohérente avec la mesure, pas une mesure de la profondeur de file elle-même.

8.5.3 C6 refait : la garde comprise

La ronde a ajouté au moteur une garde d'intégrité (section 9.9.3) active par défaut. Le chiffre honnête de C6 est donc celui du moteur gardé contre la meilleure base [calc.] sur [mes.] :

$$\frac{\bar{X}_{\text{garde}}}{\bar{X}_{\texttt{t16}}} = \frac{(1{,}703 + 1{,}900 + 2{,}126 + 2{,}131)/4}{(0{,}900 + 0{,}867)/2} = \frac{1{,}965}{0{,}884} = \mathbf{2{,}22}.$$

Le bruit de troncons, par la définition 8.1, est le plus grand bruit d'une paire : 10,97 % (passes 02 et 05) ; le seuil vaut $2 \times 10{,}97 + 3 = 24{,}9$ % et l'écart, +122 %, le dépasse de loin. Une lecture plus sévère du bruit, l'étendue des quatre passes gardées (21,8 %), porte le seuil à 46,6 % : C6 tient encore. Sans la garde, le rapport serait $2{,}054/0{,}884 = 2{,}32$. La figure 8.3 rassemble toutes les séries.

Remarque 8.3 (Deux séances, pas une). Les passes de troncons et celles de la base balayée viennent de deux séances distinctes de la même matinée (la vitesse entre 06 h 41 et 07 h 19, la base de 08 h 13 à 10 h 06), et le disque de cette machine varie d'une séance à l'autre (de 1,51 Go/s à 1,95 Go/s dans la seule séance vitesse ; 20 % plus lent en séance 2 qu'en séance 1 au lot O, cause non mesurée). Le rapport ×2,22 porte donc un bruit entre séances que la règle ne voit pas. Deux garde-fous le bornent : l'épreuve, elle, a mesuré troncons et llama.cpp dans la même séance, et trouvait ×2,89 contre `fit -t 10` (2,03 contre 0,702) ; et l'écart (122 %) dépasse de beaucoup toute variation de disque observée. Une série unique, troncons et `fit -t 16` en palindrome, reste à faire ; elle est dans les travaux futurs.

8.5.4 Le préremplissage par flux

Le coût, posé. Dans la boucle de pas, une invite de P positions lit, couche par couche, les experts manquants de chaque position :

$$B_{\text{boucle}} = \sum_{p=1}^{P} \sum_{\ell=1}^{L} (1 - h_{p,\ell})\, k\, t,$$

avec $t \approx 12{,}03$ Mo par tronçon. Par flux, chaque couche lit une fois chaque expert demandé par au moins une position : $B_{\text{flux}} = \sum_{\ell} |U_\ell \setminus V_\ell|\, t$, où U_ℓ est l'ensemble des experts choisis et V_ℓ ceux déjà en VRAM. Sous un routage uniforme, $\mathbb{E}|U_\ell| = E\,(1 - (1 - k/E)^P)$; pour $E = 128$, $k = 4$, $P = 708$: $(1 - 1/32)^{708} = e^{708 \ln 0{,}96875} = e^{-22{,}5}$, et U_ℓ est tout l'ensemble. D'où $B_{\text{flux}} \le 36 \times 128 \times 12{,}03$ Mo $= 55{,}4$ Go, contre $B_{\text{boucle}} \approx 708 \times 144 \times 0{,}55 \times 12{,}03$ Mo ≈ 675 Go [calc.], un rapport d'environ 12.

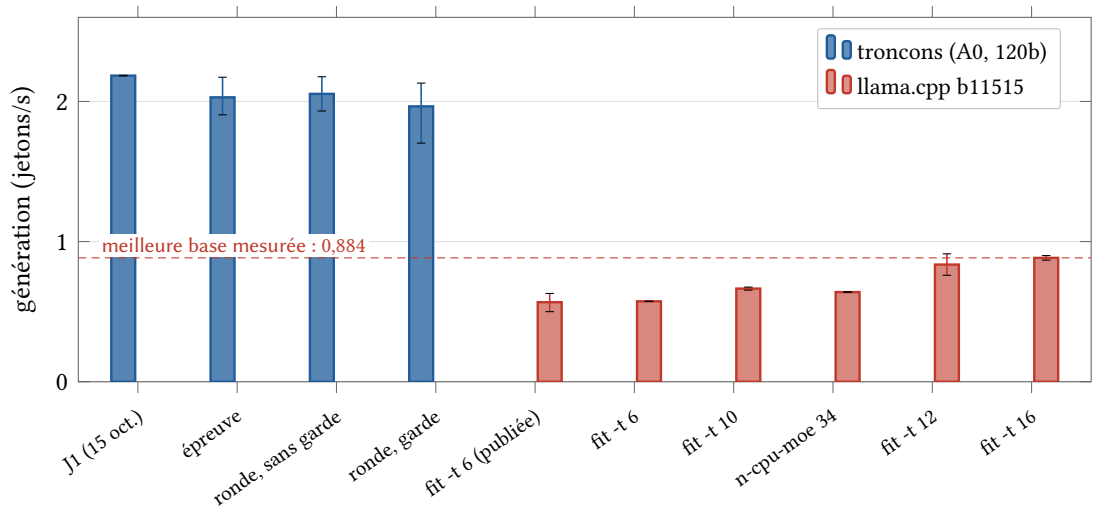


Figure 8.3 : Génération sur gpt-oss-120b (moyenne, et étendue des passes en barre d'erreur) : troncons en mode A0 dans quatre séances, llama.cpp dans sa base publiée et dans les cinq configurations du balayage de la ronde [mes.]. Le tiret est la meilleure base mesurée (`fit -t 16`). La base publiée était la plus lente des configurations balayées.

Données : `memoire/figures/tome2/donnees/res-vitesse-120b.csv`, tiré de `o4-2026-10-15-s1-passes.csv`, `s1-2026-10-16-passes.csv`, `vitesse-2026-10-16-passes.csv`, `base-2026-10-16-passes.csv` et `mesures/base-gpt-oss/120b/resume.csv`.

Mesuré. 26 s au lieu de 7 min 36 s pour les 708 jetons, soit ×17,5, rapport des débits moyens des passes `s2-11` à `s2-14` (27,15 contre 1,554 jetons/s ; par passe, 26,9 à 27,4 contre 1,46 à 1,65) [mes.]. La note du lot O écrivait ~~7 min 50 s~~, une durée qui ne vient pas de ces passes : en plus des octets, la boucle paie une synchronisation par couche et par position, que le flux fait une fois par couche. llama.cpp préremplit à 3,1 à 4,0 jetons/s : troncons le fait sept fois plus vite. L'exactitude tient par la même proposition 8.2 et par le témoin F (5 120 empreintes égales à la boucle sur le 20b ; `.kld` égaux sur le 120b à l'épreuve).

8.5.5 Le coût de la garde d'intégrité

Prédit. La garde hache chaque tronçon lu (0,71 Go par jeton) sur quatre fils, à 1,07 Go/s à 1,10 Go/s par fil [mes.], en parallèle de la copie et du décodage ; le pas attend tous ses hachages avant de rendre son jeton. Le travail, $0{,}71/1{,}07 \approx 0{,}65$ s de cœur par pas, se répartit sur quatre fils (0,17 s de mur) sous un pas de 0,46 s ; au chemin critique ne reste que le hachage du dernier tronçon, au plus

$$12{,}25\ \text{Mo}/1{,}07\ \text{Go/s} \approx 11{,}4\ \text{ms},$$

soit 2,5 % d'un pas de 457 ms [calc.]. La prédiction déclarée de la ronde écrivait 2,4 %, avec 1,1 Go/s et 460 ms (`mesures/moteur/O/corrections/seuils-declares.md`) ; au débit le plus lent mesuré, 1,07 Go/s, c'est 2,5 %.

Mesuré. L'attente de la garde vaut 5,2 ms à 5,4 ms par jeton sur les quatre passes gardées [mes.]. La série ABBA déclarée (passes 02 et 05 gardées, 03 et 04 sans garde) donne un écart de −12,3 % pour un seuil de 26,8 % : *non réel* ; le disque variait de 1,51 Go/s à 1,95 Go/s d'une passe à l'autre, et c'est lui que l'écart mesure. À débit de disque égal (passes 05 et 04, 1,708 et 1,715 Go/s), l'écart vaut

−1,6 % (publié −1,7 % sur les valeurs arrondies 1,900 et 1,932). L'outil de second chemin avait d'abord trouvé −4,3 % en comptant les quatre passes gardées : les passes 07 et 08 sont gardées aussi, mais appartiennent à la série du KV en f32 ; ce n'est pas une faute du lot, c'est le chiffre d'une autre série. L'ouverture coûte 0,95 s de plus (la table des empreintes des lignes d'enchâssement : 0,615 Go lus et hachés), et la mémoire 1,9 Mo, la même pour les deux modèles.

8.5.6 Ce que J1+ a gardé, tué, laissé ouvert

Gardé : le flux (×17,5 au préremplissage, section 8.5.4).
Tué : le préchargement par le routeur de la couche suivante. Après l'attention de ℓ, le moteur applique au résidu la norme et le routeur de $\ell + 1$ et lit d'avance les tronçons prédits. Une prédiction juste retire une lecture du chemin critique ; une fausse ajoute une lecture sur un disque qui limite déjà. Avec p la part des prédictions servies et f la part du temps d'un jeton que le recouvrement peut cacher, le gain vaut environ $f - \frac{1-p}{p}\, s_{\text{disque}}$; ici $p = 0{,}692$, les octets lus montent de 34 % (953 contre 710 Mo par jeton) et $f \approx 80/460 = 0{,}17$: la perte domine. Mesuré : −11 % (bruit 1 %) [mes.]. Le mécanisme reviendrait avec $p > 0{,}9$ environ, ou un disque qui ne limite pas.
Ouvert : les lectures de 12 Mio. Lire chaque tronçon en une requête de 12 582 912 octets fait passer la lecture dans le régime rapide du disque mesuré par le lot E : +30 % en médiane (2,18 → 2,84 jetons/s), chaque passe B au-dessus de chaque passe A, mais un bruit de 16,2 % entre les deux passes B (définition 8.1 ; ~~21 %~~, l'écart rapporté à la moyenne de A) : le seuil vaut 2 × 16,2 + 3 = 35,4 % contre +30,4 %, la règle n'est pas franchie en quatre passes. Ni établi, ni réfuté.
Une passe, à confirmer : le cache tiède. 1 Go de cache tiède divise par 2,2 le préremplissage par flux (27,1 → 12,5 jetons/s) sans rien rendre à la génération (1,90) [mes.]. Personne ne l'avait mesuré ; une passe ne fait pas une série.

8.6 Fidélité : à quelle distance des mathématiques exactes du modèle

8.6.1 L'instrument

La divergence. Pour une position, soit p la distribution du prochain jeton selon la vérité (la passe avant du modèle en double précision, sur les mêmes poids) et q celle d'un moteur. On mesure

$$\mathrm{KL}(p \,\|\, q) = \sum_{i=1}^{V} p_i \,(\log p_i - \log q_i) \;\geq 0,$$

la divergence de Kullback et Leibler [2], en nats, sur un vocabulaire de $V = 201\,088$ jetons, et la part des positions où le premier choix du moteur est celui de la vérité, avec son intervalle de Wilson à 95 % [37]. Pour une petite perturbation Δz des logits, $\mathrm{KL} \approx \frac{1}{2}\mathrm{Var}_p(\Delta z)$: la divergence est du second ordre dans l'erreur.

Pourquoi contre la vérité, et pas contre llama.cpp. Deux moteurs justes divergent : sur Qwen3.5-4B, de 0,005 5 (Tome I) ; un moteur ne peut pas être juge d'un autre. L'architecture a donc posé C4 contre les mathématiques exactes : troncons doit en être au moins aussi près que llama.cpp, mesurés tous deux par le même instrument.

Le format du fichier, et ce qu'il coûte. Tous les moteurs écrivent leurs logits au format `.kld` de llama.cpp : par position, $\log q$ quantifié sur 16 bits entre un plancher et le maximum, avec un pas d'environ $2{,}44 \times 10^{-4}$ nat. Deux conséquences, démontrées au section 9.4 : un calcul identique à la

vérité s'en écarte au plus d'un demi-pas par jeton au-dessus du plancher ; et les jetons sous le plancher reçoivent une masse en trop, si bien que la KL d'un calcul exact est *négative*, de l'ordre de $-4{,}8 \times 10^{-4}$. Ce biais est commun à tous les fichiers et fixe le zéro de l'instrument.

8.6.2 C4 sur les deux modèles

Table 8.6 : C4 : troncons et llama.cpp contre la vérité en double précision [mes.]. La fenêtre 0 compte 255 positions notées. $\bar{d}$: moyenne appariée de $\mathrm{KL}_{\text{troncons}} - \mathrm{KL}_{\text{llama}}$ par position, ± une erreur-type.

Modèle, vérité	Moteur	KL moyenne	Même premier choix [Wilson 95 %]	$\bar{d}$
20b, réf. lot R, 10 fenêtres	troncons	0,003 68	97,18 % [96,46 ; 97,75]	
	llama.cpp carte	0,022 6	89,61 % [88,36 ; 90,73]	
20b, numpy f64, fenêtre 0	troncons	0,001 74	246/255 = 96,5 % [93,4 ; 98,1]	−0,024 3 ± 0,005 3
	llama.cpp carte	0,026 1	222/255 = 87,1 % [82,4 ; 90,6]	
120b, numpy f64, fenêtre 0	**troncons**	0,003 08	244/255 = 95,7 % [92,4 ; 97,6]	−0,057 6 ± 0,012 0
	llama.cpp carte	0,060 7	205/255 = 80,4 % [75,1 ; 84,8]	
120b, numpy f64, fenêtre 0	troncons, **KV f32**	−0,000 48	255/255 = 100 %	

Données : `mesures/moteur/O/journaux/o3/c4-20b-comparer.*`, `mesures/moteur/epreuve-J1/csv/c4-numpy-\{20b,120b\}-f0.csv`, `mesures/moteur/O/corrections/journaux/verdicts-kv-2026-10-16.json` ; Wilson et $\bar{d}$ recalculés par `tome2_resultats.py`.

Le test apparié. Comparer deux moyennes de KL position par position retire la variance commune des positions difficiles : $d_i = \mathrm{KL}_{t,i} - \mathrm{KL}_{l,i}$, $\bar{d} \pm s_d/\sqrt{n}$. Sur le 120b, $\bar{d} = -0{,}057\,6$ pour une erreur-type de 0,012 0 : près de cinq erreurs-types sous zéro, et les intervalles de Wilson des premiers choix sont disjoints. C4 tient sur le 120b pour la première fois.

« Six fois plus près », requalifié. Le lot O avait écrit que troncons était « six fois plus près » de la vérité que llama.cpp (0,003 7 contre 0,022 6 sur le 20b). C'est vrai, mais l'épreuve et le lot R en ont trouvé la raison principale : llama.cpp arrondit les bornes de la rampe de YaRN [28], là où l'implémentation de référence d'OpenAI [32] ne les arrondit pas (`"truncate": false`). Sur la fenêtre 0 du 20b, la rampe seule explique 0,014 3 des 0,026 1 de llama.cpp (55 %) ; sur les 10 fenêtres, 0,014 5 des 0,022 6 (lot R). L'écart vient donc surtout d'un choix de modélisation, celui d'OpenAI, validé par un calcul indépendant ; la précision des noyaux n'en est pas la cause principale.

8.6.3 Le cache KV en f16 portait tout l'écart

Ce que l'épreuve a vu. Au-delà des moyennes, le premier jeton du texte de C12 (position 707 de l'invite) diffère : la vérité choisit le jeton 1008 (« … », $\log p = -1{,}46$), troncons le jeton 313 (« am », dont la vérité ne donne que $\log p = -10{,}03$), après une cascade de bascules de routage sur 20 couches sur 36, à partir de la couche 12. La vérité recalculée avec le seul cache KV arrondi en f16, et rien d'autre, choisit 313 elle aussi, avec la même cascade [mes.] (`premier-jeton-kvf16-120b.json`). La figure 8.4 montre l'accord de routage par couche : sur le 120b, il décroche avec la profondeur, jusqu'à 0,81 au-delà de la position 512 ; sur le 20b, il ne descend pas sous 0,96.

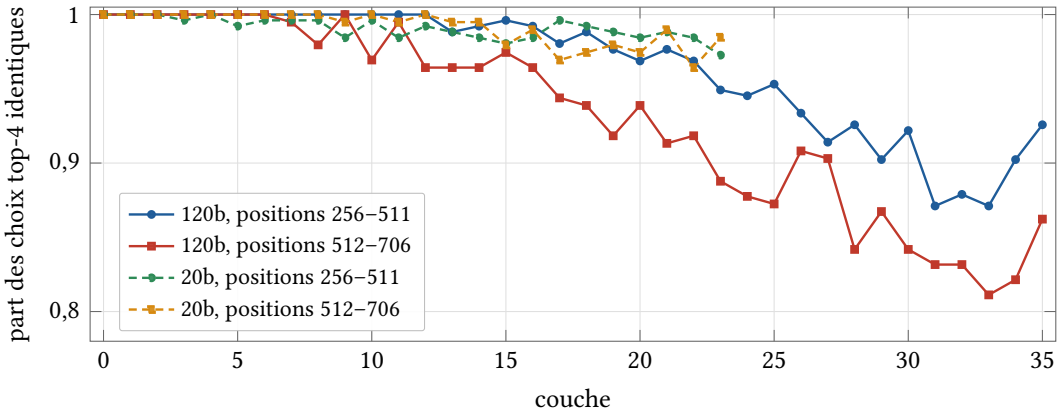


Figure 8.4 : Part des choix top-4 du routeur identiques entre troncons (KV f16) et la vérité f64, par couche, sur l'invite de C12 [mes.]. Le 120b décroche avec la profondeur, et davantage après la position 512 ; le 20b non. Avec un cache KV en f32, l'accord vaut 1 sur les 36 couches et les 708 positions.

Données : `memoire/figures/tome2/donnees/res-routage.csv`, tiré de `mesures/moteur/epreuve-J1/csv/routage-par-couche-invite.csv`.

Pourquoi l'arrondi f16 suffit, posé. Le f16 a 11 bits de mantisse : l'arrondi au plus proche donne $\tilde{k} = k(1+\delta)$ avec $|\delta| \le u_{16} = 2^{-11} \approx 4{,}9 \times 10^{-4}$ (format binary16 de la norme IEEE 754). Pour un score d'attention $s = c\sum_{j=1}^{64} q_j k_j$ ($c = 1/8$),

$$|\tilde{s} - s| \;\le\; c\,u_{16} \sum_j |q_j k_j| \;+\; \gamma_{64}\, c \sum_j |q_j \tilde{k}_j|, \qquad \gamma_{64} = \frac{64\,u_{32}}{1 - 64\,u_{32}} \approx 3{,}8 \times 10^{-6},$$

au premier ordre, le second terme étant le produit scalaire en f32 ($u_{32} = 2^{-24}$, [25]). La sortie de l'attention hérite d'une erreur relative du même ordre, par les poids du softmax et par $\tilde{v}$. Avec un cache en f32, il ne reste que l'erreur des noyaux, $u_{32}/u_{16} = 2^{-13} \approx 1/8\,192$ fois plus petite ; ~~la KL étant du second ordre, elle serait divisée par environ $8\,192^2 \approx 6{,}7 \times 10^7$: de $3{,}08 \times 10^{-3}$ vers 5×10^{-11}, bien sous le pas du format. C'est un ordre de grandeur qui suppose l'erreur linéaire ; les bascules de routage ne le sont pas, mais elles disparaissent avec elle.~~ Cette estimation du lot O est barrée par le théorème 6.9 : le terme sans bascule est divisé par $8\,192^2 \approx 6{,}7 \times 10^7$, mais celui des bascules, linéaire, ne l'est que par 8 192 ; la KL en f32 est donc au plus de l'ordre de $3{,}08 \times 10^{-3}/8\,192 \approx 3{,}8 \times 10^{-7}$ nat [calc.] (section 6.6.3), bien sous le pas du format ($\approx 2{,}4 \times 10^{-4}$ nat). La mesure ci-dessous reste le seul juge. Un routeur qui choisit 4 experts sur 128 transforme une petite erreur continue en une erreur discrète quand deux scores sont proches : c'est pourquoi le 120b, avec 128 experts, bascule plus que le 20b, avec 32.

Mesuré. La ronde a ajouté au moteur une option `--kv-f32` (noyaux ajoutés, aucun noyau existant touché ; témoin : égal au bit au chemin f16 sur des valeurs représentables en f16, et différent sur des valeurs qui ne le sont pas). Tableau 8.7 et figure 8.5 donnent le résultat.

Sur l'invite de 708 jetons en KV f32, les premiers choix égalent ceux de la vérité à 708 positions sur 708 (687 en f16), le routage est égal sur les 36×708 couples (couche, position), et **le premier jeton de C12 est 1008, celui de la vérité**. L'arrondi f16 du cache KV portait donc *tout* l'écart mesurable entre troncons et les mathématiques exactes du modèle ; l'épreuve l'avait borné par en dessous (70 % à 77 % des bascules de routage).

Table 8.7 : Cache KV en f16 contre f32, contre la vérité f64 [mes.]. « Pas » : pas de quantification du format `.kld`.

	KL moyenne	1er choix	KL > 0,1	KL p99	Écart max. des log p
120b, fenêtre 0, f16	0,003 08	95,7 %	0,39 %	0,074 7	3,07 nat
120b, fenêtre 0, f32	−0,000 48	100 %	0	$2{,}3 \times 10^{-6}$	0,69 pas
20b, fenêtre 0, f16	0,001 74	96,5 %	0,39 %	0,052 9	2,32 nat
20b, fenêtre 0, f32	−0,000 48	100 %	0	≈ 0	2,04 pas
120b, invite, positions 512–706, f16	0,009 63	—	2,05 %	0,196	—
120b, invite, positions 512–706, f32	−0,000 54	—	0	$3{,}6 \times 10^{-6}$	—

Données : `moteur/O-pour-le-memoire.md` § 13.3 ; `mesures/moteur/O/corrections/journaux/verdicts-kv-2026-10-16.json`, `m5-ecart-logp-*.json` ; moyennes recalculées depuis `kv-f16-f32-par-position.csv`.

Le prix. Le cache KV de gpt-oss-120b au contexte de 4 096 compte 18 couches pleines et 18 couches à fenêtre glissante de 128 positions :

$$18 \times (4\,096 + 128) \times 2 \times 512 \times 2\,\text{o} = 155{,}7\,\text{Mo en f16},$$

le double en f32 : +155,7 Mo de VRAM, soit $155{,}7/13{,}25 = 11{,}8$ emplacements d'experts [calc.] ; mesuré : 326 → 314 emplacements. Rejouée par le simulateur du cache sur la trace de l'invite, cette perte fait passer h de 0,459 à 0,442 et les défauts de 77,9 à 80,4 par jeton : +3,2 % d'octets lus, l'ordre du coût en vitesse quand le disque limite [calc.] sur une trace mesurée. La génération mesurée en f32 (1,50 contre 2,13 jetons/s) *n'est pas* ce coût : le texte change (`72304d85`…, qui commence par « … » comme la vérité, puis simule des appels d'outil comme la sortie gloutonne de llama.cpp), et la boucle du texte f16 relevait son succès de cache à 0,590 contre 0,435. Comparer deux vitesses sur deux textes différents mesure les textes. Le défaut reste f16 tant que l'arbitre n'a pas tranché la couture N ; la ronde propose le f32.

8.7 Ce que les résultats disent ensemble

1. **Exact** : le chemin codé égale le chemin brut au bit, sur 5 120 pas et d'une construction à l'autre ; le codage sans perte du Tome I ne coûte rien à la sortie.
2. **Borné** : 384 Mo pour 63,4 Go, +3,7 Mo quand le modèle prend 51 Go ; llama.cpp tient 28,5 fois plus de mémoire (29,7 fois dans la configuration de C6).
3. **Plus rapide que la base** : 2,22 fois la meilleure base mesurée, garde comprise ; sept fois au préremplissage.
4. **Juste** : plus près de la vérité que llama.cpp sur les deux modèles ; et, cache KV en f32, égal à la vérité au format près.
5. **Limité par le disque** : 68 % à 72 % du plancher en série ; le prochain gain est dans la file du disque (lectures plus longues, ou plus de lectures en vol) et dans le disque lui-même (chapitre 10).

Ce qui pourrait être faux

- **C6 croise deux séances** : troncons et la base balayée n'ont pas tourné dans le même palindrome ; le rapport porte un bruit entre séances que la règle ne mesure pas. La même séance, à l'épreuve, donnait ×2,89 contre une base plus faible.
- **La base peut être meilleure encore** : ni plus de 16 fils, ni `--mlock`, ni d'autres valeurs de

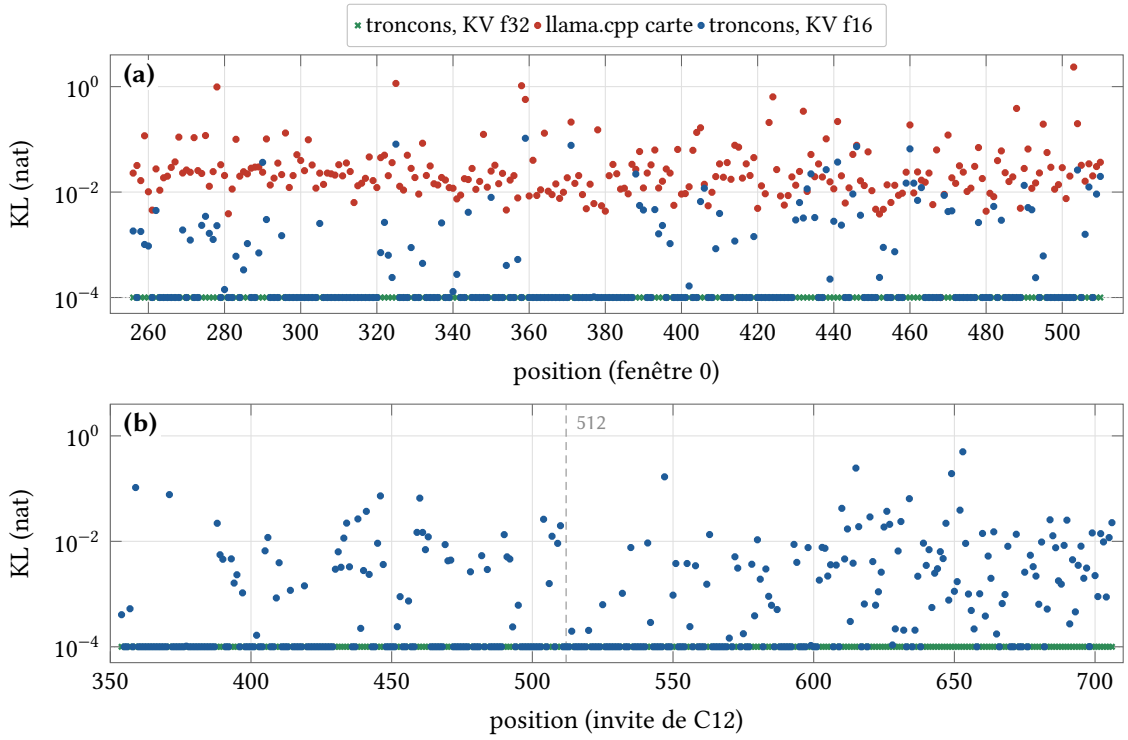


Figure 8.5 : KL par position contre la vérité f64, échelle logarithmique [mes.]. (a) Fenêtre 0 du 120b : llama.cpp (rouge), troncons en KV f16 (bleu), troncons en KV f32 (vert). (b) Invite de C12 au-delà de la fenêtre 0 : la KL du f16 monte après la position 512 ; le f32 reste au plancher. Une KL inférieure à 10^{-4}, dont toutes les KL négatives (biais du format), est dessinée sur la ligne pointillée : c'est le cas de 165 positions sur 255 pour le f16 en (a), de 255 sur 255 pour le f32, de 353 sur 353 en (b).

Données : `memoire/figures/tome2/donnees/res-kl-120b-f0.csv` et `res-kl-invite.csv`, tirés de `c4-numpy-120b-f0.csv` et `kv-f16-f32-par-position.csv`.

`--n-cpu-moe` n'ont été balayés. Le ×2,22 est un rapport à la meilleure base *mesurée*.

- **C4 sur le 120b repose sur une fenêtre** de 255 positions et sur l'invite ; une fenêtre n'est pas dix. La vérité numpy et la référence du lot R lisent le même GGUF et la même lecture du modèle : une erreur de convention commune passerait (section 9.10).
- **« KV f32 = vérité »** est mesuré sur une fenêtre de 512 jetons et une invite de 708 (120b), une fenêtre du 20b ; pas sur les dix fenêtres.
- **Le coût de la garde** repose sur une paire à disque égal (−1,6 %) ; la série déclarée dit −12,3 % sous un seuil de 26,8 % (bruit 11,9 %), non réel dans un sens comme dans l'autre.
- **La concordance avec le Tome I** ($h = 0{,}591$ contre 0,590 prédit) peut être une coïncidence d'un scénario calé sur un point.
- **Le texte de C12 et la conversation harmony** sont gloutons : ils montrent que le moteur produit un texte cohérent, pas sa qualité ; le juge, c'est C4.

Chapitre 9

L'épreuve indépendante et la ronde de corrections

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, un jalon déclaré tenu par les lots qui l'avaient construit, sur leurs propres instruments. Le SOCLE exige davantage : « le travail est relu par un juge qui ne l'a pas fait ». **Ce chapitre établit** ce qu'un juge indépendant a trouvé en une nuit (16 octobre 2026, de 02 h 12 à 06 h 10) : la spécification décode au bit près sous un troisième implémenteur ; la référence f64 est confirmée par un troisième calcul, à l'arrondi près ; C3 tient d'une construction à l'autre ; C4 tient sur le 120b ; et onze choses que personne n'avait vues, dont deux graves. Il établit ensuite ce que la ronde de corrections en a fait, le matin même : chaque constat avec sa correction, son témoin, la cassure qui fait tomber le témoin, et son verdict. **Il reste** la cause première du blocage G1 (non reproduite), et les décisions remises à l'arbitre. Sources : `mesures/epreuve-J1.md`, `moteur/epreuve-J1-pour-le-memoire.md`, `moteur/O-pour-le-memoire.md` (partie III), `moteur/REPRISE-O.md`, `mesures/moteur/epreuve-J1/` et `mesures/moteur/O/corrections/`.

9.1 Le protocole

L'architecture a écrit le protocole de l'épreuve avant toute construction (`decisions/ARCHITECTURE.md`, § 8). Le juge est un agent qui n'a écrit ni la spécification ni une ligne du code jugé ; il travaille depuis un clone propre, sur secteur, carte libre, et ne lit que la spécification, le code et les journaux. Il suit sept gestes : (1) écrire, *depuis le seul § 4* de la spécification, un décodeur du format et redonner au bit près des tronçons tirés au hasard ; (2) rejouer les cassures et vérifier que chaque témoin tombe puis tient ; (3) refaire les chiffres en séries ABBA ; (4) vérifier que la base n'est pas mal réglée ; (5) vérifier que l'instrument de mémoire voit la fenêtre ; (6) lire le texte ; (7) écrire son verdict, avec sa propre section « Ce qui pourrait être faux ».

Les conditions, notées. Secteur à chaque passe ; carte libre au départ (346 Mio) ; verrou de la carte pris à 02 h 26 et rendu à 04 h 44. Charge étrangère relevée et laissée : un `find` d'un autre agent (un cœur), une surveillance `nvidia-smi` depuis la veille, et un `nvidia-smi` orphelin né à 01 h 38 (il deviendra le constat G2). Construction depuis un clone au commit `527dff5`, en 21 s, *sans* la recette du SOCLE (qui cassait : constat m2). Le juge n'a ouvert les crates du format qu'*après* le geste 1, et son décodeur n'en a rien pris. Il restait 128 Go libres sur le disque.

9.2 Geste 1 : la spécification est-elle vraie ?

L'outil. `decodeur_s4.py` (Python 3.13, numpy) : un lecteur GGUF v3 écrit depuis la spécification publique ; l'en-tête du `.trc` champ par champ ; les sections MÉTA, RÉSIDENTS, TABLES et RÉPERTOIRE comparées au GGUF ; et, par tronçon, l'en-tête de partie, la table des flux, le décodage rANS du § 4.9.2, la table par voie du § 4.9.3 et la reconstruction exacte du § 4.11. Chaque expert reconstruit est comparé à l'empreinte d'origine du répertoire et aux octets du GGUF ; s'y ajoutent deux contrôles que le décodeur n'exige pas (médiane égale à `tri[B/2]`, fin de chaque tuile égale au début de la suivante).

Le décodeur vectorisé. Un flux rANS de la spécification est découpé en tuiles indépendantes de 32 voies ; un expert en compte 127, et chaque flux se décode en 1 024 pas séquentiels. Le juge décode les 127 tuiles à la fois, voie par voie en parallèle : 0,8 s par expert au lieu de minutes. Il faut alors prouver que cette forme vectorisée est bien le décodage normatif, qui lit les mots « pour $l = 0$ à 31, dans l'ordre ».

Lemme 9.1 (Le rANS vectorisé est le rANS du § 4.9.2)**.** *Soit, dans une tuile, le sous-pas k du macro-pas m, p la position courante dans le flux de mots de 16 bits, et pour chaque voie l l'état intermédiaire y_l après l'étape de décodage. Le décodage normatif fait renormaliser la voie l si et seulement si $y_l < L = 2^{16}$, en lisant alors un mot. Il est équivalent de calculer $b_l = [y_l < L]$ pour les 32 voies à la fois, de faire lire à chaque voie qui renormalise le mot d'indice $p + \sum_{l' \le l} b_{l'} - 1$, puis de poser $p \leftarrow p + \sum_l b_l$.*

Démonstration. Dans l'ordre normatif, la voie l lit, si elle renormalise, le mot qui suit ceux qu'ont lus les voies $l' < l$ au même sous-pas, soit le mot $p + \#\{l' < l : y_{l'} < L\} = p + \sum_{l' \le l} b_{l'} - 1$ quand $b_l = 1$. Il reste à voir qu'aucune voie ne renormalise deux fois au même sous-pas, sans quoi l'ordre normatif lirait deux mots pour une voie. Avant l'étape, l'état vérifie $x \ge L$; l'étape donne $y = f_s(x \gg 12) + (x \bmod 2^{12}) - c_s$ avec $f_s \ge 1$, donc $y \ge x \gg 12 \ge 2^{16} \gg 12 = 16$. Si $y < L$, la renormalisation donne $(y \ll 16) \mid w \ge 16 \cdot 2^{16} \ge L$: un seul mot suffit toujours. Les tuiles ont chacune leur début et leurs états, donc elles sont indépendantes. ■

Les résultats (tableau 9.1). Rien n'a manqué au § 4 pour décoder au bit près.

Table 9.1 : Geste 1 : le décodeur du juge, écrit depuis le seul § 4, sur les fichiers de production [mes.].

	gpt-oss-120b	gpt-oss-20b
Tronçons tirés (graine 20261016), égaux au GGUF au bit près	**20 / 20**	**20 / 20**
Voies hors de L ; tuiles dont la fin ≠ début de la suivante	0 ; 0	0 ; 0
Médianes égales à `tri[B/2]`	172 800 / 172 800 lignes	idem
Résidents octet pour octet (et empreinte)	**471 / 471**	**315 / 315**
MÉTA : paires du GGUF octet pour octet, plus 3 clés du codeur	oui (39 = 36 + 3)	oui
TABLES : toutes de somme 4 096 ; empreinte	432 tables ; oui	288 ; oui
RÉPERTOIRE : entrées contiguës jusqu'à la fin du fichier	4 608 ; oui	768 ; oui
En-tête : empreinte, taille, SHA-256 du source = registre des téléchargements	oui (`582bd40f`…)	oui (`27cd6c43`…)

Données : `mesures/moteur/epreuve-J1/journaux/s4-gptoss120b-20.json`, `s4-gptoss20b-20.json`.

Le témoin du décodeur tombe-t-il ? Chaque cassure est appliquée en mémoire au premier tronçon tiré (couche 12, expert 54) ; le second (couche 25, expert 109) doit tenir (tableau 9.2).

Table 9.2 : Les cassures du décodeur du juge : chacune tombe sur le seul tronçon visé [mes.].

Cassure	Voies hors de L	Octets faux	Le second tient
un bit retourné dans les mots de CODES_CTX	32 (une tuile)	31 122	oui
deux fréquences d'une table échangées	4 064	3 624 195	oui
une médiane changée d'une unité	**0**	90 (une ligne d'échelles)	oui
une fente de LUT corrompue	4 032	3 262 607	oui

Données : `journaux/s4-gptoss120b-casse-\{mot,table,mediane,lut\}.json`.

La médiane faussée laisse toutes les voies sur L : seule l'empreinte de l'expert la voit. C'est le germe du constat M1.

Les copies corrompues des lots. Sur la copie du lot E (12 bits retournés au hasard dans le 20b), les 12 tronçons touchés sont détectés, et le détail dit qui voit quoi : 6 par les voies hors de L, 2 par une erreur de format, 2 par l'en-tête de partie (dont les octets décodés égalent pourtant le GGUF), et **2 seulement par l'empreinte de l'expert** : un octet de MEDIANES (couche 23, expert 4) et un octet de BIAIS (couche 21, expert 24), qui gardent toutes leurs voies sur L. Sur la copie à médiane faussée du lot O, l'expert est faux et les voies sont sur L.

Ce que l'écriture a appris sur le § 4. Trois imprécisions, qui n'ont trompé personne mais qu'un second implémenteur devra deviner (constat m4) : les longueurs de section de l'en-tête sont les longueurs *utiles*, sans bourrage ; l'empreinte de la table des résidents est aux 8 derniers octets de la section, pas « après la zone de données » ; et « la fin de la tuile t est le début de la tuile $t + 1$ », vrai sur 100 % des tuiles décodées, est un contrôle gratuit que le § 4 ne rend pas normatif.

9.3 Les témoins tombent-ils ?

Le juge n'a pas rejoué les cassures des noyaux (K1 à K3), de la référence (R0, R1), du cache (O1) et du chemin des octets (E0, E2) : leurs journaux existent chez les lots.

9.4 Une troisième passe avant en double précision

Pourquoi un troisième calcul. C4 juge troncons contre une « vérité » : la passe avant f64 du lot R. Si cette vérité était fausse, C4 ne vaudrait rien. Le juge en a écrit une autre, en numpy f64, depuis la forme du modèle publiée par OpenAI (implémentation de référence sous Apache-2.0, [32] ; rien n'est copié), sans rien lire du lot R. Elle comprend l'enchâssement Q8_0, RMSNorm, les projections et leurs biais, RoPE par moitiés avec la rampe de YaRN réelle et la concentration $0{,}1 \ln 32 + 1$, l'attention à 64 têtes de requête et 8 de clé, l'échelle $1/8$, le puits par tête au dénominateur et une fenêtre de 128 clés sur les couches paires, le routeur top-4 puis le softmax sur les quatre, la SwiGLU bornée à 7, les experts MXFP4 déquantifiés exactement et la tête de sortie. Les bornes de rampe qu'elle trouve, 8,092 78 et 17,398 02, égalent celles du lot R. Durée : 6 min pour le 20b, 17 min pour le 120b, sur 511 positions.

Table 9.3 : Les cassures rejouées par le juge [mes.].

Cassure	Attendu	Observé
décodeur du juge : mot, table, médiane, fente de LUT	le tronçon visé tombe, l'autre tient	oui, 4 sur 4
décodeur du juge sur les copies E2 (12 bits) et O3 (médiane)	13 sur 13 détectés	oui
moteur, production, copie E2	erreur fatale nommée	« couche 2, expert 19 : décodage faux (32 voies hors de L, 0 erreurs de format, 1 pointeurs de tuile faux) »
moteur, production, copie O3	rien en production	le moteur va au bout, et son texte de 32 jetons **égale celui du fichier sain** (`48f70a45`…)
moteur, mode témoin, copie O3	empreinte nommée	« témoin : couche 0, expert 30 : empreinte `2e3544c4`… au lieu de `c9f614cc`… »
C7 : 1 Go de cache tiède	≈ +1 Go	+994 Mo

Données : `mesures/epreuve-J1.md` § 4 ; `journaux/seances/complements-2026-10-16.json`.

La référence contre la passe du juge. Sur les 255 positions notées de la fenêtre 0 du 20b, et les 201 088 jetons de chacune, l'écart maximal des log-probabilités au-dessus du plancher du format vaut 0,000 123 nat, soit 0,504 **pas de quantification** (pas médian $2{,}44 \times 10^{-4}$) ; 255 positions sur 255 sous un pas ; 100 % de premiers choix identiques [mes.]. Ce nombre a une borne théorique.

Proposition 9.2 (L'écart dû au seul format)**.** *Un fichier `.k1d` code, par position,* $\log q_i = \mathit{pas} \cdot \mathit{code}_i + \log q_{\min}$ *avec un code entier arrondi au plus proche, et donne le code 0 à tout logit sous le plancher. Si deux calculs rendent les mêmes* $\log q_i$ *réels, les valeurs lues dans leurs fichiers s'écartent d'au plus pas/2 sur chaque jeton au-dessus du plancher, plus l'arrondi f32 de pas et de* $\log q_{\min}$ *(de l'ordre de* 10^{-6} *en relatif).*

Démonstration. Un arrondi au plus proche déplace une valeur d'au plus un demi-pas ; la valeur lue est pas · code + $\log q_{\min}$, calculée à partir de deux flottants f32 stockés dans le fichier, dont l'arrondi ajoute son propre terme. ■

Mesuré : 0,504 pas, soit pas/2 plus 0,004 pas. La référence du lot R égale donc la passe du juge *à l'arrondi du format près, et à rien d'autre.* La rampe de llama.cpp, elle, s'en écarte jusqu'à 6,5 nat sur certains jetons.

Pourquoi une KL exacte est négative. Les jetons sous le plancher du format (environ 85 000 sur 201 088) reçoivent tous $\log \tilde{q} = \log q_{\min}$, plus grand que leur vraie valeur : la distribution lue $\tilde{q}$ n'est pas normalisée, $\sum_i \tilde{q}_i = e^{\delta}$, avec $\delta = 0{,}002\,0$ en moyenne [mes.].

Lemme 9.3 (La KL d'un calcul exact contre un fichier)**.** *Pour un calcul identique à la vérité,* $\mathrm{KL}(p \,\|\, \tilde{q}) = \sum_i p_i(\log p_i - \log \tilde{q}_i)$ *tend vers* $-\sum_{i \text{ sous le plancher}} p_i\,(\log q_{\min} - \log p_i) < 0$.

Démonstration. Au-dessus du plancher, $\log \tilde{q}_i = \log p_i$ au demi-pas près (proposition 9.2), et ces écarts, de signes mêlés, se compensent en moyenne. Sous le plancher, $\log \tilde{q}_i = \log q_{\min} > \log p_i$, et chaque terme $p_i(\log p_i - \log q_{\min})$ est négatif. ■

Mesuré : $-4{,}8 \times 10^{-4}$ pour la référence f64 du lot R contre la vérité du juge. Ce biais, commun à tous les fichiers, ne change aucun ordre ; il fixe le zéro de l'instrument, et c'est exactement la KL que le cache KV en f32 atteindra plus tard (section 8.6.3).

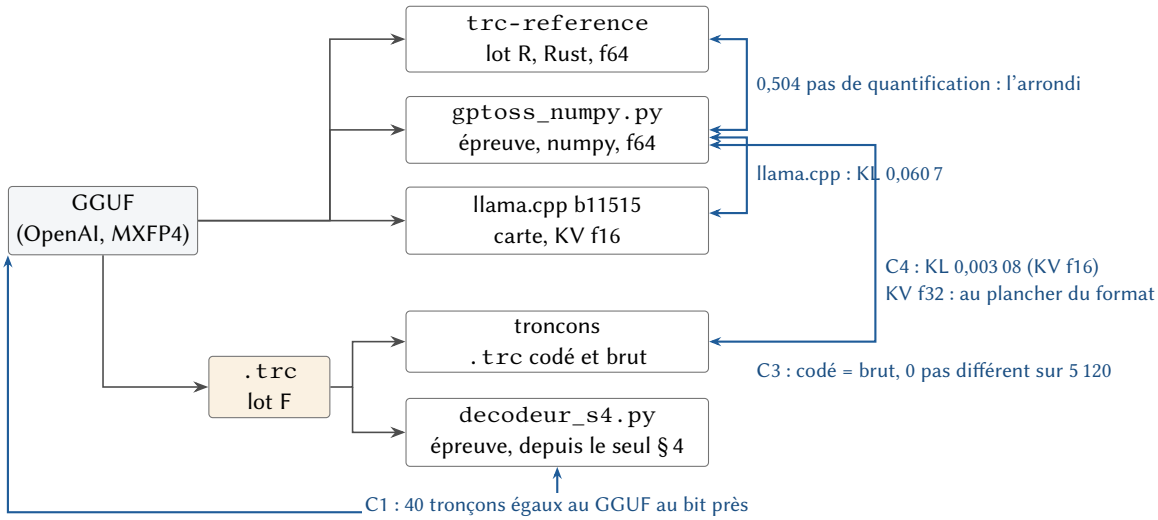


Figure 9.1 : Les chemins de la vérité : cinq programmes, trois implémenteurs indépendants (le lot R, le juge, llama.cpp) pour la passe avant, deux (le lot F, le juge) pour le format, et la mesure qui relie chaque paire.

C4, enfin jugé sur le 120b. Contre cette vérité, troncons fait 0,003 08 et 95,7 % de premiers choix sur la fenêtre 0 du 120b, llama.cpp 0,060 7 et 80,4 % ; différence appariée $-0{,}057\,6 \pm 0{,}012\,0$, intervalles de Wilson disjoints. Le détail est au tableau 8.6. Sur la même fenêtre, la rampe de YaRN seule explique 55 % de l'écart de llama.cpp sur le 20b : le « six fois plus près » du lot O est vrai, mais il porte surtout sur un choix de modélisation (constat m5).

9.5 La séance de vitesse et de mémoire

Le pilote du juge est indépendant de celui des lots : processus neufs, ordre en palindrome déclaré avant toute passe (commit `4832184`), pointes tenues par le système et lues sur la poignée après la sortie, mémoire disponible de toute la machine toutes les 100 ms, VRAM toutes les 250 ms, alimentation avant et après. La figure 9.2 trace les douze passes.

Reproduction. troncons fait 2,03 jetons/s en moyenne contre 2,18 au lot O : écart de 7,4 %, sous $2 \times 5{,}5$ % de bruit, *reproduit*. llama.cpp `fit -t 6` fait 0,627 contre 0,568 publiés : 10,4 %, sous $2 \times 9{,}5$ %, *reproduit*. Les textes égalent ceux du lot O et de la base.

C5, et le piège d'un disque froid (m1). La première passe disque seul lisait 0,725 Go/s (90e centile de 172 ms pour une lecture de 12 Mo), la seconde 1,822 Go/s ; le moteur lisait à 1,79 Go/s la minute suivante. La formule du lot O prenait la médiane des deux passes D : elle déplace le seuil de 25 % (1,10 à 1,13 au lieu de 1,43 à 1,48). Avec la seule passe chaude, pour la passe 01 [calc.] :

$$t_{\text{série}} = \underbrace{\frac{710{,}12\ \text{Mo}}{1{,}822\ \text{Go/s}}}_{389{,}7\ \text{ms}} + \underbrace{79{,}2}_{\text{PCIe}} + \underbrace{57{,}0}_{\text{déc.}} + \underbrace{\frac{1{,}686\ \text{Go} + 4 \cdot 36 \cdot 13\,254\,144\ \text{o}}{235\ \text{Go/s}}}_{15{,}3\ \text{ms}} + 3{,}5 = 544{,}7\ \text{ms},$$

soit un plancher de 1,836 jeton/s et un seuil de 1,469. Les trois passes T120 tiennent dans les deux cas. La ronde a corrigé la formule : une passe D d'échauffement, jetée.

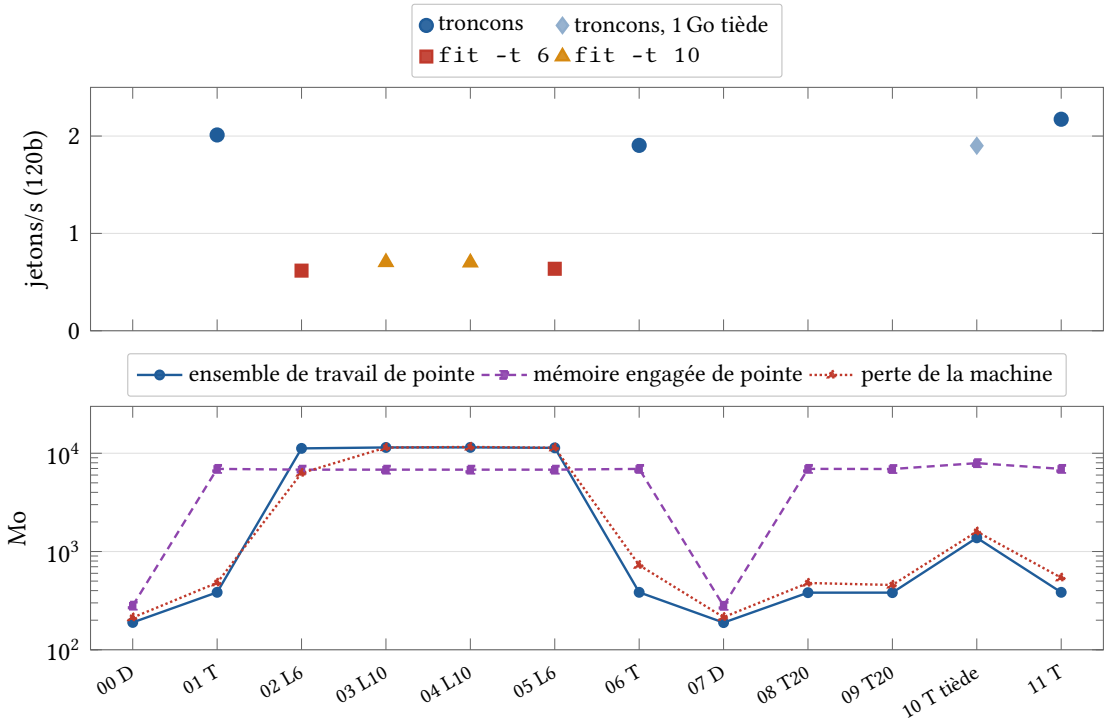


Figure 9.2 : La séance `s1-2026-10-16` de l'épreuve, dans l'ordre du palindrome [mes.]. En haut, la génération du 120b : troncons (rond), avec 1 Go de cache tiède (losange), llama.cpp à 6 fils (carré) et à 10 fils (triangle). En bas, par passe : ensemble de travail de pointe, mémoire engagée de pointe, et perte de mémoire disponible de toute la machine (échelle logarithmique). Les passes D (disque seul) et T20 (20b) n'ont pas de génération du 120b.

Données : `memoire/figures/tome2/donnees/res-epreuve-seance.csv`, tiré de `mesures/moteur/epreuve-J1/csv/s1-2026-10-16-passes.csv`.

Ce que la machine perd. Pendant une passe de troncons, la mémoire disponible de toute la machine baisse de 0,46 Go à 0,73 Go ; pendant une passe de llama.cpp, de 6,3 Go à 11,6 Go [mes.]. Le seuil déclaré d'un constat (deux fois l'ensemble de travail plus 200 Mo, 0,97 Go) n'est pas franchi. La mémoire *engagée* de troncons, 6,93 Go, est dix-huit fois son ensemble de travail : C7 ne la voit pas (constat m3).

C8, par deux instruments. Le critère comparait les octets lus selon le compteur PDH du système à ceux que le moteur compte. Le juge a ajouté les compteurs d'entrée-sortie du *processus* (`GetProcessIoCounters`), témoignés par des lectures sans tampon de 4 Kio à 64 Mio comptées à l'octet près (tableau 9.4).

Le moteur compte juste : le processus lit 22,8 Mo de plus, les lectures tamponnées de l'ouverture (MÉTA, tables, répertoire, ≈ 13,5 Mo) et environ 9 Mo d'autres. Les gros écarts du PDH sont hors du processus : sur une première passe à froid, +2,37 Go, très probablement les images des bibliothèques (CUDA, llama) chargées par fautes de page, que le PDH voit et que `ReadFile` ne compte pas [hyp.]. Constat M3 : C8 tel qu'écrit mesure la machine.

Table 9.4 : C8 par deux instruments [mes.] : le compteur global du système (PDH) et les compteurs du processus, contre le compte du moteur.

Passe	PDH − moteur	Borne déclarée	PDH	Processus − moteur
s1 01 T120 (première passe 120b)	+2 372,8 Mo	100,8	tombe	—
s1 06 T120	+314,9	244,2	tombe	—
s1 08, 09 T20	+96,5 ; +39,7	103,5 ; 121,3	tient	—
s1 10 T120 tiède	+241,9	2 689,4	tient	—
s1 11 T120	+125,4	154,4	tient	—
c3 (codé ; brut, 3 fenêtres)	—	—	—	+22,8 ; +22,6 Mo

Données : `mesures/moteur/epreuve-J1/journaux/c8-juge.json`, `temoin-compteurs-es.json`.

9.6 La base est-elle honnête ?

Le juge a fait tourner la meilleure configuration publiée (`fit`, 6 fils) et une autre de son choix, `fit -t 10`, dans la même séance ; sa raison, déclarée avant : sous `mmap`, chaque fil en faute de page est une lecture en vol, donc plus de fils font une file plus profonde sur le disque. Résultat : 0,627 jeton/s à 6 fils (bruit 3,0 %), 0,702 à 10 fils (bruit 0,6 %) ; écart de 11,9 %, réel ; mêmes textes. La base publiée sous-estimait llama.cpp, et le ×3,85 du lot O était à barrer (constat M2). Le juge a écrit lui-même ce qu'il n'avait pas fait : ni 12, ni 16 fils, ni `--n-cpu-moe`. La ronde l'a fait ; le résultat est au section 8.5.2 : à 16 fils, 0,884, et l'écart était de 54 %, pas de 12.

9.7 Le texte est-il juste ?

C12. Les 256 jetons gloutons du 120b commencent par « am », puis un résumé cohérent et exact du texte d'invite (Robert Boulter, *The Bill* en 2000, *Herons* au Royal Court en 2001), avant de tomber, après une centaine de jetons, dans une boucle (« Here is a summary » raccourci jusqu'à « Here » répété). La sortie gloutonne de llama.cpp sur la même invite boucle elle aussi, sur des appels d'outil simulés : c'est le comportement du décodage glouton de ce modèle, pas un défaut visible du moteur.

Le premier jeton, tranché par la vérité. Les deux moteurs diffèrent dès le premier jeton : troncons dit « am » (313), llama.cpp « … » (1008). Le juge a passé les 708 jetons de l'invite à sa passe f64 du 120b ; le moteur les a rejoués par flux et en boucle (égaux au bit), et la position 707 redonne bien le jeton 313.

Table 9.5 : Le premier jeton de C12 (position 707 de l'invite) selon chaque calcul [mes.].

Calcul	argmax	log p vrai de 313 (« am »)	log p vrai de 1008 (« … »)
vérité f64	**1008**	−10,03	−1,46
llama.cpp	1008		
troncons (KV f16)	**313**		
vérité f64 avec le seul KV arrondi en f16	**313**	−1,47 (sous l'imitation)	−3,84 (sous l'imitation)
troncons (KV f32, ronde)	**1008**		

Données : `journaux/premier-jeton-c12-120b.json`, `premier-jeton-kvf16-120b.json` ; `mesures/moteur/O/corrections/journaux/m5-invite-120b-f32.json`.

Ce n'est pas une rupture à 512 : les PPL des positions 354 à 706 valent 1 200,3 pour troncons et 1 221,8 pour la vérité, ce passage est difficile pour le modèle lui-même ; et `--contexte 1024` comme `--emplacements 200` laissent les 708 empreintes inchangées. C'est une cascade de bascules de routage : à la position 707, 20 couches sur 36 routent autrement que la vérité, à partir de la couche 12 ; à la position 706, aucune. Le cache KV en f16 en est la cause suffisante, et la cause principale : expérience déclarée avant sa passe (commit `549b38b`), la vérité avec le seul KV arrondi en f16 fait la même erreur, et troncons est bien plus près de cette vérité-là (routage égal à 99,3 et 97,9 % contre 96,9 et 93,1 %) : l'arrondi porte environ 70 % à 77 % des bascules. Constat M5. La suite, le cache en f32 qui rend troncons égal à la vérité, est au section 8.6.3.

La conversation harmony (publiée, non jugée) : une réponse en français, structurée en tableaux, sur l'inférence d'un mélange d'experts avec déchargement ; le canal `analysis` précède `final`, conformément au format [44]. Le fond contient des approximations du modèle, pas du moteur ; les caractères coupés entre deux jetons sortent en « ? » (défaut d'assemblage du texte, connu du lot O).

9.8 Les constats

Le juge a classé ses constats par gravité ; chacun dit où, ce qui est constaté, la preuve et une correction proposée (tableau 9.6). Toute correction est une proposition ; les coutures ne changent que par l'arbitre.

9.9 La ronde de corrections

La ronde a déclaré son premier seuil le 16 octobre à 06 h 12 (commit `ca5d338`), sept minutes après le dernier commit du rapport de l'épreuve ; son état final est daté de 10 h 15 (`moteur/REPRISE-O.md`, commit `30d8165`). Ses règles étaient celles du projet, appliquées à chaque constat : *reproduire d'abord* le défaut avec le binaire d'avant (gardé, copie figée du commit `7b288a6`), déclarer le seuil avant la passe (`mesures/moteur/O/corrections/seuils-declares.md`, un bloc commité par constat), corriger, puis montrer que le témoin tombe avec l'ancien binaire et tient avec le nouveau. La cassure du témoin, c'est l'ancien binaire.

9.9.1 G1 : la passe figée

Ce que le juge a vu. La passe `evaluer --flux --brut` des dix fenêtres du 120b, lancée à 03 h 33 pendant que la passe numpy du juge lisait 61 Go et occupait le processeur, n'a jamais fini. Au bout de 35 min : aucune lecture de plus en 10 s (1 863 lectures, 18,06 Go), carte à 0 %, fil principal à 100 %, et pas une ligne sur la sortie d'erreur. Le juge a échantillonné 300 fois le pointeur d'instruction du fil principal (300 sur 300 dans `nvcuda64.dll`) puis nommé sa pile par les symboles : `troncons::principal` → destruction du `Moteur` → `Arc<Interne>::drop_slow` → destruction des événements CUDA → `nvcuda64.dll`. Le moteur avait quitté sa boucle, très probablement sur une erreur, et sa destruction attendait un travail resté en file sur la carte ; l'erreur, elle, n'a jamais été écrite. Relancée seule, la même commande finit en 287 s.

La sortie corrigée (figure 9.3). Sur toute erreur, le moteur écrit d'abord l'erreur (sur la sortie d'erreur et dans un `<journal>.erreur.json`), puis termine son propre processus par `TerminateProcess` [45], sans détruire le moteur ni le contexte CUDA : le pilote libère tout à la fin du processus. Un battement par couche du préremplissage date tout blocage futur.

Table 9.6 : Les constats de l'épreuve, et ce que la ronde de corrections en a fait.

	Constat de l'épreuve	Correction et témoin de la ronde	Verdict
G1 grave	une passe sous charge se fige sans fin et sans message : la destruction du moteur tourne dans `nvcuda64.dll`	erreur écrite avant toute destruction, sortie sans détruire, battement par couche ; tampons d'arrivée du flux corrigés (lemme 9.4)	corrigé ; cause première non reproduite
G2 grave	après une erreur, un `nvidia-smi` orphelin tient les tuyaux du pilote, qui attend sans fin	`Drop` qui tue l'enfant, tâche « tuer à la fermeture », entrée nulle, poignées non héritables ; 3 cas, 0 orphelin	corrigé
M1	la production ne garde pas les flux bruts : une médiane fausse donne un texte identique au sain	garde de production (proposition 9.5) ; copie O3 : erreur nommée	corrigé, −1,6 % à disque égal
M2	base sous-réglée ; ×3,85 à barrer	base balayée : `fit -t 16`, 0,884 ; C6 ×2,22	fait
M3	C8 mesure la machine	compteurs du processus dans le journal : écart constant 19,92 Mo	instrument fait ; critère à l'arbitre
M4	C7-écart tombe encore	borne à termes nommés (tableau 8.4)	non fermée ; à l'arbitre
M5	le KV f16 décide du premier jeton de C12	option `--kv-f32` : égal à la vérité au format près	mesuré ; couture N à l'arbitre
m1	le seuil de C5 dépend d'une passe D froide	passe D d'échauffement jetée	fait
m2	la recette du SOCLE casse la construction	`build.rs` appelle `nvcc` sans `cmd`	corrigé
m3	C7 ne voit pas la mémoire engagée	publiée : 6,92 Go à 6,97 Go ; perte machine 0,47 Go à 0,63 Go	publiée, non jugée
m4, m6	trois imprécisions du § 4 ; où va le verdict	phrases proposées	à l'arbitre
m5	« six fois plus près » demande sa raison	requalifié : dont la rampe de YaRN	fait

Données : `mesures/epreuve-J1.md` § 8 ; `moteur/O-pour-le-memoire.md` § 14 ; `moteur/REPRISE-O.md`, état final de la ronde.

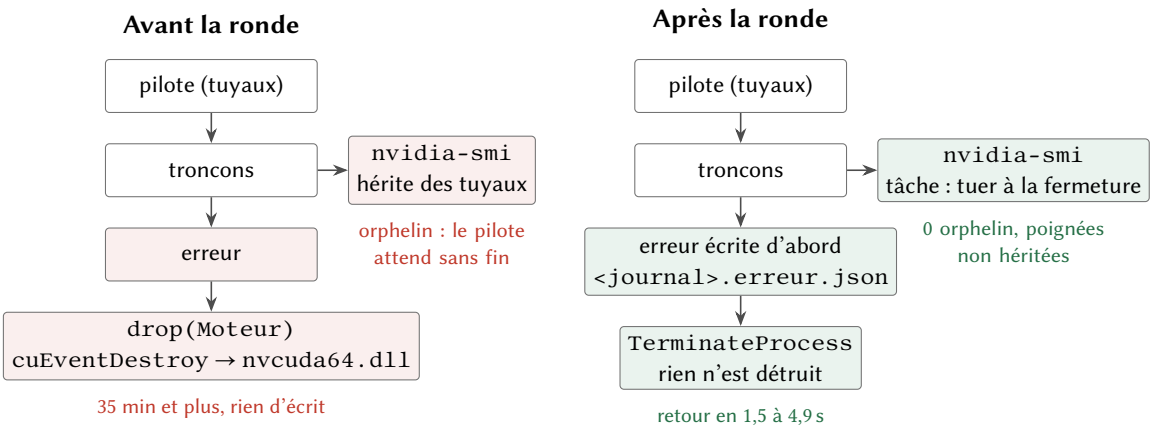


Figure 9.3 : La sortie sur erreur, avant et après la ronde (constats G1 et G2). Avant : la destruction du moteur attend la carte sans fin, et l'enfant `nvidia-smi`, qui a hérité des tuyaux du pilote, les garde ouverts. Après : l'erreur est écrite d'abord, rien n'est détruit, et l'enfant meurt avec son parent.

Une cause probable, trouvée en lisant le code. Au préremplissage par flux, les lectures d'un paquet prenaient leurs tampons d'arrivée épinglés en tourniquet, $s \bmod (d \cdot k)$ avec $d \cdot k = 12$, et au plus $\min(d \cdot k, 2 \cdot \text{profondeur}) = 8$ lectures en vol, confiées au système par paquets de quatre (figure 9.4).

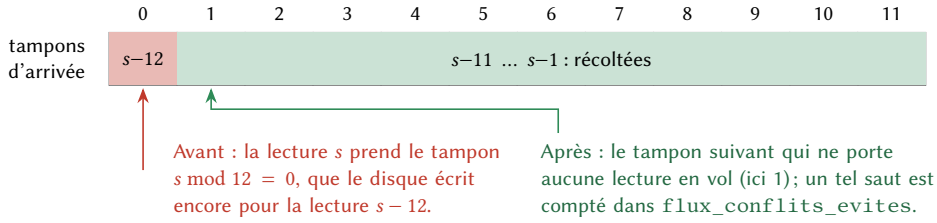


Figure 9.4 : Le tourniquet des tampons d'arrivée au préremplissage par flux, et le contre-exemple du lemme 9.4 : la lecture $s - 12$ reste en vol pendant que $s - 11, \ldots, s - 1$ passent et sont récoltées ; la lecture s prendrait son tampon.

Lemme 9.4 (Les tampons d'arrivée du préremplissage par flux)**.** *(i) Avec la règle d'avant la ronde, une lecture pouvait écrire dans un tampon qu'une autre lecture écrivait encore. (ii) Avec la règle corrigée (prendre le tampon du tourniquet s'il ne porte aucune lecture en vol, sinon le suivant qui n'en porte pas), aucune lecture n'écrit dans un tampon i tant que (a) une autre lecture y écrit, (b) la copie vers la carte qui le lisait n'est pas finie, (c) un hachage de la garde le lit encore.*

Démonstration. (i) Contre-exemple. Les achèvements arrivent dans le désordre. Que la lecture $s - 12$ reste dans le système pendant que $s - 11, \ldots, s - 1$ passent par les trois autres places et sont récoltées : la liste locale des lectures en vol ne contient plus que $s - 12$, de longueur $1 < 8$, et la lecture s est soumise dans le tampon $s \bmod 12 = (s - 12) \bmod 12$, que le disque écrit encore. La copie lancée à la récolte de $s - 12$ lit alors un mélange : contrôle des voies faux (erreur fatale, suivie de la destruction de G1), ou, si seuls des octets bruts sont touchés, un expert faux sans erreur. Il suffit qu'une lecture prenne environ $11 \times 6{,}6\,\text{ms}/3 \approx 24\,\text{ms}$ de retard sur les autres (12 Mo à 1,8 Go/s, trois places) [calc.] ; le 90e centile d'une lecture à froid relevé par le juge, 172 ms, le dépasse. (ii) Existence : au moment du choix, au plus $\min(d \cdot k, 2 \cdot \text{profondeur}) - 1 < d \cdot k$ lectures sont en vol, donc au moins un tampon n'en porte aucune. (a) suit de la règle. (b) : la soumission attend l'événement de la copie précédente

du tampon, mise en file à sa récolte. (c) : la soumission attend d'abord la fin des hachages de la zone. Le tampon de VRAM correspondant n'est récrit qu'après la fin du décodage qui le lisait. ■

Listing 9.1 : Le choix d'un tampon d'arrivée au préremplissage par flux, après la ronde (`moteur/crates/trc-ordo/src/lib.rs`, extrait).

```
// Le tampon du tourniquet, sauf s'il porte encore une lecture en vol : les
// lectures s'achèvent dans le désordre ; avant la ronde de corrections, la
// nouvelle lecture écrasait alors l'ancienne dans le même tampon (cause
// probable de G1). Il y a toujours un tampon libre : au plus
// min(d · k, 2 · profondeur) − 1 < d · k lectures sont en vol ici.
let candidat = prochaine_arrivee % self.n_arrivees;
let mut i_arr = candidat;
while en_vol.iter().any(|&(_, b)| b == i_arr) {
    i_arr = (i_arr + 1) % self.n_arrivees;
}
if i_arr != candidat {
    self.compteurs.flux_conflits_evites += 1;   // le cas du lemme, compté
}
prochaine_arrivee = i_arr + 1;
self.ev_copie[i_arr].attendre()?;               // (b) la copie précédente est
    finie
```

Rejouée sous la même charge. Le bloc 4 des seuils déclarés a rejoué la passe figée sous la charge de l'épreuve (la passe numpy f64 du 120b, 61 Go lus, lancée d'abord). Le nouveau binaire finit en 361 s, avec 360 battements, un `.kld` égal à celui du lot O et **0 conflit d'arrivée** : le cas du lemme ne s'est pas produit. L'ancien binaire, sous la même charge, **finit aussi** (338 s, `.kld` égal). Le blocage de l'épreuve n'est donc pas reproduit : il dépend d'un ordre d'achèvements ou d'un état de la machine que deux rejeux n'ont pas retrouvé. Ce que l'on sait : la destruction pouvait attendre sans fin (pile nommée) ; le nouveau binaire ne détruit plus rien sur une erreur et l'écrit d'abord ; le tourniquet pouvait écraser une lecture en vol (démontré). Ce que l'on ne sait pas : l'erreur d'origine.

9.9.2 G2 : l'orphelin qui tient les tuyaux

L'échantillonneur de VRAM du moteur lançait `nvidia-smi -lms 250` et ne le tuait que dans son arrêt normal ; aucune destruction ne le faisait. Toute erreur laissait l'enfant vivant. Or il héritait des poignées du moteur, donc des tuyaux de sortie du pilote : le pilote, qui attend la fermeture des sorties, attendait aussi longtemps que l'orphelin vivait, c'est-à-dire toujours. À l'épreuve, deux passes finies en quelques secondes n'ont « rendu la main » qu'après 518 et 630 s, quand le juge a tué leurs orphelins ; un orphelin né à 01 h 38 d'une passe du lot O tournait encore à 04 h 40.

La correction, en quatre gestes. (1) Une destruction de l'échantillonneur qui tue et attend l'enfant ; (2) l'enfant placé dans une tâche Windows « tuer à la fermeture » [46], qui le tue même si le parent meurt par `TerminateProcess`, que la destruction ne voit pas ; (3) une entrée standard nulle ; (4) les poignées standard du parent rendues non héritables, parce que `bInheritHandles` vrai donne à l'enfant toutes les poignées héritables [47], dont les tuyaux du pilote.

Le témoin. `corrections/temoin_sortie.py` lance trois cas comme le font les pilotes (fichier absent, fichier corrompu, processus tué), sorties en tuyaux, et attend leur fermeture. Seuil déclaré : retour en moins de 30 s, code 1, erreur écrite, aucun `nvidia-smi` vivant 2 s après. L'ancien binaire **tombe** sur les trois (retour au délai de 90 s, un orphelin chacun) ; le nouveau **tient** (1,5, 4,9 et 4,9 s,

0 orphelin). Le témoin lui-même a eu deux défauts avant d'être valable, publiés : la première passe tuait le processus avant la naissance de l'enfant (elle ne prouvait rien) ; la deuxième ne vidait pas les tuyaux, et l'enfant se bloquait sur sa sortie d'erreur pleine (deux fils de lecture l'ont corrigé).

9.9.3 M1 : la garde de production

Le trou. En production, le seul contrôle d'un tronçon était le retour des voies rANS à L. Il ne garde que les flux *codés* : les flux bruts d'une partie (MEDIANES, BIAIS : environ 43 ko sur 12 Mo, 0,36 %) et, en mode A0, la ligne d'enchâssement lue à chaque jeton n'étaient gardés par rien. Sur la copie à médiane faussée, le texte produit était *identique* à celui du fichier sain : une corruption invisible jusque dans la sortie.

La garde (figure 9.5). À chaque lecture achevée, les octets utiles du tronçon sont hachés par un fil de travail et comparés à l'empreinte que le répertoire porte déjà (aucune couture touchée) ; le hachage court en même temps que la copie et le décodage, qui ne font que lire le tampon ; le pas attend tous ses hachages avant de rendre son jeton. Les lignes d'enchâssement sont comparées à une table d'empreintes bâtie à l'ouverture en lisant `token_embd` une fois, son empreinte entière vérifiée au passage.

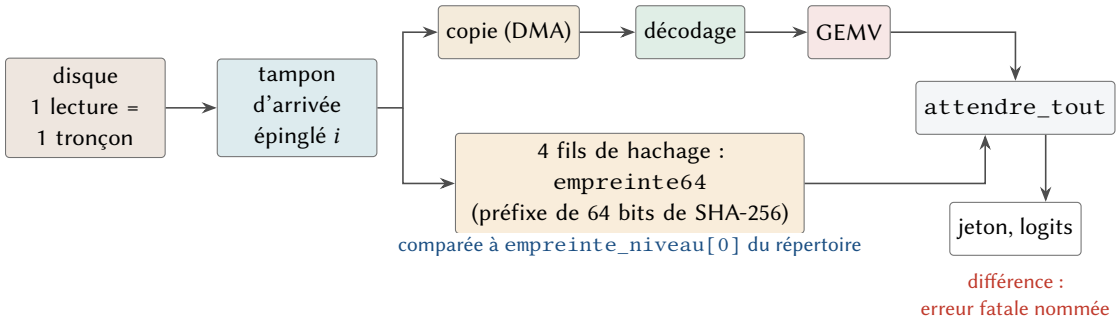


Figure 9.5 : La garde de production : le hachage court en parallèle de la copie et du décodage, sur quatre fils ; une barrière le rejoint avant que le pas ne rende son jeton.

Proposition 9.5 (Aucune sortie ne dépend d'un octet non vérifié)**.** *Garde active, le jeton et les logits que rend un pas, ou un préremplissage, ne dépendent que d'octets lus dont l'empreinte de 64 bits égale celle du répertoire (tronçons : octets utiles du niveau 0 ; lignes : table bâtie à l'ouverture), à une collision du préfixe de 64 bits de SHA-256 près [43].*

Démonstration. (1) Le décodeur et la copie ne lisent d'une partie que ses octets utiles ; le bourrage n'est lu par aucun noyau. (2) Chaque lecture achevée est confiée à la garde avant toute autre opération sur son tampon. (3) Aucune lecture n'écrit ce tampon avant la fin du hachage (lemme 9.4, (c)) : l'empreinte porte sur les octets que la copie a lus. (4) Le pas et le préremplissage attendent tous les hachages avant de lire le jeton ou les logits ; une empreinte différente est une erreur fatale nommée. (5) Une entrée tiède servie sans lecture a été hachée à sa lecture ; une corruption de la RAM après coup n'est pas couverte. (6) Chaque ligne lue est comparée à sa case avant sa copie vers la carte. Un contenu faux passe avec une probabilité 2^{-64} par vérification, si SHA-256 se comporte comme une fonction aléatoire. ■

Listing 9.2 : La garde rend un tronçon faux en erreur fatale nommée (`moteur/crates/trc-ordo/src/garde.rs`, extrait).

```rust
fn recevoir(&mut self, f: Fait) -> Result<()> {
    let t = self.en_cours.remove(&f.ticket).expect("ticket de la garde");
    if f.empreinte != t.attendu {                // l'empreinte du répertoire
        bail!(
            "garde de production : couche {}, expert {} : empreinte des {} octets utiles lus {:016x} au lieu de {:016x} (répertoire)",
            t.couche, t.expert, t.octets, f.empreinte, t.attendu
        );
    }
    ...
```

Les témoins, et leurs cassures (sur le 20b). Copie à médiane faussée : erreur fatale nommée « garde de production : couche 0, expert 30 : empreinte des 11 836 992 octets utiles lus `17d8974e`… au lieu de `9aa7c5fb`… » ; sans garde (la cassure), la passe va au bout, avec le texte du fichier sain, comme à l'épreuve. Tronçon cassé après lecture (`--casser-garde troncon:5`) : erreur nommée ; sans garde, texte changé sans erreur. Ligne cassée (`ligne:3`) : erreur nommée ; sans garde, texte inchangé (un bit d'une ligne Q8_0 ne change pas ces 32 jetons : invisible). Fichier sain : même texte au bit avec et sans garde, 920 tronçons vérifiés sur 920 lus, 739 lignes sur 739. Copie du lot E : la garde nomme couche 2, expert 19, avant le contrôle des voies. Le coût est mesuré au section 8.5.5 : −1,6 % à disque égal, 5,2 ms à 5,4 ms d'attente par jeton. C2 devient vrai en production.

9.9.4 M2 à M5, et les mineurs

M2, la base. Balayée en palindrome (tableau 8.5) : `fit -t 16`, 0,884 jeton/s ; C6 recalculé, garde comprise, ×2,22 ; ~~×3,85~~ barré dans les notes du lot O, sa note de reprise et `mesures/gpt-oss.md`.
M3, C8. Les compteurs du processus, lus sur la poignée après la sortie, sont dans le journal du moteur. Contre le compte du moteur (lecteur, résidents compris, plus les lectures tamponnées de l'ouverture, 13,57 Mo), l'écart vaut 19,92 Mo**, constant à l'octet près** sur les six passes 120b à KV f16 de la séance, dont le vocabulaire lu par `llama.dll` (12,98 Mo). Proposition à l'arbitre : juger C8 par cet instrument, borne égale à la taille du vocabulaire plus 100 Mo, le PDH publié sans être jugé. En chemin, la ronde a trouvé que la première version du compte ajoutait deux fois le chargement des résidents (+1,69 Go) : corrigé.
M4, C7-écart. La borne à termes nommés (tableau 8.4) n'est pas fermée ; proposition : juger l'écart à fenêtre de taille fixe, contre les termes hors fenêtre (≈ 0,77 Mo) plus deux bruits plus 0,5 Mo. Le verdict reste « tombe ».
M5, le cache KV. Option `--kv-f32`, hors couture : égal à la vérité au format près, 12 emplacements de moins (section 8.6.3).
m1. Une passe D d'échauffement jetée en tête de séance : la passe gardée lit 1,893 Go/s, la jetée 1,874, sans transitoire cette fois (90e centile 32,9 ms) ; seuils de C5 1,45 à 1,52, et les huit passes du 120b tiennent.
m2. Le `build.rs` des noyaux appelait `nvcc` par `cmd /C` avec des guillemets que `Command::arg` protège par des barres obliques inverses, que `cmd` ne lit pas ; il appelle maintenant `nvcc` directement. Avec et sans vcvars, la construction réussit, et les trois images `noyaux.fatbin` sont égales octet pour octet (SHA-256 `aeae5bdc`…, 295 864 octets).
m3. Mémoire engagée (6,92 Go à 6,97 Go) et perte de la machine (0,47 Go à 0,63 Go) publiées à côté de C7.
m4, m6. Trois phrases proposées pour le § 4 ; l'endroit du verdict, à l'arbitre.
m5. « Six fois plus près » requalifié partout : dont la rampe de YaRN.

9.9.5 Ce qui reste à l'arbitre

La ronde a remis douze points à l'arbitre, chacun marqué « réponse par délégation du 2026-10-16, réversible » (`moteur/REPRISE-O.md`, points 21 à 32). Les plus lourds : la couture N (cache KV en f32 par défaut, f16 en option : C4 et C12 deviendraient « égaux à la vérité au format près », pour 155,7 Mo de VRAM et 12 emplacements) ; un critère de queue pour C4 (part des positions de KL $>$ 0,1, KL au 99e centile, jugés aussi au-delà de 512 positions ; chiffres pour le fixer : f16 0,39 % et 0,075 sur la fenêtre 0, 2,05 % et 0,196 sur les positions 512 à 706 ; f32 : 0) ; la suite gloutonne de la vérité à publier à côté de C12 (non faite : la passe numpy refait tout le préfixe à chaque jeton, environ 20 min par position sur le 120b, des jours pour 256 jetons) ; C8 et C7 comme ci-dessus ; la garde inscrite au § 4.12 de la spécification ; prévenir les lots que tout enfant lancé sous un pilote à tuyaux doit avoir des poignées non héritables ou vivre dans une tâche tuée à la fermeture ; et un `nvidia-smi` orphelin du lot O (PID 13652), laissé vivant parce que ni le juge ni la ronde ne l'avaient lancé.

9.10 Ce que l'épreuve n'a pas pu vérifier

- **C1 sur les** 4 608 **experts** par le troisième décodeur : 40 tirés et 13 nommés, plus 3 394 décodages vérifiés par le témoin du moteur ; le reste repose sur le lot F et ses deux chemins.
- **C2 de bout en bout** par le banc du lot E : non relancé.
- **C4 sur les dix fenêtres du 120b** : ni la référence f64 du lot R ni une base llama.cpp de dix fenêtres n'existent sur le 120b ; le jugement porte sur une fenêtre de 255 positions, plus les 353 positions notées de l'invite.
- **La cause de G1** : l'erreur d'origine n'a jamais été écrite.
- Les cassures des lots K, R, O1 et E.
- **La séance de vitesse est une seule séance** : trois passes de troncons, deux de chaque base ; la règle ne se juge qu'à l'intérieur.

Ce qui pourrait être faux

- **La passe numpy du juge et la référence du lot R lisent le même GGUF.** Une erreur de déquantification commune aux deux (ordre des demi-octets, échelle E8M0) passerait. Garde-fous : le décodeur MXFP4 du juge suit la convention de ggml (demi-octets bas pour les éléments 0 à 15), et C3 comme les textes relient tout le reste au bit ; la convention du format n'est jugée que par l'accord avec llama.cpp, pas par OpenAI.
- **Elles suivent la même lecture du modèle** (rampe réelle, puits, fenêtre de 128). Si OpenAI entraînait avec une autre convention, les deux se tromperaient ensemble ; le `config.json` et l'implémentation de référence d'OpenAI sont les seules sources de cette lecture.
- **C4 sur le 120b repose sur une fenêtre** ; les intervalles sont disjoints, mais une fenêtre n'est pas dix.
- **La perte de mémoire de la machine est globale** : d'autres processus la font bouger. Elle est propre dans la séance `s1` ; elle était souillée (10,5 Go, la passe numpy en même temps) dans une autre passe que le juge n'a pas utilisée.
- **L'instrument du juge a eu deux défauts**, trouvés et corrigés avant tout verdict : une poignée de processus passée en entier de 32 bits (un témoin des compteurs qui rendait 0), et un premier calcul de C8 qui comptait deux fois le chargement des résidents (« le moteur compte 1,4 Go de trop »), refait juste et jamais publié.
- **La cause de G1 n'est pas établie** : le défaut des tampons est démontré possible, pas observé

(0 conflit en deux rejeux).

- **La garde suppose SHA-256 sans collision utile** sur 64 bits ; elle ne couvre ni la RAM tiède après sa lecture, ni la VRAM.
- **L'heure d'un seuil était fausse** dans le fichier du juge : ~~02 h 40~~ 02 h 26, l'heure du commit, avant l'essai du pilote (02 h 26 min 35 s) et la première passe (02 h 27 min 40 s). Les seuils ont bien été déclarés avant toute passe ; seule l'heure écrite était fausse.

Chapitre 10

Discussion, limites et travaux futurs

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, un programme qui tient neuf critères entiers sur les onze jugés (C7 tient sur son plafond et tombe sur son écart, C8 attend un autre instrument, C9 est publié sans être jugé), une épreuve indépendante et une ronde de corrections (chapitres 8 et 9). **Ce chapitre établit** ce que le Tome II dit en somme ; ce que les prédictions du Tome I valaient face aux mesures ; les chiffres barrés et les idées tuées par les nombres, rassemblés ; les limites de la méthode ; puis le travail qui vient, chaque pas avec la mesure qui le jugera : la découpe base + complément (J2), le disque `D:` sans BitLocker, la porte compatible pour relais, et l'étude biomimétique de Stone, le génome des poids. **Il reste** à le faire, dans l'ordre décidé avec Stone.

10.1 Ce que le Tome II établit

1. **Un modèle de** 63,4 Go **tourne avec** 384 Mo **de mémoire** sur une machine qui en a 16, et sa mémoire ne suit pas sa taille : +3,7 Mo pour +51,3 Go de modèle (section 8.4). La lecture de « sans RAM ni rien » que le Tome I avait retenue est vérifiée par la mesure.
2. **Le codage sans perte ne coûte rien à la sortie** : logits égaux au bit entre fichier codé et fichier brut sur 5 120 pas, d'une construction à l'autre et d'un ordre de calcul à l'autre (section 8.3).
3. **troncons est plus rapide que la meilleure base mesurée** : ×2,22 en génération, garde d'intégrité comprise, et ×7 au préremplissage (section 8.5).
4. **Il est plus près des mathématiques exactes que llama.cpp**, et, avec un cache KV en f32, égal à elles au format près : tout l'écart mesurable venait de l'arrondi f16 du cache (section 8.6.3).
5. **La méthode a fonctionné** : l'épreuve indépendante a trouvé deux défauts graves et cinq moyens que les lots n'avaient pas vus, et la ronde les a corrigés ou mesurés en une matinée, chaque correction avec un témoin qui tombe sur l'ancien binaire (section 9.9).

10.2 Les prédictions du Tome I, face aux mesures

Le Tome I a calculé avant de construire. Tableau 10.1 confronte ses chiffres à ceux du programme ; une prédiction tombée juste n'est pas une preuve, une prédiction tombée faux est une information.

Deux leçons. D'abord, le seul chiffre du Tome I qui ait vraiment bougé est celui qu'il n'avait pas mesuré lui-même : la base, estimée puis mesurée trop vite. Ensuite, le débit du disque dans le moteur est ~~le tiers de ce que le Tome I mesurait en grands blocs séquentiels~~ un peu plus de la moitié (1,55 Go/s utiles) à deux tiers (1,85 Go/s, disque seul à la longueur des tronçons) de ce que le Tome I mesurait en lectures aléatoires de 16 Mio (2,82 Go/s) [calc.] : non parce que le disque aurait changé, mais parce que le moteur lit des tronçons dans le régime lent du disque et n'a pas assez de lectures en vol (section 8.5.1). C'est le levier le plus proche.

Table 10.1 : Les prédictions du Tome I et les mesures du Tome II (gpt-oss-120b, mode A0 sauf mention).

Grandeur	Tome I (calculé ou estimé)	Tome II (mesuré)
succès du cache en VRAM	$h_v = 0{,}590$ (scénario LRU calé sur un point, cache de 4,8 Go)	$h = 0{,}591$ (326 emplacements, 4,3 Go)
vitesse en A0	repère 2,15 jetons/s (LRU calé) ; 0,99 (uniforme) ; 3,32 (Belady)	2,18 à 2,19 (lot O) ; 1,965 garde comprise
mémoire de l'hôte	A0 : 1,9 Go ; fenêtre minimale du partisan de A : 0,64 Go	0,384 Go, sous la fenêtre minimale
étage qui limite	le disque (157 contre 58 et 36 ms, scénario avec cache tiède)	le disque : 68 % à 72 % du plancher en série
débit du disque	2,82 Go/s en lectures aléatoires de 16 Mio (profondeur 1) ; 2,58 Go/s en lectures aléatoires de 4 Mio (profondeur 4)	1,85 Go/s à 1,89 Go/s à la longueur des tronçons (régime lent du lot E) ; 1,55 Go/s utiles dans le moteur
taux du codage sans perte	$r = 1{,}1026$ sur les experts	un tronçon de 12,03 Mo pour un expert de 13,25 Mo : $r \approx 1{,}10$
décodeur sur la carte	82 Go/s sous CUDA, sur un grand flux	0,85 ms à 0,92 ms par expert seul, soit 14 Go/s à 16 Go/s de sortie : un expert seul n'occupe pas la carte [hyp.] ; 10 % du temps d'un jeton
base llama.cpp	estimée ; exploration 0,49	~~0,568~~ ; 0,884 (`fit -t 16`)

Données : Tome I (résumé, chapitres « La machine », « Formulation », « Architecture »); chapitre 8.

10.3 Les chiffres barrés

Quand une mesure a contredit un chiffre publié, même le nôtre, la phrase a été barrée là où elle était, et l'on a dit ce qui l'avait remplacée (tableau 10.2). C'est la trace que la méthode a fonctionné.

10.4 Les idées tuées par les nombres

Le SOCLE demande de tuer une idée par les nombres, pas par l'opinion, et de garder les échecs avec leur raison (tableau 10.3).

10.5 Les limites

- **Une seule machine, partagée.** Tout est mesuré sur le portable de Stone, la cible ; mais d'autres agents mesuraient en même temps, et le disque a varié de 20 % d'une séance à l'autre sans cause mesurée (la température du NVMe n'est pas lisible sans droits d'administrateur).
- **Un disque chiffré.** Tous les chiffres de vitesse portent BitLocker, qui déchiffre chaque octet par le processeur et refuse le contournement d'entrée-sortie ; le disque `D:` non chiffré (section 10.6.2) dira ce qu'il coûte sur un vrai modèle.
- **Un seul modèle visé**, gpt-oss, en MXFP4 natif. Le format et le moteur sont écrits pour un mélange d'experts dont l'expert est la dimension la plus lente de chaque tenseur ; un autre modèle demandera un autre codeur, et peut-être d'autres noyaux.
- **Une seule tâche de vitesse** (une invite de 708 jetons, 256 jetons gloutons) : le succès du cache, donc la vitesse, dépend du texte (section 8.6.3).
- **Une carte NVIDIA obligatoire** : le choix de CUDA pour J1 ferme la porte aux autres cartes jusqu'au portage Vulkan.

Table 10.2 : Les chiffres barrés pendant la construction, l'épreuve et la ronde, et ce qui les a remplacés.

Barré	Remplacé par, et pourquoi	Où
~~C6 : ×3,85 sur la base~~	×2,79 à 2,89 (épreuve, `fit -t 10`), puis ×2,22 (ronde, `fit -t 16`, garde comprise) : la base tournait à 6 fils	section 8.5.2
~~×2,79 à 2,89~~	×2,22 : la base n'avait pas été balayée au-delà de 10 fils	section 8.5.3
~~meilleure base du 120b : fit, 0,568~~	`fit -t 16`, 0,884 : +54 %, écart réel	tableau 8.5
« six fois plus près de la vérité » (non barré, requalifié)	dont la rampe de YaRN de llama.cpp, 55 % de son écart : un choix de modélisation	section 8.6
~~C2 : 100 % des corruptions détectées~~ (en production)	vrai en mode témoin seulement ; vrai en production depuis la garde	section 9.9.3
~~empreinte : 2 Go/s par cœur~~ (estimé)	1,07 Go/s à 1,10 Go/s par fil, mesuré	section 8.5.5
~~KV f32 : 604 Mo au lieu de 302, −7 %~~	+156 Mo, −3,6 % d'emplacements : le premier calcul oubliait l'anneau des couches à fenêtre	section 8.6.3
~~seuil de C5 sur la médiane des deux passes D~~	une passe D d'échauffement jetée : la première, à froid, déplaçait le seuil de 25 %	section 9.5
~~compte des octets du moteur~~ (résidents comptés deux fois, +1,69 Go)	corrigé dans le code ; écart au processus constant, 19,92 Mo	section 9.9.4
~~heure du seuil de l'épreuve : 02 h 40~~	02 h 26, l'heure du commit	section 9.10

- **Une fenêtre de jugement courte sur le 120b** : C4 porte sur 255 positions et l'invite ; la référence du lot R sur les dix fenêtres du 120b manque.
- **Deux décisions en attente** : tant que l'arbitre n'a pas tranché la couture N, C7 et C8, deux critères sur douze restent « tombe » ou « instrument à changer ».

10.6 Travaux futurs

L'ordre a été décidé avec Stone le 15 octobre, puis repris le 16 au matin (« on fini une tache proprement puis ln enchaine l'autre ») : le jalon et son épreuve, ce Tome II, puis une pause complète et le redémarrage par Stone (le disque `D:`), puis l'étude du génome des poids, puis la porte compatible pour relais (`REPRISE.md`). La figure 10.1 place chaque chantier par rapport aux jalons.

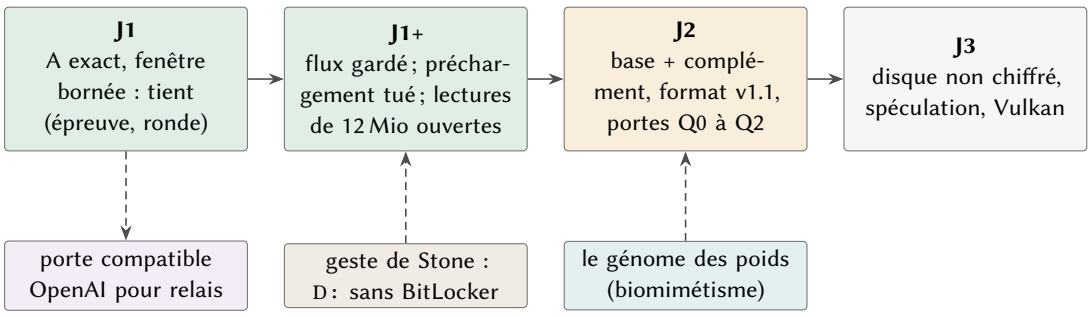


Figure 10.1 : Les jalons de troncons et les chantiers qui viennent. En vert, ce qui tient ; en orange, le jalon de la contribution ; en pointillés, ce qui alimente un jalon.

Table 10.3 : Les idées essayées pendant J1, J1+ et la ronde, et le nombre qui les a jugées.

Idée	Le nombre qui la juge
Précharger par le routeur de la couche suivante	−11 % : 69 % de prédictions justes ; les 31 % fausses coûtent plus de disque que le recouvrement ne rend [mes.]
Lectures de 12 Mio	+30 % en médiane, mais un bruit de 16,2 % entre les passes B (seuil 35,4 % ; ~~21 %~~ de dispersion) : ni établi, ni tué [mes.]
Une copie corrompue au hasard comme seule cassure de C3	insuffisante : il a fallu une médiane faussée, invisible au décodeur, pour montrer des logits faux [mes.]
Comparer deux vitesses sur deux textes différents	à ne pas faire : le texte en KV f32 ne boucle pas, h 0,435 contre 0,590 ; le coût du KV se mesure sur une même trace [calc.]
Vider les préchargements en refusant tout achèvement étranger	échec, corrigé : les lectures de la couche en cours finissent pendant le vidage
Assembler le texte en décodant chaque jeton seul	échec d'affichage : les caractères UTF-8 coupés entre deux jetons sortent mal
Construire le binaire pendant qu'un pilote le lance deux fois	à ne pas faire : les deux passes de C3 doivent tourner sur le même binaire
Appeler `nvcc` par `cmd /C` depuis `Command::arg`	échec : les guillemets protégés par des barres obliques inverses, que `cmd` ne lit pas
Vérifier la garde avant chaque produit (au lieu de la fin du pas)	non essayée : calculée à environ 6 ms × 30 couches par pas, −40 % [calc.]

Données : `moteur/REPRISE-O.md` (recettes de J1, de J1+ et de la ronde).

10.6.1 Avant tout : une série unique, et les décisions de l'arbitre

Trois gestes courts fermeraient les fragilités de ce tome : une seule série en palindrome, troncons gardé contre `fit -t 16` (et 20 fils, et `--mlock`), pour un C6 sans bruit entre séances ; la référence f64 du lot R sur les dix fenêtres du 120b, pour un C4 sur 5 120 positions ; et les décisions de l'arbitre (KV f32 par défaut, critère de queue de C4, C7 à fenêtre fixe, C8 par l'instrument du processus), qui rendraient le jalon net.

10.6.2 Le disque `D:` sans BitLocker

Le geste. C'est le seul qui revienne à Stone, sur sa machine : un outil déjà écrit et commité (`outils/preparer-disque-D.ps1`, lancé par un raccourci du Bureau qui demande les droits d'administrateur) vérifie qu'aucun agent ne tourne, coupe le fichier d'échange de `C:`, redémarre, réduit `C:` de 200 Go (au moins 150), crée un volume `D:` « Modeles » non chiffré, vérifie que `C:` reste chiffré et que `D:` ne l'est pas, remet le fichier d'échange dans tous les cas, et redémarre. Les poids des modèles sont des fichiers publics ; rien de personnel n'a besoin d'y être chiffré. `C:` reste chiffré, et la clé de récupération de BitLocker est sauvegardée depuis le 15 octobre.

Ce qu'il pourrait rendre, posé. Le disque tient 68 % à 72 % du plancher en série. Sur la passe 03 de la ronde (sans garde), le moteur lit 0,710 Go de tronçons par jeton, et les autres étages coûtent ensemble 80,7 + 50,5 + 15,3 + 3,5 = 150,0 ms. Si les octets lus restent les mêmes et que seul le débit W du disque change [calc.] :

$$t_{\text{série}}(W) = \frac{0{,}710\ \text{Go}}{W} + 150{,}0\ \text{ms},$$

$$
\begin{aligned}
W &= 1{,}893\ \text{Go/s (mesuré, C:)} && \Rightarrow 525\ \text{ms}, && 1{,}90\ \text{jeton/s},\\
W &= 2{,}82\ \text{Go/s (Tome I, grands blocs)} && \Rightarrow 402\ \text{ms}, && 2{,}49\ \text{jetons/s},\\
W &= 6{,}0\ \text{Go/s (annonce du fabricant)} && \Rightarrow 268\ \text{ms}, && 3{,}73\ \text{jetons/s}.
\end{aligned}
$$

Avec un recouvrement parfait, la borne serait l'étage le plus lent : à 6,0 Go/s, le disque prendrait 118 ms et le lien 81 ms, soit au plus 8,5 jetons/s ; le lien redeviendrait presque le maximum, comme le Tome I l'avait annoncé. Ce sont des bornes, pas des prédictions : Intel RST VMD reste sur le chemin, sa part dans les 53 % perdus n'est pas séparée, et la mesure tranchera.

Ce qui se refera sur `D:`. Les modèles et les `.trc` y seront déplacés (empreintes vérifiées), puis les bancs du disque et du jalon refaits à l'identique : la différence entre `C:` et `D:`, mêmes séances, mêmes fichiers, donnera pour la première fois le coût de BitLocker sur un modèle réel. Le débit d'une lecture sans tampon dépend de sa longueur, en deux régimes (lot E) : la longueur des lectures se rejugera sur `D:`.

10.6.3 J2 : la découpe base + complément

Ce qui est prêt. Le format `.trc` range déjà les flux d'un tronçon par niveau de raffinement, tout préfixe de niveau étant décodable seul ; les sortes de flux de la base et du complément sont spécifiées au bit près ; et la mesure F1 de l'arbitre a montré que la découpe, avec un contexte riche pour le complément, coûte +0,31 % d'octets seulement contre le niveau unique (`decisions/ARCHITECTURE.md`, § 3). Le cache connaît déjà les niveaux (`Base`, `Exact`), inutilisés avant J2. Le Tome I a démontré que le raffinement exact d'un code 4 bits natif ne coûte rien en entropie.

Ce qu'il peut rendre. Servir la base seule, sans son complément, aux défauts où l'octet est cher : B3 promettait ×1,74 en débit (×1,78 avec les bits mesurés par F1), au prix de 48,5 % d'usages d'experts servis par la base, pour une KL estimée de 0,05 à 0,12 [calc.].

Les portes, reprises sans les desserrer (§ 7.3 de l'architecture). *Q-instrument* : la ré-expression exacte doit retomber à $\mathrm{KL} \leq 7 \cdot 10^{-5}$ et 100 % de premiers choix, sinon le banc est faux. *Q1* : KL de B contre le mode exact du même moteur $\leq 0{,}005\,5$, seuil hérité de Qwen3.5-4B qui sera barré et remplacé au grand jour par l'écart entre moteurs mesuré sur gpt-oss. *Q2* : le point (KL, octets lus par jeton) de B doit passer sous la frontière des requantifications uniformes d'au moins deux erreurs-types. *Vitesse* : un mode B vaut s'il bat le mode exact du même moteur de plus de deux bruits et 3 %. Ce que ce tome change : le juge de B sera le mode exact du même moteur, que la proposition 8.2 rend reproductible au bit, et, si l'arbitre adopte le KV en f32, un mode exact lui-même égal à la vérité au format près. Le banc Q0 (la qualité de B sans moteur) et le mode base des noyaux restent à faire.

10.6.4 La porte compatible pour relais

relais est l'outil d'automatisation de Stone, piloté par un agent qui tourne sur des modèles locaux et gratuits ; c'est la mission que troncons a interrompue (« on vas continuer la suite relais plus tard on doit faire tomber se levier d'abord »). La porte serait un service local qui parle le protocole des complétions de conversation compatible OpenAI, au-dessus de `troncons generer` et du format harmony que le moteur sait déjà produire, pour que relais puisse appeler gpt-oss-120b comme n'importe quel modèle. Ce qu'elle coûterait à l'usage, posé avec les vitesses mesurées [calc.] : une invite de 1 000 jetons se préremplit en $1000/27 \approx 37$ s ; une réponse de 300 jetons se génère en $300/1{,}965 \approx 153$ s. C'est lent pour une conversation, acceptable pour une tâche d'automatisation qui tourne seule, et c'est le cas

d'usage de relais. Deux exigences en découlent : renvoyer les jetons au fil de l'eau (et assembler les octets UTF-8 coupés entre deux jetons, défaut connu), et garder le moteur ouvert entre deux requêtes, l'ouverture coûtant 2 s à 3 s.

10.6.5 Le génome des poids : l'étude biomimétique de Stone

L'idée, en ses mots. Le 15 octobre, Stone a écrit :

> *J'ai penser à du biomimetisme informatique par exemple simjler une forme de molecule d'adn virtuel qui peut stocker une grande qua tutoriel d'informations en quelque sorte creer une nouvelle convention differentes du bit une transcription et implémentation*

puis, le même jour : « j'ai la certitude que ce qu'on cherche se trouve dans du biomimetisme ». Cette conviction est l'hypothèse de travail de l'étude ; la mesure la jugera (`idees/biomimetisme-genome-des-poids.md`).

Ce qui ne marche pas : changer d'alphabet. L'ADN écrit avec quatre lettres, soit exactement $\log_2 4 = 2$ bits par lettre. Écrire avec quatre symboles plutôt que deux ne crée pas de place : tout message a une taille minimale, son entropie, et aucun alphabet ne descend en dessous (Shannon [48]). Sur un ordinateur, une « molécule virtuelle » est stockée en bits. La force de l'ADN n'est pas son alphabet ; elle est dans ses *mécanismes*.

Ce qui peut marcher : copier les mécanismes. Plusieurs sont déjà dans troncons sans en porter le nom : l'expression à la demande (la cellule ne transcrit que les gènes dont elle a besoin : les experts ne sont décodés qu'au moment du calcul, puis libérés) ; l'épigénétique (des marques disent quels gènes sont actifs : le cache garde près du calcul les experts souvent appelés) ; les familles avant le détail (purines et pyrimidines avant l'identité exacte : la base avant le complément, J2). L'étude en ajoute quatre, chacune avec une hypothèse mesurable sur gpt-oss.

Table 10.4 : Les mécanismes du vivant retenus pour l'étude, et l'hypothèse que chacun pose sur les poids de gpt-oss.

Mécanisme	Dans le vivant	Hypothèse mesurable sur gpt-oss
La recette plutôt que le résultat	l'ADN porte le programme qui construit le corps, pas sa forme	un expert se représente par une graine de générateur pseudo-aléatoire et quelques coefficients, que la carte « transcrit » au moment du calcul ; SeedLM le fait avec des registres à décalage [49], les hyperréseaux avec un réseau qui engendre les poids d'un autre [50]
Duplication de gènes	un gène nouveau naît d'une copie d'un ancien, puis mute [51]	les experts d'une couche, ou de couches voisines, sont parents : un expert « parent » et, pour chaque descendant, une « mutation » courte coûtent moins de bits que les experts entiers
Codage prédictif	le cortex ne transmet que l'écart entre ce qu'il attendait et ce qui arrive [52]	prédire chaque poids depuis ses voisins, son parent ou sa ligne, et ne stocker que l'écart : moins de bits quand la prédiction est bonne
Élagage synaptique	le cerveau jeune retire les connexions inutiles [53]	une part des poids d'un expert se retire sans changer la sortie au-delà du seuil du juge [54, 55]

Données : `idees/biomimetisme-genome-des-poids.md`, § 3 et § 5 bis ; sources relues le 2026-10-16.

Pourquoi cette machine s'y prête, posé. Le théorème du toit du Tome I dit que la vitesse est bornée par le débit de l'étage le plus lent divisé par les octets qu'il doit faire passer ; ce tome a mesuré que cet étage est le disque. Un génome des poids *dépense du calcul pour économiser des octets*, exactement l'échange que la machine demande : la carte attend le disque, avec un budget d'environ 926 opérations par octet compressé au débit du disque (Tome I ; ~~214~~, le budget au débit du lien, 12,2 Go/s, qui ne s'applique pas quand le disque limite). Chiffrons ce qu'il rapporterait s'il tenait. Aujourd'hui, un expert se lit à 3,843 1 bits par poids sans perte (mesure F1). Si une représentation atteignait b' bits par poids à qualité égale, les octets lus du disque et copiés par le lien seraient multipliés par $b'/3{,}8431$, et le reste inchangé. Pour $b' = 2$, sur la passe 03 de la ronde [calc.] :

$t_{\text{série}} = 0{,}520\times(375{,}1+80{,}7)+50{,}5+15{,}3+3{,}5 = 306{,}5\,\text{ms}$, soit un plancher de 3,26 jetons/s, contre un plancher de 1,9

Les deux nombres sont des planchers en série, sans recouvrement (section 8.5.1) : la passe 03 mesure 2,18 jetons/s, au-dessus de son plancher, et seul un moteur mesuré jugera le génome. C'est un « si », et l'étude doit le juger.

Le plan de l'étude (`idees/biomimetisme-genome-des-poids.md`, § 5). (1) Lire l'état de l'art à la source (graines de générateurs, hyperréseaux, codes en treillis, biomimétisme informatique). (2) Mesurer sur les vrais poids de gpt-oss-20b, au processeur, les bits par poids obtenus contre l'erreur de reconstruction, pour chaque mécanisme. (3) Mesurer la transcription : un noyau qui régénère un expert depuis sa recette, et son débit, contre les 1,89 Go/s du disque. (4) Juger la qualité du texte contre la référence exacte, seuil déclaré avant. (5) L'écrire, dans un chapitre propre. Le critère de réussite, à déclarer avant la mesure : moins de bits par poids lus du disque que le codage sans perte d'aujourd'hui, une perte de qualité sous le seuil du juge, et une transcription plus rapide que la lecture qu'elle remplace. Un échec compris dit où chercher : chaque échec sera analysé (quelle part des poids est du bruit, laquelle a une structure) au lieu de s'arrêter au verdict.

Ce qui pourrait être faux

- **Les gains du disque `D:`** sont des bornes calculées sous l'hypothèse que seuls les octets du disque changent ; Intel RST VMD et le régime de longueur des lectures peuvent en reprendre une part.
- **Le gain d'un génome à 2 bits** est un calcul sous un « si » : les poids d'un modèle entraîné ressemblent peut-être à du bruit, et un bruit ne se résume pas ; la recette ne pourrait alors qu'approcher les poids, avec perte.
- **La transcription peut coûter plus qu'elle ne rend** : si régénérer un expert prend plus de temps que le lire, le gain disparaît ; le décodeur d'aujourd'hui prend déjà 10 % du temps d'un jeton.
- **La porte pour relais** est chiffrée avec les vitesses d'une seule tâche ; une invite longue ou un autre texte changent le succès du cache.
- **Les gains de J2** sont ceux de l'architecture, estimés sous une linéarité valable pour de petites erreurs ; Q0 n'a pas encore mesuré la qualité de B.
- **Le bilan hérite des fragilités des deux chapitres précédents**, d'abord C6 entre deux séances et C4 sur une fenêtre.

Conclusion

Le 15 octobre 2026, Stone a demandé « une vrai solution pour prendre un modele ultra lourd quelcoque d'une ai le faire tourner proprement sans ram ni rien ». Le Tome I avait mesuré la machine, démontré le toit du problème et choisi, par le calcul, une architecture exacte qui prépare la contribution. Ce Tome II a construit le programme et l'a soumis à un juge qui ne l'avait pas écrit.

Le résultat tient en quatre nombres, chacun relu par un second chemin. Un modèle de 63,4 Go, quatre fois la RAM et huit fois la mémoire de la carte, tourne avec 384 Mo de mémoire, la même à 3,7 Mo près que pour un modèle cinq fois plus petit. Ses sorties sont identiques au bit, sur 5 120 pas, qu'on lise ses experts compressés ou bruts. Il produit 1,965 jeton par seconde avec sa garde d'intégrité, 2,22 fois la meilleure configuration mesurée de llama.cpp, et préremplit sept fois plus vite. Et, avec un cache KV en f32, il calcule ce que le modèle calcule en mathématiques exactes, au format du fichier près.

Ce que ce tome n'a pas encore fait est dit aussi clairement : la contribution de recherche, la découpe base + complément, attend son banc de qualité et le jalon J2 ; deux critères attendent une décision de l'arbitre ; le rapport à la base croise deux séances ; et la cause première du seul blocage observé n'est pas connue. Une dizaine de chiffres ont été barrés en route (tableau 10.2), dont le premier rapport de vitesse, qui comparait troncons à une base mal réglée. La méthode les a trouvés, ce qui était son travail.

La suite est décidée : une pause, le disque `D:` sans BitLocker, qui dira ce que le chiffrement coûte sur un vrai modèle ; puis l'étude du génome des poids, l'intuition de Stone que le vivant sait déjà stocker beaucoup dans peu, à juger par les nombres ; puis la porte qui rendra ce moteur à relais, la mission qu'il avait interrompue. Le levier que Stone voulait faire tomber d'abord est tombé : la mémoire d'une IA ne dépend plus de sa taille.

Annexe A

Protocoles de mesure du moteur

D'où l'on part, ce que le chapitre établit, ce qui reste

D'où l'on part. Les règles de mesure du Tome I (son annexe A, « Protocoles de mesure ») : l'étiquette de provenance de chaque nombre, le témoin et sa cassure, le seuil déclaré avant, les séries ABBA et la règle de décision, le second chemin, l'alimentation et la machine partagée, le juge de qualité. Elles valent toutes ici, telles quelles. **Ce que l'annexe établit.** Ce que la construction y a ajouté : un critère de fin pour chaque étape de chaque lot, avec son témoin, sa cassure et la dérivation de son seuil ; la preuve qu'un palindrome annule une dérive linéaire ; le verrou de la carte ; l'heure du commit comme preuve qu'un seuil précède sa passe ; le protocole de l'épreuve et celui de la ronde de corrections, bloc par bloc. **Ce qui reste.** Deux protocoles que la machine n'a pas encore permis : une série unique en palindrome qui contienne à la fois troncons gardé et la meilleure base, et une base au-delà de 16 fils et avec `--mlock` (chapitre 10).

A.1 Ce qui est hérité du Tome I

Chaque nombre porte sa provenance ([mes.], [calc.], [litt.]). Chaque mesure a son témoin, un cas dont la réponse est connue d'avance, et tout témoin neuf doit pouvoir échouer : on force sa condition à faux, on relance, on vérifie que l'instrument tombe, puis on restaure. Un seuil dérive d'une grandeur mesurée que l'on nomme, et se déclare avant le lancement. Un coût ne se fonde jamais sur une seule passe. Tout chiffre publié dont on dépend est refait par un chemin indépendant. Seules les passes sur secteur sont des débits de référence. La règle de décision est celle de l'étude 06, § 7.5, rappelée en définition 8.1 : une configuration B diffère réellement de A si

$$\left|\frac{\bar{X}_B}{\bar{X}_A} - 1\right| \;>\; 2\,\max(\beta_A, \beta_B) + 3\,\%, \qquad \beta = \frac{|X_1 - X_2|}{\bar{X}}, \tag{A.1}$$

où β est le bruit d'une configuration, l'écart de ses deux passes d'un même palindrome rapporté à leur moyenne.

A.2 Les critères de fin de chaque étape

L'architecture a écrit, avant la première ligne de code, ce qui ferait tenir chaque étape (`decisions/ARCHITECTURE.md`, § 6.5). Tableau A.1 les reprend, avec le chapitre où chacun est jugé. Le facteur 0,8 revient partout ; il n'est pas une marge choisie au jugé : c'est $1 - 2 \times 10\,\%$, où 10 % est le bruit mesuré au Tome I de la lecture de 4 Mio en profondeur 4 (étude 01). Un débit mesuré au-dessus de 0,8 fois sa référence ne peut donc pas être déclaré plus lent qu'elle par la seule faute du bruit des deux mesures.

Table A.1 : Les critères de fin des étapes des cinq lots, déclarés le 15 octobre 2026 avant la construction (`decisions/ARCHITECTURE.md`, § 6.5), et le chapitre qui les juge.

Étape	Témoin	Cassure	Seuil (et sa source)	Jugé
F1	aller-retour écriture puis lecture identique ; décodeur processeur égal aux flux de l'arbitre	un octet d'en-tête faussé : refus ; une table faussée : décodage faux	0 écart	chapitre 3
F2, F3	C1 : tous les experts décodés par le processeur égaux à l'original (empreinte et octet à octet) ; taux à 0,1 % de F1	une médiane faussée dans une copie : inexact détecté	0 expert faux	section 3.12
E0	(a) épingler 0,25, 1, 4 et 11 Go ; (b) lecture sans tampon dans cette mémoire puis DMA, octets égaux ; (c) 36 synchronisations par jeton	un octet faussé dans la source	(b) DMA $\geq$ 11,0 Go/s ; (c) $\leq$ 15,7 ms par jeton	chapitre 4
E2	C2 : empreinte de chaque expert du 20b, disque → VRAM décodé, égale au répertoire	un bit retourné dans une copie du fichier	$\geq 0{,}8 \times 2{,}58 \times r(\text{F1})$ Go/s d'experts servis	section 4.8
K1	sortie égale à `trc-codec` sur les 108 tenseurs du 120b ; voies finies sur L	une fente de table corrompue	$\geq 0{,}8 \times 41{,}6 = 33{,}28$ Go/s (référence barrée ensuite pour 45,8)	section 5.2
K2	GEMV MXFP4 exacte au bit sur entiers, 10^{-5} sur réels	référence décalée de 1 %	$\geq 0{,}8 \times 209{,}4 = 167{,}5$ Go/s	section 5.3
K3	chaque noyau de la passe avant contre `trc-reference`	chaque témoin forcé à faux	seuils de l'interface des noyaux (§ 5.6)	section 5.4
R0	plancher de l'instrument (même configuration deux fois)	un autre modèle : KL nette	publiés : G_{cfg}, G_{moteurs}	chapitre 6
R1	$\text{KL}(\text{référence} \,\|\, \text{llama.cpp}) \leq \max(G_{\text{moteurs}}, 7 \cdot 10^{-5})$ sur 10 fenêtres de 512	YaRN sans son échelle	idem	section 6.4
O1	`rejouer` égal au simulateur de l'étude 04, à l'unité	politique faussée	0 écart	section 7.4
O3	C3, C4 sur le 20b ; compte du cache égal à `rejouer` sur la trace du moteur	tronçon corrompu	critères du jalon	chapitre 8
O4	le jalon J1	critères du jalon	critères du jalon	chapitre 8

Les seuils, posés pas à pas. Chacun se refait depuis des nombres nommés [calc.] :

- **E0 (b)** : le DMA hôte → carte mesuré au Tome I vaut 12,2 Go/s ; moins son bruit de 10 %, $12{,}2 \times 0{,}9 = 10{,}98 \approx 11{,}0$ Go/s.
- **E0 (c)** : dans le scénario LRU calé du Tome I, le disque prend 157 ms par jeton du 120b ; une synchronisation qui en coûte au plus le dixième, 15,7 ms, ne change pas l'étage le plus lent.
- **E2** : 2,58 Go/s est le débit du disque en lectures de 4 Mio mesuré au Tome I (étude 01) ; servis par des tronçons compressés au taux r, ces octets valent $2{,}58\,r$ Go/s d'experts décodés ; le facteur 0,8

couvre le bruit.

- **K1** : 0,8 × 41,6 = 33,28 Go/s, où 41,6 Go/s était le débit publié par l'étude 03 du chemin que K1 remplace. Ce chiffre était lui-même composé sous Vulkan ; refait sous CUDA il vaut 45,8 Go/s, et le seuil corrigé 0,8 × 45,8 = 36,6 Go/s. Le chapitre des noyaux barre le premier et juge contre les deux (section 5.2).
- **K2** : 0,8 × 209,4 = 167,5 Go/s, où 209,4 Go/s est la GEMV MXFP4 brute de l'étude 03.
- **C5** : 0,8 fois le plancher en série $t_{\text{série}}$ recalculé *dans la même séance*, avec le h compté par le moteur et les débits que la passe `troncons disque` vient de mesurer (section 8.5).
- **C7** : 1,9 Go est la RAM de pointe du mode A0 publiée par l'étude 07, que la fenêtre minimale (0,64 Go) devait battre ; 0,5 Mo est le répertoire, le seul terme que l'on croyait croître avec le modèle (le constat M4 de l'épreuve en a trouvé d'autres, section 2.7).
- **C10** : 8 188 − 909 = 7 279 Mio, la mémoire de la carte moins les 909 Mio que l'affichage et les applications y tenaient au relevé du 15 octobre à 10 h 15, carte libre (`mesures/REPRISE-gpt-oss.md`).

A.3 Le seuil précède sa passe : l'heure du commit fait foi

Un seuil déclaré « avant » ne prouve rien si rien ne date la déclaration. Chaque bloc de seuils est donc commité *avant* la première passe qu'il juge, et c'est l'heure du commit, pas celle écrite dans le texte, qui fait foi. Tableau A.2 donne, pour chaque fichier de seuils, le commit qui l'a créé et le nombre de commits qui l'ont touché ensuite (des blocs ajoutés, chacun avant sa passe ; jamais un seuil changé après la sienne).

Table A.2 : Les fichiers de seuils déclarés et l'heure du commit qui les a créés (engendré par `memoire/outils/tome2_annexes.py` depuis `git log`).

Seuils déclarés	Créé par le commit (heure locale)	Commits
`mesures/moteur/F/00-seuils-declares.md`	`c0df2b9`, 2026-10-15 13:05:21	3
`mesures/moteur/O/corrections/seuils-declares.md`	`ca5d338`, 2026-10-16 06:12:40	6
`mesures/moteur/epreuve-J1/seuils-declares.md`	`4832184`, 2026-10-16 02:26:05	4

La règle a servi trois fois contre ses propres auteurs : les heures écrites à la main étaient fausses, et elles ont été barrées pour l'heure du commit. L'épreuve écrivait ses seuils déclarés à ~~02 h 40~~, le commit `4832184` est de 02 h 26 min 05 s ; le premier jeton de C12, à ~~05 h 50~~, le commit `549b38b` de 05 h 31 min 09 s ; le bloc 1 de la ronde, ~~vers 06 h 25~~, le commit `ca5d338` de 06 h 12 min 40 s (`mesures/moteur/epreuve-J1/seuils-declares.md`, `mesures/moteur/O/corrections/seuils-declares.md`). Les trois erreurs vont dans le même sens, celui qui aurait rendu la déclaration plus tardive qu'elle n'était : aucune n'a pu avantager un verdict.

A.4 Séries ABBA et palindromes

Les séances de ce tome ne sont pas des paires A B mais des *palindromes* : l'ordre des passes se lit de même dans les deux sens. La séance de l'épreuve commence par D120 T120 L120 L120t10 L120t10 L120 T120 D120 (troncons disque, troncons, llama.cpp à 6 puis à 10 fils) ; la base balayée de la ronde,

après une passe d'échauffement jetée, par t6 t10 t12 t16 m m t16 t12 t10 t6 (`mesures/moteur/O/corrections/seuils-declares.md`, bloc 6). La raison est un lemme.

Lemme A.1 (Un palindrome annule une dérive linéaire)**.** *Soit une séance de n passes aux instants $1, \dots, n$, et s_i la configuration de la passe i, avec $s_i = s_{n+1-i}$ pour tout i. On suppose que la passe i mesure $X_i = \mu_{s_i} + a + b\,i + \varepsilon_i$, où μ_c est la vraie valeur de la configuration c, $a + b\,i$ une dérive linéaire de la machine (échauffement, charge étrangère qui monte ou descend) et ε_i un bruit de moyenne nulle. Alors, pour deux configurations c et c', l'espérance de $\bar{X}_c - \bar{X}_{c'}$ vaut exactement $\mu_c - \mu_{c'}$: la dérive disparaît, quelles que soient a et b.*

Démonstration. Soit $I_c = \{\, i \,:\, s_i = c \,\}$, de cardinal n_c. Comme $s_i = s_{n+1-i}$, l'application $i \mapsto n+1-i$ envoie I_c dans lui-même ; c'est une involution, donc une bijection de I_c. Alors

$$\sum_{i \in I_c} i \;=\; \sum_{i \in I_c} (n+1-i) \quad\Longrightarrow\quad \frac{1}{n_c} \sum_{i \in I_c} i \;=\; \frac{n+1}{2}.$$

L'espérance de $\bar{X}_c = \frac{1}{n_c}\sum_{i \in I_c} X_i$ vaut donc $\mu_c + a + b\,\frac{n+1}{2}$. Le terme de dérive, $a + b\,(n+1)/2$, est le même pour toutes les configurations, et s'annule dans la différence. ∎

Remarque A.2. Le lemme ne vaut que pour une dérive linéaire : une dérive courbe (un disque qui se refroidit vite puis lentement) laisse un terme du second ordre. Il ne vaut que pour la partie de la séance qui est un palindrome : les quatre dernières passes de l'épreuve (T20 T20 T120tiede1 T120) ne forment pas un palindrome avec le reste, et le rapport qui les compare au reste n'est protégé que par la règle (A.1). Le bruit β d'une configuration, lui, mesure l'écart de deux passes *symétriques* : il contient la dérive entre elles, et la règle est donc plus sévère quand la machine dérive. C'est voulu.

La règle s'applique aux moyennes de chaque configuration, jamais à une passe isolée. Une passe d'échauffement est jetée quand elle précède la série. Le constat m1 de l'épreuve en donne la raison chiffrée : la première passe `troncons disque` de sa séance lisait à 0,725 Go/s (90e centile : 172 ms par lecture), la seconde à 1,822 Go/s, et la médiane des deux déplaçait le seuil de C5 de 25 % (1,12 au lieu de 1,47 jeton/s ; C5 tenait dans les deux cas) [mes.] (`mesures/epreuve-J1.md`, m1). La ronde de corrections a depuis jeté une passe D d'échauffement avant sa série (bloc 3) ; la règle générale attend la décision de l'arbitre (chapitre 9).

A.5 Le verrou de la carte et la machine partagée

La carte est une ressource unique, et d'autres agents travaillaient sur la machine. Avant toute séance qui la mesure, le pilote attend que le fichier `~/dev/verrou-carte.txt` soit absent ou vieux de plus de deux heures, puis l'écrit à son nom (lot, date, outil, numéro de processus) et le rend à la fin (`mesures/moteur/O/pilote_o4.py`, et les pilotes des lots E et K). Avant *chaque* passe, il relève et journalise l'alimentation (`Win32_Battery.BatteryStatus`, 2 : secteur), les processus `llama-*` vivants et la VRAM occupée ; il s'arrête si la machine n'est pas sur secteur. Le verrou de la ronde de corrections a été pris à 06 h 41, renouvelé à 07 h 21, 08 h 13 et 09 h 14, et rendu à 10 h 06 (`moteur/REPRISE-O.md`) ; celui de l'épreuve, pris à 02 h 26 et rendu à 04 h 44 (section 9.1).

Le verrou protège la carte, pas le reste. La charge étrangère du processeur et du disque est relevée et laissée, jamais tuée : on ne tue aucun processus que l'on n'a pas lancé (SOCLE). Les débits « au mieux » restent donc des bornes inférieures du repos, comme au Tome I. Les outils du lot F, qui ne demandent pas la carte, se lancent en priorité « sous la normale » (`mesures/moteur/F/lancer.ps1`), et les constructions se font chacune dans sa propre cible, à quatre tâches (`-j 4`), parce que cinq lots compilaient en même temps.

A.6 Le témoin des témoins

Un témoin qui ne peut pas tomber ne prouve rien. Chaque lot a donc écrit, à côté de ses témoins, un outil qui les casse un à un et vérifie qu'ils tombent, puis qu'ils tiennent une fois restaurés ; tableau A.4 les rassemble (leurs descriptions complètes sont à l'chapitre B).

Table A.4 : Les outils qui cassent les témoins, par lot.

Outil	Ce qu'il casse
`mesures/moteur/F/casser_temoins_f1.py`	les témoins de F1 (en-tête, table)
`mesures/moteur/F/casser_copie.py`	C1 sur une vraie copie : une médiane faussée doit être détectée
`mesures/moteur/F/casser_lecture.py`	la lecture sans tampon du codeur
`mesures/moteur/E/temoins_des_temoins.py`	chaque témoin de `trc-cuda` et de `trc-es`
`mesures/moteur/O/casser_mediane.py`	C3 : une seule médiane d'un seul expert change dans une copie du 20b
`mesures/moteur/O/casser_tests.py`	chaque témoin unitaire de `trc-cache`, dans une copie jetable de la crate
`mesures/moteur/O/corrections/temoin_garde.py`	la garde de production, par `--casser-garde` et `--sans-garde`
`memoire/outils/tome2_reference_cache.py`	`--casser` : les chiffres refaits des chapitres de la référence et du cache
`memoire/outils/tome2_annexes.py`	`--casser` : le témoin des options de l'chapitre B

La ronde de corrections a poussé la règle d'un cran : la cassure d'un témoin de correction, c'est *l'ancien binaire*. Avant de corriger un constat, le lot O l'a rejoué sur une copie figée du programme d'avant (commit `7b288a6`) ; le témoin devait tomber sur l'ancien et tenir sur le nouveau (section 9.9).

A.7 L'épreuve indépendante

Le protocole est écrit au § 8 de l'architecture, avant toute construction, et suivi au chapitre 9. Le juge n'a écrit ni la spécification ni le code ; il travaille depuis un clone propre (commit `527dff5`), sur secteur, carte libre, et ne lit que la spécification, le code et les journaux. Ses sept gestes : (1) un décodeur écrit depuis le seul § 4, sur 20 tronçons tirés au hasard (graine publiée) ; (2) chaque cassure rejouée ; (3) C3, C4, C5, C7, C8, C10 refaits en séries ABBA avec ses propres commandes, jugés par la règle (A.1) ; (4) la base relancée, et une autre configuration de son choix ; (5) la fenêtre : 1 Gio de cache tiède doit apparaître dans la mémoire de pointe, entre 0,9 et 1,1 fois ; (6) le texte lu ; (7) le verdict, avec sa propre section « Ce qui pourrait être faux ». Ses seuils sont dans `mesures/moteur/epreuve-J1/seuils-declares.md`, commité avant sa première passe (tableau A.2).

A.8 La ronde de corrections, bloc par bloc

Chaque constat de l'épreuve a reçu un bloc de seuils, commité avant la première passe qu'il juge (`mesures/moteur/O/corrections/seuils-declares.md`). Tableau A.5 en donne le contenu et l'issue ; le détail est au section 9.9.

Table A.5 : Les six blocs de seuils de la ronde de corrections (16 octobre 2026) et leur issue (`mesures/moteur/0/corrections/seuils-declares.md`, `moteur/REPRISE-0.md`).

Bloc	Seuil déclaré	Issue
1 : G1, G2	sur l'ancien binaire, les trois cas (fichier absent, corrompu, processus tué) doivent tomber ; sur le nouveau, retour en moins de 30 s, code 1, erreur écrite, et 0 enfant `nvidia-smi` vivant 2 s après	tombe sur l'ancien (retour à 90 s, un orphelin chacun) ; tient sur le nouveau (1,5, 4,9 et 4,9 s, 0 orphelin), après deux défauts du témoin lui-même, dits et corrigés
2 : m2	avec et sans la recette `vcvars` du SOCLE, la construction de `trc-noyaux` réussit et les trois images `noyaux.fatbin` sont égales octet pour octet	tient : même SHA-256 (`aeae5bdc…`, 295 864 octets)
3 : M1	la garde de production reste active par défaut si C5 tient avec elle ; son coût est réel selon la règle (A.1) ; prédiction [calc.] : non réel (au plus 2,4 %, déclarée avec 1,1 Go/s et 460 ms ; 2,5 % au débit le plus lent mesuré, 1,07 Go/s, section 8.5.5)	les témoins de justesse tiennent (G-XO3, G-T, G-L : erreur fatale nommée, et aucune avec `--sans-garde` ; G-= : même texte au bit avec et sans garde) ; coût non réel en série ABBA (−12,3 % pour un seuil de 26,8 %), −1,6 % à disque égal (section 8.5.5) ; garde active par défaut
4 : G1 sous charge	la passe figée de l'épreuve, rejouée sous la même charge, finit ou écrit son erreur en moins de 30 min, jamais une attente muette	les deux binaires, nouveau et ancien, finissent ; 0 conflit d'arrivée observé : la cause première de G1 n'est pas établie
5 : M5	hypothèses H5a à H5c sur le cache KV en f32 (premier jeton de C12, accord de routage, KL) ; coût en série ABBA	mesuré : le KV en f32 rejoint la vérité au format près (section 6.6) ; décision de la couture N laissée à l'arbitre ; défaut inchangé (f16)
6 : M2	base llama.cpp balayée en palindrome ; C6 recalculé contre la meilleure ; ~~×3,85~~ barré partout	meilleure base `fit -t 16`, 0,884 jeton/s ; C6 ×2,22 garde comprise, tient (section 7.11)

A.9 La reproduction des constructions

Un chiffre publié n'est reproductible que si le programme qui l'a produit se reconstruit. Trois règles l'assurent. *Une cible par lot et par juge* : chaque construction écrit dans son propre dossier de sortie, hors du dépôt (`mesures/moteur/E/construire.bat`, `mesures/moteur/epreuve-J1/construire.bat`), pour que deux lots ne s'écrasent pas. *Le juge part d'un clone propre*, au commit nommé (`527dff5`, construit en 21 s). *La recette de la machine est éprouvée par un témoin* : l'épreuve a trouvé que la recette du SOCLE (`vcvars64.bat` d'abord) cassait la construction des noyaux (constat m2) ; depuis la correction, le bloc 2 de la ronde exige que la construction avec et sans elle donne la même image des noyaux à l'octet près (`mesures/moteur/0/corrections/construire-avec-vcvars.bat`).

Ce qui pourrait être faux

- **Le bruit de référence de 10 %** vient d'une mesure du Tome I (lecture de 4 Mio en profondeur 4) ; les tronçons du 120b, de 12 Mo environ, sont lus d'une seule requête chacun. Le facteur 0,8 en hérite sans avoir été remesuré pour cette longueur ; le chapitre 4 montre que la longueur change le régime du disque.
- **Le lemme du palindrome suppose une dérive linéaire.** Le disque chiffré de la machine varie de 1,51 à 1,95 Go/s d'une passe à l'autre dans une même séance (section 8.5.5) : ce n'est

pas une dérive, c'est une dispersion, que le palindrome ne retire pas et que seule la règle couvre.
- **Le verrou est une convention, pas un mécanisme du système** : un programme qui ne le lit pas peut prendre la carte. Les pilotes relèvent la VRAM avant chaque passe pour le voir, mais un usage bref entre deux relevés passerait inaperçu.
- **C6 est jugé à travers deux séances** (troncons gardé dans celle de la ronde, la base dans la suivante) ; le lemme ne protège pas une comparaison entre séances. La série unique est à faire (chapitre 10).

Annexe B

Outils du moteur

D'où l'on part, ce que le chapitre établit, ce qui reste

D'où l'on part. L'annexe B du Tome I (« Outils ») : chaque outil des études, rangé par étude, avec la première phrase de sa propre description, et la chaîne qui construit le mémoire. **Ce que l'annexe établit.** La même chose pour le système : la chaîne qui construit ce tome et vérifie ses chiffres ; le programme `troncons`, ses quatre sous-commandes et chacune de ses options, tenues à jour par un témoin ; chaque outil commité des cinq lots, de l'épreuve et de la ronde ; chaque figure et les fichiers qu'elle lit ; chaque dossier de journaux. **Ce qui reste.** Les options d'essai (`--kv-f32`, `--longueur-lecture`) attendent la décision de l'arbitre pour devenir des défauts ou disparaître.

B.1 La chaîne du Tome II

Le Tome II se construit comme le premier, par le même script et la même configuration de `latexmk` (`memoire/construire.sh`, `memoire/latexmkrc`) :

- `./construire.sh tome2` (ou `tome2-en`, `tome2-fr`) régénère les données des figures du Tome II, refait ses chiffres par un second chemin, régénère les tables de cette annexe, puis construit les deux PDF par `latexmk` (lualatex, biber) dans `memoire/tome2/en/` et `memoire/tome2/fr/` ; `./construire.sh` sans argument construit les deux tomes. Le script échoue à la première erreur, à toute référence ou citation manquante, à toute entrée que biber ne trouve pas ou trouve deux fois, et à tout outil de données qui sort en erreur.
- `memoire/outils/tome2_format.py` : les CSV des chapitres « Architecture » et « Format » (lignes de code et témoins par membre, taux par tenseur, tronçons par couche, pointes de mémoire du codeur), tirés des `Cargo.toml` et des journaux du lot F, et les calculs du chapitre du format.
- `memoire/outils/tome2_chemin_noyaux.py` : les CSV des chapitres « Le chemin des octets » et « Les noyaux », tirés des journaux des lots E, K et O.
- `memoire/outils/tome2_reference_cache.py` : les CSV des chapitres « La référence » et « Le cache », et 31 chiffres publiés refaits depuis les journaux ; il s'arrête au premier écart, et `--casser` doit le faire tomber.
- `memoire/outils/tome2_resultats.py` : les CSV des chapitres « Résultats », « L'épreuve » et « Discussion » ; il relit chaque passe, recalcule moyennes, bruits, rapports, intervalles de Wilson et planchers, et sort en erreur si un recalcul s'écarte d'un chiffre publié.
- `memoire/outils/tome2_annexes.py` : les tables de cette annexe et celle des seuils de l'chapitre A, depuis `git ls-files` et `git log` ; témoin : les options de tableau B.1 contre celles que lit le code.

B.2 Le programme `troncons`

Le programme (`moteur/bins/troncons/src/main.rs`, lot O) a quatre sous-commandes. `generer` préremplit l'invite, puis produit n jetons gloutons, et écrit le texte et un journal JSON (commande, date, alimentation, compteurs du moteur, jetons/s, mémoire et VRAM de pointe, octets lus sur le disque par le compteur du système et par le moteur, empreinte du texte). `mesurer` est `generer` avec dix secondes de repos avant la passe, pour chiffrer le disque au repos. `evaluer` passe les fenêtres d'un fichier de base `.kld` de llama-perplexity (mêmes jetons, chacune depuis la position 0), fait passer les logits notés par la même quantification à 16 bits que llama.cpp (critère C4) et écrit l'empreinte64 des logits de chaque pas (critère C3). `disque` mesure l'étage disque seul, dans la même séance que le moteur : n tronçons tirés sans remise par une graine publiée, lus sans tampon à la longueur du moteur, avec `profondeur` lectures en vol. Tableau B.1 donne chaque option.

Table B.1 : Les options du programme `troncons`, telles que les lit `moteur/bins/troncons/src/main.rs` (vérifié par `memoire/outils/tome2_annexes.py` : aucune en trop, aucune en moins).

Option	Défaut	Effet
Le moteur (toutes les sous-commandes qui l'ouvrent)		
`--modele` *f.trc*	exigée	le fichier `.trc` servi
`--ram-tiede` *Go*	0 (mode A0)	cache tiède : tronçons codés gardés en mémoire épinglée
`--enchassement-en-ram`	non	garder la table d'enchâssement (0,6 Go) en mémoire épinglée au lieu d'en lire une ligne par jeton
`--profondeur` *n*	4	lectures confiées au système à la fois
`--couches-d-avance` *d*	3	couches dont les tampons d'arrivée tournent (fenêtre de $d \cdot k$ tampons)
`--contexte` *n*	4 096	positions au plus (cache KV des couches pleines)
`--temoin`	non	chaque expert décodé redescend vers l'hôte, son empreinte comparée au répertoire (C2)
`--brut`	non	le fichier attendu est brut (drapeau bit 2) ; il doit l'être
`--emplacements` *n*	selon la VRAM	emplacements d'experts en mémoire vidéo
`--vram-plafond` *Mio*	7 179	plafond de la VRAM totale pour calculer les emplacements : 8 188 − 909 − 100
`--longueur-lecture` *o*	celle du format	longueur de chaque lecture de tronçon, multiple de 4 096 (essai : section 4.9)
`--chronogramme` *f.csv*	aucun	dates de chaque étage de chaque défaut
`--prechargement`	non	J1+ : lecture anticipée des experts prédits par le routeur de la couche suivante (essayée, abandonnée)
`--sans-battement`	battement	retire la ligne écrite à chaque couche du préremplissage par flux (constat G1)
`--sans-garde`	garde	retire la garde de production ; sert seulement à chiffrer son coût
`--casser-garde` *troncon:N* | *ligne:N*	aucune	témoin de la garde : retourne un bit du N-ième tronçon ou de la N-ième ligne lus
`--kv-f32`	f16	essai : cache KV en f32 (constat M5 ; couture N à l'arbitre)
`generer`, `mesurer`		
`--vocab` *f.vocab.gguf*	exigée	le compagnon du découpeur

Option	Défaut	Effet
`--invite` *f.txt*	exigée sans `--harmonie`	l'invite, en UTF-8 (son SHA-256 va au journal)
`--harmonie` *« question »*	aucune	une invite au format de conversation harmony de gpt-oss
`--systeme` *texte*	message fixe	le message système de l'invite harmony
`--n` *n*	256	jetons produits (`disque` : tronçons lus, 1 000)
`--repos` *s*	0 ; 10 (`mesurer`)	secondes de repos avant la passe, pour chiffrer le disque au repos
`--dll` *f.dll*	llama.cpp b11515	la bibliothèque du découpeur
`--arret-fin`	non	s'arrêter au jeton de fin de la conversation harmony
`--sans-vram`	VRAM échantillonnée	ne pas échantillonner la VRAM
`--flux`	boucle de pas	préremplissage par flux (J1+) : chaque expert lu une fois pour toutes les positions
`--journal` *f.json*	aucun	le journal de la passe
`--sortie` *f*	aucune ; exigée par `evaluer`	le texte produit ; pour `evaluer`, le `.kld` écrit
`--trace` *f.trace*	aucune	la trace du cache (pour `rejouer`)
`evaluer`		
`--base` *f.kld*	exigée	le fichier de base de llama-perplexity (jetons et logits)
`--fenetres` *n*	toutes	fenêtres évaluées
`--positions` *n*	toutes (512)	positions évaluées par fenêtre
`--empreintes` *f.csv*	aucun	l'empreinte64 des logits f32 de chaque pas (C3)
`disque`		
`--graine` *g*	1	graine du tirage des tronçons (xorshift64*, celui des essais du lot E)

B.3 Les outils commités

Tout nombre [mes.] ou [calc.] de ce tome sort d'un outil commité. Tableau B.2 les liste, rangés par lot, chacun avec la première ligne de sa propre description, citée telle qu'écrite (en français), et son nombre de lignes. La table est engendrée par `memoire/outils/tome2_annexes.py` depuis `git ls-files` : elle suit le dépôt.

Table B.2 : Les outils du moteur et de ses mesures, par lot.

Outil	Sa propre description (première ligne, citée)	Lignes
O : le programme troncons		
`moteur/bins/troncons/src/main.rs`	troncons : le programme (ARCHITECTURE.md § 5.10).	683
F : le codeur		
`moteur/bins/trc-codeur/src/main.rs`	trc-codeur : le codeur GGUF → .trc de troncons et la vérification C1 (lot F).	125
E : le banc du chemin		
`moteur/bins/banc-chemin/src/main.rs`	banc-chemin : le banc du lot E.	223

Outil	Sa propre description (première ligne, citée)	Lignes
K : le banc des noyaux		
`moteur/crates/trc-noyaux/banc/banc_k.cu`	Banc du lot K (étapes K1 et K2) : témoins, cassures et débits des noyaux du moteur, sur les poids réels de gpt-oss, par un chemin indépendant des lots F et E.	1 376
`moteur/crates/trc-noyaux/banc/construire.bat`	Construit le banc du lot K (banc_k.exe) et l'image des noyaux seule, hors du depot. CUDA 13.1.2 depaquete sans installation (SOCLE, section La machine). …	12
O : le simulateur du cache		
`moteur/crates/trc-cache/src/bin/rejouer.rs`	rejouer : le simulateur du cache en ligne de commande (témoin O1, courbes de succès).	119
O : exemples du moteur		
`moteur/crates/trc-ordo/examples/debit_empreinte.rs`	Débit de empreinte64 (préfixe de SHA-256, crate sha2 0.10.9) sur des tampons de la taille d'un tronçon, sur un fil puis sur n fils …	47
`moteur/crates/trc-ordo/examples/memoire_unitaire.rs`	Coûts unitaires en ensemble de travail (WS) des structures du moteur qui croissent avec le modèle (constat M4 de l'épreuve J1 : le terme de 0,5 Mo de C7 ne comptait que le répertoire). …	37
R : le programme du juge		
`moteur/crates/trc-juge/src/bin/juge.rs`	juge : le programme du lot R.	572
E : mesures du chemin des octets		
`mesures/moteur/E/analyse_e0.py`	Analyse d'une séance E0 (journaux de banc-chemin) : verdicts contre les seuils déclarés, CSV.	216
`mesures/moteur/E/analyse_e1.py`	Analyse de la mesure E1 du lot E (débit du lecteur, disque seul) : verdicts contre les seuils déclarés avant la première passe mesurée (moteur/REPRISE-E.md, « E1, débit du lecteur sur le …	117
`mesures/moteur/E/analyse_e2.py`	Analyse de l'étape E2 du lot E : verdicts contre les critères déclarés avant toute ligne de code d'E2 (moteur/REPRISE-E.md, « E2 », commit 6b59ff3), CSV des figures du mémoire.	211
`mesures/moteur/E/balayage_longueurs.py`	Balayage du débit du lecteur sans tampon contre la longueur d'une lecture (lot E, diagnostic d'E2).	115
`mesures/moteur/E/construire.bat`	Construit banc-chemin (lot E) en release, cible propre au lot, -j 4 (cinq lots compilent en meme temps). Argument 1 (facultatif) : (rien) …	32
`mesures/moteur/E/pilote_e0.py`	Pilote des essais E0 du lot E (ARCHITECTURE.md § 6.5) : une séance complète, journalisée.	192
`mesures/moteur/E/pilote_e1.py`	Pilote de la mesure E1 du lot E : débit du lecteur sans tampon, disque seul (banc-chemin lecteur).	191
`mesures/moteur/E/pilote_e2.py`	Pilote de l'étape E2 du lot E : le chemin disque → VRAM décodé sur gpt-oss-20b (banc-chemin chemin).	184
`mesures/moteur/E/serie_nvvn.py`	Série N V V N du lot E (déclarée dans moteur/REPRISE-E.md avant la mesure) : banc-chemin chemin tel que spécifié (N, lecture de longueur0) contre la variante E2-V (V, –longueur-lecture), …	84

Outil	Sa propre description (première ligne, citée)	Lignes
mesures/moteur/E/temoins_des_temoins.py	Témoin des témoins de l'étape E1 (lot E) : chaque témoin de trc-cuda et de trc-es doit tomber quand on force sa condition à faux, puis tenir une fois restauré (SOCLE, « Le niveau exigé »).	67
F : mesures du format		
mesures/moteur/F/casser_copie.py	Cassure C1 sur une vraie copie du fichier (§ 6.5 : « une médiane faussée dans une copie ⟶ inexact détecté »). …	36
mesures/moteur/F/casser_lecture.py	Témoin des témoins de la lecture sans tampon de trc-codeur (gguf::lire_dans), F3.	83
mesures/moteur/F/casser_temoins_f1.py	Témoin des témoins de F1 (SOCLE : « force sa condition à faux, relance, vérifie que l'instrument tombe, puis restaure »).	72
mesures/moteur/F/comparer_f1.py	Juge des taux du lot F contre les seuils déclarés (mesures/moteur/F/00-seuils-declares.md).	70
mesures/moteur/F/lancer.ps1	Lance trc-codeur en priorité « sous la normale » (machine partagée), sorties redirigées, et note l'alimentation (Win32_Battery.BatteryStatus : 2 = sur secteur) avant et après. Usage …	17
mesures/moteur/F/lancer_py.ps1	Lance un outil Python du lot F en priorité « sous la normale » (machine partagée), sorties redirigées sous journaux/, alimentation (Win32_Battery.BatteryStatus …	18
mesures/moteur/F/second_chemin_c1.py	Second chemin de C1 (lot F), sur le fichier .trc et son GGUF source.	684
mesures/moteur/F/vers_csv.py	Données des figures du lot F, tirées des journaux de trc-codeur verifier (les octets viennent des fichiers .trc eux-mêmes) …	83
K : mesures des noyaux		
mesures/moteur/K/attendre_et_mesurer.ps1	Attend que la carte soit libre (aucun llama-*, aucun traceur, aucun verrou), puis lance le pilote des bancs de débit du lot K (pilote_bancs.ps1, qui revérifie tout et prend le verrou). …	30
mesures/moteur/K/k3_biais_attention.py	—	29
mesures/moteur/K/k3_vers_csv.py	K3 : les CSV des figures du mémoire, tirés des journaux commités sous k3/ (aucun chiffre recopié).	69
mesures/moteur/K/pilote_bancs.ps1	Pilote des bancs de débit du lot K (étapes K1 et K2), en séries A B B A de processus.	105
mesures/moteur/K/temoin-k3/src/chaine.rs	Mode chaine : la passe avant entière de gpt-oss enchaînée sur la carte par les seuls noyaux de K1 à K3, contre Reference::evaluer (f64), sur des jetons réels.	376
mesures/moteur/K/temoin-k3/src/main.rs	Témoin K3 : les douze noyaux de la passe avant, chacun contre trc-reference.	1 171
mesures/moteur/K/temoin-trc/src/main.rs	Témoin K1 sur un vrai fichier .trc du lot F.	432
mesures/moteur/K/vers_csv.py	Données des figures et tableaux du lot K pour le mémoire (CSV), tirées des journaux commités.	137
O : ronde de corrections		
mesures/moteur/O/corrections/analyse_base.py	Bloc 6 de seuils-declares.md : la base llama.cpp balayée (M2), et C6 recalculé.	71

Outil	Sa propre description (première ligne, citée)	Lignes
`mesures/moteur/O/corrections/analyse_kv.py`	Bloc 5 de seuils-declares.md (M5) : le cache KV en f32 contre f16, jugé contre la vérité f64 du juge.	114
`mesures/moteur/O/corrections/analyse_ronde.py`	Analyse de la séance « vitesse » de la ronde de corrections (blocs 3 et 5 de seuils-declares.md).	102
`mesures/moteur/O/corrections/construire-avec-vcvars.bat`	Ronde de corrections J1, constat m2 : la recette du SOCLE (vcvars64.bat d abord) sur trc-noyaux, arbre de travail du depot, cible a part. Usage : construire-avec-vcvars.bat [sans]	16
`mesures/moteur/O/corrections/pilote_ronde.py`	Pilote des séances de la ronde de corrections J1 (blocs 3, 5 et 6 de seuils-declares.md).	100
`mesures/moteur/O/corrections/rejouer_g1.py`	Bloc 4 de seuils-declares.md : la passe figée de l'épreuve (G1) rejouée sous la même charge.	91
`mesures/moteur/O/corrections/temoin_garde.py`	Témoins de la garde de production (constat M1 de l'épreuve J1), sur le 20b, passes courtes.	104
`mesures/moteur/O/corrections/temoin_sortie.py`	Témoin des constats G1 et G2 de l'épreuve J1 : le programme rend-il la main, avec son erreur, sans laisser d'enfant vivant ?	136
O : mesures du moteur		
`mesures/moteur/O/analyse_o4.py`	Verdicts de la séance O4 = J1 (C5, C7, C8, C10) depuis les journaux de pilote_o4.py.	89
`mesures/moteur/O/casser_mediane.py`	Cassure de C3 (lot O) : une copie de gpt-oss-20b.trc où une seule médiane d'un seul expert change.	62
`mesures/moteur/O/casser_tests.py`	Cassures des témoins unitaires de trc-cache : chaque témoin qui peut tomber est forcé à faux une fois (SOCLE, « le niveau exigé »), dans une copie jetable de la crate, et l'on vérifie qu'il …	137
`mesures/moteur/O/courbe_dense.py`	Courbe dense du succès du cache global (LRU et Belady) contre le nombre d'emplacements, sur les traces réelles de gpt-oss-20b, par rejouer (trc-cache) …	66
`mesures/moteur/O/pilote_j1plus.py`	Pilote de la seconde séance du 120b (lot O) : diagnostic D-C7, variante L12, préchargement (J1+), C12 sur le fichier brut, et la conversation harmony publiée.	199
`mesures/moteur/O/pilote_o3.py`	Pilote de l'étape O3 du lot O : C3 (codé contre brut, au bit), le compte du cache contre rejouer.	201
`mesures/moteur/O/pilote_o4.py`	Pilote de l'étape O4 = J1 du lot O : vitesse (C5), fenêtre bornée (C7), octets lus (C8), VRAM (C10).	126
`mesures/moteur/O/temoin-o2/src/main.rs`	Témoin O2 : la passe avant de trc-modele contre la chaîne du lot K, ligne pour ligne.	273
`mesures/moteur/O/temoin_o1.py`	Témoin O1 du lot O : le simulateur du moteur (trc-cache, rejouer) contre les simulateurs Python des études 04 et gpt-oss, compté à l'unité (ARCHITECTURE.md § 6.5, ligne O1).	236
`mesures/moteur/O/vers_csv_o.py`	Données des figures du mémoire pour les étapes O2 à O4 du lot O, tirées des journaux commités.	81
R : mesures de la référence		
`mesures/moteur/R/bascules.py`	Les bascules de routage entre deux passes de la référence (lot R, R1, reprise du 2026-10-15).	90
`mesures/moteur/R/r0_ecart.py`	R0 : l'écart entre moteurs sur gpt-oss-20b (ARCHITECTURE.md § 6.5, lot R).	295
`mesures/moteur/R/r1_reference.py`	R1 : la référence f64 de gpt-oss, et son jugement contre llama.cpp (ARCHITECTURE.md § 6.5, lot R).	226

Outil	Sa propre description (première ligne, citée)	Lignes
`mesures/moteur/R/rampe_yarn.py`	Second chemin de la rampe de YaRN de gpt-oss (figure R-M3 du mémoire ; lot R).	69
Épreuve J1 (le juge indépendant)		
`mesures/moteur/epreuve-J1/analyse_c8.py`	C8 refait par le juge, deux instruments.	36
`mesures/moteur/epreuve-J1/analyse_seance.py`	Analyse de la séance de l'épreuve (pilote_epreuve.py) : verdicts des seuils de seuils-declares.md.	134
`mesures/moteur/epreuve-J1/base_kld_invite.py`	Fabrique un .kld de base d'UNE fenêtre de n jetons donnés (format de llama-perplexity), pour que troncons evaluer passe ces jetons depuis la position 0 et écrive l'argmax de chaque …	22
`mesures/moteur/epreuve-J1/comparer_c3.py`	C3 par le chemin du juge : .kld et empreintes de logits, codé contre brut, et contre le lot O.	61
`mesures/moteur/epreuve-J1/comparer_invite.py`	Argmax position par position : vérité numpy (argmax_toutes) contre l'empreinte CSV de troncons.	39
`mesures/moteur/epreuve-J1/comparer_kld.py`	Juge de l'épreuve : KL(vérité ‖ fichier) et premier choix, position par position.	99
`mesures/moteur/epreuve-J1/comparer_routage.py`	Routage de troncons (trace du moteur, une ligne par pas, L groupes de k experts) contre celui de la vérité numpy (routage[L, P, k]) …	31
`mesures/moteur/epreuve-J1/construire-avec-vcvars.bat`	Epreuve J1, constat m2 : la recette du SOCLE (vcvars d'abord) sur la seule crate trc-noyaux.	9
`mesures/moteur/epreuve-J1/construire.bat`	Epreuve J1 : construit troncons depuis un clone propre, cible propre au juge, -j 4.	10
`mesures/moteur/epreuve-J1/decodeur_s4.py`	Épreuve indépendante du jalon J1, geste 1 (decisions/ARCHITECTURE.md § 8.1).	613
`mesures/moteur/epreuve-J1/ecart_logp.py`	Écart des log-probabilités entre la vérité numpy du juge et un .kld, hors plancher du format.	43
`mesures/moteur/epreuve-J1/echantillonner_fil.py`	Diagnostic d'un processus bloqué (lancé par le juge) : où tourne le fil donné ?	63
`mesures/moteur/epreuve-J1/gptoss_numpy.py`	Une troisième passe avant de gpt-oss, en numpy f64, écrite par le juge de l'épreuve J1.	283
`mesures/moteur/epreuve-J1/normalisation_kld.py`	Témoin de l'instrument du juge : log Σ exp(log q) de chaque entrée d'un .kld (0 si normalisé).	46
`mesures/moteur/epreuve-J1/pile_fil.py`	Diagnostic d'un fil bloqué : adresses de retour de troncons.exe trouvées sur sa pile, nommées.	89
`mesures/moteur/epreuve-J1/pilote_epreuve.py`	Épreuve indépendante de J1 (§ 8.3 à 8.5) : vitesse, RAM, VRAM et base llama.cpp, par mes commandes.	304
`mesures/moteur/epreuve-J1/temoin_compteurs_es.py`	Témoin de l'instrument C8 du juge : GetProcessIoCounters compte-t-il une lecture sans tampon ?	61
`mesures/moteur/epreuve-J1/verdict_c4.py`	C4 par la vérité du juge : comparaison appariée (troncons contre llama.cpp, mêmes positions).	35
`mesures/moteur/epreuve-J1/verifier_vocab.py`	§ 4.13 : le compagnon <nom>.vocab.gguf est un GGUF v3 valide, n_tenseurs = 0, toutes les paires du source octet pour octet, aucune donnée. Lecteur GGUF de decodeur_s4.py (écrit ici). …	19
Le mémoire du Tome II		

Outil	Sa propre description (première ligne, citée)	Lignes
`memoire/outils/tome2_annexes.py`	Produit les tables des annexes du Tome II depuis le dépôt lui-même, et vérifie l'annexe des outils.	263
`memoire/outils/tome2_chemin_noyaux.py`	Données des figures des chapitres « Le chemin des octets » et « Les noyaux sur la carte » (Tome II).	249
`memoire/outils/tome2_format.py`	Données des chapitres « Architecture » et « Format .trc » du Tome II (memoire/tome2/).	158
`memoire/outils/tome2_juge.py`	Second chemin du juge technique du Tome II (memoire/tome2/JUGEMENT.md).	152
`memoire/outils/tome2_reference_cache.py`	Données des figures et chiffres refaits des chapitres « La référence exacte et le juge » et « Le cache, l'ordonnanceur et le moteur » du Tome II.	279
`memoire/outils/tome2_reprise.py`	Reprise du jugement du Tome II : refait, depuis les CSV commités, chaque chiffre que la reprise publie ou corrige (objections G1 et m1 à m21 de memoire/tome2/JUGEMENT.md), et vérifie dans …	175
`memoire/outils/tome2_resultats.py`	Données des chapitres « Résultats », « L'épreuve » et « Discussion » du Tome II.	332
Total	83 outils	14 504

B.4 Les figures et leurs données

Chaque graphe pgfplots du tome lit un fichier commité ; chaque schéma TikZ est un fichier sous `memoire/figures/tome2/`. Tableau B.4 donne, pour chaque commande de figure qui lit des données, les fichiers qu'elle lit ; l'outil qui l'engendre sort en erreur si l'un d'eux manque.

Table B.4 : Les figures du Tome II et les fichiers de données qu'elles lisent.

Figure (commande)	Fichiers de données qu'elle lit
`\GrapheCchFalaise`	`mesures/moteur/O/csv/falaise-20b-dense.csv`
`\GrapheCchProp`	`memoire/figures/tome2/donnees/t2o-prop1.csv`
`\GrapheCchSucces`	`mesures/localite-gpt-oss/figures/succes-gpt-oss-20b-lecture.csv` `mesures/localite-gpt-oss/figures/succes-gpt-oss-120b-lecture.csv`
`\GrapheCchChrono`	`memoire/figures/tome2/donnees/t2o-chrono-couche.csv`
`\GrapheCchVitesse`	`memoire/figures/tome2/donnees/t2o-c6.csv`
`\GrapheChmSync`	`memoire/figures/tome2/donnees/t2c-sync.csv`
`\GrapheChmDma`	`memoire/figures/tome2/donnees/t2c-dma.csv`
`\GrapheChmEun`	`memoire/figures/tome2/donnees/t2c-e1-s1.csv` `memoire/figures/tome2/donnees/t2c-e1-s3.csv`
`\GrapheChmLittle`	`memoire/figures/tome2/donnees/t2c-little.csv`

`\GrapheChmEdeux`	`memoire/figures/tome2/donnees/t2c-e2-n.csv`
	`memoire/figures/tome2/donnees/t2c-e2-v.csv`
	`memoire/figures/tome2/donnees/t2c-nvvn.csv`

`\GrapheChmEtages`	`memoire/figures/tome2/donnees/t2c-etages.csv`

Figure (commande)	Fichiers de données qu'elle lit
\GrapheChmChrono	mesures/moteur/E/journaux/e2-2026-10-15-seance1/chrono-1/chrono-A-t1-r0.csv mesures/moteur/E/journaux/e2-2026-10-15-v12/chrono-v12582912-1/chrono-A-t1-r0.csv
\GrapheChmLongueurs	mesures/moteur/E/csv/e2-balayage-2026-10-15-longueurs.csv
\GrapheChmCouche	memoire/figures/tome2/donnees/t2c-couche.csv
\GrapheChmLdouze	memoire/figures/tome2/donnees/t2c-l12.csv
\TdArchGrapheLignes	memoire/figures/donnees/tome2/caisses.csv
\TdFmtGrapheTaux	mesures/moteur/F/csv/taux-par-tenseur-120b.csv mesures/moteur/F/csv/taux-par-tenseur-20b.csv
\TdFmtGrapheTroncons	mesures/moteur/F/csv/troncons-par-couche-120b.csv mesures/moteur/F/csv/troncons-par-couche-20b.csv
\TdFmtGraphePointes	mesures/moteur/F/csv/pointes-memoire.csv
\GrapheNoyDecodeur	memoire/figures/tome2/donnees/t2n-k1.csv
\GrapheNoyDecomposition	mesures/moteur/K/csv/k1-decomposition.csv
\GrapheNoyGemv	memoire/figures/tome2/donnees/t2n-k2.csv
\GrapheNoyEcarts	mesures/moteur/K/csv/k2-ecarts-reels.csv
\GrapheNoyMarges	memoire/figures/tome2/donnees/t2n-marges.csv
\GrapheNoyChaine	mesures/moteur/K/csv/k3-chaine-kl.csv
\GrapheNoyFenetre	mesures/moteur/K/csv/k3-chaine-kl.csv
\GrapheNoyRoutage	memoire/figures/tome2/donnees/t2n-routage.csv
\GrapheNoyKV	memoire/figures/tome2/donnees/t2n-kv-120b.csv
\GrapheRefRampe	mesures/moteur/R/r1/csv/rampe-yarn.csv
\GrapheRefRepartition	memoire/figures/tome2/donnees/t2r-kl-repartition.csv
\GrapheRefBascules	memoire/figures/tome2/donnees/t2r-bascules-autres.csv memoire/figures/tome2/donnees/t2r-bascules-soi.csv
\GrapheRefKv	memoire/figures/tome2/donnees/t2r-kv-glissant.csv memoire/figures/tome2/donnees/t2r-kv-invite-glissant.csv
\GrapheRefRoutage	mesures/moteur/epreuve-J1/csv/routage-par-couche-invite.csv
\GrapheResVitesse	memoire/figures/tome2/donnees/res-vitesse-120b.csv
\GrapheResPlancher	memoire/figures/tome2/donnees/res-plancher.csv
\GrapheResMemoire	memoire/figures/tome2/donnees/res-memoire.csv

Figure (commande)	Fichiers de données qu'elle lit
`\GrapheResKL`	`memoire/figures/tome2/donnees/res-kl-120b-f0.csv` `memoire/figures/tome2/donnees/res-kl-invite.csv`
`\GrapheResRoutage`	`memoire/figures/tome2/donnees/res-routage.csv`
`\GrapheResSeance`	`memoire/figures/tome2/donnees/res-epreuve-seance.csv`

B.5 Les journaux

Les journaux bruts, les CSV et les notes de chaque lot sont commités sous `mesures/moteur/`. Tableau B.6 en donne l'étendue, dossier par dossier (les outils, comptés dans tableau B.2, en sont exclus). Les fichiers lourds restent hors du dépôt : les fichiers `.trc` et leurs copies cassées sous `~/dev/modeles/trc/`, les sorties des passes (`.kld`, `.npz`) sous `~/dev/essais/troncons/`, les programmes construits sous `~/dev/cibles/` ; les journaux en gardent les empreintes SHA-256 et les commandes.

Table B.6 : Les journaux commités du moteur, par dossier.

Dossier (journaux, CSV, notes ; outils exclus)	Fichiers	Volume (Mo)
`mesures/moteur/E/csv/`	14	0,13
`mesures/moteur/E/journaux/`	223	1,82
`mesures/moteur/E/miroir/`	2	0,01
`mesures/moteur/F/`	6	0,04
`mesures/moteur/F/csv/`	9	0,03
`mesures/moteur/F/journaux/`	70	0,49
`mesures/moteur/K/`	23	0,12
`mesures/moteur/K/csv/`	9	0,03
`mesures/moteur/K/journaux/`	44	0,04
`mesures/moteur/K/k3/`	61	1,20
`mesures/moteur/K/temoin-k3/`	2	0,01
`mesures/moteur/K/temoin-trc/`	2	0,01
`mesures/moteur/O/corrections/`	94	0,82
`mesures/moteur/O/csv/`	8	0,30
`mesures/moteur/O/journaux/`	190	11,22
`mesures/moteur/O/temoin-o2/`	2	0,01
`mesures/moteur/R/r0/`	60	3,57
`mesures/moteur/R/r1/`	41	3,82
`mesures/moteur/R/r2/`	11	0,19
`mesures/moteur/epreuve-J1/`	2	0,01
`mesures/moteur/epreuve-J1/csv/`	4	0,05
`mesures/moteur/epreuve-J1/journaux/`	134	1,90
Total	1 011	25,79

Ce qui pourrait être faux

- **La première ligne d'une description n'est pas l'outil** : elle dit ce que son auteur voulait qu'il fasse. Les chapitres citent les outils par ce qu'ils ont produit, avec leurs journaux.
- **Le témoin des options compare des noms, pas des sens** : un défaut changé dans le code sans que tableau B.1 change passerait. Les défauts de la table ont été relus dans `moteur/crates/trc-ordo/src/lib.rs` (`Config::nouvelle`) et dans `main.rs` le 16 octobre 2026.
- **Les fichiers lourds hors du dépôt** ne sont reproductibles que par leurs empreintes et par les commandes des journaux ; leur dossier peut être libéré par Stone.

Bibliographie

[1] Open Compute Project. *OCP Microscaling Formats (MX) Specification, Version 1.0.* Définit MXFP4 : éléments E2M1, échelle partagée E8M0 par bloc de 32. 2023. URL : https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf (visité le 2026-10-15).

[2] S. Kullback et R. A. Leibler. « On Information and Sufficiency ». In : *The Annals of Mathematical Statistics* 22.1 (1951-03). La divergence d'information entre deux lois, positive et nulle seulement pour deux lois égales ; c'est la mesure du juge., p. 79-86. DOI : 10.1214/aoms/1177729694. URL : https://doi.org/10.1214/aoms/1177729694 (visité le 2026-10-16).

[3] J. D. C. Little. « A Proof for the Queuing Formula : $L = \lambda W$ ». In : *Operations Research* 9.3 (1961). Métadonnées relues par l'API de Crossref (l'éditeur refuse la lecture automatique). Second chemin du débit du lecteur : X = q b / R (dérivation D3 du lot E)., p. 383-387. DOI : 10.1287/opre.9.3.383. (Visité le 2026-10-15).

[4] NVIDIA Corporation. *CUDA Driver API Reference Manual, 6.27 Module Management.* Version 13.4. `cuModuleLoadData` accepte une image cubin ou fatbin produite par nvcc, ou du PTX terminé par un octet nul : c'est ce qui permet d'embarquer l'image des noyaux dans le binaire (`include_bytes!`) sans aucune DLL d'exécution de CUDA. 2026-10-12. URL : https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__MODULE.html (visité le 2026-10-16).

[5] The Cargo Team. *Workspaces, The Cargo Book.* Membres par motifs (`crates/*`), `Cargo.lock` et dossier de sortie communs, héritage des versions par `[workspace.dependencies]` et `workspace = true` ; ce que fait `moteur/Cargo.toml`. URL : https://doc.rust-lang.org/cargo/reference/workspaces.html (visité le 2026-10-16).

[6] Microsoft. *File Buffering.* FILE_FLAG_NO_BUFFERING : taille et décalage multiples du secteur du volume, adresse du tampon alignée sur le secteur physique ; VirtualAlloc aligne sur la page (4 096 sur x64). trc-es exige 4 096 pour les trois. 2025-05-31. URL : https://learn.microsoft.com/en-us/windows/win32/fileio/file-buffering (visité le 2026-10-15).

[7] ggml-org. *GGUF : file format specification.* https://github.com/ggml-org/ggml/blob/master/docs/gguf.md. Licence MIT (dépôt ggml) ; paires clé-valeur, types 0 à 12, descriptions des tenseurs, alignement `general.alignment` ; lu et réécrit de zéro par trc-format et trc-codeur ; non reconsulté par le lot F le 2026-10-15.

[8] J. Duda. *Asymmetric numeral systems : entropy coding combining speed of Huffman coding with compression rate of arithmetic coding.* 2013. arXiv : 1311.2540. URL : https://arxiv.org/abs/1311.2540 (visité le 2026-10-15).

[9] F. Giesen. *Interleaved entropy coders.* 2014. arXiv : 1402.3392. URL : https://arxiv.org/abs/1402.3392 (visité le 2026-10-15).

[10] Open Compute Project. *OCP Microscaling Formats (MX) Specification, Version 1.0*. Rapp. tech. MXFP4 : blocs de 32 éléments E2M1 et une échelle E8M0 ; non reconsulté par le lot F le 2026-10-15. 2023. URL : https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf.

[11] National Institute of Standards and Technology. *Secure Hash Standard (SHS), FIPS PUB 180-4*. Rapp. tech. SHA-256 ; empreinte64 = ses 8 premiers octets en petit-boutiste ; vecteur de test « abc » vérifié par le test empreinte64_connue ; non reconsulté le 2026-10-15. 2015. URL : https://doi.org/10.6028/NIST.FIPS.180-4.

[12] ggml-org. *llama.cpp b11515 : lecteur GGUF de ggml (gguf_init_from_file, ggml-base.dll)*. Version b11515. Licence MIT (vérifiée par l'étude 06). Binaire Windows CUDA 12.4 de la machine, empreinte vérifiée (attendu-llama.txt). Fonctions publiques appelées par ctypes depuis second_chemin_c1.py le 2026-10-15 (gguf_get_n_tensors, gguf_get_tensor_name, _offset, _type, _size, gguf_get_data_offset, gguf_get_alignment) : troisième lecture des décalages des tenseurs, aucune ligne copiée. URL : https://github.com/ggml-org/llama.cpp.

[13] NVIDIA. *CUDA Driver API Reference Manual*. Version 13.4. Gestion de la mémoire : cuMemHostAlloc (« Allocates page-locked host memory »), cuMemHostGetDevicePointer, copies asynchrones ; avertissement sur la mémoire épinglée en excès (« may degrade system performance »), sous cuMemAllocHost. Le moteur est construit contre les en-têtes de CUDA 13.1.2 et tourne sur le pilote 591.91 (13.1). URL : https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__MEM.html (visité le 2026-10-15).

[14] NVIDIA. *CUDA Driver API Reference Manual, Primary Context Management*. Version 13.4. cuDevicePrimaryCtxSetFlags : CU_CTX_SCHED_SPIN, YIELD, BLOCKING_SYNC ; AUTO attend activement tant que le nombre de contextes actifs ne dépasse pas celui des processeurs logiques (le cas de la machine : 1 contexte, 16 fils). Mesuré par l'essai E0(c). URL : https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__PRIMARY__CTX.html (visité le 2026-10-15).

[15] NVIDIA. *CUDA C++ Best Practices Guide*. Version 13.4. Recouvrement copie et calcul : asyncEngineCount ; un seul moteur de copie permet une copie asynchrone à la fois, en même temps que les noyaux ; deux moteurs permettent les deux sens à la fois. La carte de la machine rend 1 sous WDDM (relevé par trc-cuda). URL : https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html (visité le 2026-10-15).

[16] Microsoft. *I/O Completion Ports*. Paquets de complétion en file FIFO ; valeur de concurrence ; le bloc d'état n'est mis à jour qu'au retrait du paquet, sauf échec synchrone (d'où, dans trc-es, l'OVERLAPPED gardé jusqu'à la récolte, et l'annulation suivie de l'attente de chaque paquet). 2025-07-18. URL : https://learn.microsoft.com/en-us/windows/win32/fileio/i-o-completion-ports (visité le 2026-10-15).

[17] NVIDIA. *CUDA Programming Guide, 5.4 C/C++ Language Extensions*. Version 13.4.2. Fonctions de vote du warp (__ballot_sync, § 5.4.6.2 : « Evaluates predicate for all non-exited threads in mask »), d'échange (__shfl_sync, __shfl_xor_sync, § 5.4.6.5), bornes de lancement (__launch_bounds__, § 5.4.3.2). Le pas rANS du décodeur repose sur le vote (quelles voies renormalisent) et le comptage de bits (où chacune lit). URL : https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html (visité le 2026-10-15).

[18] C. Wellons. *Prospecting for Hash Functions*. Constantes de la fonction de mélange 32 bits « lowbias32 » (domaine public), utilisées par les noyaux d'essai pour tirer des indices d'experts que l'hôte sait recalculer. 2018-07-31. URL : https://nullprogram.com/blog/2018/07/31/ (visité le 2026-10-15).

[19] Microsoft. *Calculating Counter Values*. Pour PERF_COUNTER_BULK_COUNT, valeur affichée (N1 - N0) / ((D1 - D0) / F) : la donnée brute N est un compte cumulé (ici d'octets lus), d'où les octets d'un intervalle par simple différence de deux relevés, sans intégrer un débit échantillonné. 2025-07-14. URL : https://learn.microsoft.com/en-us/windows/win32/perfctrs/calculating-counter-values (visité le 2026-10-15).

[20] Microsoft. *PdhGetRawCounterValue function (pdh.h)*. Rend la valeur brute d'un compteur (structure PDH_RAW_COUNTER). Utilisé par banc-chemin (disque.rs) pour les octets lus par toute la machine pendant une passe. 2025-07-01. URL : https://learn.microsoft.com/en-us/windows/win32/api/pdh/nf-pdh-pdhgetrawcountervalue (visité le 2026-10-15).

[21] S. Pronovost. *Hardware Accelerated GPU Scheduling*. L'ordonnancement de la carte passe d'un fil du processeur à un processeur dédié de la carte ; les applications peuvent soumettre des lots plus petits plus souvent. Cadre de la variante « boîte sans vidage » de l'essai E0(c) (le pilote retient-il un lancement sans appel qui le pousse ?). Microsoft, DirectX Developer Blog. 2020-06-30. URL : https://devblogs.microsoft.com/directx/hardware-accelerated-gpu-scheduling/ (visité le 2026-10-15).

[22] NVIDIA. *NVIDIA Ada GPU Architecture Tuning Guide*. Version 13.4. Capacité de calcul 8.9 : « The shared memory capacity per SM is 100 KB », 99 Kio au plus par bloc (1 Kio réservé par bloc), 48 warps résidents au plus par SM, 64 K registres de 32 bits par SM. D'où l'occupation de decoder_codes (48 Kio de LUT par bloc de 8 warps : 2 blocs, 16 warps par SM) et le calcul de la vague unique. 2026-10-12. URL : https://docs.nvidia.com/cuda/ada-tuning-guide/index.html (visité le 2026-10-15).

[23] ggml-org. *llama.cpp, ggml/src/ggml-common.h : block_mxfp4 et kvalues_fp4*. Licence MIT (vérifiée par l'étude 06). QK_MXFP4 = 32 ; bloc de 17 octets : un octet E8M0 puis 16 octets de demi-octets ; table des valeurs 0, 1, 2, 3, 4, 6, 8, 12 et leurs opposées (grandeurs E2M1 doublées). Lu pour cadrer la disposition, rien de copié ; la GEMV de troncons la redérive (03-derivations § 1.2) et la juge contre une référence f64 écrite depuis cette seule définition. URL : https://github.com/ggml-org/llama.cpp/blob/master/ggml/src/ggml-common.h (visité le 2026-10-15).

[24] IEEE. *IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019)*. binary16 (11 bits de précision : u = 2^{-11}) et binary32 (u = 2^{-24}) ; l'arrondi du cache KV (M5). 2019. DOI : 10.1109/IEEESTD.2019.8766229. URL : https://doi.org/10.1109/IEEESTD.2019.8766229 (visité le 2026-10-16).

[25] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*. 2e éd. Bornes d'erreur d'un produit scalaire (gamma_n = n u / (1 − n u)) ; § 13.3 de O-pour-le-memoire. SIAM, 2002. DOI : 10.1137/1.9780898718027. URL : https://doi.org/10.1137/1.9780898718027 (visité le 2026-10-16).

[26] B. Zhang et R. Sennrich. « Root Mean Square Layer Normalization ». In : *Advances in Neural Information Processing Systems (NeurIPS)*. Version unique. Définition du noyau rmsnorm. 2019-10-16. arXiv : 1910.07467. URL : https://arxiv.org/abs/1910.07467 (visité le 2026-10-15).

[27] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen et Y. Liu. *RoFormer : Enhanced Transformer with Rotary Position Embedding*. v5 du 2023-11-08. Rotation par paires. gpt-oss tourne par moitiés (forme NEOX), avec les fréquences de YaRN (peng2023yarn) : c'est le noyau rope_yarn. 2021-04-20. arXiv : 2104.09864. URL : https://arxiv.org/abs/2104.09864 (visité le 2026-10-15).

[28] B. Peng, J. Quesnelle, H. Fan et E. Shippole. « YaRN : Efficient Context Window Extension of Large Language Models ». In : *arXiv* (2023-08-31). L'interpolation « NTK par parties » et le facteur de température $\sqrt{1/t} = 0{,}1 \ln s + 1$; cité par l'implémentation d'OpenAI. arXiv : 2309.00071. URL : https://arxiv.org/abs/2309.00071 (visité le 2026-10-15).

[29] G. Xiao, Y. Tian, B. Chen, S. Han et M. Lewis. « Efficient Streaming Language Models with Attention Sinks ». In : *International Conference on Learning Representations (ICLR)*. v4 du 2024-04-07. Le « puits » d'attention. Dans gpt-oss, il est appris par tête et entre dans le dénominateur du softmax (noyau attention). C'est pourquoi un biais de clé ne s'y annulerait pas, et le biais des clés vaut exactement 0 dans les deux GGUF (k3-biais-attention-*.log). 2023-09-29. arXiv : 2309.17453. URL : https://arxiv.org/abs/2309.17453 (visité le 2026-10-15).

[30] N. Shazeer. *GLU Variants Improve Transformer*. Version unique. SwiGLU. gpt-oss en emploie une forme bornée, avec α = 1,702 et u + 1 (openai2025gptossref) : c'est le noyau swiglu_oai. 2020-02-12. arXiv : 2002.05202. URL : https://arxiv.org/abs/2002.05202 (visité le 2026-10-15).

[31] NVIDIA. *CUDA Math API, Single Precision Intrinsics*. Version 13.4. Pour __fadd_rn et __fmul_rn : « This operation will never be merged into a single multiply-add instruction ». Le noyau combiner en dépend pour égaler au bit la somme de l'hôte. La cassure qui remet un fmaf fait tomber le test dès l'élément 649 (tests/passe.rs). 2026-10-12. URL : https://docs.nvidia.com/cuda/cuda-math-api/cuda_math_api/group__CUDA__MATH__INTRINSIC__SINGLE.html (visité le 2026-10-15).

[32] OpenAI. *gpt-oss : implémentation de référence en PyTorch (`gpt_oss/torch/model.py`)*. Version commit 243a1b02767da73bd2e3975be250afa801635866. Apache-2.0. Définit RMSNorm, la rotation YaRN « NTK par parties » (bornes de la rampe non arrondies, concentration $0{,}1 \ln s+1$ appliquée au cosinus et au sinus), l'attention à puits appris et fenêtre de 128 sur les couches paires, le routeur top-4 suivi d'un softmax sur les quatre valeurs, la SwiGLU bornée ($\alpha = 1{,}702$, limite 7). Lue en entier ; rien n'est copié. 2025-08-05. URL : https://github.com/openai/gpt-oss/blob/main/gpt_oss/torch/model.py (visité le 2026-10-15).

[33] OpenAI. *openai/gpt-oss-20b, `config.json`*. Version révision 6cee5e81ee83917806bbde320786a8fb61efebee. Apache-2.0. `rope_scaling` : facteur 32, contexte d'origine 4 096, β rapide 32, β lent 1, `"truncate": false` ; `layer_types` alterne fenêtre glissante (couches paires) et attention pleine. 2025-08-26. URL : https://huggingface.co/openai/gpt-oss-20b/blob/main/config.json (visité le 2026-10-15).

[34] The ggml authors. *llama.cpp, version b11515*. Version b11515, commit 3d65c90d0. MIT. Lus : `tools/perplexity/perplexity.cpp` (format du fichier `--kl-divergence-base`, codage des log-probabilités sur 16 bits, formule de la KL, quantiles), `src/models/openai-moe.cpp`, `src/llama-graph.cpp` (routage `SOFTMAX_WEIGHT`), `ggml/src/ggml.c` (`ggml_rope_yarn_corr_dims` : bornes de la rampe arrondies par `floorf` et `ceilf`), `ggml/src/ggml-cpu/ops.cpp` (`rope_yarn`, `swiglu_oai`), `src/llama-context.cpp` (facteur d'attention de YaRN), `include/llama.h` (API publique). Relus le

2026-10-15 (reprise, par raw.githubusercontent.com à l'étiquette b11515) : `ggml/src/ggml-cpu/ggml-cpu.c` (`type_traits_cpu` : MXFP4 et Q8_0 ont `vec_dot_type` = Q8_0, l'entrée d'un produit quantifié est donc ramenée en Q8_0), `ggml/src/ggml-cpu/arch/x86/quants.c` (`quantize_row_q8_0` AVX2 : d = amax/127 en f16, id = 127/amax, arrondi au plus proche), `ggml/src/ggml-cpu/ops.cpp` (attention éclair du processeur : noyau pavé quand le lot de requêtes est grand, puits appliqués au premier morceau), `ggml/src/ggml.c` (`ggml_rope_yarn_corr_dims`, revérifié). Rien n'est copié. 2026. URL : https://github.com/ggml-org/llama.cpp (visité le 2026-10-15).

[35] K. Goto et R. A. van de Geijn. « Anatomy of High-Performance Matrix Multiplication ». In : *ACM Transactions on Mathematical Software* 34.3 (2008-05). Notice vérifiée le 2026-10-15 par Crossref (api.crossref.org/works/10.1145/1356052.1356053 : titre, auteurs, revue, 34(3), pages 1–25, parution en ligne le 2008-05-16) ; le texte de l'article n'a pas été relu ce jour (dl.acm.org répond 403), p. 1-25. DOI : 10.1145/1356052.1356053. URL : https://doi.org/10.1145/1356052.1356053 (visité le 2026-10-15).

[36] T. M. Cover et J. A. Thomas. *Elements of Information Theory*. 2e éd. Chapitre 10 : théorème 10.3.2 (source gaussienne) et théorème du débit-distorsion. Hoboken, NJ : Wiley-Interscience, 2006. ISBN : 978-0-471-24195-9. DOI : 10.1002/047174882X. (Visité le 2026-10-15).

[37] E. B. Wilson. « Probable Inference, the Law of Succession, and Statistical Inference ». In : *Journal of the American Statistical Association* 22.158 (1927). Intervalle de score utilisé pour le taux de premier choix identique (C4)., p. 209-212. DOI : 10.1080/01621459.1927.10502953.

[38] S. Merity, C. Xiong, J. Bradbury et R. Socher. *Pointer Sentinel Mixture Models*. Le corpus WikiText-2 (CC BY-SA 3.0) ; `wiki.test.raw`, empreinte dans les journaux de l'étude 06. 2016-09-26. arXiv : 1609.07843. URL : https://arxiv.org/abs/1609.07843 (visité le 2026-10-15).

[39] L. A. Belady. « A Study of Replacement Algorithms for a Virtual-Storage Computer ». In : *IBM Systems Journal* 5.2 (1966), p. 78-101. DOI : 10.1147/sj.52.0078. (Visité le 2026-10-15).

[40] R. L. Mattson, J. Gecsei, D. R. Slutz et I. L. Traiger. « Evaluation techniques for storage hierarchies ». In : *IBM Systems Journal* 9.2 (1970). Les algorithmes à pile (le contenu à la capacité C est inclus dans celui à C' > C) : LRU et MIN en sont ; c'est la propriété qui rend l'étage RAM inclusif de trc-cache égal à un cache unique de sa taille (lemme 2 du lot O)., p. 78-117. DOI : 10.1147/sj.92.0078. URL : https://doi.org/10.1147/sj.92.0078 (visité le 2026-10-15).

[41] D. D. Sleator et R. E. Tarjan. « Amortized Efficiency of List Update and Paging Rules ». In : *Communications of the ACM* 28.2 (1985). Rapport compétitif de LRU : au plus N fois les défauts de l'optimum hors ligne (plus une constante), et aucune politique en ligne déterministe ne fait mieux ; la suite cyclique sur N + 1 pages en est le pire cas., p. 202-208. DOI : 10.1145/2786.2793. URL : https://doi.org/10.1145/2786.2793 (visité le 2026-10-16).

[42] NVIDIA. *CUDA Driver API Reference Manual, v13.4 : Stream Management (cuStreamWaitEvent)*. « Makes all future work submitted to hStream wait for all work captured in hEvent » : la sémantique sur laquelle repose l'ordre sûr des trois flux de trc-ordo (lemme 4). 2026-10-12. URL : https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__STREAM.html (visité le 2026-10-15).

[43] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. Federal Information Processing Standards Publication 180-4. SHA-256, dont le préfixe de 64 bits est l'empreinte64 du format ; la garde de production (M1). 2015-08. DOI : 10.6028/NIST.FIPS.180-4. URL : https://doi.org/10.6028/NIST.FIPS.180-4 (visité le 2026-10-16).

[44] OpenAI. *harmony : renderer for the harmony response format to be used with gpt-oss*. Apache-2.0. Jetons <|start|>, <|message|>, <|end|>, canaux analysis, commentary, final ; la conversation du § 8 point 6 est construite d'après ce format, les identifiants des jetons spéciaux retrouvés par leur texte dans le vocabulaire du GGUF. URL : https://github.com/openai/harmony (visité le 2026-10-15).

[45] Microsoft. *TerminateProcess function (processthreadsapi.h)*. Arrêt sans notification des bibliothèques (contrairement à ExitProcess), annulation des E/S en cours ; la sortie sur erreur sans destruction du moteur (G1). 2026-01-07. URL : https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-terminateprocess (visité le 2026-10-16).

[46] Microsoft. *JOBOBJECT_BASIC_LIMIT_INFORMATION structure (winnt.h)*. JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE : tous les processus de la tâche meurent à la fermeture de sa dernière poignée ; l'enfant nvidia-smi de l'échantillonneur (constat G2). 2024-02-22. URL : https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-jobobject_basic_limit_information (visité le 2026-10-16).

[47] Microsoft. *CreateProcessW function (processthreadsapi.h)*. bInheritHandles = TRUE : chaque poignée héritable du parent passe à l'enfant ; la cause du tuyau tenu par l'orphelin (G2) ; d'où les poignées standard rendues non héritables. 2025-07-01. URL : https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createprocessw (visité le 2026-10-16).

[48] C. E. Shannon. « A Mathematical Theory of Communication ». In : *Bell System Technical Journal* 27.3 (1948), p. 379-423. DOI : 10.1002/j.1538-7305.1948.tb01338.x. (Visité le 2026-10-15).

[49] R. Shafipour, D. Harrison, M. Horton et al. *SeedLM : Compressing LLM Weights into Seeds of Pseudo-Random Generators*. v1 du 2024-10-14, v2 du 2024-10-16 (la plus récente à la consultation). 2024-10-14. arXiv : 2410.10714. URL : https://arxiv.org/abs/2410.10714 (visité le 2026-10-16).

[50] D. Ha, A. Dai et Q. V. Le. *HyperNetworks*. v4 du 2016-12-01 (la plus récente à la consultation). 2016-09-27. arXiv : 1609.09106. URL : https://arxiv.org/abs/1609.09106 (visité le 2026-10-16).

[51] S. Ohno. *Evolution by Gene Duplication*. Métadonnées relues par l'API Crossref. Berlin, Heidelberg : Springer, 1970. DOI : 10.1007/978-3-642-86659-3. (Visité le 2026-10-16).

[52] R. P. N. Rao et D. H. Ballard. « Predictive coding in the visual cortex : a functional interpretation of some extra-classical receptive-field effects ». In : *Nature Neuroscience* 2.1 (1999). Métadonnées relues par l'API Crossref, p. 79-87. DOI : 10.1038/4580. (Visité le 2026-10-16).

[53] P. R. Huttenlocher. « Synaptic density in human frontal cortex : Developmental changes and effects of aging ». In : *Brain Research* 163.2 (1979-03). Métadonnées relues par l'API Crossref, p. 195-205. DOI : 10.1016/0006-8993(79)90349-4. (Visité le 2026-10-16).

[54] Y. LeCun, J. S. Denker et S. A. Solla. « Optimal Brain Damage ». In : *Advances in Neural Information Processing Systems 2 (NIPS 1989)*. 1989. URL : https://proceedings.neurips.cc/paper/1989/hash/6c9882bbac1c7093bd25041881277658-Abstract.html (visité le 2026-10-16).

[55] S. Han, H. Mao et W. J. Dally. *Deep Compression : Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. v5 du 2016-02-15 (la plus récente à la consultation). 2015-10-01. arXiv : 1510.00149. URL : https://arxiv.org/abs/1510.00149 (visité le 2026-10-16).